

Peng Jiang  
Liehuang Zhu

# Blockchain Technology

Cross-Chain Regulation and Privacy

 Springer

# Blockchain Technology

Peng Jiang • Liehuang Zhu

# Blockchain Technology

Cross-Chain Regulation and Privacy

 Springer

Peng Jiang  
School of Cyberspace Science  
and Technology  
Beijing Institute of Technology  
Beijing, Beijing, China

Liehuang Zhu  
School of Cyberspace Science  
and Technology  
Beijing Institute of Technology  
Beijing, Beijing, China

ISBN 978-981-96-4394-3      ISBN 978-981-96-4395-0 (eBook)  
<https://doi.org/10.1007/978-981-96-4395-0>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2025

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.  
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

If disposing of this product, please recycle the paper.



*We pay our highest respect and gratitude to the editors for their support and advice in publishing this book and thank all reviewers for their precious comments in bettering this book. We would like to thank Institute of Cryptography Technology and Data Security in School of Cyberspace Science and Technology in Beijing Institute of Technology, for their help in completing this book. We further appreciate our families for their unconditional love, care, and encouragement. We would have never completed this book without their supports. A sincere appreciation to both Peng's family members and Liehuang's family members.*

# Foreword

This book focuses on the cross-chain regulation and privacy in the blockchain technology and especially explores significant challenges and corresponding solutions. This book provides a comprehensive understanding on how to design the full life cycle regulation mechanisms from multi-dimensional detection to efficient provenance tracking, generic privacy protection cross-chain system frameworks, lightweight privacy-preserving mechanisms, and how to reach a balance between regulation and privacy in cross-chain environments. The main contents of this book are derived from the most latest technical achievements of the authors in the blockchain field. The involved solutions integrate both artificial intelligence methods and cryptographic algorithms into the cross-chain transaction systems. This book includes both theoretical and practical contents to assist readers to have a better understanding to the cross-chain regulation and privacy technologies.

This book not only facilitates researchers to acquire some basic knowledge of cross-chain technology but also supports them to find some new insights in cross-chain regulation and privacy, and is hence of particular interest to researchers in the field of blockchain, privacy concern, and network security.

# Preface

The blockchain technology's rapid development and widespread applications has brought a diverse and heterogeneous blockchain ecosystem. The independence operation, like isolated data islands, increases the difficulty of data exchange among blockchains. The cross-chain technology in state emerges, aiming to connect independent blockchains and facilitate their asset transfer and information interaction. As a newly promising technology, cross-chain with strong interoperability receives attention from both academic and industry. However, the occurrence of cross-chain financial incidents exposes significant information leakage and property loss. This makes regulation and privacy to be desirable concerns in cross-chain environments. The lack of uniformity in cross-chain architecture and common plain-format transaction increases the difficulty of both regulation and privacy preserving.

This book, for the first time, explores the challenges and solutions related to cross-chain regulation and privacy. The lack of uniformity in cross-chain architecture and common plain-format transaction increases the difficulty of both regulation and privacy preserving. This book provides a comprehensive understanding on how to design the full life cycle regulation mechanisms from multi-dimensional detection to efficient provenance tracking, generic privacy protection cross-chain system frameworks, lightweight privacy-preserving mechanisms, and how to reach a balance between regulation and privacy in cross-chain environments.

This book comprehensively summarizes the cross-chain architecture and classification, analyzes security challenges and potential requirements, and brings a number of innovative technologies and practical solutions for cross-chain regulation and privacy.

This book is of particular interest to researchers in the field of blockchain and privacy concern, and it also helps to find some new insights in cross-chain technology. We also welcome any comments and suggestions.

Beijing, China  
December, 2024

Peng Jiang  
Liehuang Zhu

# Acknowledgments

We would like to express their highest and truest gratitude to all the participants who contribute and support this work. All the supports, advice, and encouragement from peer experts and colleagues are remarkably meaningful and invaluable for the accomplishment of this book. When we completed the first manuscript, we invited some master students and PhD students to make proofreading. Specially, we would like to thank Yuan Yao, Si Yi, Guifang Zeng, Yimin Liu, Qi Liu, for their help in improving this book.

We would like to thank all authors of published references that have been cited in this book. We would like to thank the Springer editor Jingying Chen and Kamesh Senthilkumar, who gave us a lot of insightful comments and advice that have indeed reinforced and raised this book. We would also like to thank all editors of Springers who joined this book project.

This book is supported by National Key R&D Program of China (Grant No. 2022YFB2702903), NSFC (Grant No. 62232002).

Beijing, China  
December, 2024

Peng Jiang  
Liehuang Zhu

# Contents

|          |                                                                           |    |
|----------|---------------------------------------------------------------------------|----|
| <b>1</b> | <b>Introduction</b>                                                       | 1  |
| 1.1      | Background of Blockchain                                                  | 1  |
| 1.1.1    | Concept of Blockchain                                                     | 1  |
| 1.1.2    | Structure of Blockchain                                                   | 4  |
| 1.1.2.1  | Layer Architecture                                                        | 4  |
| 1.1.2.2  | Format Constitution                                                       | 5  |
| 1.1.3    | Development of Blockchain                                                 | 7  |
| 1.1.3.1  | Applications                                                              | 7  |
| 1.1.3.2  | Global Present State                                                      | 8  |
| 1.1.3.3  | Future Development Trend                                                  | 10 |
| 1.2      | Cross-Chain Overview                                                      | 12 |
| 1.2.1    | What Is Cross-Chain                                                       | 12 |
| 1.2.2    | Cross-Chain Types                                                         | 14 |
| 1.2.2.1  | Homogeneous vs. Heterogeneous                                             | 14 |
| 1.2.2.2  | Notary vs. Hash-locking vs. Sidechain/relay                               | 14 |
| 1.3      | Present Challenges and Security Requirements<br>of Cross-Chain Technology | 19 |
| 1.3.1    | Present Challenges                                                        | 19 |
| 1.3.2    | Security Requirements                                                     | 24 |
| 1.3.2.1  | Regulation                                                                | 24 |
| 1.3.2.2  | Privacy                                                                   | 25 |
|          | References                                                                | 26 |
| <b>2</b> | <b>Foundations</b>                                                        | 29 |
| 2.1      | Classical Blockchain Systems                                              | 29 |
| 2.1.1    | Bitcoin                                                                   | 29 |
| 2.1.2    | Ethereum                                                                  | 31 |
| 2.1.3    | ChainMaker                                                                | 35 |
| 2.1.4    | Monero                                                                    | 37 |
| 2.2      | Classical Cross-Chain Systems                                             | 39 |
| 2.2.1    | Cosmos                                                                    | 39 |

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| 2.2.2    | Poldadot .....                                                                        | 41        |
| 2.2.3    | Zendoo .....                                                                          | 42        |
| 2.2.4    | Horizen .....                                                                         | 45        |
| 2.3      | Preliminaries of Cross-Chain Regulation .....                                         | 46        |
| 2.3.1    | Feature Engineering .....                                                             | 46        |
| 2.3.2    | Graph Embedding Learning .....                                                        | 46        |
| 2.3.3    | Graph Neural Networks .....                                                           | 49        |
| 2.3.3.1  | Graph Convolutional Networks .....                                                    | 49        |
| 2.3.3.2  | Graph Attention Networks .....                                                        | 50        |
| 2.3.4    | Traffic Detection Methods .....                                                       | 51        |
| 2.3.4.1  | Detection Method Based on Node Traffic<br>Statistical Analysis .....                  | 51        |
| 2.3.4.2  | Detection Method Based on Machine Learning ....                                       | 52        |
| 2.4      | Preliminaries of Cryptography-Empowered Privacy Preserving.....                       | 53        |
| 2.4.1    | Accumulator .....                                                                     | 53        |
| 2.4.2    | Signature of Knowledge .....                                                          | 55        |
| 2.4.3    | Commitment .....                                                                      | 56        |
| 2.4.4    | BulletProofs .....                                                                    | 58        |
|          | References .....                                                                      | 59        |
| <b>3</b> | <b>Cross-Chain Abnormal Account Detection .....</b>                                   | <b>63</b> |
| 3.1      | Cross-Chain Account Detection via Machine Learning.....                               | 63        |
| 3.1.1    | Background and Motivation.....                                                        | 63        |
| 3.1.2    | CrossAAD Overview .....                                                               | 64        |
| 3.1.3    | Detailed Design of CrossAAD .....                                                     | 66        |
| 3.1.3.1  | Dataset Construction .....                                                            | 66        |
| 3.1.3.2  | Feature Extraction and Processing.....                                                | 68        |
| 3.1.3.3  | Model Detection.....                                                                  | 71        |
| 3.1.4    | Performance Evaluation and Analysis .....                                             | 75        |
| 3.1.4.1  | Experimental Setup.....                                                               | 75        |
| 3.1.4.2  | Evaluation Metrics .....                                                              | 76        |
| 3.1.4.3  | Classification Performance.....                                                       | 76        |
| 3.1.5    | Summary .....                                                                         | 80        |
| 3.2      | Cross-Chain Abnormal Transaction Detection<br>via Graph-Based Multi-model Fusion..... | 80        |
| 3.2.1    | Background and Motivation .....                                                       | 80        |
| 3.2.2    | GMMCCT Overview .....                                                                 | 82        |
| 3.2.3    | Detailed Design of GMMCCT .....                                                       | 83        |
| 3.2.3.1  | Source of Dataset .....                                                               | 83        |
| 3.2.3.2  | Data Preprocessing .....                                                              | 84        |
| 3.2.3.3  | Graph Construction.....                                                               | 85        |
| 3.2.3.4  | Feature Extraction .....                                                              | 85        |
| 3.2.3.5  | Multi-model Fusion .....                                                              | 86        |
| 3.2.4    | Performance Evaluation and Analysis .....                                             | 88        |
| 3.2.4.1  | Experimental Setup.....                                                               | 88        |



|            |                                                                         |            |
|------------|-------------------------------------------------------------------------|------------|
| 3.2.4.2    | Evaluation Metrics .....                                                | 88         |
| 3.2.4.3    | Classification Performance .....                                        | 90         |
| 3.2.5      | Summary .....                                                           | 91         |
| References | .....                                                                   | 92         |
| <b>4</b>   | <b>Cross-Chain Network Anomaly Detection .....</b>                      | <b>95</b>  |
| 4.1        | Background and Motivation .....                                         | 95         |
| 4.2        | Methodology Overview .....                                              | 97         |
| 4.3        | Detailed Design of Network Node Detection Mechanism .....               | 101        |
| 4.3.1      | Detection Environment for Cross-Chain Network Nodes ....                | 101        |
| 4.3.1.1    | Five-Layer Cross-Chain Protocol Stack .....                             | 101        |
| 4.3.1.2    | Cross-Chain Interaction Process .....                                   | 102        |
| 4.3.2      | Visual Network Node Monitoring System .....                             | 105        |
| 4.3.2.1    | Configure Prometheus .....                                              | 105        |
| 4.3.2.2    | Configure Grafana and set up an email<br>alerting system .....          | 106        |
| 4.4        | Performance Evaluation and Analysis .....                               | 108        |
| 4.5        | Summary .....                                                           | 111        |
| References | .....                                                                   | 111        |
| <b>5</b>   | <b>Cross-Chain Traffic Anomaly Detection .....</b>                      | <b>113</b> |
| 5.1        | Background and Motivation .....                                         | 113        |
| 5.2        | Methodology Overview of Cross-Chain Traffic Detection Model ....        | 116        |
| 5.3        | Detailed Design of Cross-Chain Traffic Detection Model .....            | 117        |
| 5.3.1      | Data Collection Module .....                                            | 118        |
| 5.3.2      | Feature Extraction Module .....                                         | 118        |
| 5.3.3      | Traffic Monitoring Module .....                                         | 119        |
| 5.4        | Performance Evaluation and Analysis .....                               | 121        |
| 5.4.1      | Indicators for Model Evaluation .....                                   | 121        |
| 5.4.2      | Assessment of Models .....                                              | 121        |
| 5.5        | Summary .....                                                           | 122        |
| References | .....                                                                   | 123        |
| <b>6</b>   | <b>Cross-Chain Provenance Tracking .....</b>                            | <b>125</b> |
| 6.1        | Background and Motivation .....                                         | 125        |
| 6.2        | Methodology Overview of Cross-Chain Provenance<br>Tracking Scheme ..... | 127        |
| 6.3        | Detailed Design of Cross-Chain Provenance Tracking Scheme .....         | 128        |
| 6.3.1      | Data Preparation Module .....                                           | 128        |
| 6.3.2      | Address Clustering Module .....                                         | 131        |
| 6.3.3      | Cross-Chain Provenance Tracking Module .....                            | 132        |
| 6.4        | Performance Evaluation and Analysis .....                               | 135        |
| 6.5        | Summary .....                                                           | 137        |
| References | .....                                                                   | 138        |

- 7 Generic Privacy-Aware Cross-Chain Framework** ..... 141
  - 7.1 Problem Statement ..... 141
  - 7.2 Sharding-Based Cross-Chain Framework..... 142
    - 7.2.1 Cross-Chain Sharding ..... 142
    - 7.2.2 Privacy-Preserving Sharding-Based Cross-Chain Framework Overview..... 144
    - 7.2.3 Framework Analysis ..... 145
    - 7.2.4 Summary ..... 146
  - 7.3 Encrypted Cross-Chain Bridge Framework..... 146
    - 7.3.1 Methodology Overview ..... 146
    - 7.3.2 System Overview ..... 147
      - 7.3.2.1 CPAP Architecture ..... 147
      - 7.3.2.2 System Goals ..... 149
    - 7.3.3 Detailed CPAP Design..... 149
      - 7.3.3.1 Building Blocks ..... 149
      - 7.3.3.2 CPAP Construction ..... 151
    - 7.3.4 Framework Analysis ..... 154
    - 7.3.5 Performance Evaluation and Analysis ..... 155
      - 7.3.5.1 Implementation Setting ..... 155
      - 7.3.5.2 Generation Test ..... 156
      - 7.3.5.3 Uploading Test ..... 157
      - 7.3.5.4 Throughput Test ..... 157
      - 7.3.5.5 Gas Discussion ..... 158
    - 7.3.6 Summary ..... 159
  - References ..... 159
- 8 Lightweight Cross-Chain Privacy-Preserving Protocol** ..... 161
  - 8.1 Background and Motivation ..... 161
  - 8.2 PTCross Overview ..... 163
  - 8.3 Detailed Design of PTCross ..... 164
  - 8.4 Performance Evaluation and Analysis ..... 166
  - 8.5 Summary ..... 168
  - References ..... 168
- 9 Balance of Privacy and Regulation for Cross-Chain Transactions** ..... 171
  - 9.1 Background and Motivation ..... 171
  - 9.2 PCP Overview ..... 172
  - 9.3 Detailed Design of PCP..... 174
  - 9.4 Security Analysis..... 179
  - 9.5 Performance Evaluation and Analysis ..... 180
  - 9.6 Summary ..... 183
  - References ..... 183

# Chapter 1

## Introduction



### 1.1 Background of Blockchain

Blockchain was invented by Satoshi Nakamoto in 2008 to serve as a core technology for Bitcoin (Nakamoto 2008). It facilitates the process of recording transactions and tracking assets in a shared and immutable way, which consists of blocks that are securely linked together via cryptographic hashes. Each block contains a cryptographic hash of the previous block, a timestamp, and transaction data (generally represented as a Merkle tree, where data nodes are represented by leaves), and all of blocks are added to the chain after verification. Since each block contains information about the previous block, they effectively form a chain (compare linked list data structure), with each additional block linking to the ones before it. Consequently, blockchain transactions are irreversible. Once they are recorded, anyone cannot backdate, deny, or destroy the information stored in these blocks. In Bitcoin, the blockchain works in a hash-like way to timestamp blocks in a decentralized network without introducing a difficulty parameter for stability. The implementation of the blockchain into Bitcoin can also solve the double-spending problem while there is no need of a central authority or a central server. The blockchain has inspired many applications including public cryptocurrencies and other distributed management systems (The Economist 2015; Times 2016; Lunn 2018; Narayanan et al. 2016; Morris 2016).

#### 1.1.1 Concept of Blockchain

Currently, it seems that there exist kinds of definitions about the blockchain. From a technical point of view, blockchain is a distributed and decentralized ledger of cryptographically signed and irrevocable blocks of transactions, and all of the information is shared by all participants within a certain ecosystem. Blockchain

maintains a continuously growing data records resistant to any changes, where all of the records should have been confirmed by other nodes in the network. A copy of the list of records is globally stored and updated on all of network nodes and is always maintained safe even in the presence of unreliable participants (IBM 2024).

**Components** The main components of the blockchain are described as follows (Shoemaker 2023).

- **Node.** Blockchain nodes play a crucial part in a blockchain network, and they together maintain the distributed ledger by processing transactions and validating new blocks. Each node is unique and identified through an attached identifier. The number of nodes also affect the capability of the blockchain network. There are various types of blockchain nodes with different functions. Each node in the blockchain network holds a copy of the distributed ledger and gets the latest updated blockchain.
- **Ledger.** A blockchain ledger is a digital record of transactions and data stored in a distributed manner across several computers globally (decentralized). This ledger is a chain of blocks, with each block containing a list of transactions. These blocks are replicated across multiple computers in the blockchain network, making it impenetrable for anyone to tamper with or hack the ledger.
- **Peer-to-Peer (P2P) network.** P2P network plays the underlying architecture of the decentralized blockchain, which enables participants to interact with each other and conduct transactions without intermediaries or centralized authorities. P2P network consists of interconnected nodes or devices that communicate with each other, performs transactions, and stores transaction records in a transparent, immutable, and unforgeable way.
- **Wallet.** Blockchain wallets (digital wallets) actively manage cryptocurrencies, enabling users to hold and perform financial transactions securely. These transactions and their associated records are cryptographically signed, making them more secure than traditional banking systems and protecting the privacy of users' data. The users' private keys are stored in their wallets, which are essential for accessing the cryptocurrency assigned to each individual. While the cryptocurrency remains on the blockchain, the users' wallets store their data.
- **Nonce.** A nonce is a number that miners must solve to add a new block to the blockchain. Before adding a new block, a miner must obtain the nonce number, which can be used only once to create the block. Once a miner adds a block with a valid nonce, they receive a reward, making it impossible for a previously recorded block to be recorded multiple times. Nonce is similar to a one-time password (OTP) that miners must solve, but instead of being sent by a financial institution, miners must solve for it. The tedious process of obtaining the number is called "proof of work" in blockchain consensus mechanisms.
- **Cryptographic hash.** The hash function can map the data into a fixed size. In a blockchain network, the hash value of one transaction is the input of another transaction. A block contains information about previous blocks via cryptographic hash, timestamp, and transactions.

- **Consensus mechanism.** Blockchain is created to reach a consensus on actions conducted in data storage systems, thus ensuring consistency that contributes to verification, atomicity, durability, and information integrity. To achieve consensus in blockchain, a stack of protocols, incentives, and systems enable a network of nodes to agree on transactions and the state of the blockchain, creating a democracy or governance approach that keeps the blockchain secure and functional. Independent nodes can verify transactions and update the ledger depending on the consensus system applied by each blockchain. Consensus mechanisms can vary, with some nodes updating the blockchain records based on work done, others based on stakes, and some based on authority. The common consensus mechanisms are like proof of work, proof of stake, etc.

**Properties** The blockchain can provide the following basic properties (Geeks for Geeks [2024b](#)).

- **Distribution.** All the nodes in the network have a copy of all the information contained in that blockchain. Information isn't processed through a central server, but is transmitted and verified by nodes in parts of the network. The network functions based on a set of rules that every client must follow exactly; if blocks are broadcast to the network and do not follow the validity rules, the block will be rejected. A blockchain's network is distributed, allowing for an egalitarian, peer-based network that can self-check.
- **Decentralization.** There is no central authority controlling the blockchain network. Instead, the network is made up of a large number of nodes that work together to verify and validate transactions. Each and every node in the blockchain network will have the same copy of the ledger.
- **Immutability.** Once a block is confirmed, the data recorded to the blockchain cannot be removed or edited. Each block is stacked upon the previous block. The next block must have the preceding block's hash in order to be added to the chain. This assures that the blockchain stays in chronological order, effectively making it tamper-proof.
- **Trust.** The underlying protocol and the consensus mechanism in the blockchain ensure the validity of transactions without the need for intermediaries. That is, no one can add any transaction blocks to the ledger without the approval of a majority of nodes, and any validated records are irreversible and cannot be changed.
- **Transparency.** All transactions on blockchain are viewable to anyone with an internet connection. Each transaction is assigned its own unique ID known as a transaction hash. These identifiers can be used to look up a public record of the transaction, also known as a transaction receipt. A transaction receipt includes the addresses involved, the amount transferred, a timestamp, transfer fees, and more.
- **Security.** Blockchains have two primary security mechanisms: network structure and cryptography. In blockchain technology, cryptography is used extensively to sign data in order to prove that a transaction was approved by the owner of the funds. The decentralized structure of network eliminates any central points

of failure. To compromise an open blockchain network, a hacker would have to control a majority of the nodes at the same time. This makes a network attack very expensive and impractical.

**Types** Depending on the use and access rights, blockchains have been categorized into three types, public, private, and consortium (Geeks for Geeks [2024a](#)). Each blockchain serves its purpose and solves particular problems. Also, each blockchain has its own set of features and advantages over one another.

- *Public blockchain.* Anyone can access the network. It is zero trust in which nodes are pseudo-anonymous, and rule compliance is enforced by incentives and by a competitive process, such as a math puzzle-solving scheme (known as proof of work).
- *Private blockchain.* Only users with specific authorization can be allowed to access the network. It is based on trusted parties and strong digital identities, so rule compliance is imposed by a number of cooperating authorities and accepted by all the participants in the network.
- *Consortium blockchain.* A restricted group of nodes instead of a single central node govern the network. This structure ensures decentralization, an element that characterizes Blockchain, making all nodes independent from each other, but subject to mutual control.

## 1.1.2 Structure of Blockchain

### 1.1.2.1 Layer Architecture

According to the functionality, the blockchain includes five layers as depicted in Fig. [1.1](#). They are the hardware layer, the data layer, the network layer, the consensus layer, and the application layer from bottom to up (Javeri [2019](#); Cyfrin Team [2024](#); John et al. [2024](#)).

- **Hardware layer.** The hardware layer provides the necessary infrastructure, such as servers, mining equipments, and mobile devices, to host a blockchain. As the physical foundation of the blockchain, it encompasses all of hardware components for executing the transaction and performing computations. It can also include some virtual machines that function like operating systems and run some contracts.
- **Data layer.** The data layer is used to store and maintain various kinds of blockchain data, such as block information, transaction records, timestamps, hash links, and ledger data, in a structured and unchangeable way. The data layer has good properties of immutability and distributed storage, with each block connected to its predecessor in a chain form. It also introduces some cryptographic algorithms for secure and persistent transaction information transmission.

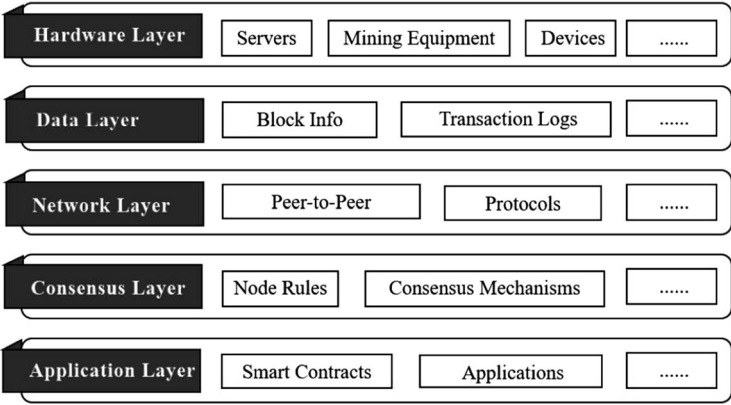


Fig. 1.1 Blockchain’s layer architecture

- Network layer. The network layer, with a peer-to-peer capability, enables the network nodes to self-organize and interact in a decentralized way. It comprises communication protocols that connect nodes within the chain and allows peer-to-peer transactions. That is, it works without a central control and, hence, averts possible risks like the single point of failure.
- Consensus layer. The consensus layer facilitates agreement among nodes on the true state of the blockchain. It sets rules for node agreement on the current chain state and various consensus algorithms (e.g., Proof of Work (PoW), Proof of Stake (PoS)), determining who can add new entries to the blockchain and how these participants are selected. Consensus mechanisms can affect both stability and defense of the blockchain network, guaranteeing both security and reliability of the whole system.
- Application layer. The application layer usually includes various blockchain use cases and specific products for the end user, such as digital currencies, lending, and business contexts. The smart contracts and apps are built on the application layer assisted with other softwares. It is also the interface between the blockchain and the end user, implementing the blockchain technology for diverse services and scenarios.

1.1.2.2 Format Constitution

As we know, a blockchain is composed of blocks that includes multiple valid transactions. Usually, the transaction data, as the basic unit, are first written as blocks, and the new block is thereafter chained to the earlier block, except the original block. Blocks can be added to the main chain only after obtaining consensus from the peer nodes. All of blocks after validation are chained based on the chronology of timestamp and cryptographic principles (Witscad 2024).

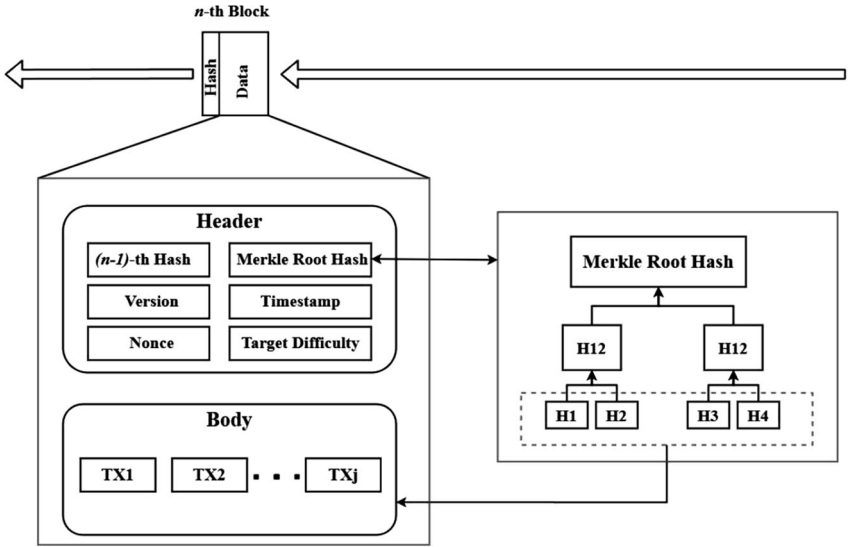


Fig. 1.2 Blockchain format

Figure 1.2 illustrates a blockchain’s generic format, which is a sequence of blocks holding a complete list of transaction records (Teimoori and Yassine 2022). Each block consists of the block header and the block body.

- The block header includes the block version, which indicates the set of block validation rules, the merkle tree root hash, timestamp, an *n*-bit string that targets threshold of a valid block hash, the nonce, the hash of the parent block.
- The block body is a collection of transaction records, including both the transaction counter and the detailed transaction information. As aforementioned, the transactions can be involved into the block as long as they are validated. In the blockchain, validating the authentication of transactions relies on an asymmetric cryptography mechanism named digital signature. For the capacity, the maximum number of transactions in a block usually depends on the size of the block and the size of each transaction.

A basic workflow from a new transaction to its inclusion into the blockchain is described as follows. The first phase is the request, i.e., a user requests a new transaction from their wallet applications and broadcasts it. The newly requested transaction then enters into a pool of unconfirmed transactions and waits the picking of the miners. The miners on the blockchain network pick up transactions and mine them to create a new block along with some associated metadata. The miner finds a hash output for the block and broadcasts the block with its signatures. The other nodes in the network verify the signature of the block to establish its validity. Once the validity is confirmed, a consensus is reached to add the block to the existing blockchain. Other blocks that have been added on top of this block are considered



as the block confirmation. As blocks get added, the blockchain grows, and already-existing blocks cannot be changed.

### ***1.1.3 Development of Blockchain***

#### **1.1.3.1 Applications**

Due to unique benefits, blockchain has wide application fields, especially in finance, healthcare, government, and inspires a multitude of use cases (Daley 2024; Kazim 2024).

- **Finance.** Blockchain technology is originated from Bitcoin and also plays a core role in the financial digital transformation. Blockchain in kinds of financial services enables reliable processes, reasonable costs, and secure networks. It has evolved from cryptocurrency to securing loans, digital transactions, and online payments, where digital currency is one of the main finance applications. In this case, financial trading and transactions are faster and more secure in a standardized and normalized environment. Blockchain has also made cross-border payments faster and safer due to real-time tracking and secure payment gateways and thereafter brings highly cost-effective method into financial services. Blockchain has enabled banks and customers to transact more frequently with reducing the element of a middleman and transaction fees. Blockchain has transparent and auditable transaction records, which helps to support financial institutions with regulatory requirements, and banks can also intercept suspicious transactions and digital banking activities on time by streamlining the auditing process.
- **Healthcare.** Blockchain technology has been applied into the healthcare field for better medical service. It can modernize medical operations, ensure healthy data integrity, enhance patient care, and enable healthy data interoperability in real time. This improves administrative standardization, reliability and efficiency. To achieve secure electronic health records (EHRs), a blockchain-based system can be linked to existing EHR softwares. The blockchain-based system stores and manages EHRs in a secure and decentralized way and provides a single view of a patient's record. It also provides patients the rights to approve changes on their EHRs, authorize who can access, and control the sharing mode with healthcare providers. Patient monitoring solutions have created one of the biggest shifts in the medical world. Healthcare practitioners are using advanced sensors to measure patients' vital signs to provide enhanced preventative and proactive care remotely. During such an application, blockchain works with additional encrypted communication and securing such devices from cyberattacks tampering with personal data.
- **Government.** Blockchain technology in the government applications is mainly to protect data, streamline processes, and reduce negative fraud or abuse, while supporting both trust and accountability of government affairs. In a blockchain-based

government model, individuals, businesses, and governments negotiate information sharing over a distributed and cryptographic ledger. Such a system structure eliminates a single point of failure and inherently protects sensitive government data. Specially, the blockchain-supported ledger can streamline regulation and enhance reaction capacity in the event of emergency, and the blockchain-supported administration system enables real-time coordination between wide-ranging participants while allowing the necessary data/behavior transparency. Also, blockchain can support voting by transparency and immutability of on-chain information in the collation period.

- **Other Industries.** Blockchain technology also has advantage applications in kinds of industry use cases. Supply chain management is one of the most common blockchain-based industrial systems. Blockchain can facilitate accurate asset tracking, enhance licensing of services, products, and software, and transparency into the provenance of consumer goods, from sourcing to the point of consumption. Thus, companies employ blockchain-based supply chain management to simplify compliance tracking, optimize maintenance, and even manage inventory. They're capitalizing on blockchain's ability to streamline operations, increase visibility into the supply chain, and facilitate trusted interactions within an ecosystem of industries. Product life-cycle monitoring is another use case. Blockchain can be used to track products and their components throughout their entire life cycle, from raw material collection to delivery to customers. All the product data can be immutably recorded, and blockchain-based product life-cycle monitoring system provides a wealth of data about the historical reliability.

### 1.1.3.2 Global Present State

Currently, the worldwide market for blockchain and cryptocurrency continues to pull interest from both public and private sectors. Some reports and studies have shown that most developing and well-established nations are dominating more possibilities of blockchain technology, and the digital market is progressively attaining momentum with growing trends, where the global Blockchain market cap has hit 3 trillion (Toti 2024). According to Zippia, there exist more than 400-million cryptocurrency users (i.e., approximately 3.9% of the global population) and 82-million blockchain wallet users in the whole world (Ariella 2023). About 400,000 Bitcoin transactions and 1 million Ethereum transactions are carried out worldwide every day (Zmudzinski 2019). The number has shown a significant growth over the past few years and still keeps increasing.

At a regional level, we summarize states of several main regions (Fortune Business Insights 2024; The World Economic Forum 2020; Markets and Markets 2024). North America and East Asia are at the forefront of technological innovation, with significant investments in both the research of blockchain technology and the development of blockchain-associated industries. These advancements have also driven the utilization of blockchain. The United States devotes itself to approve and promote the blockchain technology, from setting up research associations to

digital currency payments. Meanwhile, the federal government expects to increase investments in the blockchain development. To date, nearly 50% of blockchain-associated startup companies are set up in the United States. China has thrown its weight behind the blockchain technology. President Xi Jinping highlighted the importance of blockchain, pointing out “seize the opportunity” and “develop the digital economy.” This endorsement brings a flurry of activities targeting the topic of the blockchain. Government-led initiatives, such as blockchain service network (BSN), aim to offer a reliable and secure public infrastructure to support domestic projects about the blockchain technology. It also encourages international collaboration among both academia and industry, forging a healthy blockchain ecosystem. Japan also develops its standout blockchain with the well-defined regulatory framework. The government introduced a comprehensive legal framework to reign cryptocurrencies and the blockchain technology. This can ensure investor protection and instill confidence and benefits in business to explore blockchain-based solutions. The Japan Exchange Group’s initiatives work well and have gained international attention. EU puts a high premium on the resolution of the blockchain technology. It provides sufficient supports with investments for startups and projects for universities and research institutes. The two-side collaboration activates innovation and paves the way for cutting-edge resolutions. At the same time, the European Commission introduces a package of legislative proposals, standards, and regulation principles in promoting and normalizing the blockchain technology. The known blockchain companies include AWS (US), Oracle (US), IBM (US), Huawei (China), Accenture (Ireland), TCS (India), etc. In 2023, AWS launched the “Amazon manages blockchain (AMB)” query that offers serverless access to standardized datasets from multiple blockchains through developer-friendly APIs. In 2024, Casper Labs partnered with IBM Consulting to develop a blockchain solution from AI systems. This partnership aimed to improve transparency and auditability in generative AI, allowing users to understand the data and processes behind AI-generated outputs.

In the present blockchain market, blockchain-as-a-service (BaaS) is an enterprise-level platform service as a main segment. BaaS is the third-party creation and management of cloud-based networks for companies that build blockchain applications. According to the cloud type, there exist public cloud-based BaaS, private cloud-based BaaS, and hybrid cloud-based BaaS. By providing a cloud-based solution, BaaS eliminates the need for hefty upfront investments and streamlines the development process. Businesses can leverage the pre-built infrastructure and the development toolkit to rapidly deploy their blockchain applications, gaining a competitive edge in the marketplace. These third-party services are a relatively new directions in the growing field of the blockchain technology.

### 1.1.3.3 Future Development Trend

The global blockchain technology market size was estimated at \$10 billion by the end of 2022, and by 2030, it is expected to reach \$3.1 trillion though total amounts of investments are unpredictable because of its constant acceleration (Brezitska 2024).

- **Blockchain-as-a-Service (BaaS).** Blockchain-as-a-Service (BaaS) is a cloud-based and enterprise-level infrastructure that allows the leverage of cloud solutions to deploy kinds of blockchain applications. BaaS provides a business with access to a blockchain network of its desired configuration without developing their own blockchain, such that any business can develop and deploy its blockchain application without managing complicated infrastructure. BaaS also allows businesses to select the most suitable blockchain protocol to meet their requirements. It has been regarded as one of the key blockchain service trends due to its practical usage. Giant companies like Microsoft and Amazon are already offering similar blockchain services, which play the role of web hosting providers with the responsibility of infrastructure deployment and maintenance. For example, Hyperledger Fabric by Amazon is indeed a BaaS provider. The BaaS provider offers some common support activities to users and allows businesses to concentrate on their functions because there would be no need to create an entire blockchain environment from scratch.
- **IoT with blockchain infrastructure.** The Internet of Things (IoT) has mature development and is constantly growing with technical progress. IoT makes a good and convenient environment with compatibility of hardware, software, and cloud solutions. It generates, collects, and transmits large amounts of data, but its traditional system is centralized without enough robustness. IoT with blockchain infrastructure has been one of the key blockchain trends, which aims to create a decentralized and distributed architecture with cryptography-based security guarantees. The blockchain-based IoT systems provide new opportunities, including new applications and new solutions, to kinds of IoT services. By establishing transparent and automated environment, IoT systems integrated with blockchain can be applied to smart city field, which keeps tracking numerous indicators and ensures real-time communication between governments, citizens, and organizations. To date, there have been some cities that adopted the blockchain-empowered IoT technology and gained outstanding results.
- **Interoperability.** With prosperous developments of the blockchain technology, many new blockchains appear with different reasons that include digital payments, reliable storage, etc. As traditional work mode, each blockchain stands in a separate system with its independent running rule, and there is usually little interoperability among different blockchains due to the difficulty of connection implementation. Enabling communication and transactions among different types of blockchains is one of the key trends in the blockchain technology. The interoperability and cooperation of cross blockchains can bring an interconnected ecosystem while maintaining advantage of each blockchain

member. Such a cross blockchain can also help researchers and engineers to construct complicated applications with better features, although it might be up against more predictable and unpredictable technical challenges.

- **Asset tokenization.** Tokenizing the assets is a way of creating digital tokens via embodying certain physical token into assets. As we know, assets like real estate, commodities, art pieces have different values and properties, but all of them may be divided into little units and tokens, which become digitized. These units or tokens represent ownership and can be traded on different blockchain platforms. For example, Ethereum is one of the most popular blockchain platforms, and tokens built on the Ethereum blockchain are usually secured by the ERC-20 protocol. It allows users to claim a share or total ownership of these tokens. The digital assets provide the opportunity to buy a share of an asset without buying the entire one. Due to aforementioned advantages, asset tokenization becomes one of the current trends in the blockchain technology among investors.
- **NFT.** Non-fungible token (NFT) is a similar concept to asset tokenization. Both are digital representations of value, while NFT has some special features that NFT is unique and cannot be replaced by another token with equal value. Besides, it can connect the physical item with the digital world. NFT can be only digital or be a digital replica or evidence of ownership for the physical asset. Due to its uniqueness, NFT rises and is usually used for some special assets like artworks, which supports stronger security without tamper or counterfeiting. There are a lot of created NFTs in music, fashion, gaming, luxury goods, and ticket sales. New trends in the blockchain application represent the expansion of NFTs' usage area.
- **Metaverse.** Metaverse is envisioned as another version of the Internet that is a universal and immersive virtual world, facilitated by the use of virtual reality (VR) and augmented reality (AR) headsets. In the metaverse, people can work, relax, socialize, and do multiple digital interactions. The metaverse is supposed to be quite different from all the virtual experiences and real world we have already had. It has some common characteristics with the gaming world, but its final form is yet to be determined. Currently, most of the metaverse's core attributes can be real thanks to the blockchain technology. Blockchain-based network allows the metaverse to build secure architecture and set the universal running rules. It has to support interoperability between different worlds to communicate and exchange information. The ownership of data can be guaranteed, and the security of digital assets is provided by the blockchain ledger. All of the procurements are with cryptocurrency. The blockchain technology indeed plays a crucial and inseparable role in the implementation of the metaverse, which conversely becomes one of development trends in the blockchain's application.

## 1.2 Cross-Chain Overview

### 1.2.1 What Is Cross-Chain

Emergence of diverse blockchains promotes the birth of cross-chain technology. For the traditional blockchain, each chain operates independently and stands as a isolated field. Significant structural and principle differences among these blockchains result in limited interoperability. In the multiple blockchain scenarios, data and value are confined within their respective “data islands,” which creates the technical barriers for the information exchange and transfer. In this case, the current blockchain field faces bottlenecks in interoperability and interoperation that conversely restricts further development of blockchain technology. To address this issue, there is an urgent need for a new cross-chain technology to coordinate collaborative work among different blockchains (Chainlink 2023). Such a coordination can not only improve the overall performance and scalability of blockchain systems but also enhance the adaptation to complicated application scenarios. As illustrated in Fig. 1.3, the cross-chain technology also has the potentials to drive blockchain toward becoming another internet of value.

Cross-chain refers to that the information on chain A has been confirmed via consensus mechanism and transferred to chain B safely and reliably, and chain B can verify the information and execute subsequent operations (ZetaChain 2023). We take a blockchain-based financial platform as example, where there exists asset on Alice’s account. Supposing that Alice has a requirement of transfer and the appointed receiver Bob is only registered to another blockchain platform, they cannot carry out a valid asset transfer. If both platform have a cross-chain function, the asset on Alice’s account can be frozen or destroyed, and the blockchain that Bob belongs to can verify and create the corresponding valid asset to Bob. The key idea is that the blockchain associated with Bob can truly confirm that asset on Alice’s account is permanently frozen, such as to achieve an asset creation on Bob’s account. Therefore, the entire cross-chain process must be completed in an atomic transaction. That is, if Alice’s assets are permanently frozen in chain A, chain B has to create corresponding assets to Bob, otherwise not. Meanwhile, the states on both

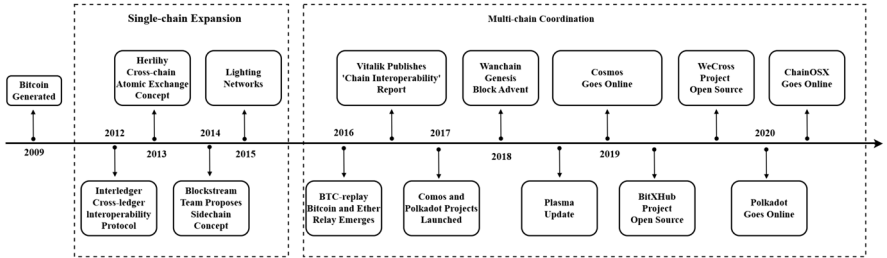
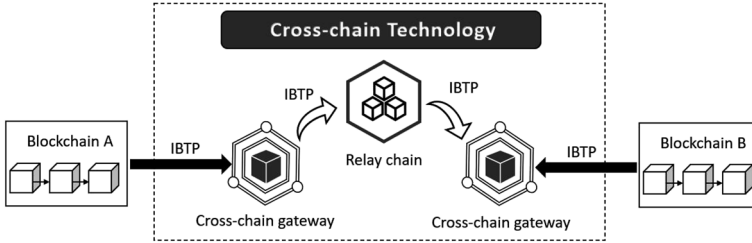


Fig. 1.3 The process of cross-chain development



**Fig. 1.4** Cross-chain structure

blockchains need to reach a consensus within a certain period of time and cannot be in an intermediate state permanently. Otherwise, some basic requirements such as the conservation of assets and no double-spend cannot be guaranteed.

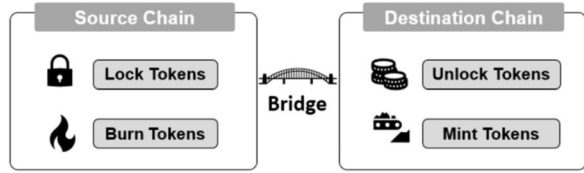
When users across multiple blockchain systems engage in the value exchange, the cross-chain technology needs to ensure data consistency across ledgers. It should prevent the basic case that one makes multiple payments while another only transmits the value once and discrepancy in exchanged amounts. During cross-chain transactions, both parties can verify each other's transaction status (Teutsch and Reitwießner 2019). As shown in Fig. 1.4, the cross-chain technology can be viewed as a platform that enables transactions between multiple blockchains. The cross-chain technology facilitates transactions between different blockchains and greatly expands the interaction range of blockchain applications with a connection of isolated blockchains into a common network. It therefore makes the cross-chain technology hold great promise as a key component to the future development of blockchain.

So far, cross-chain transactions are usually achieved via a tool named cross-chain bridge (Chainlink 2024). With economic activities siloed on isolated networks, Web3 needs robust cross-chain interoperability solutions that enable data and tokens to move across an interconnected network of blockchains in a secure and seamless manner. A cross-chain bridge is a type of decentralized application that facilitates the transfer of assets or information from one blockchain to another and plays the role of infrastructure that leverages a cross-chain messaging protocol for a specific purpose to move tokens in the Web3 ecosystem. Cross-chain bridges increase token utility by facilitating cross-chain liquidity between distinct blockchains. A cross-chain bridge typically involves locking or burning tokens on the source chain through a smart contract and unlocking or minting tokens through another smart contract on the destination chain. We present a simple overview of cross-chain bridge in Fig. 1.5.

Cross-chain bridges are powered by three main mechanisms.

- **Lock and mint.** A user locks tokens in a smart contract on the source chain, then wrapped versions of those locked tokens are minted on the destination chain. In the reverse direction, the wrapped tokens on the destination chain are burned to unlock the original coins on the source chain.

**Fig. 1.5** A simple overview of cross-chain bridge



- Burn and mint. A user burns tokens on the source chain, then the same native tokens are reissued (minted) on the destination chain.
- Lock and unlock. A user locks tokens on the source chain, then unlocks the same native tokens from a liquidity pool on the destination chain. These types of cross-chain bridges usually attract liquidity on both sides of the bridge through economic incentives such as revenue sharing.

## 1.2.2 Cross-Chain Types

### 1.2.2.1 Homogeneous vs. Heterogeneous

According to the underlying structure, the cross-chain systems can be classified into homogeneous cross-chain systems and heterogeneous cross-chain systems (Ou et al. 2022; Han et al. 2023; Herlihy 2018).

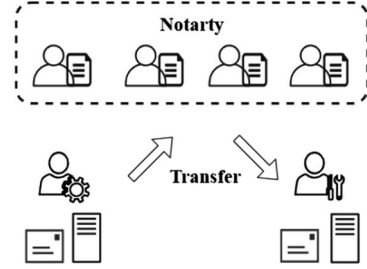
- Homogeneous cross-chain. It means that the involved blockchains have the same technical foundation and operating methods. All of chains share the identical consensus mechanism, network structure, the algorithms of block generation/verification. Due to the consistent running principle, the interaction is relatively simple in the homogeneous cross-chain systems.
- Heterogeneous cross-chain. It means that cross-chain communication among blockchains with different technical foundations. Compared with homogeneous cross-chain systems, heterogeneous cross-chain systems are more complicated with independent consensus mechanism and block component. In this case, the cross-chain interaction becomes difficult, just like the cross-domain communication in the networks, such that the third party is necessary as an assistant for the cross-chain communication.

### 1.2.2.2 Notary vs. Hash-locking vs. Sidechain/relay

According to the implementation way, the cross-chain systems can be classified into notary scheme, sidechains/relays, and hash-locking (Ou et al. 2022; Han et al. 2023; Herlihy 2018; Deng et al. 2018).



**Fig. 1.6** A structure of notary scheme for cross-chain



#### 1.2.2.2.1 Notary Scheme

The notary scheme refers to that one node or a group of nodes acting as the notary to collect data and verify transactions. The notary traces states of both blockchains and notices them, and both blockchains rely on the notary to verify and complete the transactions for transaction credibility. The notary scheme is relatively easy to deploy in a real scenario, and it is similar to a traditional centralized exchange. The scheme does not need to pay attention to the characteristics of chains' structure and consensus, but only needs to take the role of transaction confirmation and conflict arbitration, so it is very efficient and flexible to support different blockchains. Figure 1.6 shows a basic overview of notary scheme (Yi 2023).

According to constitution of the notary and the signature format, the notary schemes are categorized into centralized notary scheme, multi-signature notary scheme, and distributed signature notary scheme.

- **Centralized notary scheme.** A designated node or institution acts as the centralized notary. Similar to the traditional two-party communication via an intermediary, the notary can be compatible with multiple systems with good features of fast transaction processing speed, simple technical architecture. Its disadvantage is also apparent that compromising the notary will break down the security of the whole system.
- **Multi-signature notary scheme.** The notary team consists of multiple independent nodes or institutions. To ensure the security and reliability of cross-chain transactions, each notary has a unique key. Cross-chain transactions are only confirmed when a sufficient number of notaries reach consensus by signing on their respective ledgers. The greatest advantage is its relative safety and robustness without a centralized notary. When faced with a few attacked or downed nodes, overall stability and security remain largely unaffected because transactions can proceed as long as the remaining notaries meet the required signature threshold. Additionally, the multi-signature notary mechanism reduces reliance on specific notaries, effectively lowering centralization risks and enhancing system security. Efficiency-wise, transaction confirmation speeds up since multiple notaries participate in the verification simultaneously. In practice, the multi-signature notary mechanism often uses multi-signature algorithms, requiring participating blockchains to support and run this functionality to ensure smooth transactions.

- **Distributed signature notary scheme.** A main technical difference from the multi-signature notary scheme is that the distributed signature notary scheme applies the distributed signature over the multi-party computation. Here, it utilizes cryptographic algorithms such as distributed key generation and threshold signatures to solve the challenge of decentralized cross-chain transaction confirmation at the cryptographic level. Then, the responsibility for asset custody in cross-chain transactions is assigned to the nodes of the entire cross-chain network rather than relying on a few third-party institutions. The system generates a unique key based on cryptographic principles, which is then carefully split into multiple independent fragments. These encrypted fragments are randomly distributed among selected notaries. Only when the pre-set proportion of notaries jointly perform signature verification can these fragments be reassembled into a complete key, thereby enabling a more secure and decentralized cross-chain transaction process. Compared to the previous two mechanisms, the distributed signature notary mechanism significantly enhances security although its technical implementation is more complex.

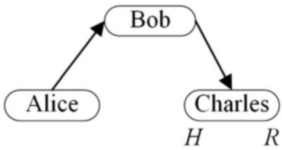
#### 1.2.2.2.2 Hash-locking

Hash-locking is also named hash time-locked contract (HTLC) from lightning network. This mechanism is originated from atomic swap and allows certain asset swap between chain A and chain B. It elaborately utilizes both time locks and hash locks to guarantee the atomic of cross-chain transactions. That is, the transaction can complete only when it can satisfy the specific time condition and hash condition. Concretely, during the transaction process, the assets involved are secured with dual protection through a combination of hash locks and time locks. The transaction can only be executed and confirmed if both parties provide the correct preimage of the hash within the allowed timeframe specified by the rules. Otherwise, the transaction will be considered invalid and will not take effect. This process requires simultaneous compliance with both the time lock and the hash lock.

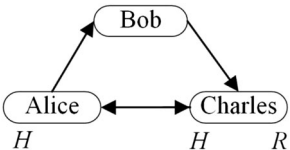
The hash lock means that a value can be locked by hashing, and the lock is only opened by the preimage, where no hash collusion is considered. Only when satisfying this condition, the transaction commitment is deemed valid. The time lock means that the lock is only opened in a specific time. That is, the transaction receiver in the transaction must complete the verification of the hash value within a specified period. If this period is exceeded, the transaction commitment becomes void and the corresponding amount will be returned to the sender. This dual-lock mechanism ensures the integrity and security of the transaction.

We take a simplified cross-chain atomic asset swap through hash-locking as the example, as shown in Figs. 1.7, 1.8, 1.9, 1.10 (0xKooKoo 2023). Alice opens a payment channel to Bob, and Bob opens a payment channel to Charles. When Alice wants to buy a product from Charles at price  $S$ , she opens a micropayment channel.

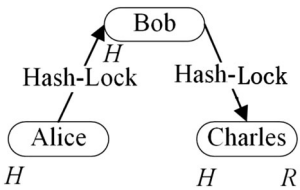
**Fig. 1.7** HTLC transaction process 1



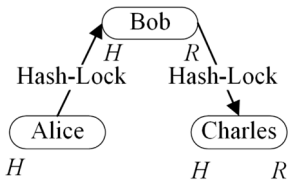
**Fig. 1.8** HTLC transaction process 2



**Fig. 1.9** HTLC transaction process 3



**Fig. 1.10** HTLC transaction process 4



- Charles first generates a random number  $R$  and then creates its SHA-256 hash  $H$  (Fig. 1.7).
- Charles sends the generated hash  $H$  to Alice. Alice pays Bob the price  $S$  and adds the condition that Bob must provide the data  $R$  that generated hash  $H$  (Fig. 1.8).
- Bob, now in possession of the hash  $H$  generated by Charles, pays Charles the price  $S$  and sends Charles a copy of Alice's additional condition (Fig. 1.9).
- Charles receives the payment from Bob and provides Bob with the data  $R$  that generated hash  $H$ . Bob then uses this  $R$  to satisfy the condition and receive payment from Alice. This completes a full HTLC transaction (Fig. 1.10).

The aforementioned process reveals some conditions in HTLC. One is that both participants in cross-chain asset swap can access the contract data, the other is to guarantee a time difference when setting the time of hash lock, such as to revoke the asset in the case of one-side cheating. The notable advantage of hash locking lies in its ability to facilitate transactions without requiring direct trust between the participating parties. By locking the assets, it achieves the effect of collateralization, eliminating the need to entrust assets to an external third-party escrow agent, thereby significantly enhancing transaction security. Additionally, the introduction of a transaction time limit optimizes the transaction process while ensuring speed, effectively curbing malicious delays, and further reducing security risks associated

with the transaction process. So far, the hash-locking mechanism just allows the cross-chain data swap, but not asset transfer, and, hence, has a restricted application scenario.

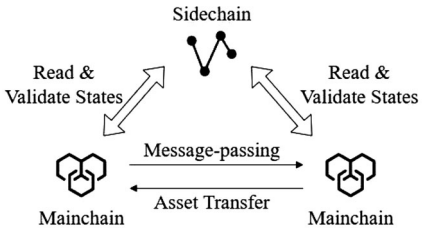
1.2.2.2.3 Sidechain/Relay

Compared with the aforementioned two modes, the sidechain/relay mechanism is more complicated and has more applications in cross-chain systems.

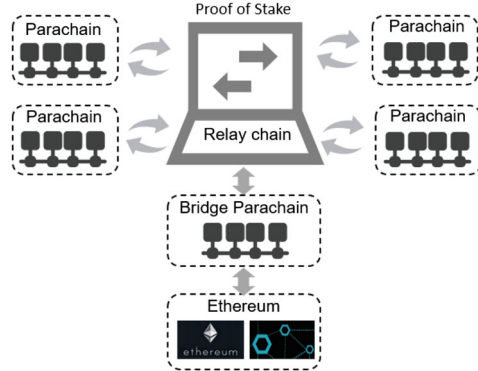
The sidechain is one of blockchain technologies that illustrates the relationship of two chains and is usually defined corresponding to the main chain. The concept and technical solutions for sidechains are primarily designed with Bitcoin. In the sidechain/relay mode, the target chain does not rely on the third part to execute the transaction verification. The sidechain can verify data from other blockchains, facilitating the transfer of digital assets between different blockchains. In cross-chain transactions, sidechains and main chains communicate through a mechanism called two-way pegging. This means that when a transaction on the main chain is locked, an equivalent value token on the sidechain is released, and vice versa. The core principle of two-way pegging is to determine whether corresponding values should be released based on actions taken on either the main or sidechain. A structure of sidechain can be seen in Fig. 1.11.

Compared with sidechains, relays are more flexible with adding a data structure between two blockchains such that both blockchains can interact via this data structure. Relays are channels connecting two blockchains. If the channel is a blockchain, it is called a relay chain. As an intermediary, it can achieve communication and coordination between blockchains over different structures and different consensus. The relay chain has four main benefits. The first is decentralization that the relay chain runs with a decentralized network of multiple nodes and ensures the security and fairness of the network. The second is generality that the relay chain is able to allow different-type blockchains such that any blockchain achieves interconnection via the relay chain. The third is flexibility that the relay chain provides cross-chain services according to distinct requirements and scenarios and supports different cross-chain protocols. The fourth is practicality that the relay chain can leverage the optimized consensus algorithm and implementation strategy to improve computation and communication efficiency of cross-chain messages.

**Fig. 1.11** Structure of sidechain



**Fig. 1.12** The work mode of relay chain



There exist the relay-based cross-chain systems like Polkadot, Cosmos, and Rocket Protocol. In Fig. 1.12, we present an overview of how the relay chain works in Polkadot.

### 1.3 Present Challenges and Security Requirements of Cross-Chain Technology

#### 1.3.1 Present Challenges

In this subsection, we analyze some technical challenges of the cross-chain technology at present (Integritee Network 2024).

*Performance and Verification of Cross-Chain Transactions* One of the present challenges in cross-chain systems is the performance of cross-chain transactions. To realize the interconnection between different blockchains and make the transactions of one blockchain can be received and verified by another blockchain, the cross-chain systems need to provide a trust mechanism among blockchain systems. Confirmation and verification of transactions involve two issues: one is to confirm that the transaction has occurred and written into a block even a blockchain ledger, and the other is to verify that the transaction has been confirmed by enough nodes in the blockchain network. At present, the common cross-chain transaction verification mechanisms include the notary mechanism and the “block header+simplified payment verification (SPV)” mechanism. The notary mechanism is to verify the reliability of the cross-chain transaction through an external notary chain. The notary needs to sign the cross-chain transaction after the verification passes. The “block header+SPV” mechanism allows the block header data of the external blockchain system provided by the notary chain in its own network and verifies the transaction according to the SPV mechanism. Some cross-chain systems, such as Cosmos and BitXHub, leverage IBC and IBTP in the cross-chain data communication

and adopt SPV into cross-chain verification. The atomic feature of cross-chain transactions is guaranteed by the application protocols. Some cross-chain systems, such as Polkadot, use XCMP and HRMP in the cross-chain communication, apply the global consensus mechanisms into cross-chain verification, while the consensus mechanism certainly limits the overall performance of the cross-chain system. In fact, as the single blockchain system, the performance of a cross-chain system mainly relies on the used consensus mechanism, the transaction verification algorithm, the communication mechanism, but not the number of the involved nodes. One of the main problems is that the transformation of the cross-chain technology. The existing systems almost adopts modularized components, and it has not yet implemented hierarchical decoupling of the basic logical functions. Meanwhile, they restrict the generic transformation and flexible configuration of some key technologies. Also, most of the existing cross-chain systems use SPV that incurs a linear-size time cost of the communication verification, which brings about low efficiency in cross-chain communication and verification, specially in the large-scale scenarios. Another problem is that the transaction efficiency. In the cross-chain systems, the transaction process is usually serial instead of parallel, which compromises the efficiency of the transaction. The sharding allows each transaction in parallel, which can assist with the performance improvement.

*Adaptation of Multi-chain Protocols and Connection* With emergence of independent blockchains and the continuous implementation of multi-function applications, the blockchain technology will develop to a multi-chain coexisting, interconnected, and interoperable ecosystem. Multi-chain interconnection needs to consider the realization of interconnection among existing blockchain systems. Indeed, multi-chain and cross-chain systems can be divided into active compatible systems and passive compatible systems. The active compatible system is implemented from top to bottom, mainly for the existing diversified blockchain systems. At present, there are different upper-layer blockchain applications, and then the underlying cross-chain mechanism is developed. Usually, the existing blockchain systems are deployed on the heterogeneous-chain architecture, which need to be connected one by one and have independent underlying structure and running mechanisms. The passive compatible system is implemented from bottom to up, mainly for the immature or unknown blockchain systems. It builds the underlying cross-chain platform and then connects the existing other blockchain systems into this platform in a secure and reliable way. It can execute communication and transactions among different blockchain systems. Due to the independence of blockchains, they might be incompatible with each other. How to communication is a challenge since there is no common and available communication methods. In this case, in order to realize the connection between chains, a specific role must be added to act as the connecting party to undertake the information exchange between the two different chains. The form of the connecting party may be various. It may be through a third party, allowing one or a group of notaries to assume this role, or only operates on two parachain participants that need to interact, in the form of a separate module or even an independent chain. It might even just add a function

to the smart contract of one chain that can read the other chain. No matter what form it evolves, the role of the connecting party is irreplaceable. Since different blockchain systems are not designed with the existence of other chains in mind, there is no common management protocol for these cross-chain systems. Therefore, it is a great challenge to realize cross-chain connection when different unknown blockchains with no trust would like to execute secure transactions. Especially, in the notary mechanism, individuals or groups can be selected via the on-chain or off-chain governance mechanism and thereafter play the role of a notary or the role of an internal neutral party. The elected notary has kinds of regulation-type functions, such as monitoring, viewing, verifying, and auditing. In the sidechain mechanism, there are also two modes that can act as the connecting party, i.e., the hosting mode and the SPV mode.

*Cross-Chain Transaction and Locked Asset Management* A complete cross-chain transaction can be split into several sub-transactions, and each sub-transaction is processed in its own blockchain system. These sub-transactions constitute a cross-chain transaction, which requires a management mechanism to ensure the consistency and the atomicity of the transaction. The cross-chain transaction management is further divided into two sub-problems, namely the final certainty of transactions and the atomicity of transactions. In cross-chain transaction management, in order to ensure the final certainty of transactions, three schemes of waiting for enough confirmations, block entanglement, and consensus algorithms can be included. Waiting for enough confirmations is the simplest and crudest method, but it has the disadvantage of longer the transaction processing time. The block entanglement aims to make blocks between two chains have a dependent relationship. When a block on one chain is revoked, related blocks on other chains are automatically revoked. Compared with the traditional PoW, consensus algorithms such as DPoS or xBFT are easier to achieve final certainty, and blockchain systems using such consensus algorithms can make cross-chain transactions more efficient. The atomicity, as the basic requirement of cross-chain transactions, is not easy to achieve but a must-have property for all the cross-chain systems. Two-way peg is the process of dual transfer of assets on the main blockchain and the sidechain according to the same exchange ratio. The key issue in two-way peg is who manages the locked account and performs operations such as locking and unlocking and how to ensure that the locked assets are securely released without double-spending attacks. In addition, how to ensure the total amount of assets in the two chains to be unchanged is also important. Regarding the management of locked assets, there are currently single hosting mode, alliance mode, and smart contract mode. The single hosting mode is that a custodian manages the locked assets, executes, and supervises the unlocking operations of the locked assets. Although the single hosting mode is simple and easy to implement, it is centralized and almost fully relies on the central custodians. Compared with the single hosting mode, the alliance mode is more decentralized. Here, a threshold idea  $(M, N)$  is applied. When receiving a cross-chain unlocking request,  $N$  notaries in the alliance independently verify the transaction and execute voting. When the number of votes reaches a threshold

$M$ , the locked assets can be disposed of. The smart contract mode is the most decentralized in these three modes, whose premise is to use the smart contract-based blockchain systems. It can store the block headers of external blockchains to verify external transaction data.

*Cross-Chain Security Risks* In the cross-chain systems, the interaction of two blockchains inevitably affects each other and incurs more security risks than the single blockchain systems. First, kinds of blockchains are not isolated, and the security of the whole cross-chain network will be broken down if one involved blockchain is attacked. The security of interaction should be guaranteed from at least three aspects, they are moderate isolation, detection of security incidents, and correctness of cross-chain transactions. Generally, the cross-chain systems employ a third-party node or another chain to carry out the transactions. If each blockchain in this system can maintain its independence, it can reduce the affected risks. If the third-party node or chain (like the sidechain or the relay chain) can detect and respond to security incidents, it can have a function similar to firewalls and improve the capability against risks. At the same time, some invalid cross-chain transactions are issued, and the transaction information is leaked, which directly brings about security problems for the cross-chain systems. If all the transmitted transactions are correct, it can enhance the security of the cross-chain systems. Different cross-chain mechanisms also have their own security issues. The first is the trust issue of the notary mechanism. Due to the participation of third-party organizations, although there are mature election strategies, value transfer or information exchange mainly depends on the honesty of notaries. It thus has a high centralization. The notary's multi-signature enhances the security to a certain extent with random selection, but does not completely eliminate the relevant dependencies, and there is still a risk of collusion. Some cross-chain projects based on the notary mechanism are also seeking to combine with other technologies, such as Interledger, which incorporates hash-locking mechanism in the protocol to provide more complete security. The second is the security issue of hash-locking. Its technical security is mainly related to the fund locking mechanism and the timeout of the locking time. For example, the hash-locking-based Lightning Network might lead to three security risks. The first is that malicious actors create a large number of transaction channels and cause all channels to time out at the same time, letting spam transaction information to be broadcast and even the network blockages, affecting normal transactions. The second is that a certain amount of funds is locked in the opening phase of the transaction channel. That is, it uses the "hot wallet," rather than "cold wallet" or offline storage, to stay connected to the blockchain network for a long time, such as to sign transactions, which will increase the risk of hackers' stealing users' private keys. The third is, if one party to the transaction loses data or does not broadcast the transaction at the correct time, there may be a risk of funds being stolen by the other party. In addition, cross-chain technology also has security issues such as orphan block, long-range attack, eclipse attack, cross-chain replay attacks, and compatibility issues after upgrade. It can be seen that in the design process of the cross-chain system, it is necessary to adopt the protection mechanism in time and at



the same time fully understand the security issues that the blockchain technology needs to solve in an all-round way. Some key concerns might also plague the evolving cross-chain technology, such as unauthorized access, privacy leakage, and lack of traceability.

Fully anonymity of cross-chain exchange provides a great space for the crime when assets are transferred across different chains, since the anonymity of cryptocurrencies also affords a cover for various illegal activities, and the abuse of anonymous cryptocurrencies for illegal purposes is gaining momentum. Tracing the movement of illicit funds across bridges can be an arduous task for law enforcement and regulatory bodies. Despite the availability of several well-conceived cross-chain solutions, significant challenges persist in the areas of cross-chain privacy and auditing, including cross-chain linkability exposure (CLE), incompatibility of privacy and auditing (IPA), and full auditing inefficiency (FAI).

- CLE. Some solutions have been proposed to address cross-chain privacy concerns. Specifically, the schemes in Yi et al. (2022), Zhang et al. (2021) are highly tied to the application scenarios. The privacy protection in Baldimtsi et al. (2021), Li et al. (2022) benefits from its underlying blockchain platform, such as Monero and Zcash, which makes it incompatible with other blockchain systems. The mechanism in Deshpande and Herlihy (2020) provides a solution for cross-chain exchange but requires both parties to trust the same secret as a prerequisite, which creates challenges for two individuals who lack trust in one another to reach an agreement. Based on the aforementioned analysis, it is evident that the current research on cross-chain privacy protection is insufficient, and there also exist concerns regarding their generality and security risks. Additionally, existing works overlook the significant differences between cross-chain transfer and cross-chain exchange, attempting to use a single protocol to support both activities, which needs third-party entities and thereby poses security risks (such as the bridge-based scheme). One can see that the difficulty in addressing the CLE problem lies in comprehending and differentiating the activities and characteristics of blockchain interactions while using more adaptable techniques to obscure the relations between the participants. More importantly, the entire process cannot introduce third parties to disrupt the security of the system.
- IPA. The demands for both privacy protection and auditing often exist concurrently in a system, and these two requirements have conflicting ultimate goals. To the best of our knowledge, no existing cross-chain work can solve this conflict. Current cross-chain privacy-preserving solutions (Kietzmann et al. 2022) are not initially designed with auditing support in mind. Their primary focus is directed toward simultaneously minimizing the disclosure of information and ensuring the integrity of the protocols, disregarding considerations for information verifiability. Moreover, considering that in permissioned blockchains, auditors are external to a blockchain, they may not have access to the transaction information. For example, in a multi-chain system comprising several companies, internal nodes engage in frequent interactions, while third-party entities, such as banks and insurance companies, assume the role of auditors. The privacy of

internal data, including receipts and employee information, cannot be disclosed to external entities. However, this restriction prevents banks from conducting audits, consequently impeding their ability to perform operations such as lending. To effectively address the aforementioned issues, it becomes crucial to enable the validation of private information by leveraging publicly accessible data.

- FAI. Current auditing works can be further broken down into two distinct sub-categories: incentive-based and relay-based. The former, such as the fisherman in Polkadot (Wood 2016), utilizes an incentive mechanism in which auditors selectively verify a subset of transactions to get the corresponding reward. The latter employs a relay mechanism, such as an entity or a network, to facilitate the thorough examination of all transactions in order to mitigate the risks of malicious activities (Wang et al. 2021; Zhang et al. 2023). In general, a relay-based auditing scheme offers better security as it audits all transactions (full auditing); however, it may be less efficient than an incentive-based audit scheme. Specifically, assuming  $k$  blockchains and an average of  $n$  transactions processed per chain in a unit of time, the workload responsibility of each audit node in a relay-based scheme can be estimated as  $O(k \times n)$ . This poses a high computational demand for auditors and leads to issues with inefficient verification. More seriously, as the number of blockchains continues to grow, the limitations of both methods in terms of auditing efficiency pose a challenge to their adoption in real-world settings. To address this issue, it is crucial to design an auditor-friendly scheme that guarantees security and efficiency while providing full auditing.

### 1.3.2 Security Requirements

In this part, we list potential requirements of cross-chain systems in terms of regulation and privacy.

#### 1.3.2.1 Regulation

Cross-chain technology enables interoperability of data and assets between different blockchain networks, significantly expanding kinds of blockchain applications. However, as the cross-chain technology and the legal transactions evolve, regulation has becoming increasingly important. For cross-chain regulation, we consider three requirements, i.e., account, traffic, and network.

- Account. The cross-chain systems should ensure the basic security of cross-chain transactions. All of transactions are carried out by accounts, and assets are transferred among different accounts. Accounts in the cross-chain systems can be divided into normal accounts and abnormal accounts. Normal accounts are usually valid, and abnormal accounts might deviate from expected pat-

terns or suspicious characteristics, potentially indicating fraudulent or malicious behavior. In recent years, due to the lack of the account regulation, many criminals associated with cross-chain technologies have happened. The account regulation can identify illegal cross-chain transactions from abnormal account activities. However, existing abnormal detection methods in blockchain systems mainly focus on the single-chain transactions and cannot be directly applied into the cross-chain transactions. This is because cross-chain transactions have nonuniform values, and the network heterogeneity lacks standardized transaction protocols.

- **Traffic.** The communication in the cross-chain systems generates traffic information. Regulating blockchains based on traffic have not kept pace with the development of related blockchain technologies. Existing research primarily focuses on consensus mechanisms, security protocols, and smart contract optimization, while neglecting in-depth analysis of abnormal traffic data transmission. However, abnormal traffic analysis is crucial for identifying potential security threats, abnormal behaviors, and performance bottlenecks. Through comprehensive traffic monitoring and analysis, it can effectively enhance the overall security and reliability of blockchain systems, ensuring stable operation of cross-chain technology across various applications. Therefore, there is an urgent need to develop traffic regulation that is synchronized with blockchain advancements to address increasingly complex network environments and security challenges.
- **Network.** Network nodes play important role in the cross-chain network. A honest network node can correctly execute the given protocol and commands while it might launch malicious behaviors once it is compromised. This will incur kinds of security risks and even the whole system crash. In fact, the network nodes are characterized by their massive, distributed, autonomous, open, and freely accessible nature. They are more susceptible to malicious attacks and manipulation, potentially causing irreparable damage to the cross-chain system. Checking whether the network nodes are abnormal is necessary for a reliable cross-chain system. Traditional network detection mechanisms are labor-intensive and time-consuming and might be infeasible to work in the cross-chain systems. The effective regulation of network nodes can help to promote the health development of cross-chain technology. How to real-time detect abnormal network nodes is a desirable issue for cross-chain regulation.

### 1.3.2.2 Privacy

Most of cross-chain transactions are still with plain-text format and expose associated data to the whole network. Therefore, the privacy concern is a crucial issue in the cross-chain systems. We take a consideration from the following aspects.

- **Multiple targets.** From a perspective of the transaction user, she/he would like to keep its sensitive information secret, such as the identity, the transaction account, the transaction amount, the deposit, and the balance. The leakage of

these sensitive information might endanger the user's personal and property safety. Different privacy targets aim to support different requirements and need their corresponding protection methods. For a cross-chain system, single-mode privacy protection and multi-mode privacy protection have enormous discrepancy.

- **Generic mechanism.** For the heterogeneous cross-chain network, each blockchain has its privacy-preserving mechanism, and different mechanisms have independent underlying structure, the used technologies, and the serviceability scenarios. The incompatibility among different privacy-preserving mechanism is a technical challenge. Also, due to the format independence and the protection target difference, it is difficult to directly transfer one to another. A generic privacy-preserving mechanism is important for the cross-chain system.
- **Performance of privacy methods.** The common privacy-preserving mechanisms are based on cryptographic algorithms, such as zero-knowledge proof. For example, ZCash applies zero-knowledge succinct non-interactive arguments of knowledge (zkSNARK) into the single blockchain system for the transaction privacy. However, these zero-knowledge type schemes have high computation cost, communication cost, and storage cost, even including a required trust setup, which costs much in the deployment. Whether the basic structure of these cryptographic algorithms can match the cross-chain network is unknown. Thus, decentralized and lightweight privacy preserving is a demand for the cross-chain systems.
- **Collaboration of privacy and traceability.** For information, the privacy protection is to hide it such that outsiders cannot access. However, when the information is related to some malicious attributes, we need to open it, audit it, and trace the origin, such as to prevent potential damages. It becomes apparent that there is a contradiction between the privacy and the traceability. How to balance this contradiction and allow a privacy-traceability collaboration is a hard problem in the cross-chain systems.

## References

- 0xKooKoo (2023) What to do for the cross-chain bridge design? <https://foresightnews.pro/article/detail/48168>
- Ariella S (2023) 30 striking cryptocurrency statistics. <https://www.zippia.com/advice/cryptocurrency-statistics/>
- Baldimtsi F, Miers I, Zhang X (2021) Anonymous sidechains. In: García-Alfaro J, Muñoz-Tapia JL, Navarro-Arribas G, Soriano M (eds) Data Privacy Management, Cryptocurrencies and Blockchain Technology - ESORICS 2021 International Workshops, DPM 2021 and CBT 2021, Darmstadt, Germany, October 8, 2021, Revised Selected Papers. Springer, Lecture Notes in Computer Science, vol 13140, pp 262–277
- Brezitska M (2024) Exploring the latest trends in blockchain technology. <https://binariks.com/blog/emerging-blockchain-technology-trends/>
- Chainlink (2023) What is cross-chain? <https://chain.link/education/cross-chain>

- Chainlink (2024) What is a cross-chain bridge? <https://chain.link/education-hub/cross-chain-bridge>
- Cyfrin Team (2024) Blockchain architecture layers: Guide and topology. <https://www.cyfrin.io/blog/blockchain-architecture-layers-what-is-it#blockchain-architecture-layers>
- Daley S (2024) 16 blockchain applications and real-world use cases. <https://builtin.com/blockchain/blockchain-applications>
- Deng L, Chen H, Zeng J, Zhang L (2018) Research on cross-chain technology based on sidechain and hash-locking. In: Liu S, Tekinerdogan B, Aoyama M, Zhang L (eds) Edge Computing - EDGE 2018 - Second International Conference, Held as Part of the Services Conference Federation, SCF 2018, Seattle, WA, USA, June 25–30, 2018, Proceedings. Springer, Lecture Notes in Computer Science, vol 10973, pp 144–151
- Deshpande A, Herlihy M (2020) Privacy-preserving cross-chain atomic swaps. In: Bernhard M, Bracciali A, Camp LJ, Matsuo S, Maurushat A, Rønne PB, Sala M (eds) FC 2020. Springer, Lecture Notes in Computer Science, vol 12063, pp 540–549
- Fortune Business Insights (2024) Blockchain technology market size, share & industry analysis. <https://www.fortunebusinessinsights.com/industry-reports/blockchain-market-100072>
- Geeks for Geeks (2024a) Types of blockchain. <https://www.geeksforgeeks.org/types-of-blockchain/>
- Geeks for Geeks (2024b) Components of blockchain network. <https://www.geeksforgeeks.org/components-of-blockchain-network/#key-features>
- Han P, Yan Z, Ding W, Fei S, Wan Z (2023) A survey on cross-chain technologies. *Distrib Ledger Technol Res Pract* 2(2):1–30
- Herlihy M (2018) Atomic cross-chain swaps. In: Newport C, Keidar I (eds) Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing, PODC 2018, Egham, United Kingdom, July 23–27, 2018. ACM, pp 245–254
- IBM (2024) What is blockchain? <https://www.ibm.com/topics/blockchain>
- Integritee Network (2024) Cross-chain interoperability: Major issues & how to tackle them. <https://medium.com/integritee/cross-chain-interoperability-major-issues-how-to-tackle-them-be8b8f3bbf63>
- Javeri P (2019) Blockchain architecture. <https://medium.com/@prashunjaveri/blockchain-architecture-3f9f1c6dac5e>
- John F, Oleh M, Luciano C (2024) Blockchain architecture layers: A comprehensive guide. <https://hacken.io/discover/blockchain-architecture-layers/>
- Kazim W (2024) 26 top blockchain applications and use cases in 2024. <https://learn.g2.com/blockchain-applications>
- Kietzmann P, Schmidt TC, Wählisch M (2022) A guideline on pseudorandom number generation (PRNG) in the iot. *ACM Comput Surv* 54(6):112:1–112:38
- Li Y, Weng J, Li M, Wu W, Weng J, Liu J, Hu S (2022) Zerocross: A sidechain-based privacy-preserving cross-chain solution for monero. *J Parallel Distrib Comput* 169:301–316
- Lunn B (2018) Blockchain may finally disrupt payments from micropayments to credit cards to swift. Tech. rep. <https://dailyfintech.com/2018/02/10/bitcoin-will-finally-disrupt-the-credit-card-rails/>
- Markets and Markets (2024) Blockchain market report: Tc 4638. <https://www.marketsandmarkets.com/Market-Reports/blockchain-technology-market-90100890.html>
- Morris DZ (2016) Leaderless, blockchain-based venture capital fund raises \$100 million, and counting. Tech. rep. <https://fortune.com/2016/05/15/leaderless-blockchain-vc-fund/>
- Nakamoto S (2008) Bitcoin: a peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, online; 2008
- Narayanan A, Bonneau J, Felten EW, Miller A, Goldfeder S (2016) Bitcoin and cryptocurrency technologies - A comprehensive introduction. Princeton University Press
- Ou W, Huang S, Zheng J, Zhang Q, Zeng G, Han W (2022) An overview on cross-chain: Mechanism, platforms, challenges and advances. *Comput Networks* 218:109378
- Shoemaker P (2023) Key components of a blockchain network. <https://www.identity.com/key-components-of-a-blockchain-network/>

- Teimoori Z, Yassine A (2022) A review on intelligent energy management systems for future electric vehicle transportation. *Sustainability* 14(21)
- Teutsch J, Reitwießner C (2019) A scalable verification solution for blockchains. *CoRR* abs/1908.04756
- The Economist (2015) The great chain of being sure about things. Tech. rep. <https://www.economist.com/briefing/2015/10/31/the-great-chain-of-being-sure-about-things>
- The World Economic Forum (2020) These are the challenges blockchain faces in 2020. <https://www.weforum.org/stories/2020/01/blockchain-in-2020-epic-changes-and-monumental-challenges/>
- Times TNY (2016) A venture fund with plenty of virtual capital, but no capitalist. Tech. rep. <https://www.nytimes.com/2016/05/22/business/dealbook/crypto-ether-bitcoin-currency.html>
- Toti B (2024) Crypto market cap hits 3trillionasbtcmashespast85k. <https://crypto.news/crypto-market-cap-hits-3-trillion-as-btc-smashes-past-85k/>
- Wang X, Qiu W, Zeng L, Wang H, Yao Y, He D (2021) A supervisory and governance mechanism for power master-slave chain architecture. In: 2021 IEEE International Conference on Electronic Technology, Communication and Information (ICETCI). IEEE, pp 172–175
- Witscad (2024) Blockchain generation. <https://witscad.com/course/blockchain-fundamentals/chapter/blockchain-generation>
- Wood G (2016) Polkadot: Vision for a heterogeneous multi-chain framework. White Paper
- Yi H (2023) A post-quantum blockchain notary scheme for cross-blockchain exchange. *Comput Electr Eng* 110:108832
- Yi L, Sun Y, Wang B, Duan L, Ma H, Wang B, Han Z, Wang W (2022) CCUBI: A cross-chain based premium competition scheme with privacy preservation for usage-based insurance. *Int J Intell Syst* 37(12):11522–11546
- ZetaChain (2023) Zetachain: A blockchain with native cross-chain smart contracts. <https://github.com/etherclust/etherclust>
- Zhang X, Chen J, Zhou Y, Jiang S (2021) Privacy-preserving cross-chain payment scheme for blockchain-enabled energy trading. In: 10th IEEE/CIC International Conference on Communications in China, ICC3 2021, Xiamen, China, July 28–30, 2021. IEEE, pp 109–114
- Zhang Y, Jiang J, Dong X, Wang L, Xiang Y (2023) Bedcv: Blockchain-enabled decentralized consistency verification for cross-chain calculation. *IEEE Trans Cloud Comput* 11(3):2273–2284
- Zmudzinski A (2019) Daily ethereum transactions exceed one million, a first since may 2018. <https://cointelegraph.com/news/daily-ethereum-transactions-exceed-one-million-a-first-since-may-2018>

# Chapter 2

## Foundations



### 2.1 Classical Blockchain Systems

#### 2.1.1 Bitcoin

Bitcoin (BTC) is the first decentralized cryptocurrency. Nodes in the peer-to-peer Bitcoin network verify transactions through cryptography and record them in a public distributed ledger, called a blockchain, without central oversight. Consensus between nodes is achieved using a computationally intensive process based on proof of work, called mining, which guarantees the security of the Bitcoin blockchain. Mining consumes large quantities of electricity and has been criticized for its environmental effects (The New York Times 2021).

Based on a free market ideology, Bitcoin was invented in 2008 by Satoshi Nakamoto (The Economist 2015). Use of Bitcoin as a currency began in 2009 (Yorker 2014), with the release of its open-source implementation (Antonopoulos 2014). In 2021, El Salvador adopted it as the legal tender (Financial Times 2021). Bitcoin is currently used more as a store of value and less as a medium of exchange or unit of account. It is mostly seen as an investment and has been described by many scholars as an economic bubble (Wolff-Mann 2018). As Bitcoin is pseudonymous, its use by criminals has attracted the attention of regulators, leading to its ban by several countries as of 2021 (Sun Yin et al. 2019).

The Bitcoin peer-to-peer network stores the entire transaction history in the “blockchain,” which can be viewed as a ledger recording all Bitcoin transactions. The blockchain is a distributed database composed of a group of decentralized client nodes and maintained by all participants, serving as a record of all Bitcoin transaction history. Satoshi Nakamoto anticipated that as the volume of data increased, users would prefer not to store all these data on their own nodes. To achieve this, he introduced the mechanism of hash functions. This allows clients to automatically discard parts they will never need, such as very early Bitcoin

transaction records. In 2009, Satoshi Nakamoto created the first block of the Bitcoin system, known as the “Genesis Block.”

In the Bitcoin network, the process of confirming a transaction is achieved through a proof-of-work mechanism that involves solving a series of computational problems. The proof-of-work mechanism requires a certain amount of computational power and time to solve these problems, making it infeasible for attackers to rewrite or alter the transaction history unless they possess computational power greater than that of the entire Bitcoin peer-to-peer network, enabling them to generate blocks faster (known as a “51% attack” (Aponte-Novoa et al. 2021)). The difficulty of the proof-of-work mechanism is automatically adjusted by the system so that new blocks are generated approximately every 10 minutes. Nodes across the Bitcoin peer-to-peer network automatically verify the validity of transactions and blocks, ignoring any transactions and blocks that violate the rules, such as those generating an incorrect number of Bitcoins or double-spending the same Bitcoins.

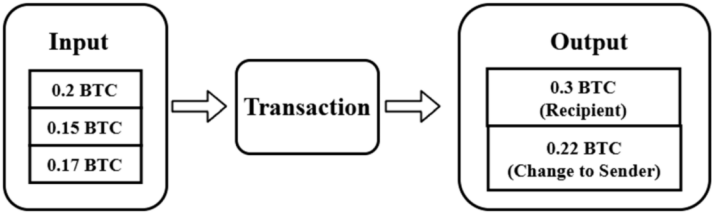
During this process, clients participating in block processing can earn newly issued Bitcoins and related transaction fees. To obtain these newly minted Bitcoins, users need to invest significant time and computational power (specialized mining machines have been developed to replace lower-spec network devices like standard computers). This process is very similar to mining natural resources, which is why Satoshi Nakamoto referred to data processors as “miners” and the data processing activity as “mining.” These newly created Bitcoins reward data processors in the system, whose computational work ensures the proper functioning of the Bitcoin peer-to-peer network. Consequently, other similar cryptocurrencies (Altcoins) use the same concept to handle peer-to-peer transaction data but make adjustments to their proof-of-work mechanisms, such as using Proof of Stake (Deuber et al. 2020) or Script algorithms.

### **Bitcoin Model**

Bitcoin operates on a unique model known as the Unspent Transaction Outputs model. This model plays a critical role in how transactions are processed and verified on the Bitcoin network.

Unspent Transaction Outputs, or UTXOs, represent specific amounts of Bitcoin that you have received but not yet spent. Each UTXO is like an individual bill in your wallet, each with a unique value. For example, a balance of 0.52 BTC in your wallet may consist of several UTXOs like 0.2, 0.15, and 0.17 BTC. Each UTXO is distinct and can hold any amount of Bitcoin. They are the pieces of Bitcoin you haven’t spent yet, and you use them to make new payments. As its name suggests, a UTXO is an output of a Bitcoin transaction. An output exists as a UTXO until it is used as an input in a future transaction. When someone sends you Bitcoin, what you receive is a UTXO. The Bitcoin balance in your wallet is the total of all the UTXOs received. The UTXO set is the collection of all UTXOs on the Bitcoin blockchain at any moment. Bitcoin nodes track this set to agree on identifying existing coins and their owners, helping to prevent the issue of double spending, which had plagued previous attempts at a decentralized digital currency. An example of Bitcoin UTXO model can be seen in Fig. 2.1.





**Fig. 2.1** Example of a Bitcoin UTXO trading model

The fee of a Bitcoin transaction is determined by the number and data size of the UTXOs involved. For example, using many small UTXOs makes a large payment with a longer transaction size and more cost. You can think of this like paying a large bill with many small coins. Usually, when transactions get more complicated, there can be several outputs. On the other hand, the amount of UTXOs your wallet uses as inputs is determined by how you received those UTXOs from previous transactions.

**2.1.2 Ethereum**

Ethereum (Vitalik Buterin 2014) is an open-source blockchain platform designed to enable the programming of smart contracts and the deployment of decentralized applications (dApps). It provides a decentralized virtual machine environment called the Ethereum Virtual Machine (EVM), which allows developers to write smart contracts and execute them on the blockchain. The cryptocurrency of Ethereum is Ether, used for paying transaction fees and rewarding miners. Key features of Ethereum include smart contract functionality, support for Decentralized Autonomous Organizations (DAOs), and programmable blockchain technology, making it an ideal platform for building decentralized applications and digital assets.

Blockchain 2.0 represents the evolution and advancement of blockchain technology, highlighting new features and improvements built upon the first-generation blockchain. Key characteristics of this stage include smart contracts, DAOs, programmable blockchain technology, and more advanced cryptocurrency functionalities. A smart contract is a self-executing contract with terms written in computer code that can run on the blockchain, enabling various business logics. DAOs are a form of decentralized organization managed and governed through smart contracts, promoting a more democratic and transparent organizational model. The development of Blockchain 2.0 has expanded the use of blockchain technology beyond cryptocurrencies, applying it to fields such as finance, supply chain management, identity verification, and more, bringing about significant innovation and transformation to society.

Meanwhile, Ether differs from Bitcoin in several ways. Ether supports more complex applications due to its improved consensus algorithm, support for smart

contracts, mining reward schemes, and content encryption. Although blockchain technology has been widely accepted and research on Ethereum's smart contracts and algorithms is growing rapidly, Ethereum has developed private protocols for encrypted content transmission, making the study of Ethereum's traffic more challenging.

The following parts will introduce the basic concepts and layered structure of Ethereum.

### **Basic Concepts of Ethereum**

Ethereum encompasses many fundamental concepts, including accounts, transactions, blocks, mining, and consensus. Ethereum accounts are core components of the network used for storing Ether and executing smart contracts, broadly categorized into two types.

Firstly, external accounts are generated by a pair of public and private keys encrypted through elliptic curve cryptography. These accounts are controlled by users who can sign transactions. Each external account has an associated Ether balance, allowing it to perform transfers and invoke smart contracts. Secondly, contract accounts are created by smart contracts and function as accounts without private keys. Instead, they execute code logic via transactions, store state, and perform complex computations. Smart contracts on Ethereum are deployed within these contract accounts. The state of Ethereum accounts is updated through transactions, which include transfers, smart contract calls, and smart contract deployments. Each account has a unique address that serves as its identifier and access point.

Regarding transactions, Ethereum supports several types, including regular transactions, contract creation transactions, and message call transactions. Regular transactions transfer Ether between external accounts, contract creation transactions deploy new smart contracts on the blockchain, and message call transactions invoke existing smart contracts. Once a transaction is sent to the Ethereum network, it gets included in a block, confirmed, and executed by miners through the process of mining; after confirmation, the transaction becomes irreversible and is written onto the blockchain. Each transaction has a post-execution status—success, failure, or cancellation—that can be queried using the transaction's hash on the blockchain.

Ethereum blocks are data units on the Ethereum blockchain containing multiple transactions occurring over a certain period, along with other essential information. Each block has a unique hash and a set of transaction records. These blocks link together to form an ever-growing chain, constituting Ethereum's distributed ledger. After validation, transactions within a block are written onto the blockchain, verified, and added by miners through mining. Each block must meet specific consensus rules and security standards to ensure the stable operation of the network and the security of the data. Ethereum enhances the proof-of-work (PoW) (Liu and Camp 2006) mechanism with additional consensus algorithms such as proof of stake (PoS) (Barhanpure et al. 2018) and proof of authority (PoA) (Angelis et al. 2018) to address global consistency issues (Awerbuch et al. 2002). Mining involves competing computational power to validate and add transactions to the blockchain. Miners use computing resources to solve mathematical problems, i.e.,

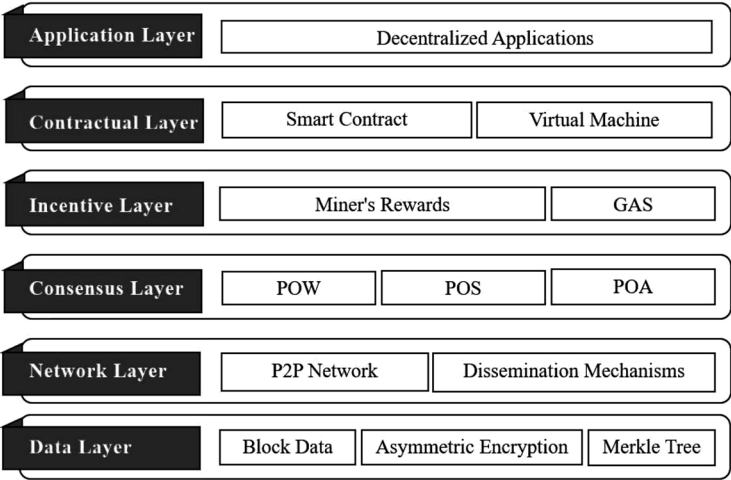


Fig. 2.2 Layered structure of Ethereum

PoW, to compete for creating new blocks. The miner who successfully finds the correct solution is rewarded with a certain amount of Ether and packages their transaction records into a new block. This process not only creates new blocks but also ensures network security and transaction validity.

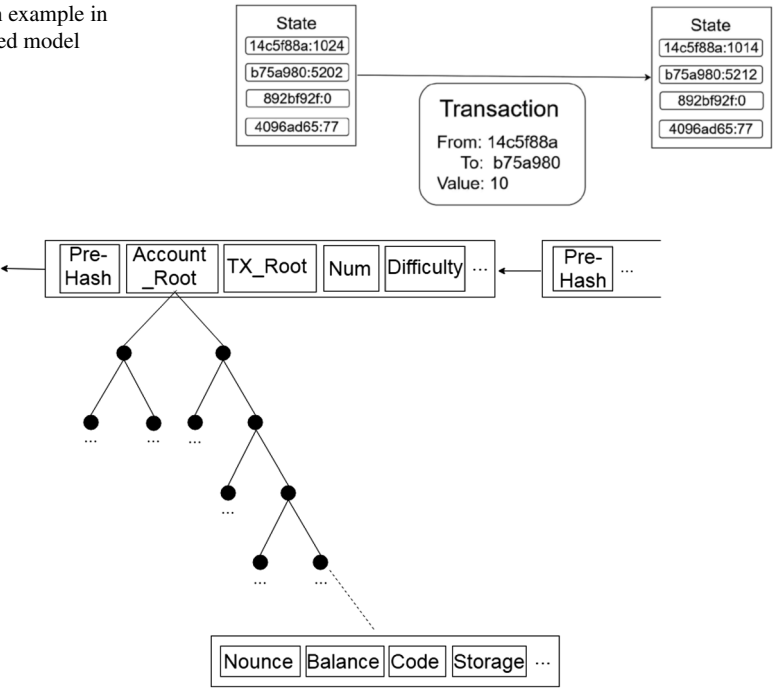
**Layered Structure of Ethereum**

The structure of Ethereum can be broadly divided into six layers, each with its own technologies. Together, they form the complete functionality of Ethereum, and the absence of any part would prevent the execution of smart contracts as well as the storage, transmission, and verification of data. Figure 2.2 outlines the general hierarchical structure of Ethereum.

Among these, the Data Layer refers to the fundamental infrastructure that stores, manages, and maintains transaction data within the blockchain, ensuring its security, integrity, and immutability. The Network Layer is responsible for communication and information transfer between nodes, ensuring the connectivity and stability of the blockchain network. The Consensus Layer implements the consensus mechanism, determining data writing permissions and synchronization methods. The Contract Layer includes the Ethereum Virtual Machine (EVM) and smart contracts, enabling automatic execution. The Incentive Layer achieves incentive mechanisms by rewarding miners and paying transaction fees. Finally, the Application Layer encapsulates various functionalities, allowing users to interact with Ethereum’s content through it.

Different from Bitcoin, Ethereum executes the account-based model where the system first checks whether the balance in the payer’s account is enough. If yes, the transacted amounts are directly subtracted from the payer’s account and added into the payer’s account. After each transaction, corresponding accounts update their

**Fig. 2.3** An example in account-based model



**Fig. 2.4** Basic structure of account-based transaction system

states. Figure 2.3 shows an example of an account-based transaction. Each item contains the account and its balance. 14c5f88a transfers 10 coins to b75a980. Then 14c5f88a’s balance becomes 1014 coins and b75a980’s balance becomes 5212 coins while other accounts remain unchanged.

Figure 2.4 shows a basic account-based structure, where the global state on the blockchain is made up of objects called “accounts.” The blockchain uses Merkle tree to store all accounts. Each account has a 4-tuple containing as its state, i.e., nonce, balance, code, and internal storage. The nonce is a counter used to make sure each transaction can only be processed once. The balance is an accumulation of balances of all transactions, where the balance of a transaction is what the account receives (if it is the receiving account) minus what it spends (if it is the sending account). The code is the contract code under smart contract. The internal storage is used to store the data.

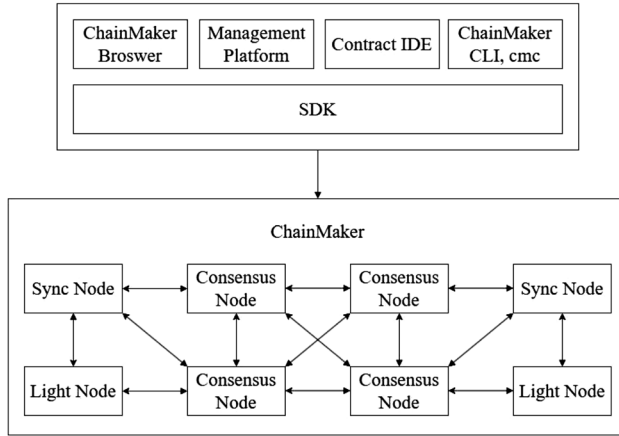
A transaction transfers the transacted amount from one account to another. Once a transaction is issued, the transacted amount is debited from the sending account. If the sending account has enough balance and this transaction is unique, the transaction is valid. Only when this transaction is confirmed, the transacted amount is credited into the receiving account. After each transaction, the corresponding accounts’ states will be updated. This is more like a bank account system in the real world.

### 2.1.3 ChainMaker

The ChainMaker (Beijing Microchip Research Institute et al. 2023) is China's first blockchain project that can be independently controlled. It is a distributed ledger system based on blockchain technology, aimed at building a secure and trustworthy commercial application platform. Its name is derived from the prosperous ancient city of Chang'an in Chinese history, symbolizing prosperity and reliability. Therefore, its main design goal is to provide a highly secure, high-performance, and scalable blockchain solution to meet the needs of enterprise-level applications. The ChainMaker employs core technologies such as consensus algorithms, smart contracts, and distributed storage, striving to create an open, transparent, and efficient business ecosystem. It offers reliable services for data storage, contract execution, and digital asset management for enterprises.

ChainMaker has the following six advantages (Chainmaker 2024):

- **Autonomous and Controllable.** The core modules of the ChainMaker are entirely self-developed, establishing a comprehensive national cryptographic algorithm system. It features a robust permission control mechanism and comprehensive privacy protection measures. The chain supports secure storage modes and is compatible with mainstream domestic processors and operating systems, thereby building an autonomous and controllable technological foundation.
- **Open Source and Inclusive.** Adhering to the principles of open source and inclusiveness, the ChainMaker brings together a wide range of research forces from academia, industry, and application sectors. It is co-developed by top universities, renowned enterprises, and other leading entities. Additionally, it warmly welcomes both individual and enterprise developers to collaboratively build a vibrant blockchain technology ecosystem.
- **Performance Leadership.** The ChainMaker boasts a self-developed efficient contract engine, high-performance pipeline consensus algorithm MaxBFT, and high-speed P2P network Liquid. These enable an efficient storage method with pre-write logging and asynchronous disk writing, offering high concurrency and low-latency technical advantages. Its transaction throughput can reach up to 100,000 TPS.
- **Flexible Assembly.** The ChainMaker standardizes and modularizes blockchain execution processes, advancing from manual operations to automated assembly production modes. This flexibility allows users to build blockchain systems tailored to different business needs, laying the foundation for large-scale application of the technology.
- **Efficient and User-Friendly.** With multi-language SDK tools and browser plugins for convenient access, one-click deployment tools, a visual management console, and various chain-building tools ensure low entry barriers. The alerting and monitoring system enables efficient operations and maintenance, making development, debugging, deployment, and maintenance user-friendly throughout the entire process.



**Fig. 2.5** Logical architecture of ChainMaker

- **Rich Functionality.** The ChainMaker supports three identity permission systems, four types of databases, five consensus algorithms adapted to different use cases. It also supports five contract engines and two P2P networks to meet diverse developer needs. Additionally, it provides PB-level storage engines capable of supporting large-scale scenarios with tens of thousands of nodes.

The logical architecture of ChainMaker is shown in Fig. 2.5. It mainly contains the following elements:

- **Consensus Node:** A node that participates in consensus voting, transaction execution, block validation, and bookkeeping within a blockchain network.
- **Sync Node:** Also known as a witness node, it participates in block and transaction synchronization, block validation, transaction execution, and records a complete ledger of data but does not participate in consensus voting.
- **Light Node:** A node that participates in synchronizing and verifying block header information and validating the existence of transactions.
- **SDK:** Helps users connect to the blockchain network via RPC to perform functions such as contract creation, invocation, and chain management.
- **ChainMaker Browser:** A visual interface that displays blockchain information such as block details, transaction details, and node information for users.
- **Management Platform:** A visual interface that allows users to manage the chain, browse information, and monitor resources conveniently.
- **Contract IDE:** An online development environment for smart contracts, supporting all contract languages of ChainMaker for development and compilation.
- **ChainMaker CLI, cmc:** A command-line toolset that allows users to deploy and manage the chain using command-line operations, such as certificate generation, chain configuration, and transaction submission.

2.1.4 Monero

Monero (Thankful for Today 2014) is a cryptocurrency that specializes in private and censorship-resistant transactions. Its various privacy-enhancing technologies help ensure the anonymity of its users. It is the only major cryptocurrency where every user is anonymous by default. The sender, receiver, and amount of every single transaction are hidden to all other participants on the network.

Most major cryptocurrency networks, including Bitcoin and Ethereum, operate using transparent blockchains, meaning that transactions can be verified and traced by anyone in the world. While cryptocurrencies are often touted as being anonymous, in reality most are only pseudonymous. With enough effort, a network observer would be able to trace an entity back to a real-world person. Monero is different. Its core development team’s hyper-focus on privacy means every transaction on Monero is untraceable, making it a truly fungible currency. Merchants and individuals accepting Monero do not need to worry about blacklisted or tainted coins.

It achieves this increased level of privacy through three specific features: Ring Signatures, Confidential Addresses, and Ring Confidential Transactions (Noether et al. 2016), as shown in Fig. 2.6 (Feng et al. 2019).

- **Ring Signatures.** A ring signature is a cryptographic technique that allows a group of individuals to digitally sign a message anonymously. Ring signatures conceal the true sender of a transaction by creating a “ring” of potential senders, including the actual sender, requiring each ring member to sign the transaction. The signature itself does not reveal which ring member signed the message,

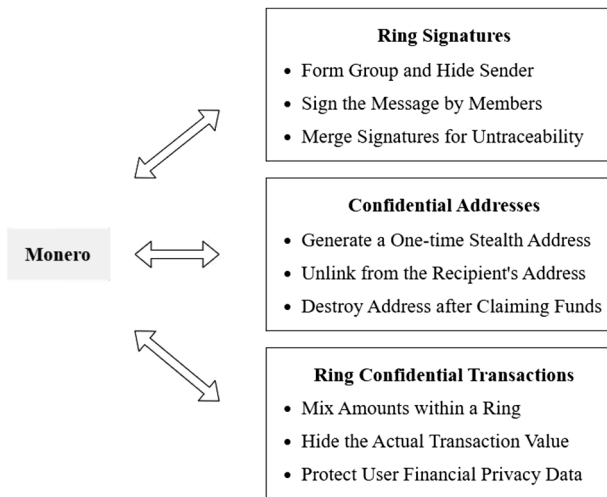


Fig. 2.6 Monero

making it impossible to identify the true sender. To send XMR, the sender first selects a group of potential signers, including themselves. Each member of the ring generates a unique key pair for the transaction. The private key will be used for signing, while the public key will be used to verify the signature. The sender then creates a transaction, signs it using their private key, and includes the public keys of all the other members of the ring. Anyone can verify the transaction by checking the signatures of all the public keys in the ring. Since each key can create a valid signature, it is impossible to determine which ring member signed the transaction.

- **Confidential Addresses.** Confidential addresses are one-time addresses generated uniquely for the recipient of a transaction. They further enhance privacy by preventing the association of a user's public address with the transactions they make, unlike addresses used on other blockchains, which are linked to a specific user's wallet. When a user sends XMR to another user, their wallet generates a unique stealth address for the recipient. This address is derived from the recipient's public key but is not directly linked to them. The sender then creates a transaction using the recipient's stealth address as the destination. Once the funds have been claimed, the stealth address is automatically destroyed, preventing it from being reused for future transactions.
- **Ring Confidential Transactions.** RingCTs hide the transaction amounts, adding another layer of privacy by making it impossible to determine how much Monero is being sent in each transaction and preventing funds from being tracked. RingCTs create a "ring" of potential transaction amounts, including the actual amount being sent, and mixes the amounts, effectively hiding the financial details of the transaction. Prior to RingCTs, anyone analyzing the Monero blockchain could see the transaction amounts, even though the senders and receivers were hidden. This allowed for some level of transaction tracking and analysis, potentially compromising users' financial privacy.

The advantages of Monero are as follows:

- **Fungibility.** For a currency to be fungible, its value can't be influenced by its source or usage. As many cryptocurrencies can be blacklisted if, for example, they were stolen, they aren't fungible. However, as Monero is untraceable, it is also truly fungible.
- **ASIC-resistant.** The majority of many types of cryptocurrencies are mined by purpose-built units known as "ASIC miners." This makes them more centralized as the supply is controlled by those who possess ASIC miners. To prevent this, Monero mining can only be completed via CPUs and GPUs.
- **Privacy.** This is Monero's main selling point. It offers users true anonymity by preventing others from seeing your balance. This is something that many other cryptocurrencies lack.
- **Scalability.** There is no limit to Monero's blockchain. This scalability should prevent fees from skyrocketing as usage increases.



The disadvantages of Monero are as follows:

- **Reputation.** The anonymity Monero offers has led to its adoption by black markets, and it is widely used on the dark web. This association may limit Monero's widespread adoption.
- **Centralisation of miners.** In spite of the fact that Monero is ASIC-resistant, it still hasn't solved the problem of disproportionate control of cryptocurrencies. Three pools control over 40% of the hash rate.
- **Lack of wallets.** It's more difficult to develop wallets to work on Monero than for other cryptocurrencies. Therefore, there aren't a lot of wallets that support Monero.
- **Lack of adoption.** Monero isn't as widely adopted as other cryptocurrencies like Bitcoin, which limits its use.

Monero's privacy-enhancing features make it more complex than other cryptocurrencies, potentially hindering adoption among less technical users. The intricate nature of ring signatures, stealth addresses, and ring confidential transactions can be daunting for those unfamiliar with cryptography.

While Monero's privacy features protect user anonymity, they have also raised concerns and made it attractive for individuals and entities engaged in drug trafficking, gambling, hacking, and other illicit activities. According to blockchain analysis firm Chainalysis, XMR has become favored over BTC on darknet markets. These online underground economies sell illegal goods and operate on the dark web, an encrypted portion of the internet inaccessible through traditional search engines (Chainalysis 2023).

Monero's strong privacy features have attracted scrutiny from concerned regulators and governments. It has made it more difficult for law enforcement to identify suspects and the flow of illicit funds. For example, in 2021, the U.S. Department of the Treasury's Office of Foreign Assets Control (OFAC) designated Sodinokibi and REvil ransomware operators as sanctioned entities. The OFAC alleged that the cybercriminals used BTC and XMR to launder money stolen in the ransomware attacks. U.S. Department of the Treasury, Office of Foreign Assets Control. "Treasury Continues to Counter Ransomware as Part of Whole-of-Government Effort; Sanctions Ransomware Operators and Virtual Currency Exchange" (Janet L. Yellen 2022).

## 2.2 Classical Cross-Chain Systems

### 2.2.1 *Cosmos*

Cosmos (Cosmos 2023) is an innovative blockchain network architecture that supports heterogeneous networks, utilizing the Tendermint consensus algorithm. Tendermint Core provides a high-performance, secure, and consistent consensus protocol. Cosmos has established the first cross-chain zone, the Cosmos Hub,

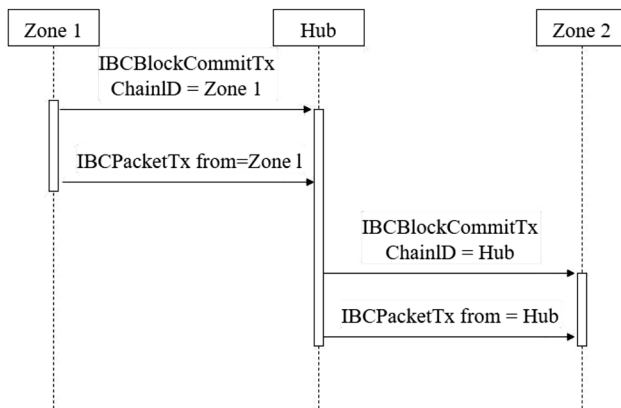
where different blockchains adhere to the Inter-Blockchain Communication (IBC) protocol.

Cosmos offers the IBC protocol and mechanism, which connects various zones in the cross-chain system through the Cosmos Hub. All token transfers between zones are conducted via the Cosmos Hub, which also tracks the total number of tokens held in each zone. This allows Cosmos to achieve interoperability between blockchains, enabling asset transfers without relying on a trusted third party.

Tendermint is considered one of the most viable alternatives to high-energy-consuming PoW blockchains like Bitcoin and Ethereum. This strategy combines practical Byzantine fault-tolerant consensus with a PoS proof mechanism, dynamically selecting nodes as validators for the next block. Tendermint Core can turn any application black box into a distributed blockchain without needing to understand its specific operation, as long as it produces deterministic results. By connecting to blockchain applications through the Application Blockchain Interface (ABCI), users can program blockchain applications in any language.

Cosmos is a network of many interconnected blockchains supported by Tendermint, allowing these blockchains to run concurrently while maintaining interoperability. The Cosmos Hub manages numerous independent blockchains, with each zone continuously submitting the latest block data to keep the Hub's state synchronized with each zone. It then publishes Merkle proofs, which serve as evidence of information sent and received, facilitating the flow of information between zones. This mechanism is also known as Inter-Blockchain Communication (IBC).

For example, Fig. 2.7 illustrates the communication process where data are transferred from one blockchain zone (Zone 1) through the Hub to another blockchain zone (Zone 2) (Guo et al. 2020). The sending chain freely manages the published data, with the sender attaching block header information for the receiving chain



**Fig. 2.7** Communication process between each region in Cosmos and Hub interconnection

to confirm the received data. The IBC mechanism splits this process into two independent transactions:

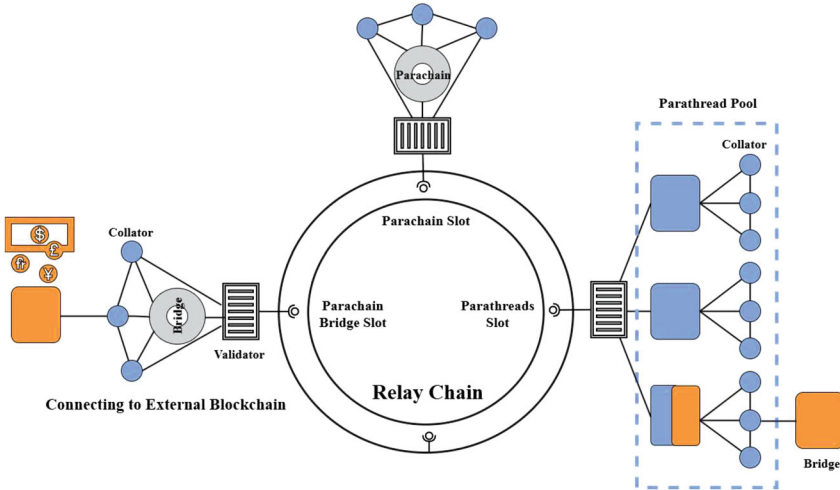
- **IBCBlockCommitTx:** Allows the blockchain to prove its latest hash value to any observer.
- **IBCPacketTx:** Allows the blockchain to prove to any observer that given data were published by the sender's application, using Merkle proof of the recent block hash.

### 2.2.2 *Polkadot*

Polkadot (Wood 2016), whose network was only launched mid-May 2020, yet had successfully secured a spot amongst the top 10 cryptocurrencies, owning a market capitalization of around 18 billion USD [2]. Polkadot is developed by the Web3 Foundation and Parity Technologies, both founded by Dr. Gavin Wood, who is also known for being a cofounder and former CTO of Ethereum. In 2016, Dr. Gavin Wood shared his vision of Polkadot and its technical outline through the white paper [3], which was, in his words, creating “the next version of Ethereum” that would complement the aspects for which Ethereum 2.0 is less optimal [4].

Polkadot is a fully “sharded” blockchain whose design is based on sharding, a database splitting technique, which enables multiple chains to process their transactions in parallel. Each blockchain shard is called a “parachain,” which connects to the Relay Chain. The Relay Chain is effectively the heart of the blockchain, acting as the main hub of the system. Furthermore, Polkadot serves as an interoperability platform; i.e., it allows cross-communication between heterogeneous blockchains including external ones, such as Bitcoin and Ethereum. While the Relay Chain orchestrates and ensures correct functioning of the entire network, parachains can be customized as needed, including to host smart contracts and a myriad of other use cases. Essentially, Polkadot was developed with the goal to address some shortcomings of existing blockchain technologies, namely, scalability, interoperability, and achieving standard security guarantees across differing trust models.

Polkadot is popular for being a multi-chain (sharded) blockchain network, specifically built to support the integration of heterogeneous blockchains in a trust-free environment and under common security guarantees. It also enables secured communication with external sovereign blockchains (e.g., Bitcoin or Ethereum), thus, laying the foundations for establishing a truly decentralized web. In addition to interoperability, Polkadot's multi-chain design allows transactions to be processed in parallel instead of sequentially, leading to a higher throughput and scalability. All these properties are provided owing to the Relay Chain. It unites a network of heterogeneous blockchains (or arbitrary state machines), operating in parallel, which include parachains, parathreads, and bridges. The Relay Chain provides shared security to the entire network through its consensus mechanisms.



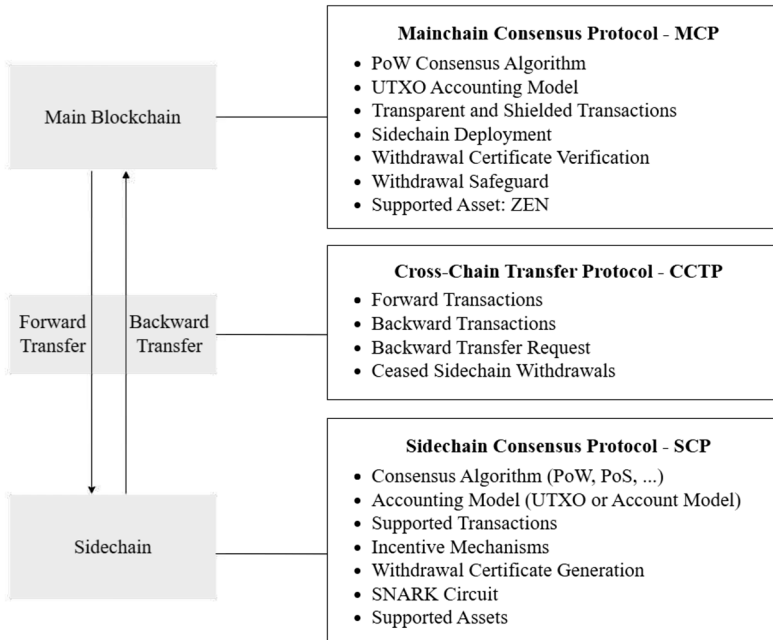
**Fig. 2.8** Polkadot data structures and participants

Polkadot adopts Proof-of-Stake (PoS) consensus, namely a variation of it called the Nominated Proof of Stake (NPoS). The scheme is referred to as “nominated” because the validators (Relay Chain maintainers: nodes that are responsible for authoring and verifying new blocks) are backed up by nominators who lock their DOT—Polkadot’s native currency—as collateral in exchange for staking rewards. However, if validators misbehave, their corresponding nominators are also slashed (i.e., DOT deduction from their accounts). While validators maintain the Relay Chain, a collator is a node responsible for maintaining its parachain. The basic structures and roles are shown in Fig. 2.8 (Abbas et al. 2022).

For a long time, Polkadot has maintained a steady position in terms of coin market capitalization, ranking among the top 10, meanwhile its network and architecture are still developing and not fully mature yet. In addition, even though Polkadot was introduced as a multi-chain interoperability platform, parachains were only added in November 2021, a year and half after the network’s launch, and it is not known yet when bridges will become available to fulfill the interoperability claims.

### 2.2.3 Zendoo

Zendoo (Sompolinsky et al. 2016) is a verifiable cross-chain transfer protocol that allows for the forward transfer of data and assets, meaning the movement of assets from the original blockchain A (main chain) to the target sidechain B. Figure 2.9 presents the main components (Horizon 2023). The main chain generates a special type of operation that removes the corresponding amount of currency and provides



**Fig. 2.9** Main components of Zennod

the sidechain with the metadata of the equivalent amount. The sidechain then accepts this metadata and issues the corresponding amount of currency, thereby achieving cross-chain asset transfer. The forward transfer process can be represented by a tuple of parts:

- **Ledger ID:** The unique identifier of the target sidechain must be specified before any transactions or asset transfers.
- **Amount of Currency:** The amount of currency to be transferred between the main chain and the sidechain.
- **Metadata:** This structure is not fixed in the main chain. Its semantic meaning is composed of different predefined types based on the structure of different sidechains. Therefore, the main chain does not need to understand its semantic structure to construct the metadata.

There are four cross-chain modes in total:

- **Forward Transfer (FT)** operates on the main blockchain: it transfers the assets on the main blockchain to the sidechains. The protocol is also quite simple: the main blockchain treats FT as a special transaction, destroying coins on the main blockchain and providing metadata concerning the payment on the sidechains. The side chain needs to monitor the main blockchain in real-time. If the latest block of the main chain does not contain FT transactions, the side chain will only

use the hash of the block in the main blockchain as a reference and add it to the side chain; if the latest block of the main blockchain contains FT transactions, the entire block header, all FT transactions, and the Merkle path of all FT transactions will be introduced into the side chain as a reference. In this way, anyone on the side chain is able to verify whether the transaction is legal instead of going to the main blockchain for query verification.

- **Backward Transfer (BT)** operates on the side chain and transfers the assets on the side chain to the main blockchain. As mentioned before, IOHK is also one of the authors of the Zendoo white paper, so we can see the resemblance of the Ouroboros protocol in the design of BT. Like the Ouroboros protocol, the protocol design of BT also involves the concepts of epoch and slot, as shown in the figure above. A withdrawal epoch is composed of len and slots, and the len has different parameters for different side chains. If a withdrawal certificate is not submitted to the main chain within this period of time, the side chain will be considered as ceased (for instance, a withdrawal certificate in epoch  $i$  is not submitted to the main blockchain in the first len blocks in epoch  $i+1$ ), and subsequent withdrawal certificates will not be accepted by the main blockchain, but the funds of the side chain can still be withdrawn. The  $B$  in the given figure is a slot. The side chain forger (block producer) will package the slot with BT signature and submit the withdrawal application certificate to the main blockchain within the specified epoch time.
- **Backward Transfer Request (BTR)** is possible that users cannot perform BT operations on the sidechain. For example, the RPC address on the sidechain refuses to access. At this time, if one wants to retrieve the funds on the sidechain to the main blockchain, one can only initiate a BTR operation on the main blockchain. BTR will not directly transfer funds on the main chain, but synchronize this BTR to the sidechain, and go through the process analogous to BT (excluding the user's active part) on the sidechain.
- **Cessed Sidechain Withdrawal (CSW)**. When the sidechain does not respond, for example, the sidechain is attacked or the heartbeat message is not sent to the main blockchain within the specified time. At this time, the funds on the sidechain need to be withdrawn from the main blockchain.

In contrast to forward transfers, the reverse transfer protocol allows currency to move from the sidechain back to the main chain, relying on the idea of batch transfers. This means that if all requests submitted to the sidechain during a certain period are transferred back, this period is called the “withdrawal epoch.” The withdrawal information is collected in a special withdrawal certificate and sent to the main chain for processing.

The withdrawal certificate is not only a container for reverse transfers but also a heartbeat mechanism for the sidechain. It periodically sends feedback to the main chain to help confirm the status of each sidechain. Additionally, the withdrawal times of different sidechains are not aligned and may vary in length, allowing the entire system to operate asynchronously.

### 2.2.4 Horizon

Horizon (Horizon 2018) is an efficient cross-chain bridge protocol that transfers assets from a BFT blockchain to another blockchain (like Ethereum), supporting basic smart contract execution. The cross-chain bridge protocol enables on-chain cryptocurrency exchanges, asset transfers to the periphery, and cross-shard verification of events in segmented blockchains, mainly relying on Byzantine Fault Tolerance (BFT) protocols to achieve scalability. Unfortunately, existing bridge protocols that transfer funds from BFT blockchains to target blockchains generate significant computational overhead, making smart contract verification costly.

As shown in Fig. 2.10, a cross-chain transaction  $A \rightarrow B$  aims to transfer currency from blockchain A to blockchain B (such as the Ethereum blockchain) (Lan et al. 2021). The Horizon cross-chain bridge protocol includes a smart contract S deployed on blockchain B and a relay node R, which periodically syncs with block headers from A. Horizon consists of two key components:

- **Relay/Contract Synchronization:** At the end of each period (every 24 hours), R sends the latest block header to the contract, which contains enough information to allow the contract to verify any transaction included on A. In the cross-chain bridge protocol, contract verification is represented by Tburn, submitted by the client. Tburn essentially transfers currency from A to an empty address, permanently deleting the currency.
- **Cross-Chain Transaction:** Horizon protocol's cross-chain transaction is initiated by client C when submitting Tburn to blockchain A. Once the transaction is confirmed on A, C sends a request to a full node F, which maintains a complete and up-to-date copy of chain A. After receiving the request, F locates the block Bk containing Tburn, checks, and verifies its Merkle tree root. Finally, F sends the verified data to C, who creates an unlock transaction and submits it to blockchain B. Since the full node is untrusted, this transaction might fail. In such a case, the client replaces F with another full node and repeats the process.

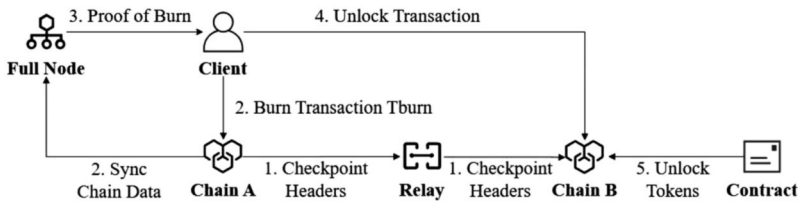


Fig. 2.10 Horizon cross-chain protocol

## 2.3 Preliminaries of Cross-Chain Regulation

### 2.3.1 Feature Engineering

Features are information extracted from data that is useful for outcome prediction. Feature engineering (Zhang et al. 2020) is the process of transforming raw data into features that better express the nature of the problem for use by algorithms and models. Better features means flexibility, simpler models, and better results. We go through a series of steps to complete the entire feature engineering. Figure 2.11 shows the whole procedure.

Many machine learning algorithms (Alves et al. 2022; Qi et al. 2022) usually perform better when numerical variables have a Gaussian probability distribution. We use Yeo–Johnson transformation

$$Z = \begin{cases} \log(X + 1), & \text{if } \lambda = 0, X \geq 0, \\ \frac{(X+1)^\lambda - 1}{\lambda}, & \text{if } \lambda \neq 0, X \geq 0, \end{cases}$$

$$Z = \begin{cases} -\log(-X + 1), & \text{if } \lambda = 2, X \leq 0, \\ \frac{-[(-X+1)^{2-\lambda} - 1]}{2-\lambda}, & \text{if } \lambda \neq 2, X \leq 0, \end{cases}$$

to make data more Gaussian-like. Here  $\lambda$  is the transformation factor,  $X$  is the data to be transformed, and  $Z$  means the final result of Yeo–Johnson transformation.

### 2.3.2 Graph Embedding Learning

Graph embedding learning (de Barros et al. 2023; Wu et al. 2023) is an important branch of graph learning, aimed at mapping nodes in a graph to a low-dimensional vector space to better capture the topological structure of the graph, vertex-to-vertex relationships, and other relevant information about the graph, subgraphs, and

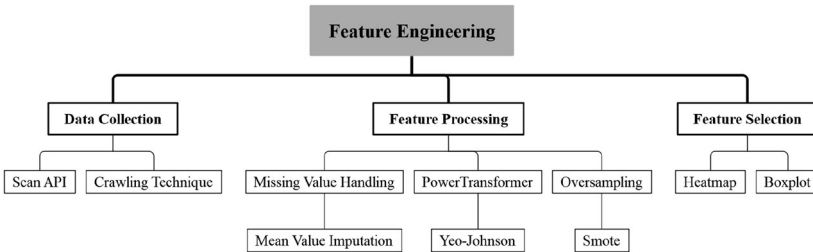


Fig. 2.11 Feature engineering procedure



vertices. The development in this field enables us to better understand and utilize information in complex network structures. A graph is a data structure composed of nodes and edges, used to represent relationships between entities. In graph embedding learning, we represent a graph in the form of an adjacency matrix or edge list, where nodes represent entities and edges represent the relationships between them. The goal of graph embedding learning is to map nodes in a graph to a low-dimensional vector space, preserving the similarity between nodes in the original space. Each node is assigned a vector, where the position of this vector reflects the node's position in the graph and its relationships with other nodes.

The approach adopted in this book is based on the neighborhood-based node2vec algorithm (Grover and Leskovec 2016). It captures neighborhood information between nodes by performing random walks in the graph and uses algorithms like Skip-gram (Li et al. 2016) to learn node embeddings, thereby better capturing the structure of the graph. Its flexible random walk strategy allows for a balance between BFS (breadth-first search) and DFS (depth-first search). In the context of cross-chain abnormal transaction detection, the strength of node2vec lies in effectively learning the relationships between nodes in the cross-chain transaction graph, providing richer information for detecting abnormal transactions.

The core idea is to treat nodes in the graph as vocabulary, node sequences (generated through random walks) as sentences, and then utilize the Word2Vec model (Adewumi et al. 2022) (specifically the Skip-gram model) to learn the vector representations of nodes. The algorithmic steps of node2vec mainly involve the following four stages:

**Stage 1: Generating Node Sequences Through Random Walks** Initiating from a particular node in the graph, a node sequence is generated based on a random walk strategy. Node2vec first samples multiple node sequences in the graph through a specific random walk strategy. This random walk strategy is controlled by two parameters,  $p$  and  $q$ , representing the return parameter and the in-out parameter, respectively. A lower value of  $p$  tends to make random walks explore previously visited nodes, emphasizing the structural properties of the network in node2vec. A lower value of  $q$  tends to make random walks explore new nodes far from the previous node, emphasizing the homophily properties of the network in node2vec. Specifically, when a walk transitions from node  $v$  to node  $u$ , the next step transition probability is defined as:

$$P(c_i = x | c_{i-1} = u) = \begin{cases} \frac{1}{p}, & \text{if } x = v, \\ 1, & \text{if } x \text{ is a neighbor of } u \text{ and } x \neq v, \\ \frac{1}{q}, & \text{otherwise,} \end{cases}$$

Here,  $c_i$  represents the node at step  $i$  of the walk, and  $x$  is a neighbor node of node  $u$ . By adjusting the values of  $p$  and  $q$ , the exploration scope and diversity of the walk can be controlled.

**Stage 2: Formulating the Objective Function** Using the generated node sequences, the Skip-gram objective function is constructed to bring nodes that appear in the same random walk sequences closer in the embedding space. The objective of the Skip-gram model is to maximize the sum of the logarithm of the probabilities of the dot product between the feature vector of a specific node and the feature vectors of its context nodes:

$$\max \sum_{u \in V} \sum_{v \in N(u)} \log P(v|u).$$

Where,  $V$  is the set of nodes in the graph,  $N(u)$  is the set of context nodes of node  $u$  (i.e., the set of nodes that appear in the random walk sequence along with  $u$ ), and  $P(v|u)$  is the conditional probability of observing node  $v$  given node  $u$ . The conditional probability  $P(v|u)$  is typically defined using the following softmax function.  $(\mathbf{v}_u)$  and  $(\mathbf{v}_v)$  are the vector representations of nodes  $u$  and  $v$ , respectively.

$$P(v|u) = \frac{\exp(\mathbf{v}_u \cdot \mathbf{v}_v)}{\sum_{w \in V} \exp(\mathbf{v}_u \cdot \mathbf{v}_w)}.$$

**Stage 3: Optimizing the Loss Function** Due to the computationally intensive nature of calculating the denominator involving all nodes in the softmax function, direct computation is often impractical. Therefore, techniques such as negative sampling or hierarchical softmax are commonly used for approximate optimization. The concept of negative sampling involves randomly selecting a certain number of negative samples (i.e., node pairs that are not neighbors) for each positive sample (i.e., neighbor node pairs in the sequence), and modifying the optimization objective to maximize the sum of the logarithm of the probabilities of distinguishing between positive and negative samples:

$$\log \sigma(\Phi(u) \cdot \Phi(v)) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(v)} [\log \sigma(-\Phi(w_i) \cdot \Phi(v))].$$

where  $\sigma$  is the sigmoid function,  $k$  is the number of negative samples, and  $P_n(v)$  is the distribution of negative samples for node  $v$ . The learning rate is gradually adjusted during the training process, usually decreasingly, to ensure model convergence. The appropriate number of iterations is chosen based on the size and complexity of the dataset to ensure adequate training while avoiding overfitting.

**Stage 4: Obtaining Node Embeddings** After training, each node is mapped to a low-dimensional vector space, forming node embeddings. These node embeddings can be used for subsequent tasks such as similarity computation or node classification.

### 2.3.3 Graph Neural Networks

Graph Neural Networks (GNNs) are a class of deep learning methods designed to perform inference on data described by graphs, which have achieved remarkable results in various fields such as social network analysis, recommender systems, bioinformatics (Zhang et al. 2021). Graph data are a common form of unstructured data, such as social networks, knowledge graphs, and chemical molecular structures. Traditional neural networks cannot deal with graph data directly, so graph neural networks have emerged to effectively capture the relationships and features between graph data. It implements the processing and analysis of graph data through node representation learning, graph convolution operations, and graph pooling operations. The graph neural networks used in this book are mainly graph convolutional networks and graph attention networks.

#### 2.3.3.1 Graph Convolutional Networks

Graph Convolutional Networks (GCNs) (Mesgaran and Hamza 2024) leverage the convolution operation on graphs to perform node-level or graph-level tasks. The key idea behind GCNs is to generalize the convolution operation, which is typically applied on regular grids, to graph-structured data. This allows GCNs to propagate information across the graph while taking into account the connectivity and topology of the graph. Here's a brief overview of how GCNs work:

- **Graph Representation:** Let  $G = (V, E)$  be an undirected graph, where  $V$  represents the set of nodes (with  $N$  nodes), and  $E$  represents the set of edges. We can represent the graph structure using an adjacency matrix  $A$ , where  $A_{ij} = 1$  if there is an edge between nodes  $i$  and  $j$ , and  $A_{ij} = 0$  otherwise.
- **Node Features:** Each node  $v_i$  is associated with a feature vector  $x_i$  of dimension  $F$ , representing its attributes. We can stack these feature vectors into a node feature matrix  $X$ . Its size is  $N \times F$ .
- **Graph Convolution:** The core operation in GCNs is the graph convolution, which aggregates information from a node's neighborhood and updates its representation. Given an input node feature matrix  $X$  and adjacency matrix  $A$ , the output of a single graph convolutional layer can be defined:

$$H^{(l+1)} = f \left( A \cdot H^{(l)} \cdot W^{(l)} \right).$$

Here,  $H^{(l)}$  is the node representation matrix at layer  $l$  (initialized as  $X$  at layer 0),  $W^{(l)}$  is the weight matrix of the convolutional operation, and  $f$  is the activation function.  $A \cdot H^{(l)}$  performs message passing, aggregating information from neighboring nodes for each node.  $W^{(l)}$  applies linear transformation to the aggregated messages. And  $f$  applies a nonlinear activation function element-wise to the result.

- **Propagation:** Multiple graph convolutional layers can be stacked to propagate information across the graph and refine node representations:

$$H^{(l+1)} = f\left(A \cdot H^{(l)} \cdot W^{(l)}\right).$$

By iteratively applying graph convolutional operations, GCNs effectively capture the structural information of the graph and learn meaningful representations for nodes, enabling them to perform well on a wide range of tasks.

### 2.3.3.2 Graph Attention Networks

A Graph Attention Network (GAT) (Tian et al. 2021) is an architectural design of a neural network that functions on data structured in graphs. It utilizes masked self-attention layers to tackle the limitations observed in previous approaches reliant on graph convolutions or their estimations. Through the sequential arrangement of layers, wherein nodes possess the capability to focus on the features of their neighboring nodes, a GAT allows for the implicit assignment of varying weights to nodes within a neighborhood. Notably, this is achieved without necessitating computationally expensive matrix operations, like inversion, or prior knowledge of the graph structure. Graph Attention Networks (GATs) extend the capabilities of Graph Convolutional Networks (GCNs) by incorporating an attention mechanism, allowing nodes to selectively aggregate information from their neighbors. A few core concepts are then introduced:

- **Graph Representation:** Just like in GCNs, the input data are represented as a graph  $G = (V, E)$ , where  $V$  represents the set of nodes and  $E$  represents the set of edges.
- **Node Features:** Each node  $v_i$  in the graph is associated with a feature vector  $x_i$  of dimension  $F$ , forming a node feature matrix  $X$  of size  $N \times F$ , where  $N$  is the number of nodes.
- **Attention Mechanism:** GATs incorporate an attention mechanism to compute weighted combinations of neighboring node features. For each pair of nodes  $v_i$  and  $v_j$ , attention scores are computed based on their features. The attention scores are computed as a function of the nodes' features using a learnable attention mechanism.
- **Graph Attention:** Given the node feature matrix  $X$ , attention mechanism computes attention coefficients for each pair of nodes. Let  $e_{ij}$  be the attention score between node  $v_i$  and node  $v_j$ , computed as a function of their features:

$$e_{ij} = \text{LeakyReLU}\left(\mathbf{a}^\top [W \cdot \mathbf{h}_i \parallel W \cdot \mathbf{h}_j]\right).$$

Here,  $\mathbf{h}_i$  and  $\mathbf{h}_j$  are the hidden representations of nodes  $v_i$  and  $v_j$ , respectively,  $W$  is a learnable weight matrix,  $\parallel$  denotes concatenation, and  $\mathbf{a}$  is a learnable

attention vector. LeakyReLU is a leaky rectified linear unit activation function.  $\mathbf{a}$  is shared across all nodes and serves as the attention mechanism's parameters.

- **Attention Coefficients:** The attention scores  $e_{ij}$  are normalized using the softmax function to obtain attention coefficients  $\alpha_{ij}$ . Here,  $\mathcal{N}_i$  represents the neighbors of node  $v_i$ .

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}.$$

- **Aggregation:** Once attention coefficients are computed, they are used to compute a linear combination of neighboring node features:

$$\mathbf{h}_i' = \sigma \left( \sum_{j \in \mathcal{N}_i} \alpha_{ij} \cdot W \cdot \mathbf{h}_j \right).$$

Here,  $\mathbf{h}_i'$  is the updated representation of node  $v_i$ ,  $W$  is a learnable weight matrix, and  $\sigma$  is an activation function.

By selectively attending to different parts of the graph, GATs effectively capture complex relationships between nodes and have demonstrated state-of-the-art performance on various tasks involving graph-structured data.

### 2.3.4 Traffic Detection Methods

There are many kinds of intrusion methods in blockchain network, and abnormal traffic is one of them. Abnormal traffic is widely used for its easy operation, low cost, and unequal benefit of both sides. The key to improving the security of blockchain networks is the monitoring of abnormal traffic, which can effectively sense abnormal behavior within the network. This section classifies traffic detection methods according to different attack modes and describes various traffic detection methods in detail.

In the blockchain network layer, it is typically necessary to collect datasets to distinguish between normal traffic and DDoS attack traffic. Currently, there are two main methods for detecting DDoS attacks at the blockchain network layer:

#### 2.3.4.1 Detection Method Based on Node Traffic Statistical Analysis

The first detection method is based on node traffic statistical analysis, which sets two indicators to determine whether the traffic is a network layer DDoS attack: traffic statistics and network congestion levels (Rüth et al. 2018). Relevant feature thresholds are then used to separate and detect DDoS attack traffic (Baek et al. 2019). This statistical classification of traffic datasets is achieved by setting appropriate time intervals at the terminal routers of the blockchain network.

However, this detection technique has several drawbacks:

- First, attackers might exploit the fact that the blockchain consensus process allows for redundant traffic transmission, spreading through flooding mechanisms (Essaid et al. 2019). Such attack traffic often employs feature evasion strategies to avoid detection, reducing the accuracy of statistical detection methods (Mirkin et al. 2020).
- Additionally, because the characteristics of such attack traffic are very similar to Flash Crowd traffic, this detection technique struggles to distinguish between these types of traffic and DDoS attack traffic, thereby increasing the false-positive rate (Idé 2018).

#### 2.3.4.2 Detection Method Based on Machine Learning

This approach primarily involves collecting various traffic data from the blockchain network layer (Hu et al. 2022), including transaction information, inter-node communication, etc. Machine learning algorithms are then used to analyze and process this data to extract fundamental features of DDoS attack traffic, such as abnormal traffic patterns, frequency, and sources. The trained machine learning model is then used to detect anomalies by comparing and analyzing input traffic samples to identify potential DDoS attack traffic (Baek et al. 2019). These models are typically trained using numerous real and simulated attack scenarios to improve their accuracy and reliability. Once anomalous traffic matching DDoS attack characteristics is detected, the system takes corresponding defensive measures, such as traffic filtering, blocking, or alerting, to ensure network security and stability. This machine learning approach helps network administrators promptly identify and respond to DDoS attacks, ensuring the normal operation and data security of the blockchain network.

However, this strategy also has its limitations:

- The complex environment of the blockchain network layer, which includes multiple consensus mechanisms, weakens the generalization capability of detection models (Saad et al. 2018).
- Current machine learning models struggle to distinguish key components of dynamically changing mixed DDoS attack traffic because the characteristics of rumor-based attack traffic within the blockchain network layer are very similar to those of regular network traffic (Rodrigues and Stiller 2019), leading to lower detection accuracy with current methods.

Specifically, although the P2P network structure forms the underlying foundation of the blockchain network layer, existing P2P network DDoS detection algorithms are not entirely suitable for the blockchain network layer environment due to unsolved issues arising from differences in traffic propagation algorithms between the two networks. The inherent weaknesses of network redundancy in blockchain traffic propagation allow attackers to masquerade as P2P or rumor-based transaction

traffic, making it impossible to directly apply suppression strategies based on harmful repeated P2P traffic to blockchain networks (Ghaffar et al. 2019; Gao and Gao 2020). Furthermore, rumor-based DDoS attack traffic may use traffic obfuscation to evade some anomaly detection methods, posing significant practical challenges in detecting DDoS attack traffic in blockchain networks.

## 2.4 Preliminaries of Cryptography-Empowered Privacy Preserving

### 2.4.1 Accumulator

An accumulator (Au et al. 2009) is a cryptographic data structure used to quickly verify that an element belongs to a set without enumerating all the elements in the set individually. Specifically, denote  $\mathcal{F} = \{F_\lambda\}$  as families of functions and  $\mathcal{Y} = \{Y_\lambda\}$  as families of finite sets, which satisfy  $F_\lambda = \{f : U_f \times Y_f \rightarrow U_f\}$  and  $Y_\lambda \subseteq Y_f$  for all  $\lambda \in \mathbb{N}$ . An accumulator includes the following algorithms:

- **ACC.GEN**( $1^\lambda$ ). It takes as input a security parameter  $\lambda$  and outputs a description **desc**.
- **ACC.Eval**(**desc**,  $Y$ ). It takes as input the description **desc** and  $Y \subset Y_\lambda$  and outputs an accumulated value  $v = f(u, Y)$ , where  $f \in F_\lambda$ ,  $u \in U_f$  and  $f(u, Y) = f(\dots f(u, y_1) \dots y_n)$ ,  $Y = \{y_1, \dots, y_n\} \subset Y_\lambda$ .
- **ACC.Wit**(**desc**,  $y$ ,  $Y$ ). It takes as input the description **desc**,  $Y \subset Y_\lambda$  and  $y \in Y$ , and outputs a witness  $\omega$ , where  $v = f(\omega, y)$ .
- **ACC.Verify**(**desc**,  $\omega$ ,  $y$ ,  $Y$ ). It takes as input the description **desc**, the witness  $\omega$ ,  $Y \subset Y_\lambda$ , and  $y$ , and certifies whether  $y \in Y$  and outputs 1 for yes, 0 for no.

We call the pair  $(\mathcal{F}, \mathcal{Y})$  an accumulator family with one-way domain if the following conditions hold:

- quasi-commutativity: for all  $\lambda \in \mathbb{N}$ ,  $f \in F_\lambda$ ,  $u \in U_f$ ,  $x_1, x_2 \in X_\lambda$ , it holds that  $f(f(u, x_1), x_2) = f(f(u, x_2), x_1)$ .  $X_\lambda$  is always referred to as the domain of this accumulator. For any  $X = \{x_1, x_2, \dots, x_n\} \in X_\lambda$ , we further refer to  $f(\dots f(u, x_1) \dots x_n)$  as the accumulated value of  $X$  over  $u$ , which will be denoted by  $f(u, X)$  thanks to this quasi-commutative property.
- collision-resistance: for all  $\lambda \in \mathbb{N}$  and efficient adversaries  $\mathcal{A}$ , it holds that

$$\Pr \left[ \begin{array}{l} X \subset X_\lambda \wedge (x \in X_f \setminus X) \\ (w \in U_f) \wedge (f(w, x) = f(u, X)) \end{array} \right] : f \leftarrow F_\lambda; \\ u \leftarrow U_f; (x, w, X) \leftarrow \mathcal{A}(f, U_f, u)$$

- one-way domain: let  $\{Y_\lambda\}$ ,  $\{R_\lambda\}$  be two sequences of families of sets associated with  $\{X_\lambda\}$ , such that each  $R_\lambda$  is an efficiently *verifiable, samplable relation* over

$Y_\lambda \times X_\lambda$  and it is infeasible to efficiently compute a witness  $y' \in Y_\lambda$  for an  $x$  sampled from  $X_\lambda$ . That is,

$$\Pr[(y', x) \in R_\lambda : (y, x) \leftarrow \text{Samp}(1^\lambda); y' \leftarrow \mathcal{A}(1^\lambda, x)] \leq \text{negl}(\lambda),$$

where  $\text{Samp}$  denotes the efficient sampling algorithm over  $R_\lambda$ .

- **efficient generation:** there exists an efficient algorithm denoted by  $\text{ACC.Gen}$  that on input a security parameter  $\lambda$  outputs a description  $\text{desc}$  of a random element of  $F_\lambda$ , possibly including some auxiliary information.
- **efficient evaluation:** for  $\lambda \in \mathbb{N}$ ,  $f \in F_\lambda$ ,  $u \in U_f$  and  $X \subset X_\lambda$ ,  $w \in U_f$  is called a witness for the fact that  $x \in X$  has been accumulated within  $v \doteq f(u, X) \in U_f$  iff  $f(w, x) = v$ . There exist two algorithms denoted by  $\text{ACC.Eval}$  and  $\text{ACC.Wit}$  that on input  $(\text{desc}, X)$  and  $(\text{desc}, x, X)$  can efficiently evaluate the accumulated value  $f(u, X)$  and the witness for  $x$  in  $f(u, X)$ , respectively.

For many applications of cryptographic accumulators, the set of interest  $\mathcal{Y}$  evolves with the time, after the accumulator  $Y$  has been computed. A naive way of handling such situations would be to recompute the accumulator from scratch each time the set  $\mathcal{Y}$  changes: however, this would be highly impractical, especially when the set  $\mathcal{Y}$  is rather large. For this reason, recently, Camenisch and Lysyanskaya (2002) introduced the notion of dynamic accumulators, where the cost of computing the new accumulator  $Y'$ , resulting after adding or deleting an element from  $\mathcal{Y}$ , is independent of the number of accumulated values.

A dynamic accumulator scheme is a 7-tuple of polynomial time algorithms  $(\text{Gen}, \text{Eval}, \text{Wit}, \text{Verify}, \text{Add}, \text{Del}, \text{Upd})$ , where

- **Gen, Eval, Wit, Verify** are as in a regular accumulator scheme;
- **Add**, the element addition algorithm, is a (usually deterministic) algorithm that given an accumulator key  $k$ , a value  $y$  obtained as the accumulation of some set  $Y$ , and another element  $y'$ , returns a new accumulator  $Y'$ , along with a witness  $w'$  for  $Y'$  and some update information  $\text{aux}_{\text{Add}}$ , which will be used by the **Upd** algorithm;
- **Del**, the element deletion algorithm, is a (usually deterministic) algorithm that given an accumulator key  $k$ , a value  $y$  obtained as the accumulation of elements of  $Y_k$ , and an element  $y'$ , returns a new accumulator  $Y'$ , along with some update information  $\text{aux}_{\text{Del}}$ , which will be used by the **Upd** algorithm;
- **Upd**, the witness update algorithm, is a deterministic algorithm used to update the witness  $w$  for an element  $y$  previously accumulated within an accumulator  $Y$ , after the addition (or deletion) of an element  $y'$  in (or from)  $Y$ . **Upd** takes as input  $(k, y, w, b, \text{aux}_{\text{op}})$  (where  $\text{op}$  is either **Add** or **Del**), and returns an updated witness  $w$  that “proves” the presence of  $y$  within the updated accumulator  $Y'$ .

The definition presented earlier is actually slightly different from that of Camenisch and Lysyanskaya (2002): a few changes were made to attain a formalization more adherent to the original motivation of Benaloh and de Mare (1993), i.e., avoiding the need for a trusted central authority. In fact, to meet the



efficiency requirement for the element deletion algorithm **Del**, in Camenisch and Lysyanskaya (2002), Camenisch and Lysyanskaya considered schemes where the accumulator key generation algorithm **Gen** outputs, along with the accumulator key  $k$ , some secret information  $tk$  that enables an efficient implementation of the **Del** algorithm, but at the same time opens a potential hole in the security of the scheme itself. Thus, the trapdoor  $tk$  should only be available to an “accumulator manager,” who is trusted to use this knowledge exclusively for the purpose of updating the accumulator after the removal of some elements and not for deriving fake witnesses for values that have not been accumulated. Such schemes, in which the presence of a trapdoor information is essential for the proper functioning of the system, will be referred to as trapdoor dynamic accumulator schemes.

### 2.4.2 Signature of Knowledge

Signature of knowledge (SoK) (Chase and Lysyanskaya 2006) allows for a payer to prove that he/she has some specific knowledge, like a NP-relation  $\mathcal{R}$  with the corresponding language  $L = \{y : \exists x, s.t.(x, y) \in \mathcal{R}\}$ , without having to reveal any additional information. It contains the following algorithms:

- **Gen**( $1^\lambda$ ). It takes as input a security parameter  $\lambda$ , and outputs a public parameter  $\text{par}$ ;
- **Sign**( $m, x, y$ ). It takes as input a message  $m$  and a pair  $(x, y) \in \mathcal{R}$ , and outputs an SoK  $\pi$ ;
- **Verf**( $m, \pi, y$ ). It takes as input a message  $m$ , an SoK  $\pi$  and a statement  $y$ , and outputs 0/1.

An SoK is SimExt-secure (Chase and Lysyanskaya 2006) if it satisfies correctness, simulatability, and extractability.

*Correctness* For any message  $m$  and a pair  $(x, y) \in \mathcal{R}$ , it holds if

$$\left| \Pr \left[ \text{Verf}(m, \pi, y) = 1 : \begin{array}{l} \text{par} \leftarrow \text{Gen}(1^\lambda); \\ \pi \leftarrow \text{Sign}(m, x, y) \end{array} \right] \right| \geq 1 - \text{negl}(\lambda).$$

*Simulatability* There exists a polynomial-time simulator **Sim** such that for any PPT adversary  $\mathcal{A}$ ,

$$\begin{aligned} & |\Pr[b = 1 : (\text{par}, td) \leftarrow \text{Gen}_{\text{Sim}}(1^\lambda); b \leftarrow \mathcal{A}^{\text{Sign}_{\text{Sim}}(\text{par})}] \\ & - \Pr[b = 1 : \text{par} \leftarrow \text{Gen}(1^\lambda); b \leftarrow \mathcal{A}^{\text{Sign}(\text{par})}]| \leq \text{negl}(\lambda), \end{aligned}$$

where  $td$  is an additional trapdoor in **Sim** to simulate the signatures without the witness.

*Extractability* There exists an extractor  $\text{Ext}$  such that for any PPT adversary  $\mathcal{A}$ ,

$$\Pr \left[ \begin{array}{l} (x, y) \in \mathcal{R} \vee (m, y) \in \mathcal{Q} \\ \vee \text{Verf}(m, y, \pi) = 0 : \\ (\text{par}, td) \leftarrow \text{Gen}_{\text{sim}}(1^\lambda); \\ (m, y, \pi) \leftarrow \mathcal{A}^{\text{Sim}}(\text{par}); \\ x \leftarrow \text{Ext}(\text{par}, td, m, y, \pi); \end{array} \right] \leq \text{negl}(\lambda),$$

where  $\mathcal{Q}$  is a list of queries to  $\text{Sign}$  Oracle of simulator that  $\mathcal{A}$  made.

Next, let us show how signatures of knowledge give rise to a simple group signature scheme. In a group signature scheme, we have group members, a group manager, and an anonymity revocation manager. Each member can sign on behalf of the group, and a signature reveals no information about who signed it, unless the anonymity revocation manager gets involved. The anonymity revocation manager can trace the signature to the group member who issued it; moreover it is impossible, even if the group manager and the revocation manager collude, to create a signature that will be traced to a group member who did not issue it.

Consider the following simple construction. The group's public key consists of  $(PK_S, PK_E, f)$ , where  $PK_S$  is a signature verification key for which the group manager knows the corresponding secret key;  $PK_E$  is an encryption public key for which the anonymity revocation manager knows the corresponding decryption key; and  $f$  is a one-way function. To become a group member, a user picks a secret  $x$ , gives  $f(x)$  to the group manager and obtains a group membership certificate  $g = \sigma_{PK_S}(f(x))$ . To issue a group signature, the user picks a random string  $R$ , encrypts his identity using randomness  $R : c = \text{Enc}(PK_E, f(x); R)$  and produces a signature of knowledge  $\pi = \text{Sign}(x, g, R)$  such that  $c$  is an encryption of  $f(x)$  using randomness  $R$ , and  $g$  is a signature on  $f(x)$ . The resulting group signature consists of  $(c, \pi)$ . To trace a group signature, the revocation manager decrypts  $c$ . It is not hard to see (only intuitively, since we haven't given any formal definitions) that this construction is a group signature scheme. Indeed, at a high level, this is how existing practical and provably secure group signatures work.

### 2.4.3 Commitment

A commitment scheme (Pedersen 1991) is a basic protocol in cryptography for binding information to a value without disclosing it until it is needed to be revealed. Concretely, a commitment scheme has the following three polynomial-time algorithms:

- **CGEN**( $1^\lambda$ ). This polynomial-time algorithm takes as input a security parameter  $\lambda$  and outputs a public commitment key  $ctk$ ;

- **Com**( $ctk, m, r$ ). This polynomial-time algorithm takes as input the commitment key  $ctk$ , a message to be committed  $m \in \{0, 1\}^*$ , and a randomness  $r$  and outputs a commitment  $c$ ;
- **Open**( $c, m, r$ ). This polynomial-time algorithm takes as input a commitment  $c$ , the message  $m$ , and the randomness  $r$  and checks whether  $c = \text{Com}(ctk, m, r)$ .

Commitment scheme is security if it has the following two specific properties:

*Hiding* Hiding requires that the commitment  $c$  reveals nothing about  $m$ . Specifically, for any PPT adversary  $\mathcal{A}$ , we have

$$\left| \Pr \left[ \begin{array}{l} ctk \leftarrow \text{CGen}(1^\lambda); \\ (m_0, m_1) \leftarrow \mathcal{A}(ctk); \\ b \leftarrow 0, 1; \\ c \leftarrow \text{Com}(ctk, m_b) \end{array} \right] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

*Binding* The prover can't modify  $m$  to produce same  $c$ .

Pedersen commitments were introduced by Pedersen in (1991). The scheme is built in the subgroup of  $\mathbb{Z}_p^*$  with prime order  $q$ , from which an element  $g$  is chosen as generator. Let  $h = g^a$ , so that  $h$  is the element of the same subgroup as  $g$ . The committer binds herself to the message  $m$  by computing  $c = g^m * h^r$ , where  $r$  is sampled randomly from  $\mathbb{Z}_q^*$ . Such commitment should be revealed later as the pair  $(m, r)$ . It is easy to see that the opening algorithm is efficient, knowing  $(g, h)$ . The paper (Pedersen 1991) proves formally that this commitment is perfectly hiding. We mention additionally that the same  $m$  can be used in more than one commitment without revealing it due to random input  $r$ , which leads to different  $c$ .

Pedersen's commitment has become very useful tool in many complex cryptographic protocols, in particular, in multi-party computations, where parties have to commit to the values they will share later, to guarantee that none of the parties forges its value adaptively, on the observation of an input of other parties. The commitment scheme can also be used in various interactive zero-knowledge proof protocols, where a Prover commits typically to a secret input, adding randomness too, but does not open according to the scheme, as a secret is not shared in a zero-knowledge protocol. The computational binding property of Pedersen commitment is used by such protocol for the soundness, opening simultaneously an efficient algorithm to the Verifier. The notable example of using Pedersen commitments in ZKP for MPC threshold signing is the work (Canetti et al. 2020), where the scheme is used in critical ZK range proofs. An experienced reader may notice that Pedersen commitments in Canetti et al. (2020) are built upon RSA group (a multiplicative group modulo product of two primes) instead of a group of prime order; therefore, they are given the name “Ring Pedersen” by authors. Admittedly, the change does not affect the binding property—the Prover is bound to  $(m, r)$ , this time by strong RSA assumption, unless the Prover knows the discrete logarithm of  $h$  to  $g$ .

### 2.4.4 *BulletProofs*

Bulletproofs (Bünz et al. 2018) are a relatively advanced NIZK proof technique in recent years; its biggest feature is that it provides native support for submitted values such as Pedersen commitments and public keys and can perform scope proofs in this general zero-knowledge proof framework without requiring installing large and complex elliptic curve arithmetic in it. The main building block of Bulletproofs is the inner product argument. Using Bulletproofs, the prover convinces a verifier that she knows vectors whose inner product is equal to a determined public value.

It contains the following main algorithms:

- **BP.Prove:** input a secret value  $v$ , output a Pedersen commitment  $Com$  and a proof  $\pi$ .
- **BP.Verify:** input the Pedersen commitment  $Com$  and the proof  $\pi$ , output 0/1.

Bulletproof protocols have multiple applications, most of which are discussed in Bulletproofs and Mimblewimble. The following list links these use cases to the different Bulletproof protocols:

- Bulletproofs provide short, non-interactive, zero-knowledge proofs without a trusted setup. The small size of Bulletproofs reduces overall cost. This has applicability in distributed systems where proofs are transmitted over a network or stored for a long time.
- Range Proof Protocol with Logarithmic Size provides short, single and aggregatable range proofs, and can be used with multiple blockchain protocols including Mimblewimble. These can be applied to normal transactions or to some smart contract, where committed values need to be proven to be in a specific range without revealing the values.
- The protocol presented in Aggregating Logarithmic Proofs can be used for Merkle proofs and proof of solvency without revealing any additional information.
- Range proofs can be compiled for a multi-party, single, joined confidential transaction for their known outputs using MPC Protocol for Bulletproofs. Users do not have to reveal their secret transaction values.
- Verifiable shuffles and multi-signatures with deterministic nonces can be implemented.
- Bulletproofs present an efficient and short zero-knowledge proof for arbitrary Arithmetic Circuits using Zero-knowledge Proof for Arithmetic Circuits.
- Various Bulletproof protocols can be applied to scriptless scripts, to make them non-interactive and not have to use Sigma protocols.
- Batch verifications can be done using Optimized Verifier using Multi-exponentiation and Batch Verification, e.g. a blockchain full node receiving a block of transactions needs to verify all transactions as well as range proofs.

## References

- Abbas H, Caprolu M, Pietro RD (2022) Analysis of polkadot: Architecture, internals, and contradictions. In: IEEE International Conference on Blockchain, Blockchain 2022, Espoo, Finland, August 22–25, 2022. IEEE, pp 61–70
- Adewumi TP, Liwicki F, Liwicki M (2022) Word2vec: Optimal hyperparameters and their impact on natural language processing downstream tasks. *Open Comput Sci* 12(1):134–141
- Alves V, Gazzaneo V, Lima FV (2022) A machine learning-based process operability framework using gaussian processes. *Comput Chem Eng* 163:107835
- Angelis SD, Aniello L, Baldoni R, Lombardi F, Margheri A, Sassone V (2018) PBFT vs proof-of-authority: Applying the CAP theorem to permissioned blockchain. In: Ferrari E, Baldi M, Baldoni R (eds) *Proceedings of the Second Italian Conference on Cyber Security*, Milan, Italy, February 6th - to - 9th, 2018, CEUR-WS.org, CEUR Workshop Proceedings, vol 2058
- Antonopoulos AM (2014) *Mastering Bitcoin: unlocking digital cryptocurrencies*. O'Reilly Media, Inc.
- Aponte-Novoa FA, Orozco ALS, Villanueva-Polanco R, Wightman PM (2021) The 51% attack on blockchains: A mining behavior study. *IEEE Access* 9:140549–140564
- Au MH, Tsang PP, Susilo W, Mu Y (2009) Dynamic universal accumulators for DDH groups and their application to attribute-based anonymous credential systems. In: Fischlin M (ed) *CT-RSA 2009*. Springer, Lecture Notes in Computer Science, vol 5473, pp 295–308
- Awerbuch B, Holmer D, Nita-Rotaru C, Rubens H (2002) An on-demand secure routing protocol resilient to byzantine failures. In: Maughan WD, Vaidya NH (eds) *Proceedings of the 2002 ACM Workshop on Wireless Security*, Atlanta, GA, USA, September 28, 2002. ACM, pp 21–30
- Baek U, Ji S, Park J, Lee M, Park J, Kim M (2019) Ddos attack detection on bitcoin ecosystem using deep-learning. In: 20th Asia-Pacific Network Operations and Management Symposium, APNOMS 2019, Matsue, Japan, September 18–20, 2019. IEEE, pp 1–4
- Barhanpure A, Belandor P, Das B (2018) Proof of stack consensus for blockchain networks. In: Thampi SM, Madria S, Wang G, Rawat DB, Calero JMA (eds) *Security in Computing and Communications - 6th International Symposium, SSCC 2018*, Bangalore, India, September 19–22, 2018, Revised Selected Papers. Springer, Communications in Computer and Information Science, vol 969, pp 104–116
- Beijing Microchip Research Institute et al. (2023) Introduction to the Chang'an Chain Underlying Technology Platform. <https://docs.chainmaker.org.cn/v2.3.6/html/quickstart/changan-chain-introduction.html>, Accessed on 2024-11-27
- Benaloh JC, de Mare M (1993) One-way accumulators: A decentralized alternative to digital sinatures (extended abstract). In: Helleseht T (ed) *Advances in Cryptology - EUROCRYPT '93*, Workshop on the Theory and Application of Cryptographic Techniques, Lofthus, Norway, May 23–27, 1993, Proceedings. Springer, Lecture Notes in Computer Science, vol 765, pp 274–285
- Bünz B, Bootle J, Boneh D, Poelstra A, Wuille P, Maxwell G (2018) Bulletproofs: Short proofs for confidential transactions and more. In: SP 2018. IEEE Computer Society, pp 315–334
- Camenisch J, Lysyanskaya A (2002) Dynamic accumulators and application to efficient revocation of anonymous credentials. In: Yung M (ed) *Advances in Cryptology - CRYPTO 2002*, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18–22, 2002, Proceedings. Springer, Lecture Notes in Computer Science, vol 2442, pp 61–76
- Canetti R, Gennaro R, Goldfeder S, Makriyannis N, Peled U (2020) UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In: Ligatti J, Ou X, Katz J, Vigna G (eds) *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security*, Virtual Event, USA, November 9–13, 2020. ACM, pp 1769–1787
- Chainalysis (2023) Monero: All about the top privacy coin. Tech. rep. <https://www.chainalysis.com/blog/all-about-monero/>
- Chainmaker (2024) Chainmaker. Tech. rep. <https://chainmaker.org.cn>

- Chase M, Lysyanskaya A (2006) On signatures of knowledge. In: Dwork C (ed) CRYPTO 2006. Springer, Lecture Notes in Computer Science, vol 4117, pp 78–96
- Cosmos (2023) Cosmos whitepaper 2023. <https://cosmos.network/whitepaper>
- de Barros CDT, Mendonça MRF, Vieira AB, Ziviani A (2023) A survey on embedding dynamic graphs. *ACM Comput Surv* 55(2):10:1–10:37
- Deuber D, Döttling N, Magri B, Malavolta G, Thyagarajan SAK (2020) Minting mechanism for proof of stake blockchains. In: Conti M, Zhou J, Casalicchio E, Spognardi A (eds) Applied Cryptography and Network Security - 18th International Conference, ACNS 2020, Rome, Italy, October 19–22, 2020, Proceedings, Part I. Springer, Lecture Notes in Computer Science, vol 12146, pp 315–334
- Essaid M, Kim D, Maeng SH, Park S, Ju H (2019) A collaborative ddos mitigation solution based on ethereum smart contract and RNN-LSTM. In: 20th Asia-Pacific Network Operations and Management Symposium, APNOMS 2019, Matsue, Japan, September 18–20, 2019. IEEE, pp 1–6
- Feng Q, He D, Zeadally S, Khan MK, Kumar N (2019) A survey on privacy protection in blockchain system. *J Netw Comput Appl* 126:45–58
- Financial Times (2021) El salvador's dangerous gamble on bitcoin. Tech. rep. <https://www.ft.com/content/c257a925-c864-4495-9149-d8956d786310>
- Gao H, Gao T (2020) Prevention of rumor spreading based on blockchain. In: 20th IEEE International Conference on Communication Technology, ICCT 2020, Nanning, China, October 28–31, 2020. IEEE, pp 1174–1178
- Ghaffar A, Sarwar MA, Abubaker Z, Gurmani MU, Sultana T, Shehzad F, Javaid N (2019) Smart contracts for research lab sharing scholars data rights management over the ethereum blockchain network. In: Barolli L, Hellinckx P, Natwichai J (eds) Advances on P2P, Parallel, Grid, Cloud and Internet Computing - Proceedings of the 14th International Conference on P2P, Parallel, Grid, Cloud and Internet Computing, 3PGCIC-2019, Antwerp, Belgium, November 7–9, 2019. Springer, Lecture Notes in Networks and Systems, vol 96, pp 70–81
- Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: Krishnapuram B, Shah M, Smola AJ, Aggarwal CC, Shen D, Rastogi R (eds) SIGKDD 2016. ACM, pp 855–864
- Guo Z, Guo S, Zhang S, Song L, Wang H (2020) Analysis of cross-chain technology of blockchain. *Chinese J Internet Things* 4(2):35
- Horizon (2018) Horizon: A scalable and secure blockchain platform. <https://horizon.io/whitepaper>, accessed on 2024-11-27
- Horizon (2023) Zendo - cross-chain platform. <https://www.horizon.io/academy/zendo/>
- Hu X, Zhu C, Tong Z, Gao W, Cheng G, Li R, Wu H, Gong J (2022) Identifying ethereum traffic based on an active node library and devp2p features. *Future Gener Comput Syst* 132:162–177
- Idé T (2018) Collaborative anomaly detection on blockchain from noisy sensor data. In: Tong H, Li ZJ, Zhu F, Yu J (eds) 2018 IEEE International Conference on Data Mining Workshops, ICDM Workshops, Singapore, Singapore, November 17–20, 2018. IEEE, pp 120–127
- Janet L. Yellen (2022) Remarks by Secretary of the Treasury Janet L. Yellen on Way Forward for the Global Economy. Tech. rep. <https://home.treasury.gov/news/press-releases/jy0714>
- Lan R, Upadhyaya G, Tse S, Zamani M (2021) Horizon: A gas-efficient, trustless bridge for cross-chain transactions. *CoRR abs/2101.06000*
- Li C, Wang B, Pavlu V, Aslam JA (2016) An empirical study of skip-gram features and regularization for learning on sentiment analysis. In: Ferro N, Crestani F, Moens M, Mothe J, Silvestri F, Nunzio GMD, Hauff C, Silvello G (eds) ECIR 2016. Springer, Lecture Notes in Computer Science, vol 9626, pp 72–87
- Liu D, Camp LJ (2006) Proof of work can work. In: 5th Annual Workshop on the Economics of Information Security, WEIS 2006, Robinson College, University of Cambridge, England, UK, June 26–28, 2006
- Mesgaran M, Hamza AB (2024) Graph fairing convolutional networks for anomaly detection. *Pattern Recognit* 145:109960

- Mirkin M, Ji Y, Pang J, Klages-Mundt A, Eyal I, Juels A (2020) Bdos: Blockchain denial-of-service. In: Ligatti J, Ou X, Katz J, Vigna G (eds) CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9–13, 2020. ACM, pp 601–619
- Noether S, Mackenzie A, Lab T (2016) Ring confidential transactions. *Ledger* 1:1–18. <https://doi.org/10.5195/LEDGER.2016.34>
- Pedersen TP (1991) Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum J (ed) CRYPTO 1991. Springer, Lecture Notes in Computer Science, vol 576, pp 129–140
- Qi S, Liu L, Kumar BS, Prathik A (2022) An english teaching quality evaluation model based on gaussian process machine learning. *Expert Syst J Knowl Eng* 39(6)
- Rodrigues B, Stiller B (2019) Cooperative signaling of ddos attacks in a blockchain-based network. In: Proceedings of the ACM SIGCOMM 2019 Conference Posters and Demos, SIGCOMM 2019, Beijing, China, August 19–23, 2019. ACM, pp 39–41
- Rüth J, Zimmermann T, Wolsing K, Hohlfeld O (2018) Digging into browser-based crypto mining. In: Proceedings of the Internet Measurement Conference 2018, IMC 2018, Boston, MA, USA, October 31–November 02, 2018. ACM, pp 70–76
- Saad M, Thai MT, Mohaisen A (2018) POSTER: deterring ddos attacks on blockchain-based cryptocurrencies through mempool optimization. In: Kim J, Ahn G, Kim S, Kim Y, López J, Kim T (eds) Proceedings of the 2018 on Asia Conference on Computer and Communications Security, AsiaCCS 2018, Incheon, Republic of Korea, June 04–08, 2018. ACM, pp 809–811
- Sompolinsky Y, Lewenberg Y, Zohar A (2016) Zendo: Divide and conquer bitcoin. In: 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16). USENIX Association, Santa Clara, CA
- Sun Yin HH, Langenheldt K, Harlev M, Mukkamala RR, Vatrpu R (2019) Regulating cryptocurrencies: a supervised machine learning approach to de-anonymizing the bitcoin blockchain. *J Manag Inf Syst* 36(1):37–73
- Thankful for Today (2014) Monero (Bitmonero) - Cryptonote based cryptocurrency. <https://bitcointalk.org/index.php?topic=625878.0>, Accessed on 2024-11-27
- The Economist (2015) The great chain of being sure about things. Tech. rep., <https://www.economist.com/briefing/2015/10/31/the-great-chain-of-being-sure-about-things>
- The New York Times (2021) Bitcoin uses more electricity than many countries. How is that possible? Tech. rep. <https://www.nytimes.com/interactive/2021/09/03/climate/bitcoin-carbon-footprint-electricity.html>
- Tian H, Li Y, Cai Y, Shi X, Zheng Z (2021) Attention-based graph neural network for identifying illicit bitcoin addresses. In: Dai H, Liu X, Luo DX, Xiao J, Chen X (eds) BlockSys 2021. Springer, Communications in Computer and Information Science, vol 1490, pp 147–162
- Vitalik Buterin (2014) Ethereum Whitepaper. <https://ethereum.org/en/whitepaper/>, Accessed on 2024-11-27
- Wolff-Mann E (2018) ‘only good for drug dealers’: More nobel prize winners snub bitcoin. Tech. rep. <https://finance.yahoo.com/news/good-drug-dealers-nobel-prize-winners-snob-bitcoin-184903784.html>
- Wood G (2016) Polkadot: Vision for a heterogeneous multi-chain framework. Website. <https://polkadot.network/PolkaDotPaper.pdf>
- Wu Y, Chen Y, Yin Z, Ding W, King I (2023) A survey on graph embedding techniques for biomedical data: Methods and applications. *Inf Fusion* 100:101909
- Yorker TN (2014) The crypto-currency: Bitcoin and its mysterious inventor. Tech. rep. <https://www.newyorker.com/magazine/2011/10/10/the-crypto-currency>
- Zhang B, Li J, Chen C, Lee K, Lee I (2021) A practical botnet traffic detection system using GNN. In: Meng W, Conti M (eds) CSS 2021. Springer, Lecture Notes in Computer Science, vol 13172, pp 66–78
- Zhang Z, Qi P, Wang W (2020) Dynamic malware analysis with feature engineering and feature learning. In: AAAI 2020. AAAI Press, pp 1210–1217

# Chapter 3

## Cross-Chain Abnormal Account Detection



### 3.1 Cross-Chain Account Detection via Machine Learning

#### 3.1.1 Background and Motivation

The lack of regulation and the ease with which funds can be transferred across networks raise the cross-chain security issues. Since 2020, criminals have laundered at least \$4 billion worth of illicit crypto proceeds using cross-chain technologies (Elliptic 2022). In fact, above security issues, even criminals can be identified from abnormal account activities. Abnormal account activity refers to transactions that deviate from expected patterns or suspicious characteristics, potentially indicating fraudulent or malicious behavior. To identify abnormal transactions, there are mainly two types of solutions, i.e., machine learning algorithms (Chen et al. 2021a; Hu et al. 2023; Wu et al. 2022) and graph-based approaches (Chen et al. 2020; Yu et al. 2021). Machine learning algorithms use statistical techniques to identify patterns and relationships within datasets, allowing the system to generalize and make predictions on unseen data. Graph-based approaches leverage graph structures to represent and analyze relationships and connections between entities. Previous research in abnormal account detection has predominantly focused on applying these techniques to single-chain networks (Li et al. 2022a; Zhang et al. 2020), such as the Bitcoin blockchain and Ethereum. However, applying these techniques directly to cross-chain transactions poses unique challenges, which can be described as follows:

- The decentralized and distributed nature of cross-chain transactions hinders the establishment of a reliable baseline for normal transaction behavior.
- Multiple currencies in cross-chain transactions have nonuniform values.
- Different chains may have the same address, which may cause some confusion for abnormal account detection. Additionally, the heterogeneity of blockchain



networks and the lack of standardized protocols further complicate the detection process.

To address aforementioned problems, Zhang et al. (2020) considered anomaly detection particular to the Bitcoin transaction network. They set time constraints to process Bitcoin data and constructed static networks and dynamic networks respectively according to time features. They used the  $k$ -step neighborhood method for specific users or transactions to screen out anomalous candidate sets and reference sets. Because Ethereum supports smart contracts and has better scalability, some works (Chen et al. 2020; Kim et al. 2023; Wen et al. 2021) focus on Ethereum, which is under the account-based model, and supervised learning was introduced to detect fraudulent accounts. Chen et al. (2020) can effectively identify Ethereum phishing scams by proposing a graph-based cascade feature extraction method based on transaction records and a lightGBM-based dual-sampling Ensemble algorithm. To further expand the feature set, Wen et al. (2021) proposed a phishing scam accounts detecting method using totally 25 features. The extensive experimental results show that the selected features are effective. To further improve the detection effectiveness of the multi-edge blockchain transaction, Kim et al. (2023) proposed a novel heterogeneous graph structure called an account-transaction graph, ATGraph, which allows graph learning more efficiently by eliminating multi-edges. The previous works aforementioned only support the transaction detection in the single-chain environment. This chapter focuses on cross-chain transactions, and the proposed CrossAAD extracts cross-chain transaction-related features and performs abnormal account detection with a mix of machine learning models.

To address the transaction traceability problem in the cross-chain system, Hinteregger and Haslhofer (2019) considered Monero and launched a (passive) traceability analysis on data from Monero, MoneroV, and Monero Original blockchains, where only a small amount of inputs are traceable. However, this method cannot support privacy protection. Li et al. (2022b) proposed ZeroCross, a novel privacy-preserving sidechain-based scheme that guarantees transaction unlinkability, exchanging fairness, and value confidentiality. To allow identity transformation with supervision, Ding et al. (2023) proposed a scalable cross-chain access control and identity authentication scheme, which ensures that all cross-chain operations are carried out by verified users. To protect the security across on-chain smart contracts and off-chain programs, Zhang et al. (2022) proposed an automatic tool named Xscope, to find security violations in cross-chain bridges. In this chapter, we focus on leveraging machine learning and feature engineering to achieve efficient abnormal account detection for cross-chain transactions.

### 3.1.2 CrossAAD Overview

Motivated by these challenges, we have to address the diversities and complexities of abnormal account detection in cross-chain transactions. To fit the cross-chain

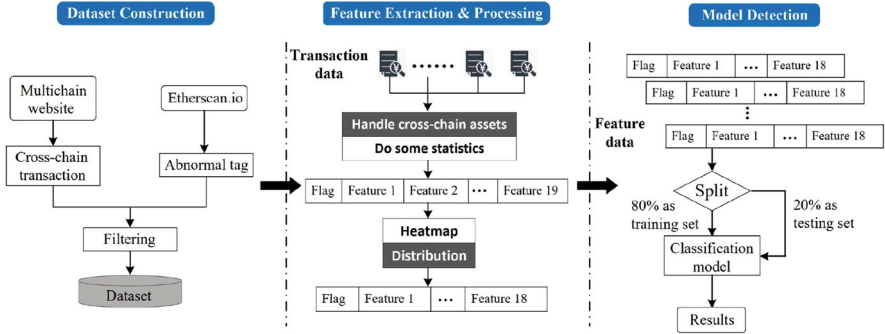


Fig. 3.1 Overview of cross-chain abnormal account detection

structure, we adopt machine learning to achieve a better cross-chain abnormal account detection performance. We present CrossAAD, an abnormal account detection method for cross-chain transactions. It integrates manual feature engineering, transaction records analysis, and classification models. An obstacle is that the raw data are not compatible with the machine learning algorithms leading to low predictive value. Our used feature engineering method allows relevant information input to machine learning such as to improve the predictive performance. Also, few labels and data imbalance in the cross-chain transactions increase the difficulty of detection. CrossAAD narrows the transaction scope to account-based model with a combination of SMOTE. We also delve into feature engineering to obtain better features of the training data. To extract more cross-chain transaction-related features, different types of cross-chain currencies are converted to account-type currency.

Figure 3.1 shows an overview of cross-chain abnormal account detection based on transaction information. It is divided into three phases: dataset construction, feature extraction and processing, and model detection. For the first phase, we use crawler techniques to obtain a large amount of cross-chain transaction data from the multi-chain website. Then we make a pre-filtering of the dataset and tag the dataset based on the anomaly address reports on etherscan website. During the feature extraction and processing phase, multiple statistical features are extracted based on the dataset from the previous phase. The extracted features are then further processed, such as the unification of assets across chains. Then, we use the features and labels of cross-chain accounts to train a classification model.

The overall process of model detection includes two parts: training and testing phases. A training set of normal and abnormal accounts is provided to the classifier model. The features of the accounts and their labels are the input for the training model. In the testing phase, a collection of new account data that did not appear in the training phase will be classified by the training model. The performance of the generated training model is evaluated by metrics (Rajkomar et al. 2018), including precision, recall, F1-score, and AUC score, which have been discussed in Sect. 3.1.4.

3.1.3 Detailed Design of CrossAAD

3.1.3.1 Dataset Construction

**Source of Dataset** The cross-chain transaction dataset used in this article was crawled from the multi-chain website.<sup>1</sup> Multi-chain, formerly known as Anyswap, specializes in cross-chain bridges or multi-chain asset exchanges. Due to its many integrations and supported blockchains, multi-chain has become a major bridging solution. As of June 1, 2023, multi-chain has been able to support 93 different chains and 3352 assets across chains. It also has reached 947,900 users and has a transaction volume of 104.4B.

The dataset spans from 2020-08-17 to 2023-04-28 and contains over 5.5-million transactions with 11 attributes per transaction, as shown in Table 3.1.

**Acquisition of Dataset** We obtain data by crawling technique, whose structure is shown in Fig. 3.2:

- Seed URL (Uniform Resource Locator): The entry address of the crawler.
- URL to be crawled: The seed URL and the URL of the crawled web page that have been de-duplicated.
- Crawling web page: Set the retry mechanism to 20 to prevent the data download from failing due to network fluctuations and other problems, and improve the efficiency of web page crawling.
- Parse the data: Get the web page content of the URL. Parse the cross-chain transaction data of interest to this chapter from the web page content.
- Save the data: Sort through the parsed data, convert it to csv in the specified format, and export it.

Table 3.1 Transaction details

| Attribute           | Value                                           |
|---------------------|-------------------------------------------------|
| Source chain hash   | 9fb2e68d4aaf8cb766b6a516b50269a63850b6c01.....  |
| Target chain hash   | 0x96a8b26e96d69aa6f65cf0382ea24c48f88bf633..... |
| Source chain        | Bitcoin mainnet                                 |
| Target chain        | Ethereum mainnet                                |
| From                | 3DSVaLoJBEXg8h4byEVcKJX6qN8YajyFKD              |
| To                  | 0x79F49449A106D290E697C3492694b80171052a8c      |
| Date                | 2023/03/23 09:13                                |
| CoinType            | BTC                                             |
| Send the number     | 0.125263                                        |
| Number of receiving | 0.125263                                        |
| Status              | Success                                         |

<sup>1</sup> <https://scan.multichain.org>

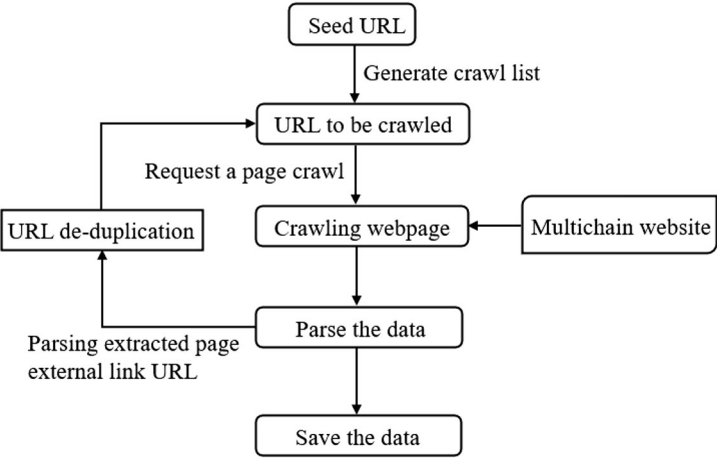


Fig. 3.2 Crawling structure

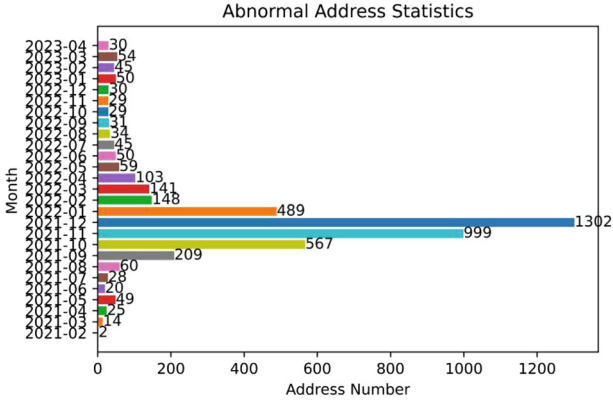


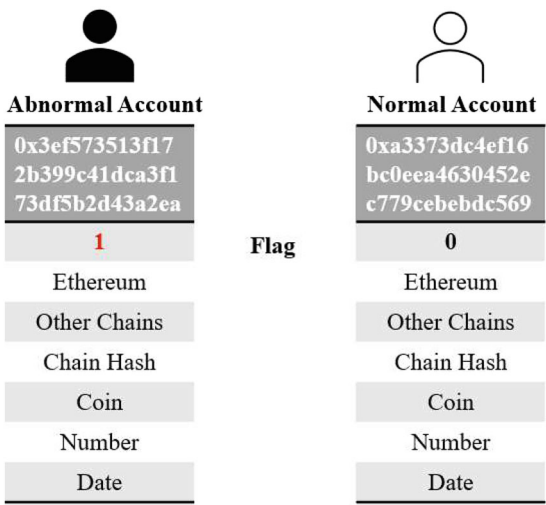
Fig. 3.3 Statistics of abnormal address per month

**Filtering of Cross-Chain Transactions** After preliminary analysis of the crawled data, it is found that there are some transactions with empty attributes. Then we remove these data from the original dataset. The cross-chain transactions related to Ethereum are filtered out, totaling 1,310,888 transactions, involving 370,000 addresses. According to Etherscan.io browser reports, multi-chain’s abnormal addresses include 14 exploit addresses and 2789 addresses with risks.

By comparing with the reported anomalous addresses, it is found that there are 2413 abnormal addresses in the crawled data. To figure out the distribution of the abnormal addresses, we count the number of abnormal addresses existing in each time period, and the results are shown in Fig. 3.3:

By analyzing the aforementioned graph and referring to the time of cross-chain bridge attack events, the four months of data from 2021-10 to 2022-01 are finally

**Fig. 3.4** Abnormal account vs. normal account



selected as the final dataset, which is defined as *CC Dataset*. These four months of data contain 2322 multi-chain addresses that may be at risk and a total of 425,889 cross-chain transactions. From them, 6000 normal addresses are randomly selected to form the benign account sample set, and 2322 abnormal addresses are used to form the abnormal account sample set. Abnormal accounts will be tagged as 1, and normal accounts will be tagged as 0. We can see it in Fig. 3.4. Then we get a list of accounts and their tags.  $Accountlist = \{v_1, v_2, \dots, v_n\}$ ,  $n$  is the number of accounts, and  $v_i (i = 1, 2, \dots, n)$  represents account.  $Tags = \{f_1, f_2, \dots, f_n\}$ ,  $f_i = 0$  or  $1$  means normal or abnormal. Abnormal account detection in cross-chain transactions may be a more complex task. This is because the amount of transaction data with anomaly labels is too small compared to normal transactions, and there is a data imbalance problem. Directly using machine learning methods to classify the imbalanced data will be less effective. Therefore, downsampling, oversampling, and synthetic sampling algorithms are considered to alleviate the data imbalance problem and thus improve the learning effect.

**3.1.3.2 Feature Extraction and Processing**

Based on tagging accounts during the dataset construction phase, we intend to extract and process multiple features by doing statistics on account-related trading data.

**Feature Extraction** In order to describe the behavior of different accounts and achieve a better classification effect, 19 features are extracted based on the cross-chain transaction dataset. All extracted features are shown in Table 3.2.

**Table 3.2** Detailed description of 19 extracted features

| Feature name | Feature description                        |
|--------------|--------------------------------------------|
| IT           | Amount of incoming transactions            |
| OT           | Amount of outgoing transactions            |
| UIT          | Amount of unique incoming transactions     |
| UOT          | Amount of unique outgoing transactions     |
| TAS          | Total amount sent                          |
| TAR          | Total amount received                      |
| AVR          | Average value received                     |
| AVS          | Average value sent                         |
| MIVSF        | Minimum value of the sending funds         |
| MIVRF        | Minimum value of the receiving funds       |
| MAVSF        | Maximum value of the sending funds         |
| MAVRF        | Maximum value of the receiving funds       |
| ATIT         | Average time between incoming transactions |
| ATOT         | Average time between outgoing transactions |
| MAT          | Maximum time between transactions          |
| MIT          | Minimum time between transactions          |
| DUR          | First and last transaction time interval   |
| BIO          | IT-OT                                      |
| NCC          | Number of cross-chains                     |

All of the aforementioned transaction attribute characteristics can be calculated by the method of data statistics. For the  $Accountlist = \{v_1, v_2, \dots, v_n\}$  and  $Tags = \{f_1, f_2, \dots, f_n\}$  obtained during the data collection phase, we will extract the aforementioned features for each account. According to all the properties of cross-chain transactions, the specific method is shown as follows:

- (a) Iterate through each  $v_i \in Accountlist, i = 1, 2, \dots, n$ .
- (b) Count the amount of incoming transactions of  $v_i$ , define it as  $IT_i$ .
- (c) Count the amount of outgoing transactions of  $v_i$ , define it as  $OT_i$ .
- (d) Count the unique amount of incoming transactions of  $v_i$ , define it as  $UIT_i$ .
- (e) Count the unique amount of outgoing transactions of  $v_i$ , define it as  $UOT_i$ .
- (f) Count the total number of sent assets of  $v_i$ , define it as  $TAS_i$ .
- (g) Count the total number of received assets of  $v_i$ , define it as  $TAR_i$ .
- (h) Define  $AVR_i = TAR_i/IT_i$ ,  $AVS_i = TAS_i/OT_i$ .
- (i) Count the minimum value of the sending funds of  $v_i$ , define it as  $MIVSF_i$ .
- (j) Count the minimum value of the receiving funds of  $v_i$ , define it as  $MIVRF_i$ .
- (k) Count the maximum value of the sending funds of  $v_i$ , define it as  $MAVSF_i$ .
- (l) Count the maximum value of the receiving funds of  $v_i$ , define it as  $MAVRF_i$ .
- (m) Calculate average time between incoming and outgoing transactions of  $v_i$ , define it as  $ATIT_i$  and  $ATOT_i$ .
- (n) Count the maximum time between transactions of  $v_i$ , define it as  $MAT_i$ .
- (o) Count the minimum time between transactions of  $v_i$ , define it as  $MIT_i$ .

**Table 3.3** Feature data example of each cross-chain account

| Address       | Flag | IO | OT | UIT | ... | DUR      | BIO | NCC |
|---------------|------|----|----|-----|-----|----------|-----|-----|
| 0xe7da19..... | 0    | 1  | 3  | 1   | ... | 127029.3 | - 2 | 1   |
| ⋮             | ⋮    | ⋮  | ⋮  | ⋮   | ⋮   | ⋮        | ⋮   | ⋮   |
| 0x3ef573..... | 1    | 0  | 2  | 0   | ... | 73.95    | - 2 | 1   |

**Table 3.4** Top-6 assets in cross-chain transaction dataset

| Asset       | ETH  | USDC | MIM  | FANTOM | USDT | WETH |
|-------------|------|------|------|--------|------|------|
| Rank        | 1    | 2    | 3    | 4      | 5    | 6    |
| Total value | 6606 | 6393 | 4546 | 3760   | 3072 | 2396 |

- (p) Calculate the interval time of  $v_i$  from the first transaction to the last transaction, define it as  $DUR_i$ .
- (q) Define  $BIO_i = IT_i - OT_i$ .
- (r) Count the number of different chains traded of  $v_i$ , define it as  $NCC_i$ .

We get the feature data of each account, as Table 3.3.

**Feature Processing** According to the statistics, assets of cross-chain transactions in *CCDataset* are up to 287 types. Top-6 assets are shown in Table 3.4. The value of different assets varies. Therefore, assets of different kinds of cross-chain transactions are transformed into a uniform value before amount-dependent features are extracted. In this chapter, they are all converted to Ether.

The presence of multicollinearity between variables affects the accuracy of the model. In this chapter, we use a heat map to visualize the correlation between variables, and then determine whether the independent variables should be removed, or the operation of dimensionality reduction should be performed. There are too many fields in the correlation matrix, and it is not easy to find the fields with strong correlation. So a heat map should be used to take a very visual look at correlations, as shown in Fig. 3.5.

The scale on the right side of the heat map shows the color shades corresponding to the different correlation coefficients. Figure 3.5 shows that the correlation between *Max\_val\_received* and *Total\_received\_number* is high at 1.0, i.e., there is strong multicollinearity. Consideration can be given to eliminating one of the two variables when performing feature engineering (Chicco et al. 2022) to avoid causing overfitting due to multicollinearity. After dropping *Total\_received\_number* feature, the processed correlation matrix is shown in Fig. 3.6.

Then we investigate the distribution of our features by using boxplots, which is shown in Fig. 3.7. It can be observed that the case where most of the values of the variables are 0 does not occur. Thus, we don't have to discard any feature. In the training process, we found that our model was easier to train on a balanced dataset. We divided the data into 80% training and 20% testing. Also, we used SMOTE (Chawla et al. 2002) method to oversample the training set to an equal number to

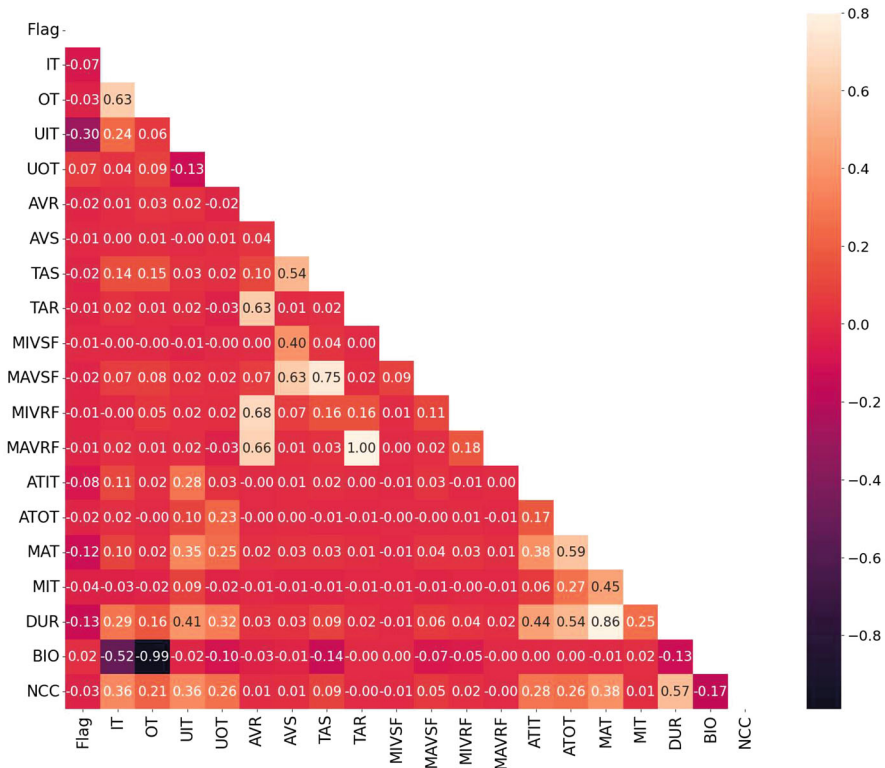


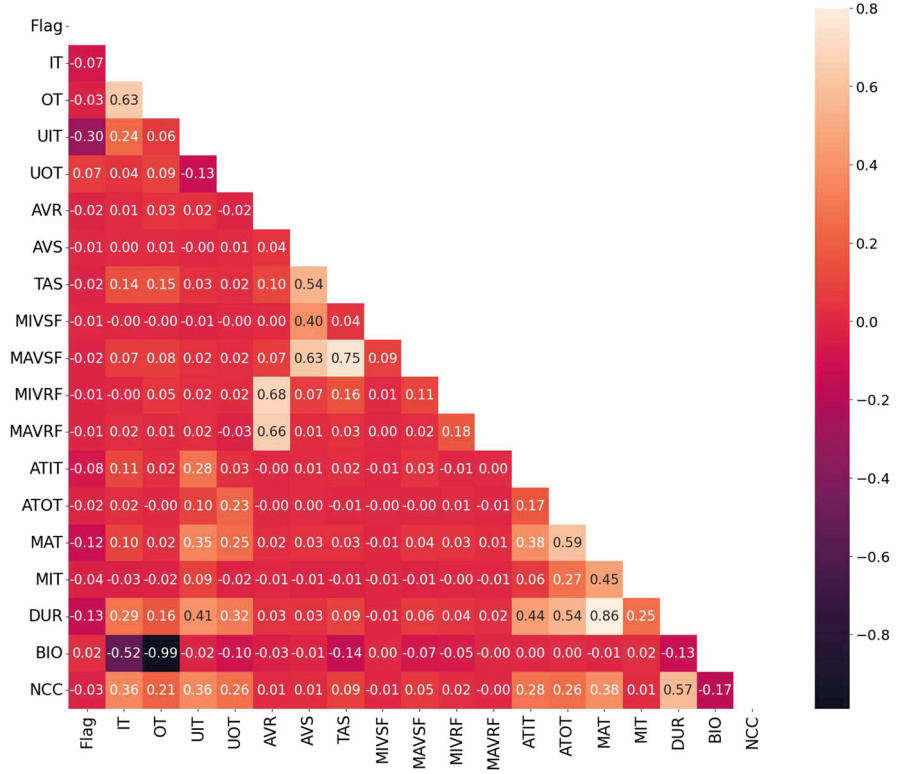
Fig. 3.5 Heatmap of 19 extracted features

obtain a balanced training dataset. The number of abnormal samples was expanded from 1869 to 4789. SMOTE is an oversampling technique, which focuses on the feature space to generate new instances with the help of interpolation between the positive instances that lie together.

3.1.3.3 Model Detection

**Support Vector Machines (SVM)** SVM (Lu et al. 2023) is a powerful and widely used supervised machine learning algorithm. It is a binary classification algorithm that aims to find an optimal hyperplane in a high-dimensional feature space to separate data points into different classes. In the context of abnormal account detection, SVM can learn patterns and features from labeled training data to distinguish between normal and abnormal account behavior. SVM aims to find the hyperplane that maximizes the margin, which is the distance between the hyperplane and the closest data points of each class. By maximizing the margin, SVM is more robust to noise and can generalize well to unseen data. This property





**Fig. 3.6** New heatmap after dropping the feature with strong multicollinearity

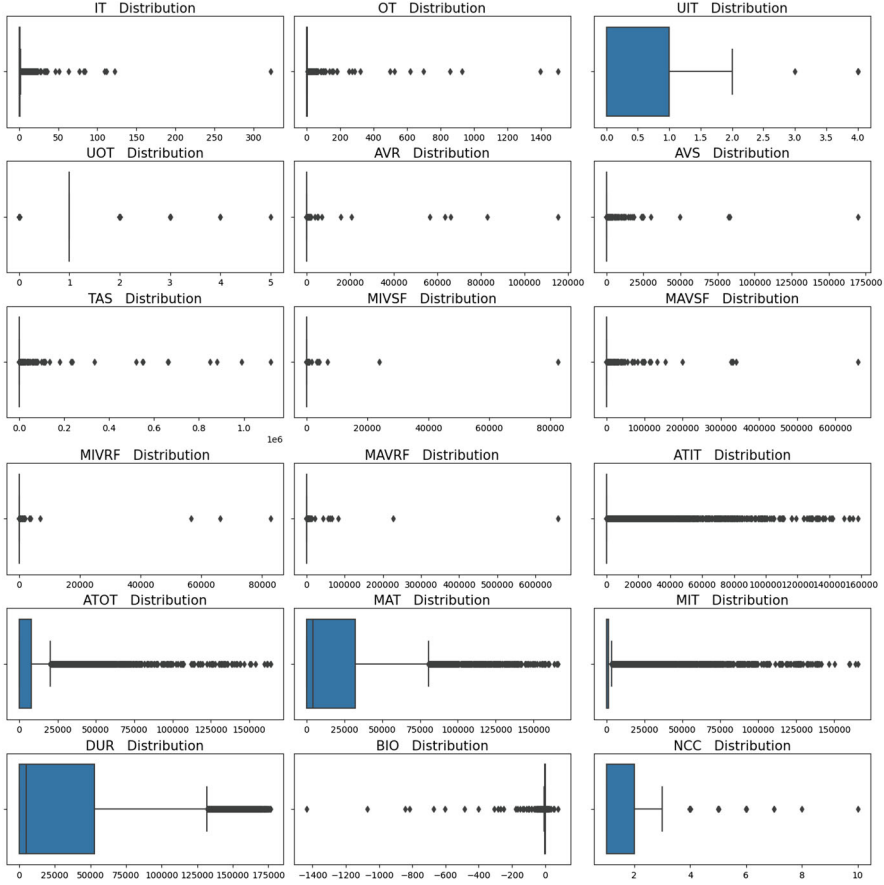
helps in identifying abnormal accounts that exhibit subtle deviations from normal behavior. Given a training dataset  $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ,  $m$  is the number of training samples,  $x_i (i = 1, 2, \dots, m)$  is the input feature, and  $y_i$  is the corresponding label. The objective function and constraints of SVM are shown as follows:

$$\min_{\omega, b} \frac{1}{2} \|\omega\|^2,$$

$$s.t. \quad y_i(\omega \cdot x_i - b) \geq 1.$$

The classification problem of the SVM is then converted into a problem of solving for the two parameters  $\omega$  and  $b$ . Here  $\omega = [\omega_1, \omega_2, \dots, \omega_m]^T$ . And  $\|\omega\|$  denotes the two-paradigm number of  $w$ . This is computed by summing the squares of all the elements and then squaring the result.

**Logistic Regression (LR)** LR (Selvi et al. 2022) is a widely used and interpretable statistical learning algorithm that is commonly employed for binary classification



**Fig. 3.7** Distribution of features by using boxplots

tasks, including abnormal account detection in blockchain transactions. It models the relationship between the input features and the probability of an account being classified as normal or abnormal. The Logistic Regression model uses the logistic function (also known as the sigmoid function) to map the linear combination of input features to a probability value between 0 and 1. The classification is then based on this probability value. The mathematical expression for the logistic function is given as follows:

$$P(Y = 1 | x) = \frac{1}{1 + e^{-(\omega^T x + b)}}.$$

Given a training dataset  $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ,  $m$  is the number of training samples,  $x_i$  ( $i = 1, 2, \dots, m$ ) is the input feature and  $y_i$  is the corresponding label. Let  $P(Y = 1 | x_i) = h(x_i)$ . The logistic function value

for each sample  $x_i$  is  $h(x_i)$ . The training objective of logistic regression is to find the optimal weighting parameter  $\omega$  such that the predictions of the model are as consistent as possible with the actual observations. This is usually achieved using a minimised log-loss function. Solving for the parameters in the model requires a loss function, which is defined as follows:

$$J = -\frac{1}{m} \sum_{i=1}^m [y_i \log h(x_i) + (1 - y_i) \log(1 - h(x_i))].$$

The parameters of the model are learned by minimizing the loss function  $J$  through optimization algorithms such as gradient descent. Once training is complete, the learned parameters can be used to determine decision boundaries, i.e., boundaries that separate two categories in the feature space.

**Random Forest (RF)** RF (Purwanto et al. 2023) is a versatile and powerful ensemble learning algorithm commonly used for classification and regression tasks, including abnormal account detection in blockchain transactions. It combines the predictions of multiple decision trees to make accurate and robust predictions. The Random Forest algorithm operates by constructing a multitude of decision trees during the training phase. Each decision tree is built using a randomly selected subset of the training data and a subset of the input features. The input is the training set  $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ,  $m$  is the number of training samples,  $x_i$  ( $i = 1, 2, \dots, m$ ) is the input feature, and  $y_i$  is the corresponding label.  $K$  is the number of weak classifier iterations. For  $j = 1, 2, \dots, K$ , the training set is sampled for the  $j$ th time, and a total of  $p$  times are collected to obtain a sampling set  $Dt$  containing  $p$  samples. Train the  $j$ th decision tree model  $Gt(x)$  with the sampling set  $Dt$ . The category or one of the categories with the most votes from the  $K$  weak learners is the final category. In summary, the Random Forest model is a powerful ensemble learning algorithm that combines the predictions of multiple decision trees to perform abnormal account detection in blockchain transactions.

**Extreme Gradient Boosting (XGBoost)** XGBoost (Extreme Gradient Boosting) (Chen and Guestrin 2016) is a powerful and widely adopted gradient boosting framework that excels in various machine learning tasks, including abnormal account detection in blockchain transactions. It is known for its scalability, speed, and ability to handle diverse data types and complex relationships. XGBoost operates by sequentially adding weak learners (decision trees) to an ensemble model, where each subsequent tree corrects the errors made by the previous ones. Given a training dataset  $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ,  $m$  is the number of training samples,  $x_i$  ( $i = 1, 2, \dots, m$ ) is the input feature, and  $y_i$  is the corresponding label. The final predicted value for the  $i$ th sample is  $\hat{y}_i$ . Here  $\hat{y}_i = \sum_{k=1}^K f_k(x_i)$ ,  $f_k(x)$  is the function of the  $k$ th tree,  $K$  is the final number of rounds. The new decision tree is built by means of an objective function, which is constructed as follows:

$$Obj = \sum_{i=1}^m l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k).$$

$l$  is the loss function, and  $\Omega(f_k)$  is a regularity term to control the complexity of the model and prevent overfitting. This iterative process optimizes an objective function, aiming to minimize the overall prediction error. The XGBoost model combines the predictions of multiple decision trees to make accurate and robust predictions. In summary, the XGBoost model is a powerful gradient boosting framework that excels in abnormal account detection tasks.

3.1.4 Performance Evaluation and Analysis

Abnormal account detection can be viewed as a binary problem with only two categories, normal and abnormal. Then mixing all the results of the predicted situation with the actual situation in pairs, brings about the following four cases, which form the confusion matrix in Table 3.5. Among them, 1 represents abnormal, and 0 represents normal.

Since 1 and 0 are numbers and are not well readable, we denote the two outcomes 1 and 0 by P and N, respectively. P denotes the abnormal prediction, and N denotes the normal prediction. True (T) and false (F) then denote the predicted outcomes, with T indicating a correct prediction where the predicted value matches the actual value and, conversely, F indicating an incorrect prediction. By reassigning the matrix according to the character representation aforementioned, the confusion matrix is shown in Table 3.6.

3.1.4.1 Experimental Setup

We use the sklearn library for the model construction in Sect. 3.2.3.5. The overall prototype includes around 800 lines of Python language codes, including all of the test programs. For all our experiments, we use 80% of the dataset (i.e., both normal and abnormal) to train the machine learning algorithms, and the remaining of the 20% is used to test the performance. To solve the problem of data imbalance, we use SMOTE as the oversampling solution.

Table 3.5 Confusion matrix with 0 and 1

|                 |   | Actual result |    |
|-----------------|---|---------------|----|
|                 |   | 1             | 0  |
| Predict Outcome | 1 | 11            | 10 |
|                 | 0 | 01            | 00 |

Table 3.6 Confusion matrix represented by meaningful letters

|                 |   | Actual result |    |
|-----------------|---|---------------|----|
|                 |   | 1             | 0  |
| Predict Outcome | 1 | TP            | FP |
|                 | 0 | FN            | TN |

**Datasets** This work is performed in Sect. 3.1.3.1. The final dataset includes a total of 425,889 cross-chain transactions. There are 6000 normal addresses and 2322 abnormal addresses. Abnormal addresses will be tagged as 1, and normal addresses will be tagged as 0. For each address, 18 features are extracted.

### 3.1.4.2 Evaluation Metrics

In this chapter, precision rate, recall rate, F1 value, and AUC score are used as evaluation indicators for abnormal transaction monitoring. According to the Tag list:  $Tags = \{f_1, f_2, \dots, f_n\}$ ,  $f_i = 0$  or  $1$ , and the corresponding prediction result list:  $Predicting\ outcome = \{y_1, y_2, \dots, y_n\}$ , we then calculate the precision rate, recall rate, and F1 value. For  $i = 1, 2, \dots, n$ ,

1. If  $f_i == y_i == 1$ , then count plus one.  $TP$  is the final accumulated value.
2. If  $f_i == 0, y_i == 1$ , then count plus one.  $FP$  is the final accumulated value.
3. If  $f_i == 1, y_i == 0$ , then count plus one.  $FN$  is the final accumulated value.

Among them, the precision rate is for the prediction results, and its meaning is the probability of the actual positive samples among all the samples that are predicted to be positive. Its formula is as follows:

$$Precision = \frac{TP}{TP + FP}.$$

The recall is also known as the check-all rate, which is for the original sample, and it means the probability of being predicted as a positive sample out of an actual positive sample. Its formula is as follows:

$$Recall = \frac{TP}{TP + FN}.$$

F1 value is the summed average of precision rate and recall rate, taking into account both precision rate and recall rate, allowing both to reach the highest at the same time and taking a balance. Its formula is as follows:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

### 3.1.4.3 Classification Performance

We run the experiments on a Colab virtual machine, and the specific configuration can be seen in Table 3.7.

**Comparison of Experimental Results of Various Models** The processed feature data are used as model inputs to compare the classification performance of

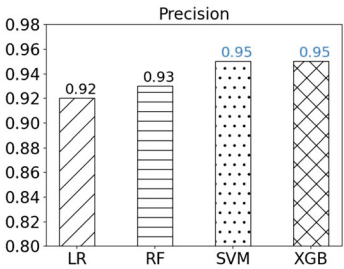
**Table 3.7** Software and hardware configurations of experiments

| Configuration      | Specific parameter information |
|--------------------|--------------------------------|
| CPU                | 2 Intel(R) Xeon(R) 2.2GHz      |
| RAM                | 12.68 GB                       |
| Disk Space         | 107.72 GB                      |
| Operating system   | Ubuntu 20.04.5 LTS             |
| Learning framework | Sklearn                        |

**Table 3.8** Detection performance of abnormal accounts

| Method                   | Precision | Recall | F1-score |
|--------------------------|-----------|--------|----------|
| Logistic regression      | 0.65      | 0.79   | 0.71     |
| Random forest classifier | 0.70      | 0.83   | 0.76     |
| XGBoost classifier       | 0.71      | 0.87   | 0.78     |
| SVM                      | 0.67      | 0.87   | 0.76     |

**Fig. 3.8** Normal class precision

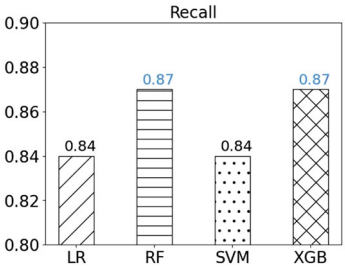


four models, namely LR (Logistic Regression), RF (Random Forest), XGBoost Classifier, and SVM (Support Vector Machine). Table 3.8 shows the results of the model comparison experiments in the same experimental environment.

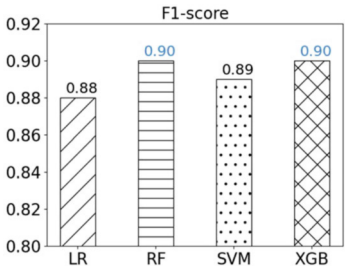
After analyzing the experimental results shown in Table 3.8, it can be seen that the XGBoost Classifier used in this chapter has 71% precision, 87% recall, and 78% F1-score in the experiment, which has achieved good classification results. Compared with Logistic Regression, Random Forest, and SVM models, the F1 value of XGBoost is improved by 7%, 2%, and 2%, respectively. Among them, the recall value of the SVM-based model is as high as that of XGBoost. However, the precision of SVM is only 67%, which is 3% lower than that of XGBoost. In a comprehensive comparison, only the XGBoost model has all three metrics higher and currently has the best performance.

The results of the experiment for the normal account category are shown in Figs. 3.8, 3.9, and 3.10. The XGBoost model is the one with the best performance in all three metrics. The XGBoost Classifier has 95% precision, 87% recall, and 90% F1-score in the detection performance of normal accounts. The RF model achieves the same high recall and F1-score as the XGBoost Classifier, but its precision is 93%, which is a bit lower than the XGBoost. The LR model has the worst performance of the four models. Its scores of precision, recall, and F1-score are 92%, 84%, and 88%.

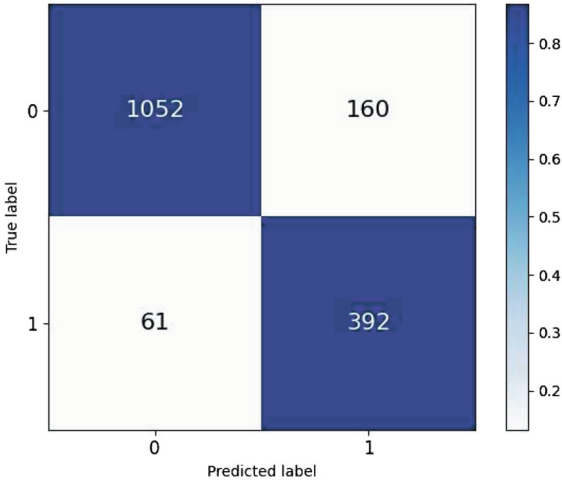
**Fig. 3.9** Normal account recall



**Fig. 3.10** Normal class f1-score



**Fig. 3.11** Confusion matrix of the XGBoost model detection results



In order to show the detection results of the XGBoost model more clearly, a confusion matrix is used to visualize the final classification results, as shown in Fig. 3.11. In the confusion matrix, each row represents the true category of the data, and each column represents the category predicted by the model.

Due to the uneven number of abnormal samples in the experiment, the ROC curves of the four models were then compared for comprehensive consideration to avoid the influence of sample distribution, which can be seen in Fig. 3.12. The ROC curve is a graphical tool used to represent the performance of a classification model. It depicts the performance of a classifier at different thresholds by using TPR (True-

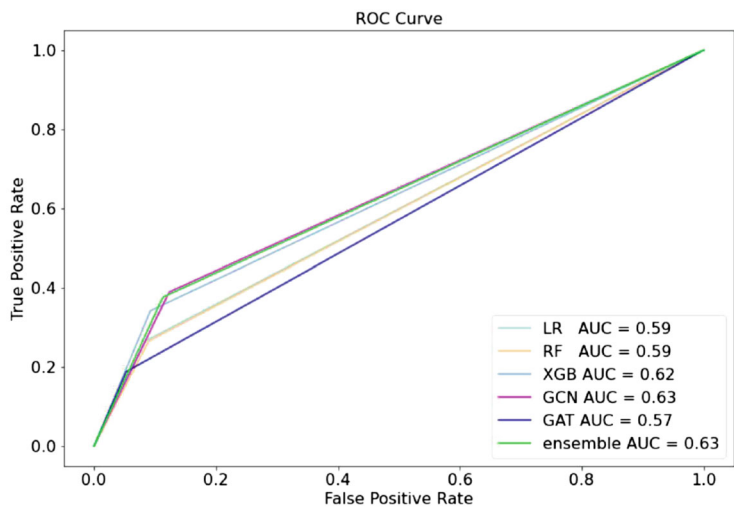


Fig. 3.12 ROC curve of four models

Table 3.9 Running time of four models

| Models                   | Running time(seconds) |
|--------------------------|-----------------------|
| Logistic regression      | 0.12                  |
| Random forest classifier | 3.13                  |
| XGBoost classifier       | 1.83                  |
| SVM                      | 4.62                  |

Positive Rate) and FPR (False-Positive Rate) as horizontal and vertical coordinates. AUC is the area under the ROC curve and is used to measure classifier performance. The closer the AUC value is to 1, the better the classifier performance is. Besides, the higher the AUC, the better the model is at predicting 0 to 0 and 1 to 1. We can observe that the XGBoost model has an AUC score of 0.87, which is the highest score among the four models. The AUC scores of LR, RF, and SVM are 0.82, 0.85, and 0.86, respectively.

Under the condition of approximate operation results, the computational speed of the four models, LR, RF, XGBoost Classifier, and SVM, is compared, and the running times of different models are compared as shown in Table 3.9. According to the running time comparison, the fastest training speed of the Logistic Regression model is 0.12s. XGBoost model is the second fastest with a running time of 1.83s. While the training time of Random Forest Classifier and SVM models is longer at 3.13s and 4.62s. In summary, the XGBoost model can produce fast classification results while improving the performance of abnormal account detection.

**Effect of the Number of Features on the Experiment** In this chapter, we extract features based on transaction behavior and compare the various machine learning models. We can see that the XGBoost model can obtain a better detection rate. In order to further explore whether the low-dimensional features combined with



**Table 3.10** Performance comparisons of different features

| Methods                           | Precision | Recall | F1-score |
|-----------------------------------|-----------|--------|----------|
| Time and degree features          | 0.43      | 0.72   | 0.54     |
| Amount features only              | 0.69      | 0.86   | 0.76     |
| Time and degree + amount features | 0.71      | 0.87   | 0.78     |

the XGBoost model can still achieve better detection results. Table 3.10 shows the experimental results of different numbers of selected features as the input of the training and testing model.

The analysis shows that when combining the time, degree, and amount features with the XGBoost classifier, the F1 value reaches 78%. As the number of features decreases, the precision, recall, and F1 value gradually decrease. Compared with time and degree features, the XGBoost classifier with only the amount features has a better result of 69% precision, 86% recall, and 76% F1-score. Therefore, the model combined with low-dimensional features does not obtain a better result.

3.1.5 Summary

We introduce CrossAAD, an abnormal account detection method that can effectively identify abnormal cross-chain transactions via a technical combination of feature engineering. The proposed scheme is built on top of cross-chain bridge and employs feature extraction to capture transaction features and cross-chain properties. We consider four classification models to determine the final training model. We implement CrossAAD with the multi-chain dataset on a Colab virtual machine in Python. Experimental results show that the proposed CrossAAD achieves a practical performance for cross-chain transactions with 95% precision and 87% recall on normal labels, 71% precision, and 87% recall on abnormal labels, which is comparable to existing single-chain detection schemes. This work has been published in ACISP 2024 (Lin et al. 2024a).

3.2 Cross-Chain Abnormal Transaction Detection via Graph-Based Multi-model Fusion

3.2.1 Background and Motivation

Due to the involvement of multiple parties and interactions, cross-chain transactions inherently possess the characteristics of a graph structure. In the common graph-based transaction network, information is hidden in the graph, where the account and the transaction can be abstracted as the node and the edge, respectively.

Regulation for graph-based transactions still depends on transaction record mining and tends to prefer behavioral pattern analysis via graph topology (Di et al. 2022). Due to complicity of graph-based structure, both efficiency and accuracy are important in graph-based transaction regulation.

To improve efficiency of illegal node detection in time measurements, a fraud detection model (Ibrahim et al. 2021) was proposed based on three different machine learning algorithms, selecting the six most relevant features of transactions to improve the detection time of illegal Ethereum accounts. To detect more granular anomalies, study (Stefánsson et al. 2022) examined methodologies applicable for identifying potential instances of money laundering in Bitcoin transactions through machine learning techniques. Their aim is to equip investigators with enhanced tools to accurately and efficiently discern whether suspicious activities may involve money laundering. However, this method cannot track money laundering activities. To address the problem of tracking money laundering activities on the Bitcoin network, Lorenz et al. (2020) proposed an active learning solution based on machine learning methods. The approach mimics a common real-life scenario where only a restricted number of labels can be obtained through manual annotation by experts. To figure out if presented ML models are biased toward a specific malicious activity, Agarwal et al. (2021) pinpointed a robust supervised machine learning (ML) algorithm that remains impervious to biases stemming from an overrepresentation of specific malicious activities within the dataset. Additionally, it demonstrates resilience against adversarial attacks.

Recently, graph-based anomaly detection methods have gained prominence as effective solutions for analyzing the expanding data within blockchain ledgers, including those of Bitcoin (Weber et al. 2019; Poursafaei et al. 2021) and Ethereum (Yuan et al. 2020; Chen et al. 2021b; Poursafaei et al. 2021). Weber et al. (2019) conceptualized the Bitcoin transaction network as a directed acyclic graph, where the nodes represent individual transactions, and the edges depict the movement of cryptocurrency between them. They used variations of Logistic Regression, Random Forest, Multilayer Perceptrons, and Graph Convolutional Networks to achieve anti-money laundering on Bitcoin. To detect phishing scams on Ethereum, Yuan et al. (2020) used support vector machines for phishing detection based on the features extracted by the graph embedding technique node2vec. Chen et al. (2021b) treated accounts and transactions as nodes and edges. They introduced a detection method leveraging Graph Convolutional Networks and autoencoders to accurately differentiate phishing accounts. To detect illicit nodes on blockchain networks, Poursafaei et al. (2021) proposed an efficient graph-based method. They transformed the blockchain network into a simple graph and then extracted carefully designed features, which explain structural, transactional, regional, and neighborhood features of the nodes. They also achieved great performance on Bitcoin and Ethereum.

For graph-based cross-chain transaction regulation mechanisms, we first consider the anomaly detection via node classification. The current node classification methods are almost built on top of conventional machines learning classifiers (Wen et al. 2023; Chen et al. 2021a; Wu et al. 2022) and graph representation learning

(Liu et al. 2022; Chen et al. 2020; Yu et al. 2021). The former needs crafted training features and expert knowledge, which is incongruous with the usual automation extraction. The latter inlays graph neural network, which cannot support large-scale cross-chain system. Despite aggregating one-hop and two-hop neighbors in training, it still struggles to extract sufficient and accurate information. Employing traditional node classification is still technically challenging for graph-based cross-chain abnormal transaction detection.

### 3.2.2 *GMMCCT Overview*

We introduce GMMCCT, a graph-based abnormal transaction detection method that integrates graph embedding learning and multi-model fusion based on graph data. To accommodate the cross-chain structure, we combine graph embedding algorithms and graph neural networks to enhance the detection performance of abnormal transactions. GMMCCT extracts high-order feature information from the graph structure and uses a model that integrates machine learning and graph neural networks to identify abnormal transactions. Through a comparison of various machine learning models and graph neural networks, we assess their suitability for cross-chain scenarios and recognize the data processing challenges. To create a dataset of cross-chain transactions with abnormal account labels, we extract data from a cross-chain bridge. By introducing graph embedding technology, we gain a comprehensive understanding of the behavioral patterns of abnormal transactions by examining the relationships between different nodes in the cross-chain network. This approach enhances the accuracy of abnormal transaction detection. We have employed a model fusion strategy that combines LR, XGBoost, and GCN models to generate final prediction results based on the structural features extracted by node2vec. Our approach to multi-model fusion leverages the strengths of each model to enhance predictive performance.

The fundamental concept behind GMMCCT revolves around depicting cross-chain transactions as a graph structure. Here, accounts are depicted as nodes featuring low-dimensional feature vectors derived through graph embedding techniques. The pivotal aspect lies in amalgamating various models, such as graph neural networks, to enhance classification outcomes. Subsequently, node identities are deduced based on a labeled training dataset. Figure 3.13 provides an overview of GMMCCT, delineated into three stages: graph establishment, feature extraction, and multi-model fusion. In the first stage, we preprocess the collected cross-chain transaction data. Then, we construct graph data with cross-chain transactions. In the second stage, based on the constructed graph data, we use the graph embedding algorithm node2vec to extract network structural features, mapping transactions network to a low-dimensional vector space. The final stage involves ensemble modeling using LR, XGBoost, and GCN, where multiple predictive probabilities are integrated to achieve model fusion. The features extracted by node2vec serve as input, with the output being the classification results for nodes.

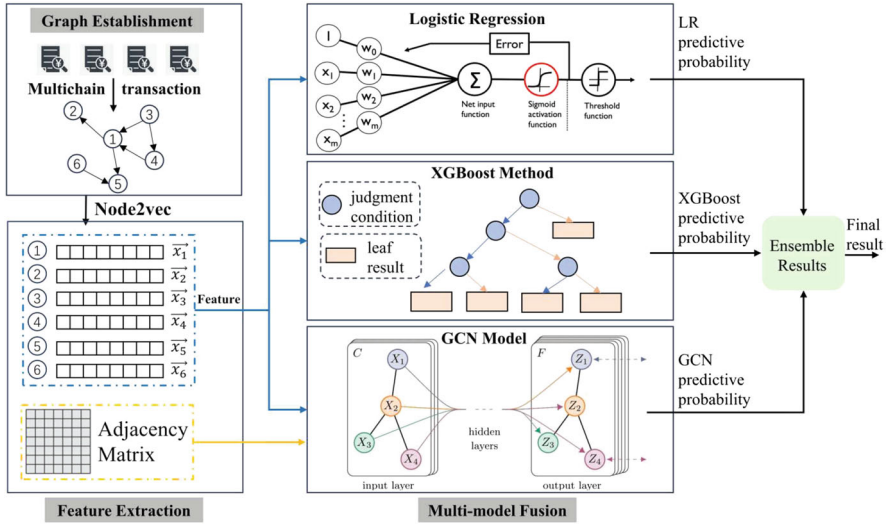


Fig. 3.13 Overview of GMMCCT design

3.2.3 Detailed Design of GMMCCT

3.2.3.1 Source of Dataset

The cross-chain transaction data in this study are sourced from the multi-chain cross-chain bridge.<sup>2</sup> Cross-chain bridges are essential tools for connecting different blockchains, allowing assets and information to be securely and transparently transferred between different chains. There are three types of cross-chain bridges: anchored cross-chain bridges, synthetic cross-chain bridges, and proxy cross-chain bridges. Multi-chain (formerly known as Anyswap) is generally classified as a synthetic cross-chain bridge. It is a well-known platform that provides decentralized cross-chain bridge services, and its official browser displays cross-chain transactions involving different chains and amounts. Multi-chain aims to address the interoperability challenges between different blockchain platforms, enabling users to seamlessly transfer and exchange encrypted assets across different blockchains. Multi-chain supports popular EVM chains such as Ethereum, BNB, Polygon, as well as non-EVM compatible chains like Solana, and even non-smart contract chains like Bitcoin. Users can transfer assets across these chains. The variety of cross-chain assets exceeds 3000, with nearly 1,000,000 users and over 100 billion cross-chain transaction data. Using the official interface provided by multi-chain, we programmatically implemented the automatic downloading of cross-chain

<sup>2</sup> <https://scan.multichain.org>

**Table 3.11** Transaction details

| Attribute           | Value                                           |
|---------------------|-------------------------------------------------|
| Source chain hash   | 9fb2e68d4aaf8cb766b6a516b50269a63850b6c01.....  |
| Target chain hash   | 0x96a8b26e96d69aa6f65cf0382ea24c48f88bf633..... |
| Source chain        | Bitcoin mainnet                                 |
| Target chain        | Ethereum mainnet                                |
| From                | 3DSVaLoJBEXg8h4byEVcKJX6qN8YajyFKD              |
| To                  | 0x79F49449A106D290E697C3492694b80171052a8c      |
| Date                | 2023/03/23 09:13                                |
| CoinType            | BTC                                             |
| Send the number     | 0.125263                                        |
| Number of receiving | 0.125263                                        |
| Status              | Success                                         |

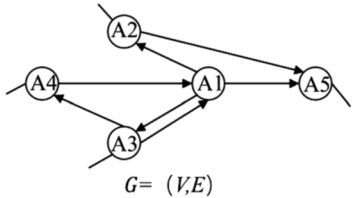
transaction data, amounting to approximately 1.5GB. Cross-chain transactions involve 11 attributes, as shown in Table 3.11.

3.2.3.2 Data Preprocessing

After collecting information online, there is a total of 2413 abnormal nodes in cross-chain transaction network. Nodes with very low degrees are not conducive to extracting network structural features, so this study removes nodes with a degree of 1. The remaining 1077 nodes are designated as the abnormal set. Additionally, 3500 normal nodes are randomly selected as the normal dataset. The final dataset consists of normal and abnormal data. Subsequently, data cleaning is performed, and the detailed process is outlined as follows:

- **Data Review:** A comprehensive review of the data is conducted, covering a time span of four months from October 2021 to January 2022. There are a total of 425,889 cross-chain transactions involving several different blockchains such as Ethereum and Bitcoin. This helps in understanding the overall structure and content of the data.
- **Handling Missing Values:** Upon initial examination of the dataset, it is observed that some transactions had missing attribute values. Analysis indicates that this is likely due to issues arising from frequent access to the cross-chain bridge data interface. These missing values are then removed.
- **Dealing with Anomalies:** The dataset contains instances of failed cross-chain transactions, which can result from various reasons such as inability to connect to the cross-chain client, incorrect chain codes or function information, non-existent recipient accounts, or failure to obtain cross-chain responses. Unsuccessful cross-chain transactions are not useful for constructing the graph dataset; Hence, we remove them.

**Fig. 3.14** Cross-chain transaction network diagram



By rigorously following the detailed steps outlined earlier, we can effectively cleanse the raw data, improve the quality and reliability of the data, and establish a solid foundation for subsequent analysis and modeling work.

**3.2.3.3 Graph Construction**

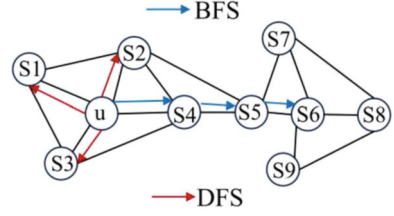
After ensuring the data quality, we also need to transform the cross-chain transaction data into a form suitable for modeling. Before conducting cross-chain transaction network analysis, the first step is to represent the cross-chain transaction data in the form of network structure data, i.e., constructing the cross-chain transaction graph. This study constructs a directed graph for cross-chain transactions, where different cross-chain account addresses serve as nodes, and the flow of funds between accounts serves as edges. The graph is denoted as  $G = (V, E)$ , where  $V$  represents the node set, i.e., the collection of cross-chain account addresses, and  $E$  represents the edge set in the cross-chain transaction network, i.e., the flow of cross-chain transactions. Figure 3.14 shows the schematic diagram of the cross-chain transaction network.

Where A1, A2, A3, A4, A5 represent different nodes, and the directed edges between the nodes indicate a transaction between these nodes, with the edge representing the transfer of a certain amount of currency from the source node to the destination node. Each directed edge must have two key attributes: the transaction amount and the timestamp. To facilitate input to the classification model, nodes need to be represented by numbers, eventually forming data files for nodes and edges.

**3.2.3.4 Feature Extraction**

Based on tagging Graph dataset during the graph establishment phase, we intend to extract multiple features by graph embedding techniques. To better extract the topological structure information of cross-chain transaction networks, this chapter employs node2vec to extract network structural features, mapping accounts, and transactions to a low-dimensional vector space, thereby preserving cross-chain relationships and multimodal information. Node2vec improves upon DeepWalk by modifying the random walk strategy during the process. As illustrated in Fig. 3.15, node2vec random walks primarily adopt depth-first and breadth-first strategies to capture the network’s topology.

**Fig. 3.15** Schematic diagram of the randomized wandering process of node2vec



Depth-first search tends to traverse along a single path in the graph until it reaches a dead end, then backtracks to the previous level to continue the search. It reflects the network's homophily, where embeddings of nearby nodes should be as close as possible. In Fig. 3.15, the embeddings of node  $u$  and its connected nodes  $s1$ ,  $s2$ ,  $s3$ , and  $s4$  should be close, demonstrating homophily. Breadth-first search, on the other hand, traverses the nodes in layers, first visiting all neighboring nodes of a node, then exploring outward layer by layer. It reflects the network's structural equivalence, where embeddings of structurally similar nodes should be close. Both node  $u$  and node  $s6$  are central nodes in their respective local networks, exhibiting structural similarity, thus their embeddings should be close, demonstrating structural equivalence.

During the random walk process, the balance between BFS and DFS is adjusted through parameters  $p$  and  $q$ . When  $p = q = 1$ , it indicates random selection of the walking direction, which is equivalent to the DeepWalk method. Figure 3.16 shows the core pseudocode of node2vec.

The extracted feature vectors are as Fig. 3.17. Each node  $v_i$  can obtain a feature vector  $x = (x_1, x_2, \dots, x_d)$ , where  $d$  is the dimensionality of the feature vector, which can be adjusted. The feature vectors of all nodes constitute the entire feature matrix.

### 3.2.3.5 Multi-model Fusion

This chapter selects three classic models for fusion, namely logistic regression (Selvi et al. 2022), XGBoost (Chen et al. 2015), and Graph Convolutional Networks. Both logistic regression and XGBoost require the node features as input to train the models. For GCN, in addition to the node features, the adjacency matrix of the nodes is required as the second input. The structure of the GCN used is composed of two graph convolutional layers, a normalization layer, and an activation function layer. The input to the first graph convolutional layer is the node feature vectors with a dimensionality of 64. After normalization and activation function processing, the output is passed as input to the second layer. Before outputting the final result, the output of the second layer is passed through a softmax function to ensure that the output represents the probability values of the predicted classes. This step facilitates model ensemble and fusion in subsequent processes. If the class probability predictions of the logistic regression model are denoted as

---

**Require:** Graph:  $G = (V, E, W)$   
**Ensure:** d-dimensional vector

```

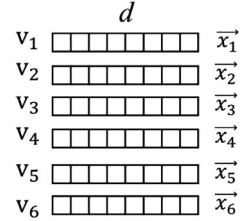
1: Begin
2: (1) Feature Learning:
3: function LEARNFEATURES(Graph  $G = (V, E, W)$ , Dimensions  $d$ , Walks per node  $r$ , Walk length  $l$ , Context size
    $k$ , return  $p$ , In-out  $q$ )
4:    $\pi \leftarrow \text{PREPROCESSMODIFIEDWEIGHTS}(G, p, q)$ 
5:    $G' \leftarrow (V, E, \pi)$ 
6:   Initialize  $walks$  to Empty
7:   for  $iter \leftarrow 1$  to  $r$  do
8:     for all nodes  $u \in V$  do
9:        $walk \leftarrow \text{NODE2VECWALK}(G', u, l)$ 
10:      Append  $walk$  to  $walks$ 
11:     end for
12:   end for
13:    $f \leftarrow \text{STOCHASTICGRADIENTDESCENT}(k, d, walks)$ 
14:   return  $f$ 
15: end function
16: (2) Nodal Roaming:
17: function NODE2VECWALK( $G' = (V, E, \pi)$ , Start node  $u$ , walk length  $l$ )
18:   Initialize  $walk[0]$  to  $u$ 
19:   for  $walk\_iter \leftarrow 1$  to  $l$  do
20:      $curr \leftarrow walk[-1]$ 
21:      $V_{curr} \leftarrow \text{GETNEIGHBORS}(curr, G')$ 
22:      $s \leftarrow \text{ALIASAMPLE}(V_{curr}, \pi)$ 
23:     Append  $s$  to  $walk$ 
24:   end for
25:   return  $walk$ 
26: end function
27: End

```

---

Fig. 3.16 Node2vec pseudocode

Fig. 3.17 The extracted feature vectors diagram



$LR\_prob = (LRp_{00}, LRp_{01}), \dots (LRp_{m0}, LRp_{m1})$ ,  $m$  is the number of nodes to be predicted. The XGBoost output probabilities are denoted as:  $XGB\_prob = (XGBp_{00}, XGBp_{01}), \dots (XGBp_{m0}, XGBp_{m1})$ , and the GCN output probabilities are denoted as  $GCN\_prob = (GCNp_{00}, GCNp_{01}), \dots (GCNp_{m0}, GCNp_{m1})$ , then the final predicted probability values after combining the three models can be calculated as follows:  $Ensemble\_prob = \frac{1}{3} * (LR\_prob + XGB\_prob + GCN\_prob)$ . For each node, the final predicted result is based on the class with the highest predicted probability value among the combined probabilities from the three models.



3.2.4 Performance Evaluation and Analysis

3.2.4.1 Experimental Setup

To conduct comparative experiments and validate the efficacy of our proposed GMMCCT, we have chosen several conventional methods as baselines for experimentation. These include commonly used machine learning techniques as well as Graph Neural Networks. Specifically, we have selected representatives such as Logistic Regression, Random Forest, XGBoost, GCN, and GAT for comparison. We utilize the sklearn library and the PyTorch Geometric library for constructing our models. The prototype comprises approximately 1500 lines of Python code, covering both the model construction and test programs.

For all experiments conducted, we adopt an 80/20 split strategy for the dataset, where 80% of the data, encompassing both normal and abnormal instances, are utilized for training the machine learning algorithms. The remaining 20% is reserved for testing the performance of the trained models. The construction of the dataset is described. The final dataset contains 4577 cross-chain nodes, of which 1077 are labeled as anomalous and 3500 are labeled as normal. For each node, network structural features are extracted by the graph embedding algorithm node2vec, and the dimensionality of the features is 64 dimensions. The graph learning and classification programs are executed on a cloud server with the following configuration information in Table 3.12.

3.2.4.2 Evaluation Metrics

In this chapter, precision rate, recall rate, F1 value, accuracy, and AUC score are used as evaluation indicators for abnormal transaction monitoring. According to the Tag list:  $Tags = \{f_1, f_2, \dots, f_n\}$ ,  $f_i = 0$  or  $1$ , and the corresponding prediction result list:

$$Predicting\ outcome = \{y_1, y_2, \dots, y_n\}.$$

Table 3.12 Software and hardware configurations of experiments

| Configuration      | Specific parameter information                 |
|--------------------|------------------------------------------------|
| GPU                | RTX 3080(10GB)                                 |
| CPUs               | 12vCPU Intel(R) Xeon(R) Platinum 8255C @2.5GHz |
| RAM                | 40 GB                                          |
| Disk space         | 30 GB                                          |
| Operating system   | ubuntu 20.04 cuda 11.3                         |
| Learning framework | Sklearn+PyTorch                                |

we then calculate the precision rate, recall rate, and F1 value. For  $i = 1, 2, \dots, n$ ,

1. If  $f_i == y_i == 1$ , then count plus one.  $TP$  is the final accumulated value.
2. If  $f_i == 0, y_i == 1$ , then count plus one.  $FP$  is the final accumulated value.
3. If  $f_i == 1, y_i == 0$ , then count plus one.  $FN$  is the final accumulated value.

Among them, accuracy is the proportion of correctly predicted data to all data. Its formula is as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}.$$

The precision rate is for the prediction results, and its meaning is the probability of the actual positive samples among all the samples that are predicted to be positive. Its formula is as follows:

$$Precision = \frac{TP}{TP + FP}.$$

The recall is also known as the check-all rate, which is for the original sample. It means the probability of being predicted as a positive sample out of an actual positive sample.

$$Recall = \frac{TP}{TP + FN}.$$

F1 value is the summed average of precision rate and recall rate, taking into account both precision rate and recall rate, allowing both to reach the highest at the same time and taking a balance. Its formula is as follows:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

The Receiver Operating Characteristic (ROC) curve is a visual representation of the performance of a classification model. It illustrates the trade-off between the True-Positive Rate (TPR) and the False-Positive Rate (FPR) at various classification thresholds. TPR, also known as sensitivity or recall, measures the proportion of actual positive cases that are correctly identified by the classifier as positive. It is calculated as:  $TPR = \frac{TP}{TP + FN}$ . FPR, on the other hand, measures the proportion of actual negative cases that are incorrectly identified by the classifier as positive. It is calculated as:  $FPR = \frac{FP}{FP + TN}$ . The formula of AUC is as follows:

$$AUC = \int_0^1 TPR(FPR^{-1}(t))dt.$$

**Table 3.13** Detection performance of abnormal nodes

| Input              | Methods       | Precision   | Recall      | F1-score    | Accuracy    | AUC         |
|--------------------|---------------|-------------|-------------|-------------|-------------|-------------|
| Features           | LR            | 0.52        | 0.26        | 0.35        | 0.75        | 0.59        |
| Features           | RF            | 0.5         | 0.26        | 0.36        | 0.75        | 0.59        |
| Features           | XGBoost       | 0.56        | 0.34        | 0.42        | 0.76        | 0.62        |
| Features+Adjacency | GCN           | 0.52        | 0.39        | 0.44        | 0.75        | 0.63        |
| Features+Adjacency | GAT           | 0.55        | 0.19        | 0.28        | 0.75        | 0.57        |
| Features+Adjacency | <b>GMMCCT</b> | <b>0.57</b> | <b>0.43</b> | <b>0.49</b> | <b>0.77</b> | <b>0.63</b> |

The bold represents the proposed scheme in this chapter

**Table 3.14** Detection performance of normal nodes

| Input              | Methods       | Precision   | Recall      | F1-score    | Accuracy    | AUC         |
|--------------------|---------------|-------------|-------------|-------------|-------------|-------------|
| Features           | LR            | 0.78        | 0.92        | 0.85        | 0.75        | 0.59        |
| Features           | RF            | 0.78        | 0.91        | 0.84        | 0.75        | 0.59        |
| Features           | XGBoost       | 0.8         | 0.91        | 0.85        | 0.76        | 0.62        |
| Features+Adjacency | GCN           | 0.81        | 0.88        | 0.84        | 0.75        | 0.63        |
| Features+Adjacency | GAT           | 0.77        | <b>0.95</b> | 0.85        | 0.75        | 0.57        |
| Features+Adjacency | <b>GMMCCT</b> | <b>0.82</b> | 0.89        | <b>0.85</b> | <b>0.77</b> | <b>0.63</b> |

The bold represents the proposed scheme in this chapter

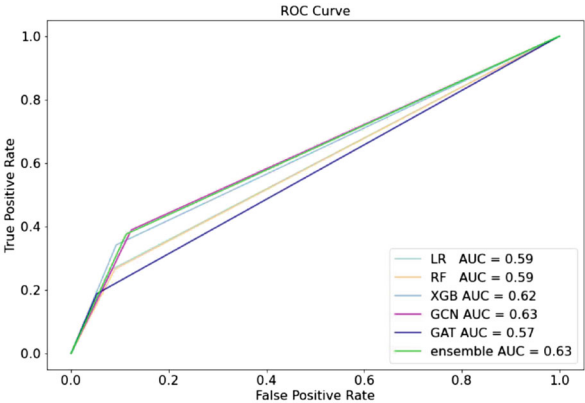
3.2.4.3 Classification Performance

We use the five metrics (Rajkomar et al. 2018) described earlier to measure the performance of the model on cross-chain anomalous transaction detection, where here the inputs to the traditional machine learning model are structural features extracted via node2vec. The inputs of other graph neural networks are structural features plus adjacency matrix. Table 3.13 shows the results of the model comparison experiments in the same experimental environment.

Our proposed GMMCCT method achieved the highest F1 score of 0.49 in abnormal node detection, outperforming GAT by 21% and GCN by 5%. GMMCCT also has 57% precision, 43% recall, and 77% accuracy. These metrics are also the highest among other models. The GCN model demonstrates the second-highest performance in terms of F1 score and recall, with an F1 score of 0.44 and a recall of 0.39. The precision value ranks fourth, with a value of 0.52. It can be observed that Graph Neural Networks outperform these traditional machine learning models on this dataset. In a comprehensive comparison, only the GMMCCT method has all five metrics higher and currently has the best performance.

The performance of normal node detection is shown in Table 3.14. GMMCCT method is the one with the best performance in four metrics. It has 82% precision, 89% recall, 85% F1-score, 77% accuracy, and 63% AUC. Only the recall performance ranks second, following the GAT model. The GAT model achieved the highest recall value of 0.95, but its precision is only 0.77, which is the lowest among the six methods. The GCN model’s performance is average, with a precision

**Fig. 3.18** ROC curve of six models



of 0.81, ranking second, but its recall is only 0.88, which is also the lowest. To ensure comprehensive analysis and mitigate the impact of sample distribution, we compare the ROC curves of the six models in the experiment. This comparison is particularly crucial due to the uneven number of abnormal samples. Please refer to Fig. 3.18 for visual representation.

The Area Under the ROC Curve (AUC) is a scalar value that quantifies the overall performance of the classifier. A perfect classifier would have an AUC of 1.0, while a random classifier would have an AUC of 0.5. A higher AUC value indicates better discrimination capability of the classifier across different threshold values. It is evident that both GMMCCT and GAT exhibit the joint highest AUC value of 0.63. In the second position is the XGBoost model with an AUC value of 0.62. The AUC scores for LR, RF, and GAT are 0.59, 0.59, and 0.57, respectively.

### 3.2.5 Summary

We present GMMCCT, a graph-based method for detecting abnormal transactions, which effectively identifies abnormal cross-chain transactions through a technical fusion of models. The proposed approach leverages a cross-chain bridge and utilizes node2vec for feature extraction to capture the structural properties of cross-chain transactions. We integrate GCN, LR, and XGBoost models to enhance performance. GMMCCT is implemented with a multi-chain dataset on a Python-based cloud server. Experimental results demonstrate that GMMCCT achieves practical performance for cross-chain transactions, with 82% precision and 89% recall on normal labels, and 57% precision and 43% recall on abnormal labels, comparable to existing single-chain detection schemes. This work has been accepted by BSCI 2024 (Lin et al. 2024b).

## References

- Agarwal R, Thapliyal T, Shukla SK (2021) Detecting malicious accounts showing adversarial behavior in permissionless blockchains. CoRR abs/2101.11915. <http://doi.org/2101.11915>
- Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) SMOTE: synthetic minority over-sampling technique. *J Artif Intell Res* 16:321–357
- Chen B, Wei F, Gu C (2021a) Bitcoin theft detection based on supervised machine learning algorithms. *Secur Commun Networks* 2021:6643763:1–6643763:10
- Chen L, Peng J, Liu Y, Li J, Xie F, Zheng Z (2021b) Phishing scams detection in ethereum transaction network. *ACM Trans Internet Tech* 21(1):10:1–10:16
- Chen T, Guestrin C (2016) Xgboost: A scalable tree boosting system. In: Krishnapuram B, Shah M, Smola AJ, Aggarwal CC, Shen D, Rastogi R (eds) SIGKDD 2016. ACM, pp 785–794
- Chen T, He T, Benesty M, Khotilovich V, Tang Y, Cho H, Chen K, Mitchell R, Cano I, Zhou T, et al. (2015) Xgboost: extreme gradient boosting. *R Pack Ver 04-2* 1(4):1–4
- Chen W, Guo X, Chen Z, Zheng Z, Lu Y (2020) Phishing scam detection on ethereum: Towards financial security for blockchain ecosystem. In: Bessiere C (ed) IJCAI 2020, [ijcai.org](http://ijcai.org), pp 4506–4512
- Chicco D, Oneto L, Tavazzi E (2022) Eleven quick tips for data cleaning and feature engineering. *PLoS Comput Biol* 18(12):1010718
- Di Z, Wang G, Jia L, Chen Z (2022) Bitcoin transactions as a graph. *IET Blockchain* 2(3–4):57–66
- Ding Y, Zhang Y, Qin B, Wang Q, Yang Z, Shi W (2023) A scalable cross-chain access control and identity authentication scheme. *Sensors* 23(4):2000
- Elliptic (2022) The state of cross-chain crime. Tech. rep. <https://www.elliptic.co/hubfs/Cross%20Chain%20Report%20exec.pdf>
- Hinteregger A, Haslhofer B (2019) Short paper: An empirical analysis of monero cross-chain traceability. In: Goldberg I, Moore T (eds) FC 2019. Springer, Lecture Notes in Computer Science, vol 11598, pp 150–157
- Hu S, Zhang Z, Luo B, Lu S, He B, Liu L (2023) BERT4ETH: A pre-trained transformer for ethereum fraud detection. In: Ding Y, Tang J, Sequeda JF, Aroyo L, Castillo C, Houben G (eds) WWW 2023. ACM, pp 2189–2197
- Ibrahim RF, Elian AM, Ababneh M (2021) Illicit account detection in the ethereum blockchain using machine learning. In: ICIT 2021. IEEE, pp 488–493
- Kim J, Lee S, Kim Y, Ahn S, Cho S (2023) Graph learning-based blockchain phishing account detection with a heterogeneous transaction graph. *Sensors* 23(1):463
- Li S, Gou G, Liu C, Hou C, Li Z, Xiong G (2022a) TTAGN: temporal transaction aggregation graph network for ethereum phishing scams detection. In: Laforest F, Troncy R, Simperl E, Agarwal D, Gionis A, Herman I, Médini L (eds) WWW '22. ACM, pp 661–669
- Li Y, Weng J, Li M, Wu W, Weng J, Liu J, Hu S (2022b) Zerocross: A sidechain-based privacy-preserving cross-chain solution for monero. *J Parallel Distrib Comput* 169:301–316
- Lin Y, Jiang P, Guo F, Zhu L (2024a) Crossaad: Cross-chain abnormal account detection. In: Zhu T, Li Y (eds) Information Security and Privacy - 29th Australasian Conference, ACISP 2024, Sydney, NSW, Australia, July 15–17, 2024, Proceedings, Part III. Springer, Lecture Notes in Computer Science, vol 14897, pp 84–104
- Lin Y, Jiang P, Zhu L (2024b) Cross-chain abnormal transaction detection via graph-based multi-model fusion. <https://www.cloud-conf.net/bsci/2024/program.html>
- Liu X, Tang Z, Li P, Guo S, Fan X, Zhang J (2022) A graph learning based approach for identity inference in dapp platform blockchain. *IEEE Trans Emerg Top Comput* 10(1):438–449
- Lorenz J, Silva MI, Aparício D, Ascensão JT, Bizarro P (2020) Machine learning methods to detect money laundering in the bitcoin blockchain in the presence of label scarcity. In: Balch T (ed) ICAIF '20. ACM, pp 23:1–23:8
- Lu C, Huang J, Huang L (2023) Detecting urban anomalies using factor analysis and one class support vector machine. *Comput J* 66(2):373–383

- Poursafaei F, Rabbany R, Zilic Z (2021) Sigtran: Signature vectors for detecting illicit activities in blockchain transaction networks. In: Karlapalem K, Cheng H, Ramakrishnan N, Agrawal RK, Reddy PK, Srivastava J, Chakraborty T (eds) PAKDD 2021. Springer, Lecture Notes in Computer Science, vol 12712, pp 27–39
- Purwanto AD, Wikantika K, Deliar A, Darmawan S (2023) Decision tree and random forest classification algorithms for mangrove forest mapping in sembilang national park, indonesia. *Remote Sens* 15(1):16
- Rajkomar A, Dai AM, Sun M, Hardt M, Chen K, Rough K, Dean J (2018) Reply: metrics to assess machine learning models. *npj Digit Medicine* 1
- Selvi A, Belbasi MR, Haugh M, Wiesemann W (2022) Wasserstein logistic regression with mixed features. In: Koyejo S, Mohamed S, Agarwal A, Belgrave D, Cho K, Oh A (eds) *NeurIPS 2022*
- Stefánsson HP, Grímsson HS, Órason JK, Óskarsdóttir M (2022) Detecting potential money laundering addresses in the bitcoin blockchain using unsupervised machine learning. In: *HICSS 2022*. ScholarSpace, pp 1–10
- Weber M, Domeniconi G, Chen J, Weidele DKI, Bellei C, Robinson T, Leiserson CE (2019) Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *CoRR abs/1908.02591*
- Wen H, Fang J, Wu J, Zheng Z (2021) Transaction-based hidden strategies against general phishing detection framework on ethereum. In: *ISCAS 2021*. IEEE, pp 1–5
- Wen T, Xiao Y, Wang A, Wang H (2023) A novel hybrid feature fusion model for detecting phishing scam on ethereum using deep neural network. *Expert Syst Appl* 211:118463
- Wu J, Yuan Q, Lin D, You W, Chen W, Chen C, Zheng Z (2022) Who are the phishers? phishing scam detection on ethereum via network embedding. *IEEE Trans Syst Man Cybern Syst* 52(2):1156–1166
- Yu L, Zhang N, Wen W (2021) Abnormal transaction detection based on graph networks. In: *COMPSAC 2021*. IEEE, pp 312–317
- Yuan Q, Huang B, Zhang J, Wu J, Zhang H, Zhang X (2020) Detecting phishing scams on ethereum based on transaction records. In: *ISCAS 2020*. IEEE, pp 1–5
- Zhang J, Gao J, Li Y, Chen Z, Guan Z, Chen Z (2022) Xscope: Hunting for cross-chain bridge attacks. In: *ASE 2022*. ACM, pp 171:1–171:4
- Zhang R, Zhang G, Liu L, Wang C, Wan S (2020) Anomaly detection in bitcoin information networks with multi-constrained meta path. *J Syst Archit* 110:101829

# Chapter 4

## Cross-Chain Network Anomaly Detection



### 4.1 Background and Motivation

The network nodes of cross-chain systems, with their massive, distributed, autonomous, open, and freely accessible characteristics, are more susceptible to malicious attacks and manipulation, causing irreparable damage to the cross-chain system. Due to the regulatory challenges associated with blockchain network nodes, data released by relevant technical departments indicate that from 2011 to April 2018, global losses from attacks, damage, and other anomalies on blockchains reached \$2.864 billion (Xuehai et al. 2020).

The reasons for anomalies in cross-chain network nodes are often very complex, but most of the time, they are due to interference and attacks from external malicious factors, leading to varying degrees of damage to the nodes. If anomalies are not detected promptly, allowing a large number of nodes to fail can significantly impact the normal operation of transactions and, in severe cases, can cause the entire cross-chain system to collapse or to result in substantial asset losses. Attackers exploit the characteristics of cross-chain network nodes using various attack methods, such as double spending, hash power attacks, and resource theft, to disrupt the blockchain network and undermine the normal operation of cross-chain nodes (Chuannian 2022). These attacks can lead to communication disruptions between nodes, data synchronization errors, and failures in reaching consensus, thereby affecting the overall stability and credibility of the cross-chain system (Cai et al. 2021). Once cross-chain network nodes are attacked, their symptoms may include data transmission delays, node disconnections, and abnormal on-chain transactions. These anomalies not only affect user experience but can also lead to a serious crisis of trust in the cross-chain system.

In addition to external malicious attacks, cross-chain systems may also face issues due to inherent technical flaws, such as transaction delays caused by system bottlenecks, resulting in prolonged transaction times. Some cross-chain technologies implement delay mechanisms to ensure that cross-chain transactions

and interactive data are accurately recorded on the blockchain, which can mitigate certain long-range attack risks. However, considering the current limitations of blockchain technology, especially its constrained transaction processing capacity, when faced with a surge of spam transactions or a sudden increase in transaction volume, blockchain nodes may experience downtime, leading to network congestion. For instance, both Bitcoin and Ethereum, two of the most representative public blockchain systems, have encountered situations where transactions were blocked for hours or even days without confirmation. When cross-chain technology forms a network of interconnected chains, if one side's blockchain becomes congested, cross-chain transactions may time out and be difficult to quickly reverse, potentially leading to a blockage of the entire cross-chain interaction network and risking the paralysis of blockchain nodes (Qian et al. 2022).

Checking whether the network nodes of the cross-chain system are alive requires manually sending ping messages for confirmation. If the recipient does not respond to the ping message after 20 minutes, it is considered disconnected from the network (Yuan and Wang 2019). Even if a confirmation is received, it cannot fully determine whether it is in normal working condition. It still requires manual regular checks, which is very labor-intensive and time-consuming, and it is impossible to constantly confirm the health status of the network nodes and lacks the ability to take corresponding measures before potential problems erupt.

In current research, scholars have made relentless efforts to achieve real-time dynamic monitoring and anomaly detection of blockchain network nodes. Zheng et al. (2019) proposed a mechanism for automatically discovering blockchain nodes using Kademlia Distributed Hash Table (DHT) technology. Compared to other methods, their approach constructs a novel DHT topology based on XOR, significantly improving the efficiency of routing queries and enabling rapid identification and localization of nodes. Bandyopadhyay et al. (2019) employed an automated code-writing method based on machine learning. By using unsupervised deep learning to analyze the behavioral characteristics of anomalous nodes, they can automatically exclude detected anomalous nodes, thereby eliminating potential risks promptly. In their study, the authors summarized the features of anomalous nodes in attribute graphs: first, some traits of these anomalous nodes are often almost indistinguishable from those of normal nodes; secondly, they tend to be connected to different organizations; thirdly, they may come from organizations where their characteristics do not align with those of other nodes. Akoglu et al. (2010) proposed an unsupervised anomaly detection algorithm for weighted graphs called OddBall. This algorithm uses EgoNet to identify node anomalies and observed that node features exhibit a power-law distribution, marking those deviating from this distribution as anomalies. Ding et al. (2012) addressed the high false-positive rate issue in traditional out-degree detection methods for network intrusion detection by proposing a community-based anomaly detection algorithm. By clustering the internal nodes within communities, nodes entirely within a community are considered normal users, while bridge nodes or edges between communities are viewed as anomalous users. Wu et al. (2022) aimed to detect phishing scams in the Ethereum network using network representation learning (Trans2Vec) to extract



relevant data features. Combined with a binary classification model, they effectively identified phishing scams within the Ethereum network. Yuan (2022), in their deep exploration of food safety regulation, proposed an evaluation method for potential malicious nodes in blockchain networks. By enabling regional node cooperation, they layered the blockchain network into a multi-region network and conducted secondary inspections to promptly identify and replace malicious nodes. Through comprehensive performance evaluations of the nodes, they optimized regulatory node decisions and achieved non-static replacement of regulatory nodes.

Real-time detection and visualization of abnormal states of network nodes in the blockchain cross-chain system are crucial. This not only helps to timely discover and deal with potential security risks but also enhances the overall performance and stability of the cross-chain system, providing a strong guarantee for the widespread application of blockchain technology.

## 4.2 Methodology Overview

The network node count of the cross-chain system is vast and widely distributed, enhancing the system's robustness and decentralization significantly. Furthermore, each node's ability to independently participate in the cross-chain process adds flexibility and scalability. However, these features also present management challenges and security risks. Malicious nodes may disguise their identity, tamper with data, or launch attacks, disrupting the system's normal operation and posing a serious threat to the security and stability of cross-chain transactions. More importantly, due to the autonomy of nodes, inconsistencies or conflicts may arise between nodes. The openness and freedom of nodes, while making the system more flexible and inclusive, also provide opportunities for malicious users. They might launch attacks or steal sensitive information by creating new nodes, threatening system security. Coupled with the high resource space occupancy rate of cross-chain technology on network node servers, there are risks of memory overflow, node downtime, etc., which may lead to the collapse of the cross-chain system. Therefore, it's crucial to detect anomalies in network nodes of the cross-chain system. It's one of the key measures to maintain the safe and stable operation of the cross-chain system, timely detect and deal with potential risks, and ensure the smooth progress of the cross-chain process.

This chapter researches and designs a network node anomaly detection mechanism for cross-chain systems. The mechanism consists of two crucial parts: a cross-chain system based on ChainMaker relay and a visual anomaly detection alarm system based on Prometheus and Grafana. With the continuous development of blockchain technology, cross-chain interaction has become the key to achieving interconnection and interoperability between different blockchains. ChainMaker, as a high-performance, open-source blockchain solution, has five major technical advantages: a controllable underlying platform, leading international processing performance, complete and portable supporting tools, flexible and efficient assembly

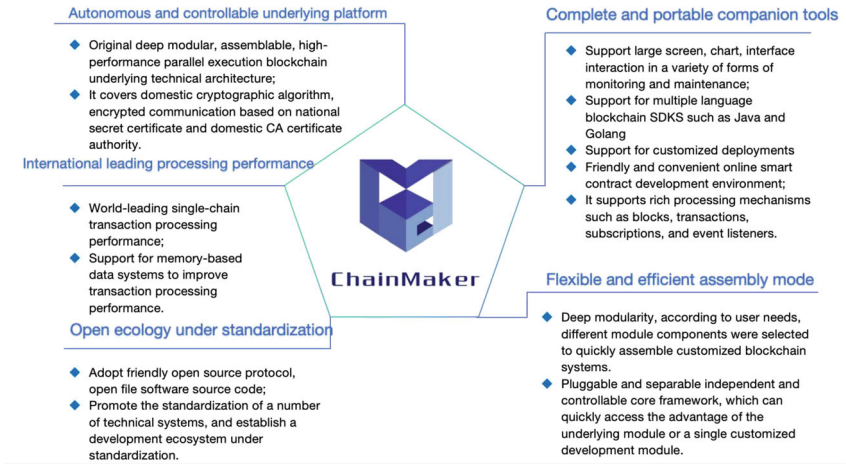


Fig. 4.1 Technological advantages of ChainMaker

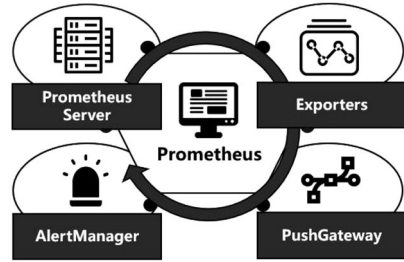
mode, and standardized open ecology. It has a wide range of application prospects (Beijing Microchip Research Institute et al. 2023), as shown in Fig. 4.1. The proposed cross-chain system based on ChainMaker relay, through a reasonable protocol stack and deployment mode, has achieved mutual connection between different blockchains. The system fully considers the needs of performance, controllability, and privacy protection, providing strong support for the development of cross-chain applications.

Prometheus (2017), as an open-source monitoring and alerting tool, can collect and process the running data of the cross-chain system in real time on the network node server's node\_exporter and issue email alerts through Alertmanager based on the abnormal threshold values of various indicators in the set rules. Grafana provides powerful data visualization capabilities, presenting complex data in an intuitive and understandable way and archiving historical anomalies. The visual anomaly detection system based on these two can monitor the operation of the cross-chain system in real time, timely detect potential problems and anomalies, and provide strong support for the system's stable operation.

As shown in Fig. 4.2, Prometheus is primarily composed of four core components. Prometheus Server, Exporters, AlertManager, and PushGateway. Through these components, Prometheus forms a complete operational monitoring ecosystem that includes data collection, data processing, visualization, data analysis, and alert handling.

The Prometheus Server, as the cornerstone of the Prometheus components, encompasses core functionalities including the acquisition, storage, and querying of monitoring data. It can manage monitoring targets through static configuration, as well as achieve dynamic target management in conjunction with Service Discovery, allowing it to extract data from various monitoring targets. Importantly,

**Fig. 4.2** Four core components of prometheus



the Prometheus Server itself functions as a time-series database and provides a custom query language called PromQL, enabling users to flexibly query and analyze resources. Additionally, the Prometheus Server features a built-in Express Browser UI, which allows users to perform data searches and visualizations directly using PromQL, greatly enhancing the readability and usability of monitoring data. In large-scale monitoring scenarios, the federated cluster capability of the Prometheus Server is particularly significant. Through this capability, the Server can aggregate data from other Prometheus servers, enabling flexible system expansion through federation and functional partitioning strategies.

The Exporter is a tool used by Prometheus to collect monitoring information. The data resources generated by monitoring targets are continuously delivered to the Prometheus server by the Exporter. In other words, the Exporter serves as a conduit between the monitored object's data and the Prometheus server. It acts like a switch: turning it on means running the Exporter, allowing the Prometheus server to collect information; turning it off, or stopping the Exporter, disconnects Prometheus from the monitoring target, preventing data collection. Exporters are typically categorized into two types: direct and indirect collection methods. Direct collection Exporters are those that inherently support Prometheus monitoring, while indirect collection is applicable when the monitoring target is not compatible with Prometheus. In such cases, a custom monitoring collector needs to be developed to accommodate the target.

The AlertManager is the alert management component of the Prometheus server. When certain rules are met, alerts are triggered, and the subsequent handling of these alerts is managed by the AlertManager. It not only supports notification methods such as email and DingTalk but also allows for custom alerting methods.

Considering the complexity of network environments, Prometheus data collection utilizes a Pull model. In certain situations where the Prometheus server cannot directly communicate with the Exporter, the PushGateway plays a crucial role. It actively pushes monitoring data from the internal network to the gateway, allowing the Prometheus server to pull data from the gateway, thereby obtaining complete and real-time data.

Grafana, as a versatile visualization tool, demonstrates its "universality" by adapting to various data visualization needs, not limited to just Prometheus monitoring data. In Grafana's architecture, data providers like Prometheus are referred

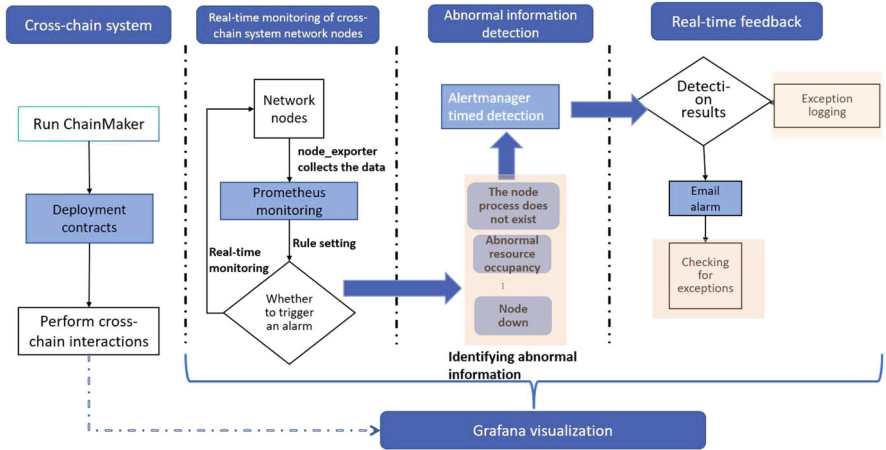


Fig. 4.3 Network node exception identification mechanism flow of cross-chain system

to as data sources, meaning that Grafana can integrate with multiple data sources, offering users a wealth of visualization options. The officially supported data source platforms encompass nearly all data monitoring tools, greatly facilitating those who wish to convert collected data into visual charts. Users simply need to adjust the interface, access the data and analysis results stored by the monitoring tools, connect the data sources to Grafana, and design the layout and specific display content of the panels to easily achieve data visualization.

The operation process of the entire anomaly detection mechanism is shown in Fig. 4.3 and can be divided into four major stages:

- Stage One: Start the ChainMaker chain in the cross-chain system, deploy contracts, and carry out cross-chain interaction.
- Stage Two: Collect data from the cross-chain system, monitor the operating status of network nodes, obtain chain data information in real time through buried underlying chain data, and store it.
- Stage Three: Anomaly information detection stage, the Alertmanager program periodically determines whether a specified alarm rule is triggered and waits for an alarm.
- Stage Four: Feedback the detection results. If the alarm rule is triggered and the buffer time is exceeded, an alarm email will be sent to the specified mailbox, and the abnormal event and time will be recorded.

### 4.3 Detailed Design of Network Node Detection Mechanism

#### 4.3.1 *Detection Environment for Cross-Chain Network Nodes*

The cross-chain solution is based on a relay-based cross-chain architecture, aiming to achieve layered decoupling, enhance scalability, and ensure minimal intrusion principles. It designs a five-layer cross-chain interaction protocol stack based on on-chain smart contracts and the adapter pattern. This protocol stack clearly defines the functional divisions of each layer, ensuring that protocols at each level can evolve independently while supporting flexible horizontal scaling of multiple protocols within the same layer. This approach meets the varying degrees of requirements for performance optimization, privacy protection, and other aspects for different types of blockchains, all while minimizing changes to the existing blockchain architecture, thereby enabling broader applications of blockchain technology.

##### 4.3.1.1 Five-Layer Cross-Chain Protocol Stack

The cross-chain protocol stack is designed based on fundamental functions such as transaction assurance, information transmission, and trust transfer within the cross-chain interaction process. Information transmission refers to the accurate and secure forwarding of state data from the source blockchain to the destination blockchain. Trust transfer means that the destination blockchain can effectively verify the validity of the state data from the source blockchain, ensuring that the data have reached consensus across the entire network of the source blockchain. Transaction assurance involves implementing a consistent cross-chain transaction execution plan to ensure the correctness and consistency of the global state after the execution of cross-chain transactions.

The cross-chain protocol stack is divided into five layers: the application layer, transaction layer, transport layer, verification layer, and forwarding layer. Each layer has its specific functions, as follows:

- **Application Layer:** The application layer plays a crucial role in the cross-chain interaction protocol stack. It is primarily responsible for the delivery process of cross-chain data packets from application to application. The core task of this layer is to address how cross-chain applications can adapt to the protocol stack and effectively implement the cross-chain interaction process, ensuring that applications can seamlessly utilize cross-chain functionalities.
- **Transaction Layer:** The transaction layer focuses on data consistency issues during the cross-chain interaction process. By separating the application's business logic from the transaction processing logic, it provides robust transaction processing security for cross-chain applications, ensuring that cross-chain data remain accurate and consistent throughout the interaction process.

- **Transport Layer:** The transport layer is responsible for the end-to-end delivery of cross-chain data packets during cross-chain interactions. This layer closely integrates with the upper application layer, effectively addressing the reliability issues of information transmission in cross-chain interactions, such as ordered data delivery and packet loss handling, thereby ensuring the integrity and accuracy of information during the transmission process.
- **Verification Layer:** The verification layer primarily addresses the transfer of trust in cross-chain interactions. It is responsible for verifying the authenticity and validity of cross-chain data, ensuring that trust can be effectively transmitted between different blockchains and providing a solid foundation of trust for cross-chain interactions.
- **Forwarding Layer:** The forwarding layer focuses on the transmission of cross-chain data packets between different blockchains during cross-chain interactions. It primarily addresses the issues of on-chain and off-chain communication, enabling cross-chain data packets to be accurately and efficiently transmitted across various blockchain networks, thereby facilitating smooth cross-chain interactions.

#### 4.3.1.2 Cross-Chain Interaction Process

The cross-chain system is deployed based on a relay-based cross-chain architecture. The main components of the system include the source parallel chain, destination parallel chain, source gateway, destination gateway, and interlink chain. These components work together to ensure the smooth progress of cross-chain interactions. Specifically, the cross-chain interaction protocol stack is deployed in the form of on-chain contract programs on the source parallel chain, destination parallel chain, and interlink chain. The source parallel chain and destination parallel chain both fully include the application layer, transaction layer, transport layer, verification layer, and forwarding layer of the protocol stack. These layers work together to ensure the smooth progress of cross-chain interactions. The interlink chain mainly deploys the verification and forwarding layers of the protocol stack, responsible for the verification and forwarding of cross-chain data between different parallel chains. On the gateway nodes, the cross-chain protocol stack is deployed in the form of traditional off-chain programs. These nodes mainly cover the transport layer, verification layer, and forwarding layer of the protocol stack, and they are responsible for reliable data transmission, verification, and forwarding between different chains during cross-chain interactions.

The overall interaction process of the native cross-chain interaction protocol stack has the characteristic of two-way interaction. It mainly divides into two core stages: the request state and the response state. These two stages together constitute a complete cross-chain interaction, as shown in Fig. 4.4.

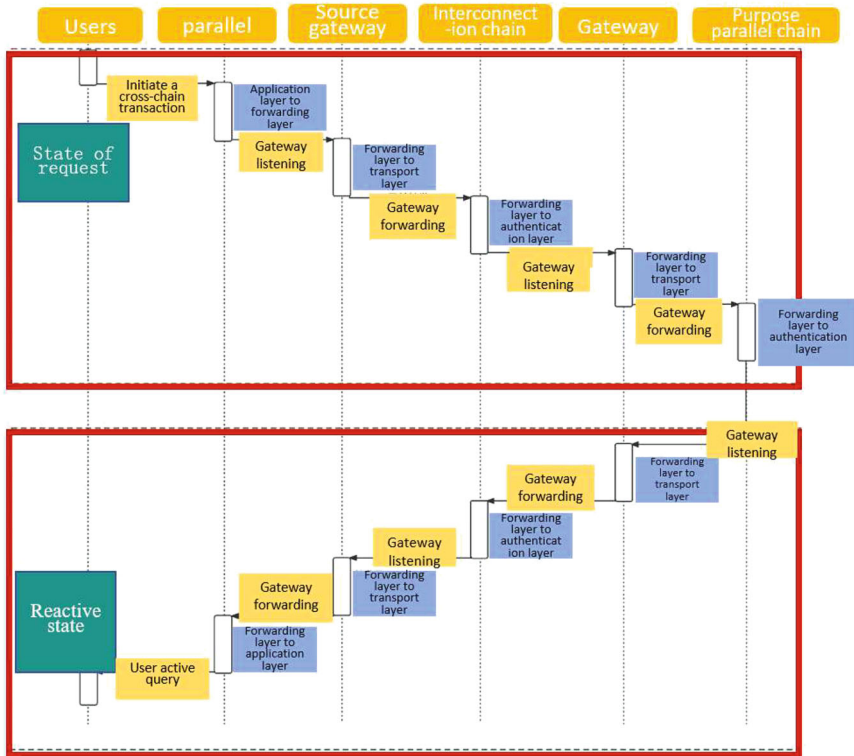


Fig. 4.4 Cross-chain interaction process based on relay architecture

In the request state phase:

- The user initiates a cross-chain transaction request to the source parallel chain, and the source parallel chain protocol stack builds a complete cross-chain request message in the execution order of the application layer, transaction layer, transport layer, verification layer, and forwarding layer and stores it on the chain;
- The source gateway monitors the cross-chain request message on the source parallel chain. This request is processed by the forwarding layer, verification layer, transport layer, etc., and finally forwarded to the interchain by the forwarding layer in the form of a transaction;
- The destination parallel chain protocol stack parses and processes the cross-chain request message in the execution order of the forwarding layer, verification layer, transport layer, transaction layer, and application layer. Each layer writes the execution result into the cross-chain response message and finally stores the cross-chain response message on the chain;
- The destination gateway monitors the cross-chain request message on the interchain, processes it through the forwarding layer, verification layer, transport

layer, etc., and finally forwards it to the destination parallel chain in the form of a transaction by the forwarding layer;

- The destination parallel chain protocol stack parses and processes the cross-chain request message in the execution order of the forwarding layer, verification layer, transport layer, transaction layer, and application layer. Each layer writes the execution result into the cross-chain response message and finally stores the cross-chain response message on the chain, finally forming a valid cross-chain transaction. This process ensures that the cross-chain interaction request initiated from the source parallel chain can be safely and reliably transmitted to the destination parallel chain, and the destination parallel chain can respond based on the content of the request.

In the response state phase:

- It starts with the destination gateway monitoring the cross-chain response message on the destination parallel chain, processing it through the forwarding layer, verification layer, transport layer, etc., and forwarding it to the interchain in the form of a transaction;
- The interchain protocol stack parses and verifies the cross-chain request message in the execution order of the forwarding layer and the verification layer, finally stores it on the chain, and waits for the source gateway to monitor;
- The source gateway monitors the cross-chain request message on the interchain, processes it through the forwarding layer, verification layer, transport layer, etc., and finally forwards it to the source parallel chain in the form of a transaction by the forwarding layer;
- The source parallel chain protocol stack parses and processes the cross-chain response message in the execution order of the forwarding layer, verification layer, transport layer, transaction layer, and application layer. Each layer performs the next operation based on the response result. After all operations are completed, the cross-chain request message will eventually be deleted.

The process of the response state accurately and reliably transmits the response results of the destination parallel chain back to the source parallel chain, and the source parallel chain then performs subsequent operations based on these response results.

The relay-based cross-chain architecture establishes an efficient and secure cross-chain interaction mechanism among source parallel chains, destination parallel chains, source gateways, destination gateways, and interchain by reasonably deploying various levels of the cross-chain interaction protocol stack. This deployment mode ensures the smooth progress of cross-chain interactions and satisfies the technical requirements of different types of blockchains to varying degrees.



### 4.3.2 Visual Network Node Monitoring System

To establish a visual network node data monitoring system, it is necessary to install the corresponding versions of Prometheus, Alertmanager, and Grafana on the monitoring network server, and to install the appropriate version of Node Exporter on the monitored network node servers.

#### 4.3.2.1 Configure Prometheus

The main configuration is in the `prometheus.yml` file. As shown in the code example in Fig. 4.5, the primary task is to add jobs for cross-chain network nodes, specifically the `job_name`. The IP addresses and port numbers of the nodes need to be mapped one by one, with 9090 and 9100 being the default port numbers for Prometheus and Node Exporter, respectively. Here, `node-1` and `node-2` are the network server nodes.

After the configuration is complete, restart the server. In a browser, access the Prometheus UI at [http://monitoring\\_server\\_ip:9090](http://monitoring_server_ip:9090). If the connection is successful, we will be able to retrieve relevant data resource information from the network server nodes. By clicking on the Status/Targets interface, we can see information

```
# A scrape configuration containing exactly one endpoint to
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any
  # from this config.
  - job_name: "prometheus"
    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.
    static_configs:
      - targets: ["0.0.0.0:9090"]

  - job_name: "node-1"
    static_configs:
      - targets: ["123.249.20.xx:9100"]

  - job_name: "node-2"
    static_configs:
      - targets: ["1.92.78.xx:9100"]
```

Fig. 4.5 Configuration of the `prometheus.yml` file

including the IP addresses of the network node servers, the access endpoints, connection status, and the labels configured in `prometheus.yml`, among other details. If the connection fails, the Error section will indicate the reason for the connection error.

#### 4.3.2.2 Configure Grafana and set up an email alerting system

Access [http://monitoring\\_server\\_ip:3000](http://monitoring_server_ip:3000) to enter the main interface of Grafana. By default, we can log in using the credentials `admin/admin`. By adding a data source (this study selects data from Prometheus) and creating a dashboard, we can obtain a visual resource interface for the monitored servers. From this visual interface, we can clearly and intuitively see the IP addresses of the network server nodes and resource usage values, such as CPU usage, memory usage, and disk usage. You can also configure line charts, to reflect the changes in resource utilization of the servers during the monitoring period. When anomalies occur in the network nodes, we can trace back to the specific time point when the abnormality happened.

Create and configure the `email.tmp` file in the corresponding path of the installed Alertmanager folder. This file serves as the alert email template. You need to define an external call name, such as `email.html`, and then set the format and content of the email as required. Here, the alerting program is set to `prometheus_alert`, indicating that the alert originates from this program; the alerting machine is the IP address of the detected anomalous network node; the alert time is when the cross-chain network node anomaly alert email is sent. The alert level, alert name, alert subject, and alert details are all customized during the subsequent alert rule setup. For example, the alert level can be set to “warning,” the alert name to “CPU usage,” the alert subject to “Usage too high,” and the alert details will include not only the predefined anomaly threshold but also the current specific usage rate.

Next, we need to configure the `alertmanager.yml` file, which is generally divided into four key sections: sender email information, alert email sending rules, recipient email information, and alert suppression rules.

- Set up the sender email information: This needs to be configured in the global settings section. If no alert notifications are received for a certain period, Alertmanager will change the previously marked alert status to resolved. You can use `resolve_timeout` to set the duration of this waiting period. Find the corresponding SMTP port on the email service provider’s website and set it using `smtp_smarthost`. The `smtp_from` parameter represents the email address associated with the previous `smtp` parameter. Then, set the corresponding email username and email authorization code (password); note that this password is not for logging into the email account. Finally, add the template for sending alert emails, following the path of the configured `email.tmp` file strictly as described earlier.

- Set up the alert email sending rules: Configure the grouping basis for alert emails using the `group_by` label set to the alert name `alertname`. This means that if different cross-chain network nodes generate the same alert information, they will be sent in the same email to the recipient. Therefore, we also need to set a `group_wait` time for the system to wait until the grouping is complete before sending. To prevent the email from being flooded with repeated alert emails and missing the latest alert information, we need to set the sending interval for the same group of alert emails using `group_interval`. Since the system monitors the health status of cross-chain network nodes 24 hours a day, we should set the interval for sending the next alert email using `repeat_interval`. Finally, configure the alerting method using the sender's information in receiver, which can be email, WeChat, etc.
- Set up the recipient email information: First, we need to set a name for the alert source, which can be customized based on the sender's information. If alerts are sent via email, we must specify the recipient's email address and bind the alert template.
- Set up alert suppression rules: The purpose of configuring alert suppression rules is to reduce alert convergence and minimize the number of alerts, focusing on sending only critical alert information to ensure that significant anomalies can be addressed in a timely manner. If the severity level of the source alert is critical and the severity level of the target alert is warning, with the same `alertname`, `dev`, and instance label values, then the target alert will be suppressed and not sent, thereby reducing alert redundancy. This ensures that only critical alert information is transmitted.

Create a `rules.yml` file in the Prometheus folder to set up the rules for anomaly alerts. You need to specify the group name `name`, the alert name for the alerting rule `alert`, the resource calculation expression `expr`, the duration for which the anomaly must persist before triggering the alert, the severity level `severity`, and the alert description annotations. It is particularly important to note that the job in the code must correspond to the node names configured in `prometheus.yml`, matching one to one.

Taking the alert rules for memory usage, node downtime, CPU usage, and disk usage as examples, as shown in Table 4.1, it is assumed that the normal resource utilization for cross-chain network nodes is below 60%. If the system cannot connect to a node for more than 5 minutes, it indicates that the node has gone down. The

**Table 4.1** Example of alert rules table

| Alert rule name | Monitored resource information | Resource anomaly threshold        | Duration before alert |
|-----------------|--------------------------------|-----------------------------------|-----------------------|
| Memory usage    | Memory usage status            | Greater than 60%                  | 15s                   |
| Node down       | Node downtime status           | No response for over five minutes | 10s                   |
| CPU usage       | CPU usage status               | Greater than 60%                  | 15s                   |
| Disk usage      | Disk usage status              | Greater than 60%                  | 15s                   |

resource calculation expressions monitored in the example of alert rules are as follows:

(a) Memory Usage:

$$\text{round}(100 - \text{node\_memory\_MemAvailable\_bytes} / \text{node\_memory\_MemTotal\_bytes} * 100) > 60$$

(b) Node-Down:

$$\text{up}\{\text{job} = \text{"node"} - 1\} == 0$$

(c) CPU Usage:

$$\text{ceil}(100 - \text{sumby}(\text{instance})(\text{increase}(\text{node\_cpu\_seconds\_total}\{\text{job} = \text{"node"} - 1, \text{mode} = \text{"idle"}\}[5m])) / \text{sumby}(\text{instance})(\text{increase}(\text{node\_cpu\_seconds\_total}\{\text{job} = \text{"node"} - 1\}[5m])) * 100) > 60$$

(d) Disk Usage:

$$\text{round}((1 - (\text{node\_filesystem\_avail\_bytes}\{\text{fstype} = \text{"ext3|ext4|xfs|nfs"}, \text{job} = \text{"node"} - 1\} / \text{node\_filesystem\_size\_bytes}\{\text{fstype} = \text{"ext3|ext4|xfs|nfs"}, \text{job} = \text{"node"} - 1\}))) * 100) > 60$$

Finally, reconfigure the prometheus.yml file to include the Alertmanager section, as shown in the code in Fig. 4.6. The additional configuration includes three parts: the alert manager, the path to the alert rule file, and the target addresses and ports for data scraping. The default port number for Alertmanager is 9093.

## 4.4 Performance Evaluation and Analysis

The entire experiment is deployed on the cloud server, using the configuration versions shown in the Table 4.2.

After starting the cross-chain program, wait for it to run and for the system to confirm whether the anomalous situation continues to trigger the alert rules. After the preset waiting time, we can check the Prometheus interface to see if any alert rules have been triggered, indicating whether any network nodes are in an abnormal state, as shown in Fig. 4.7. Here, Node-1Down and Node-2Down are deliberately set to be down to facilitate testing the accuracy of anomaly detection, as the system cannot monitor the resource status of nodes that are down.

```
# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets: ["127.0.0.1:9093"]

# Load rules once and periodically evaluate them according to the global '
  evaluation_interval'.
rule_files: #Location of the alert rule configuration file
  - "/root/prometheus/rules/rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped
  # from this config.
  - job_name: "prometheus"
    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.
    static_configs:
      - targets: ["0.0.0.0:9090"]
  - job_name: "alertmanager"
    static_configs:
      - targets: ["127.0.0.1:9093"]
```

Fig. 4.6 Configuration of the prometheus.yml file

Table 4.2 Software and hardware configurations of experiments

| Configuration | Specific parameter information |
|---------------|--------------------------------|
| CentOSLinux   | 7.5.1804 (Core)                |
| ChainMaker    | v2.3.1                         |
| geth          | v1.10.25                       |
| solc          | v0.8.7                         |
| Prometheus    | 2.54.1.linux – amd64           |
| Grafana       | 10.4.0 – 1.x86_64              |
| Nodeexporter  | 1.6.1.linux – amd64            |
| Alertmanager  | 0.27.0.linux – amd64           |

As shown in the given figure, the two pre-set down nodes can be detected, and their specific IP addresses, along with the details of the triggered alert rules, are displayed, as illustrated in Fig. 4.8. Other predefined alert rules will also trigger as specified and will similarly display the IP addresses of the nodes that experienced anomalies, along with the details of the triggered rules.

Meanwhile, the email alert system setup in Sect. 4.3.2.2 is also functioning as intended. It can be observed that the email alert system is delivering alert emails to the recipient’s inbox in a timely and quantitative manner, according to the specified schedule and predefined content, as illustrated in Fig. 4.9.

By opening the email to check the specific content, as shown in Fig. 4.10, we can see that the alert system displays the specific alert level, name, node IP address, alert

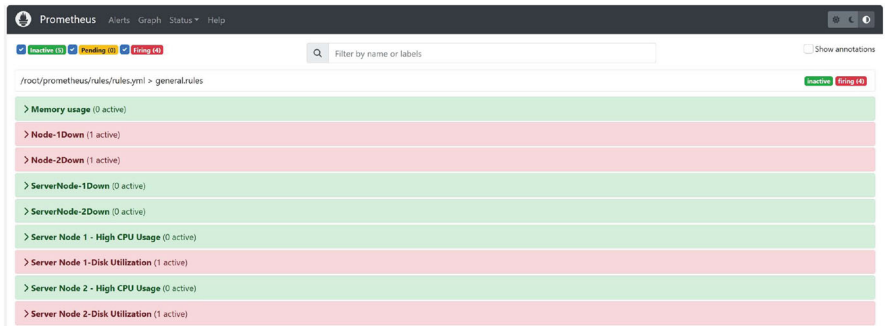


Fig. 4.7 Overall status of alert rule triggers



Fig. 4.8 Trigger status of node downtime alert rules

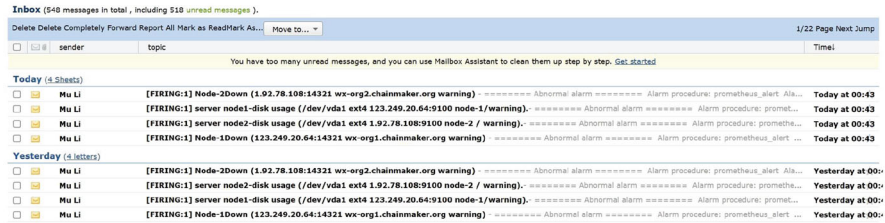


Fig. 4.9 Email alert status

=====  
Abnormal alarm =====  
Alarm Procedure: prometheus\_alert  
Alarm Level: Warning level  
Alarm Name: Server Node 1-Disk Utilization  
Alarm machine: 123.249.20.64:9100 /dev/vda1  
Alarm subject: 123.249.20.64:9100/ The partition usage is too high  
Alarm Details: 123.249.20.64:9100/ partition usage greater than 60% (current value: 100%)  
Alert time: 2024-07-17 10:12:09  
===== END =====

Fig. 4.10 Email alert information

reason, exact usage rate, and the time the alert was triggered, according to the alert email template.

Based on the aforementioned test results, it can be concluded that this mechanism is able to effectively identify anomalies in the network nodes of the cross-chain system according to the established rules.

## 4.5 Summary

We begin by addressing the security challenges faced by current cross-chain systems and the management requirements for cross-chain network nodes. It extensively explores the development process, theoretical frameworks, and technical foundations and then drafts a preliminary design proposal. After analyzing and researching existing methods for anomaly detection in blockchain network nodes both domestically and internationally, we provide a detailed introduction to the relevant theories and technologies that underpin this research as preparatory knowledge. It then presents a comprehensive design scheme for detecting anomalies in network nodes specific to cross-chain systems, detailing the architecture of the cross-chain system used in this study, the functions of the cross-chain protocol stack, and the process of cross-chain interactions. Finally, the design scheme is implemented and tested, demonstrating its effectiveness and feasibility.

## References

- Akoglu L, McGlohon M, Faloutsos C (2010) oddball: Spotting anomalies in weighted graphs. In: Zaki MJ, Yu JX, Ravindran B, Pudi V (eds) *Advances in Knowledge Discovery and Data Mining*, 14th Pacific-Asia Conference, PAKDD 2010, Hyderabad, India, June 21–24, 2010. Proceedings. Part II. Springer, Lecture Notes in Computer Science, vol 6119, pp 410–421
- Bandyopadhyay S, Lokesh N, Murty MN (2019) Outlier aware network embedding for attributed networks. In: *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27–February 1, 2019*. AAAI Press, pp 12–19
- Beijing Microchip Research Institute et al. (2023) Introduction to the Chang'an Chain Underlying Technology Platform. <https://docs.chainmaker.org.cn/v2.3.6/html/quickstart/changan-chain-introduction.html>, Accessed on 2024-11-27
- Cai XQ, Deng Y, Zhang L (2021) The principle and core technology of blockchain. *Chinese J Comput* 44(1):84–131
- Chuannian S (2022) Research review on blockchain cross-chain technology. *J Internet Things* 6(4):183. <https://doi.org/10.11959/j.issn.2096-3750.2022.00301>. <https://www.infocomm-journal.com/wlw/CN/abstract/article172989.shtml>
- Ding Q, Katenka N, Barford P, Kolaczyk ED, Crovella M (2012) Intrusion as (anti)social communication: characterization and detection. In: Yang Q, Agarwal D, Pei J (eds) *The 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '12, Beijing, China, August 12–16, 2012*. ACM, pp 886–894

- Prometheus (2017) Prometheus introduction. Cloud Native Computing Foundation, available at: <https://prometheus.io/docs/introduction/overview/>
- Qian P, Liu Z, He Q, Huang B, Tian D, Wang X (2022) Smart contract vulnerability detection technique: a survey. *J Software* 33(8):3059. <https://doi.org/10.13328/j.cnki.jos.006375>
- Wu J, Yuan Q, Lin D, You W, Chen W, Chen C, Zheng Z (2022) Who are the phishers? phishing scam detection on ethereum via network embedding. *IEEE Trans Syst Man Cybern Syst* 52(2):1156–1166
- Xuehai H, Yang W, Fangyu L (2020) Research review on blockchain security supervision technology. *China Sci Found* 34(1):18–24
- Yuan G (2022) Research on food safety supervision system based on computer big data visualization analysis system. In: 5th IEEE International Conference on Information Systems and Computer Aided Education, ICISCAE 2022, Dalian, China, September 23–25, 2022. IEEE, pp 941–945
- Yuan Y, Wang F (2019) Blockchain theory and methods. Tsinghua University Press, Beijing
- Zheng L, Helu X, Li M, Lu H (2019) Automatic discovery mechanism of blockchain nodes based on the kadmlia algorithm. In: Sun X, Pan Z, Bertino E (eds) Artificial Intelligence and Security - 5th International Conference, ICAIS 2019, New York, NY, USA, July 26–28, 2019, Proceedings, Part I. Springer, Lecture Notes in Computer Science, vol 11632, pp 605–616



# Chapter 5

## Cross-Chain Traffic Anomaly Detection



### 5.1 Background and Motivation

Besides the node detection, traffic analysis serves as one of the critical tools for identifying anomalous behavior in the network. This is achieved by monitoring data flows and identifying patterns that deviate from established norms. Traffic-based anomaly detection (Chandola et al. 2009) is effective due to its ability to capture real-time network activity and spot irregularities, making it a valuable tool for detecting cyber threats, such as DDoS attacks, malware propagation, and unauthorized access attempts (Signorini et al. 2018; Baqer et al. 2016). In traditional single-chain or standard network environments, traffic detection methods typically involve approaches like signature-based detection, statistical anomaly detection, machine learning-based methods, and behavioral analytics (The UCI KDD Archive 1999). However, in the context of cross-chain systems, where multiple blockchains interact, traffic-based anomaly detection presents unique challenges. The heterogeneous and decentralized nature of cross-chain interactions requires the development of new approaches that can effectively analyze and identify anomalies across interconnected blockchain ecosystems, as traditional network monitoring methods may not directly apply (Moustafa and Slay 2015). Understanding existing techniques in single-chain networks is essential for adapting them to the more complex cross-chain environment.

Current traffic detection techniques broadly focus on DDoS attacks and eclipse attacks. Due to the differences in the composition of normal traffic and DDoS attack traffic in the blockchain network layer regarding dataset distribution and traffic characteristics, traffic in the blockchain network layer is often collected and processed, transforming it into datasets for classification or anomaly detection studies. Currently, DDoS attack detection methods in the blockchain network layer can be categorized into two types based on different detection approaches. The first type consists of statistical analysis methods based on node traffic (Jamader et al. 2019), which classify traffic datasets in blockchain network routers over set

time intervals, using network congestion levels and statistical indicators as the basis for identifying DDoS attack traffic. However, this method has drawbacks: attackers can exploit the blockchain consensus mechanism that allows redundant traffic propagation, disguising DDoS attack traffic as redundant P2P traffic and using flooding mechanisms. Since such attack traffic often employs feature evasion strategies, the core statistical features of anomalous traffic may not be significantly different, leading to insufficient accuracy in statistical detection methods. Additionally, the blockchain consensus mechanism processes traffic load information sent by all miner nodes indiscriminately, allowing attackers to encapsulate and propagate rumor-type DDoS attack traffic. The characteristics of such traffic are highly similar to Flash Crowd traffic, making it easy for statistical detection methods to misidentify Flash Crowd traffic as DDoS attack anomalies, resulting in a high false-positive rate. The second type involves detection methods based on machine learning (Kim et al. 2018; Hu et al. 2022), which capture traffic data at the end nodes of the blockchain network and use machine learning algorithms to mine and extract core features of DDoS attack traffic, inputting the traffic to be detected into a trained machine learning model for anomaly detection to identify whether the input traffic samples contain DDoS attack traffic. However, this method also has some shortcomings. First, the coexistence of various consensus mechanisms (such as PoW, PoS, DPoS, and BFT) in the blockchain network layer leads to a mixture of underlying traffic protocols (such as RLPx, GHOST, Gossip, DEVp2p, and Gnutella), and the significant differences in encapsulation formats across different platforms (such as Ethereum, Hyperledger Fabric, and IOTA) increase the complexity of the traffic data's nonlinear feature dimensions and model detection outputs, resulting in weak generalization capability of existing blockchain layer DDoS traffic detection models. Moreover, the characteristics of rumor-type attack traffic in the blockchain network layer are highly similar to normal network traffic, leading to a weak perceptual ability of existing machine learning models regarding the core features of such dynamically changing mixed DDoS attack traffic, causing low detection accuracy in current methods.

As for the Eclipse attack, its current anomalous traffic detection methods can be divided into two categories. The first type is based on routing topology awareness (Rottondi et al. 2016; Fantacci et al. 2009). This method analyzes and models the routing structure based on parameters such as the topology of the blockchain network layer and the forwarding states of key nodes' routing tables. By detecting changes in routing states and analyzing blockchain network routing messages and traffic information, it determines whether an Eclipse attack has occurred. This detection method typically has high reliability in detecting Eclipse attacks and provides significant insights into structural vulnerabilities in the blockchain network layer. However, the complexity of these detection models often results in weak generalization and scalability, making real-time detection capabilities insufficient to meet the security demands of actual blockchain network layers. Furthermore, this approach has limited perceptual capability for dynamically non-fixed Eclipse attack paths, struggling to adapt to the changing traffic environment of the blockchain network. The second type is based on link traffic state analysis. Since Eclipse attack

traffic aims to disrupt the routing structure, attackers need to send a large volume of traffic to achieve routing suppression, resulting in specific distribution patterns for Eclipse attack traffic (Zheng et al. 2021). This method captures blockchain network layer traffic, analyzes real-time traffic states, and builds statistical or machine learning models to extract core behavioral patterns of Eclipse attacks, allowing for the determination of whether an Eclipse attack is occurring. This approach has strong real-time detection capabilities and model generalization, enabling dynamic monitoring and identification of Eclipse attack traffic based on changes in traffic features. However, considering that Eclipse attackers may opt not to directly attack the blockchain subnet, they often employ traffic disguises to send erroneous routing requests to the blockchain subnet. This leads to a weak perceptual capability of existing detection methods for dynamic multi-path Eclipse attack traffic, revealing deficiencies in their ability to dynamically perceive such attack traffic characteristics. Additionally, the similarity in behavioral characteristics between Eclipse attack traffic and normal traffic is quite high, making the core distinguishing features less apparent. Consequently, the detection accuracy and performance of existing methods for such attack traffic still require improvement, and there remains a significant gap between anomaly detection capabilities and the actual security needs of the blockchain network environment.

When existing DDoS and Eclipse attack detection technologies are applied directly to cross-chain environments, several challenges arise due to the differences between single-chain and cross-chain networks. Cross-chain environments involve multiple blockchains with distinct consensus mechanisms (e.g., PoW, PoS, DPoS, BFT) and protocol layers (e.g., Ethereum's RLPx, Hyperledger's GRPC, IOTA's Tangle), leading to difficulties in adapting detection models to handle the heterogeneous encapsulation formats and traffic protocols, which reduces detection accuracy. The increased complexity of traffic patterns in cross-chain networks, where multiple blockchains interact, further complicates the distinction between normal and malicious traffic. For instance, rumor-type DDoS or multi-path Eclipse attacks can easily be masked within legitimate cross-chain communication. Moreover, cross-chain networks rely on efficient interoperability mechanisms for data exchange, such as intermediary chains and relayers, which introduce new attack vectors that are not accounted for in detection models designed for single-chain environments. The distributed nature of cross-chain networks also allows attackers to exploit multiple vulnerabilities simultaneously, overwhelming relayers or gateway nodes in DDoS attacks or disguising Eclipse attack traffic across multiple chains. These dynamically changing attack vectors are difficult for existing detection models to track. Finally, the higher transaction volumes and complexity of cross-chain networks exacerbate scalability issues, limiting the real-time detection capabilities of current statistical and machine learning-based methods, which fail to meet the security demands of such environments.

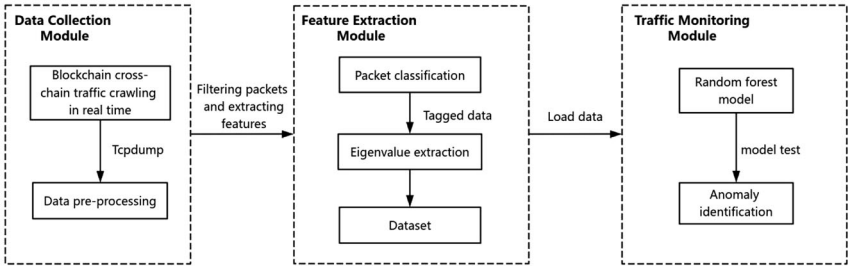
The motivation for cross-chain traffic detection research stems from the need to address several limitations in current DDoS and Eclipse attack detection methods, which are typically tailored to specific blockchain platforms and protocols. To enhance security in cross-chain environments, it is crucial to develop detection

models that can generalize across multiple blockchains by learning the core features of different consensus mechanisms and traffic encapsulation formats. Cross-chain networks also introduce more complex and dynamic traffic patterns, necessitating new methods capable of analyzing multi-layered traffic flows and accurately detecting anomalies in both legitimate and malicious cross-chain interactions. As these networks grow, scalable and real-time detection solutions become critical, prompting research into lightweight, efficient models that can operate across multiple blockchains without sacrificing performance. Moreover, cross-chain environments bring new attack vectors, such as vulnerabilities in intermediary nodes and relayers, which current detection methods fail to address. Research in cross-chain traffic detection aims to close this gap by protecting the entire ecosystem from emerging threats. Additionally, the high similarity between legitimate cross-chain traffic (e.g., Flash Crowds) and attack traffic (e.g., rumor-type DDoS or Eclipse attacks) motivates the development of refined feature extraction techniques to distinguish between them more effectively, reducing false positives. Addressing these challenges through targeted research will significantly improve the security of cross-chain environments, ensuring robust protection against various attack types.

## 5.2 Methodology Overview of Cross-Chain Traffic Detection Model

Among various types of machine learning algorithms, decision tree is a classification model based on tree structure, by recursively partitioning the dataset in order to construct a tree model, which is easy to understand, but is susceptible to the influence of data noise and overfitting problem. Random forest is an integrated learning algorithm based on multiple decision trees, which improves the prediction accuracy and robustness by integrating multiple tree models, and can effectively reduce the risk of overfitting. SVM is a binary classification model, which realizes classification by finding the optimal hyperplane in the high-dimensional space, and it has strong generalization ability, but it has a large amount of computational effort in dealing with large-scale datasets. KNN is an instance-based learning algorithm that classifies new samples by measuring the distance between them and training samples, which is simple and intuitive but requires high computational resources. Simple Bayesian algorithm is a classification algorithm based on Bayes' theorem and the assumption of feature independence, which is suitable for large-scale datasets, but may be limited by the assumption of feature independence and lead to performance degradation.

For the goal of this chapter, which aims to quickly and accurately classify unknown traffic on the blockchain, the random forest algorithm has certain advantages. Due to its high accuracy and robustness, the Random Forest algorithm effectively handles large-scale, high-dimensional datasets and demonstrates a strong ability to manage missing values and outliers. As a result, it is suitable for various



**Fig. 5.1** Traffic detection model based on random forest algorithm

data types and complex problems. Therefore, this chapter designs and builds a blockchain traffic anomaly detection model based on the random forest algorithm, as shown in Fig. 5.1.

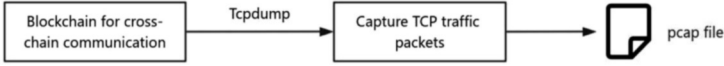
The model is mainly composed of three core components: data acquisition module, feature extraction module, and detection module. First, the data acquisition module is responsible for capturing various traffic data generated in the cross-chain environment in real time. This process not only involves continuous monitoring of network traffic but also preprocessing of the data to convert the originally chaotic and unorganized traffic packets into a collection of data with clear attributes and structure for subsequent analysis.

After the data preprocessing is completed, the feature extraction module begins its work. The task of this module is to classify the traffic on the organized data collection and extract the specific features of each traffic packet. These features may include information such as packet size, transmission rate, and protocol type, which are essential for understanding the nature and behavior of the traffic. By effectively categorizing traffic and extracting features, we can construct an inspection dataset that contains rich information.

Finally, this detection dataset will be imported into the random forest model. Random forest is a powerful machine learning algorithm to improve the accuracy and reliability of traffic anomaly detection by constructing multiple decision trees for voting. At this stage, the model will analyze the traffic using the extracted features to identify the presence of anomalous activity. The tight integration of this series of steps ensures that the model is efficient and accurate in monitoring and identifying potential threats in real time.

### 5.3 Detailed Design of Cross-Chain Traffic Detection Model

The cross-chain environment of this design has been presented in Sect. 4.3.1, and the specific details of the design are as follows.



**Fig. 5.2** Data collection module

### 5.3.1 Data Collection Module

In order to analyze traffic in a blockchain cross-chain environment, it is necessary to collect various types of network communication traffic data in a real cross-chain environment. Based on the previously selected cross-chain protocol stack, a large amount of network communication is carried out in this environment, and all traffic data packets passing through a specific gateway are collected. In this process, this chapter collects abnormal communication traffic by simulating DoS attacks. The purpose of a DoS attack is to overload the target server, service, or network resource by sending a large number of requests or data packets, thereby causing its performance to decline, response to slow down, or even fail to operate normally. Therefore, this chapter simulates a DoS attack by sending a large number of requests to occupy server resources, such as memory, CPU, or causing database connections to be exhausted. The data collection module mainly consists of real-time capture of blockchain cross-chain traffic and preprocessing of traffic packets. The module's workflow is shown in Fig. 5.2.

In the data collection module, this chapter uses the Tcpdump tool to capture traffic data packets. In order to facilitate subsequent feature extraction and label classification, this chapter adopts a strategy: each time the blockchain cross-chain protocol stack communicates, a traffic data packet is captured and saved as a pcap file. This is beneficial for the subsequent feature extraction module to carry out more in-depth analysis and operations.

### 5.3.2 Feature Extraction Module

To distinguish between normal network traffic and potentially abnormal traffic, multi-level filtering processing of the data is necessary to ensure that the information in the dataset is accurate and meaningful. Following is the filtering process applied in normal and abnormal traffic data scenarios:

1. Filtering non-TCP packets: Exclude all non-TCP protocol packets, as mainstream blockchain protocols such as Bitcoin and Ethereum mainly use TCP to establish connections between nodes and for data transmission.
2. Filtering unestablished data streams: Exclude those data streams that have not completed the connection, i.e., those packets sent only unilaterally.

3. Filtering data streams that do not match node features: Delete data streams where the source IP address and port number do not match the predefined node list, in order to accurately identify the required traffic data packets.

After filtering, basic traffic features can be extracted from the blockchain cross-chain traffic dataset. However, blockchain abnormal communication data have characteristics such as high traffic, high frequency, and a wide distribution of attack sources. Moreover, the data packets of attacks such as DDoS seem no different from ordinary network traffic. Therefore, based on the principles and common patterns of various blockchain attacks, this chapter selects several statistical features that can distinguish between normal and abnormal cross-chain traffic, including source IP address and destination IP address, source port number and destination port number, timestamp, network protocol, packet size, etc.

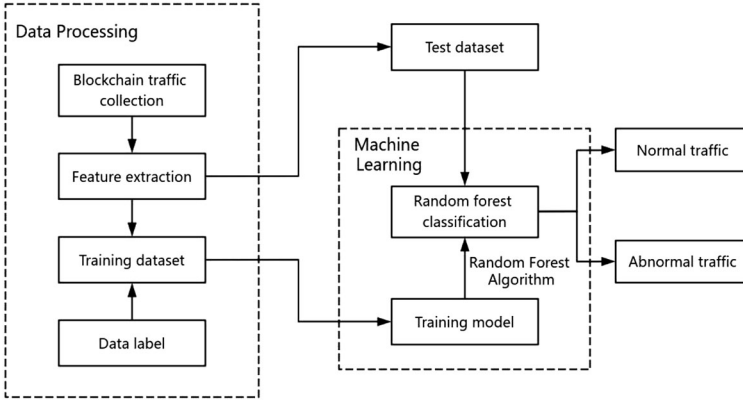
To facilitate the extraction and analysis of the traffic features in the data packets collected in Sect. 5.3.2, this chapter uses the Wireshark tool for further operations on the data packets. As a powerful network protocol analysis tool, Wireshark provides various functions to extract and analyze the features of network traffic, helping to gain a deep understanding of the components of network traffic, and thus troubleshoot network faults, optimize performance, or conduct security analysis, which is convenient for subsequent training of the traffic monitoring model.

Therefore, the feature extraction module, after capturing a sufficient amount of traffic data packets, parses the completed pcap files for extraction. Since the data collection module stores a pcap file each time a cross-chain communication is conducted, this effectively prevents the file size from increasing over time and reducing the efficiency of selecting specific data packets; and this method facilitates the labeling of abnormal communication traffic packets with abnormal labels for normal and abnormal packet classification. After classification, Wireshark is used to extract the basic traffic features of each traffic data packet and save them as CSV files.

### 5.3.3 Traffic Monitoring Module

To effectively detect abnormal communication traffic, this chapter implements advanced classification detection methods grounded in machine learning techniques. The first step involves the collection of a substantial volume of traffic data packets generated from cross-chain communication within blockchain environments. These data serve as a rich source for extracting essential traffic features that are critical for constructing a robust dataset.

Once the necessary features are identified, the dataset is meticulously divided into two distinct subsets: a training dataset and a testing dataset. The training dataset comprises feature values paired with their corresponding labels, which facilitate the training of various machine learning models. During this phase, different algorithms are employed to ensure a comprehensive analysis and to optimize the model's ability to recognize patterns indicative of normal and abnormal traffic.



**Fig. 5.3** Training process of random forest module

After the models are trained, they undergo rigorous testing using the unlabeled testing dataset. This evaluation process aims to assess the classification performance of the trained models in detecting abnormal traffic patterns. By analyzing the models' accuracy, precision, and recall, the study aims to identify the most effective machine learning approaches for safeguarding blockchain communications against potential threats and anomalies. Through this systematic methodology, the research seeks to contribute valuable insights into enhancing security measures in blockchain networks.

Using the random forest model as an example, Fig. 5.3 illustrates the training process of machine learning models.

The specific process of model construction for the Random Forest algorithm is roughly as follows:

1. Data sampling: random forest uses self-service sampling method to generate the training dataset for each decision tree. The training dataset for each tree is randomly selected from the original dataset, allowing sample repetition.
2. Feature random selection: when constructing each decision tree, Random Forest randomly selects a subset of features at each node split instead of using all features. This random selection of features increases the diversity of the model and reduces the risk of overfitting.
3. Constructing decision trees: on the basis of the generated training dataset and feature subsets, each decision tree is constructed. For classification tasks, the decision tree splits the data according to the features until a specific stopping condition is met; for regression tasks, the leaf nodes of the tree return the predicted values.
4. Integrated Prediction: when the random forest needs to make a prediction, new input data will be passed into each decision tree, and each tree will give a prediction. In the classification task, the random forest determines the final classification result by "majority voting," i.e., selecting the classification with



the highest number of occurrences. In regression tasks, the final result is usually the average prediction of all decision trees.

## 5.4 Performance Evaluation and Analysis

### 5.4.1 *Indicators for Model Evaluation*

In this experiment, normal blockchain cross-chain communication traffic is taken as normal samples, and DoS attack traffic is taken as abnormal samples, and Accuracy, Precision, Recall, and F-measure are used as the performance indexes for evaluating the anomaly detection model. The specific calculations are as follows:

1. Accuracy is the percentage of correctly predicted results out of the total sample. While accuracy is a measure of the overall performance of a model, the validity of accuracy can be severely limited in the presence of an unbalanced sample distribution. For example, consider a sample with 95% positive samples and 5% negative samples. In this case, if a model predicts all samples as positive, it will have an accuracy rate as high as 95%. However, this high accuracy is misleading because it ignores the model's ability to predict a small number of categories.
2. Precision is used to measure the proportion of results that are actually positive in a sample predicted to be positive. Thus, the precision rate serves to measure the accuracy of the model's predictions, to help reduce false positives, and to provide a reliable performance metric in scenarios with unbalanced data and high precision requirements.
3. Recall is used to measure the proportion of results that are actually positive in a sample predicted to be positive. Recall measures the model's ability to identify positive samples, ensures that the model has high coverage, reduces the risk of underreporting, and provides an effective performance assessment when dealing with unbalanced data.
4. F-measure is used to measure the proportion of results that are actually positive in a sample predicted to be positive. The F1 score is a composite metric used to measure the performance of machine learning models, especially when dealing with classification tasks. It combines precision and recall to provide a balanced assessment in the form of a reconciled mean. The F1 score is useful in scenarios with unbalanced datasets or where there is a need to weigh precision and recall.

### 5.4.2 *Assessment of Models*

Based on the principles and common patterns of various types of blockchain attacks, we select several statistical features that can distinguish normal and abnormal cross-chain traffic, including source and destination IP addresses, source and destination

**Table 5.1** Evaluation results of multiple machine learning

| Feature                | Description                                                         |
|------------------------|---------------------------------------------------------------------|
| Network protocol       | Types of protocols used for cross-chain communication               |
| Sequence number        | Sequence number that identifies the stream of bytes in a TCP packet |
| Acknowledgement number | Confirm the sequence number of the packet that has been received    |
| Window size            | Window size in TCP messages                                         |
| Frame length           | Total length of the packet on the network                           |
| Timestamp value        | Timestamp value set by the sender when sending TCP messages         |
| Timestamp echo rep     | The receiver sends back to the sender after receiving a TCP message |

**Table 5.2** Evaluation results of multiple machine learning

| Algorithm          | Precision | Recall | F-measure | Accuracy |
|--------------------|-----------|--------|-----------|----------|
| Random forest      | 0.975     | 0.990  | 0.982     | 0.968    |
| SVM                | 0.898     | 0.997  | 0.946     | 0.898    |
| C4.5 Decision tree | 0.970     | 0.985  | 0.977     | 0.958    |
| KNN                | 0.974     | 0.985  | 0.979     | 0.963    |
| Naive bayes        | 0.906     | 0.995  | 0.948     | 0.903    |

port numbers, timestamps, network protocols, packet sizes, and other relevant information, as shown in Table 5.1.

To compare the performance differences of existing machine learning algorithms on this topic, we conduct comparative tests on Random Forest, SVM, C4.5 Decision Tree, KNN, and Naive Bayes algorithms. The evaluation results are shown in Table 5.2 as follows.

Based the aforementioned data, we can compare various indicators of multiple machine learning methods, as shown in Fig. 5.4. As can be seen, although SVM and Naive Bayes perform better on the evaluation indicator of recall rate than Random Forest, considering the accuracy and precision comprehensively, Random Forest is still the best performing model in classification.

### 5.5 Summary

We aim to conduct an in-depth study of anomaly detection methods based on traffic analysis, with the objective of effectively identifying abnormal traffic and potential threats in real-world blockchain cross-chain technology applications. Specifically, we first set up an experimental environment for the cross-chain protocol stack and collected a large amount of traffic data during the blockchain cross-chain process within this environment, including both normal and abnormal traffic. Subsequently, we conducted a detailed analysis of this data and extracted relevant features to



Fig. 5.4 Line charts of multiple machine learning models

better understand the security risks in a cross-chain environment. Building on this foundation, we designed and constructed a traffic detection model based on the Random Forest algorithm to identify abnormal traffic in blockchain cross-chain environments. We provide a detailed overview of the model’s overall framework, including the data collection module, feature extraction module, and anomaly detection module. The results indicate that the Random Forest-based traffic detection model performs exceptionally well in detecting abnormal traffic in blockchain cross-chain environments, offering a reliable method for enhancing the security of blockchain systems.

References

Baquer K, Huang DY, McCoy D, Weaver N (2016) Stressing out: Bitcoin “stress testing”. In: Financial Cryptography and Data Security: FC 2016 International Workshops, BITCOIN, VOTING, and WAHC, Christ Church, Barbados, February 26, 2016, Revised Selected Papers 20. Springer, pp 3–18

Chandola V, Banerjee A, Kumar V (2009) Anomaly detection: A survey. *ACM Comput Surv (CSUR)* 41(3):1–58

Fantacci R, Maccari L, Rosi M, Chisci L, Aiello LM, Milanesio M (2009) Avoiding eclipse attacks on kad/kademlia: an identity based approach. In: 2009 IEEE International Conference on Communications. IEEE, pp 1–5

Hu X, Zhu C, Tong Z, Gao W, Cheng G, Li R, Wu H, Gong J (2022) Identifying ethereum traffic based on an active node library and devp2p features. *Future Gener Comput Syst* 132:162–177

- Jamader AR, Das P, Acharya BR (2019) Bciot: blockchain based ddos prevention architecture for iot. In: 2019 International Conference on Intelligent Computing and Control Systems (ICCS). IEEE, pp 377–382
- Kim SK, Ma Z, Murali S, Mason J, Miller A, Bailey M (2018) Measuring ethereum network peers. In: Proceedings of the Internet Measurement Conference 2018, pp 91–104
- Moustafa N, Slay J (2015) Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). IEEE, pp 1–6
- Rottondi C, Panzeri A, Yagne CT, Verticale G (2016) Detection and mitigation of the eclipse attack in chord overlays. *Int J Comput Sci Eng* 13(2):111–121
- Signorini M, Pontecorvi M, Kanoun W, Di Pietro R (2018) Bad: blockchain anomaly detection. arXiv preprint arXiv:180703833
- The UCI KDD Archive (1999) KDD Cup 1999 Data. <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- Zheng H, Tran T, Arden O (2021) Total eclipse of the enclave: detecting eclipse attacks from inside tees. In: 2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). IEEE, pp 1–5

# Chapter 6

## Cross-Chain Provenance Tracking



### 6.1 Background and Motivation

Provenance tracking technology plays an important role in the blockchain field (Palamara 2016). Due to the untamperable, transparent, and verifiable nature, blockchain can be used for the entire life cycle tracking and ensure the authenticity of its source, ownership, and history. This mechanism is commonly used in scenarios such as supply chain management, digital rights management, luxury goods anti-counterfeiting, and food safety (Dutta et al. 2020; Siddiqui et al. 2020; Wang et al. 2021; de Boissieu et al. 2021). Due to the anonymity of blockchain, some criminals carry out illegal activities such as money laundering and coin cashing through blockchain (Bjelajac and Bajac 2022). Provenance tracking technology provides clues for regulators to track down illegal transactions.

However, in the cross-chain environment, the provenance tracking technology has to face the compatibility challenges. In areas such as supply chain management, if asset tracking involves different blockchains used between different industries, how to achieve data interoperability and unified traceability is an issue that limits its development (Longfei et al. 2023). On the other hand, in the past, the traditional way for criminals to launder money and cash out coins through blockchain systems was through centralized exchanges. Because users would hold their coins in escrow in the exchange's address, most provenance tracking techniques relied on identifying the address associated with the exchange (Meiklejohn et al. 2013). Required by regulatory liability, most centralized exchanges implement strict Know Your Customer/Anti-Money Laundering (KYC/AML) policies (Poskriakov et al. 2020), which means that criminals and even ordinary users risk exposing their true identities when using centralized exchanges (Yousaf et al. 2019). The decentralized cross-chain exchanges thus emerged, such as ShapeShift, Changelly2, and WRDEX. With no centralized operator, these platforms typically do not have strict KYC/AML requirements (Altschuler 2022), meaning users are generally not required to provide personal information. While facilitating cross-chain transactions, criminals also use

them to move coins from one blockchain to another in an attempt to obfuscate the flow of coins (Zhang et al. 2022). As traditional tracking technologies for a single chain or centralized exchange are less effective, cross-chain transactions require specialized tracking technology for regulatory purposes.

Existing tracking technologies vary depending on the type of cryptocurrency. For Bitcoin, the relationship between the transaction address and the mixed currency transaction can be analyzed through taint analysis (Möser et al. 2013). And the operating mechanism of the mixed currency transaction can be analyzed based on the number and amount characteristics of the mixed currency transaction output (Wu et al. 2021). For Zcash, the correlation between the participants of the shielded transaction can be found by observing the number of shielded transaction inputs and outputs (Biryukov and Feher 2019). For Monero, 0-mixin transactions can be used to perform cascade analysis on transactions with more obfuscated inputs (Möser et al. 2017). These methods are only applicable to a certain cryptocurrency and cannot be applied to cross-chain transactions. Chen et al. (2017) restored the relationship between the participants of the mixing transaction by matching the transaction amount, which achieved the provenance tracking of cross-chain transactions for all cryptocurrencies. However, there may be multiple possible transactions in a similar time period, making it hard to determine which one is the actual cross-chain transaction. Our work will increase the consideration of transaction time to achieve a smaller false-positive rate.

The address clustering heuristics have been proposed to address the traceability issue of cryptocurrencies. They have been applied into Bitcoin (Haslhofer et al. 2016; Meiklejohn et al. 2013), Litecoin, and Zcash (Kappos et al. 2018; Zhang et al. 2020), which are all UTXO-based. These methods cannot be applied to cross-chain transactions. Yousaf et al. (2019) used crawlers to collect 2 million cross-ledger transaction flows from 2017 to 2018 on ShapeShift. They also proposed a common-relationship heuristic to observe relationships of individual users and identified transaction patterns of ShapeShift. They tracked cross-ledger money by analyzing particular patterns of behavior. But their clustering scheme can only be applied to the ShapeShift platform and requires real-time monitoring to obtain transaction flows. Zhang et al. (2022) made improvement with the deposit and withdrawal mechanisms of other 19 trading platforms. However, their address clustering algorithm does not distinguish between cryptocurrencies, i.e., it works well for UTXO-based cryptocurrencies but performs poorly for account-based cryptocurrencies such as Ethereum. Friedhelm (Victor 2020a) proposed three address clustering heuristic for Ethereum: deposit address reuse, airdrop multi-participation, and token transfer authorization. However, he did not test the parameters in detail to reduce false positives and did not extend the algorithm to cross-chain and traceability.

Existing clustering and tracking schemes mainly target cross-chain system including a UTXO-based model, which allows a single transaction to have multiple inputs and outputs, but are not applicable to account-based cryptocurrencies such as Ethereum, which instead employs an account model where a regular transaction has one source and one destination account address. In this chapter, we propose a new provenance tracking scheme for cross-chain systems including an account-based model via deposit address reuse clustering heuristic.

## 6.2 Methodology Overview of Cross-Chain Provenance Tracking Scheme

We propose a provenance tracking scheme for cross-chain transactions based on address clustering. The overview of our scheme is shown in Fig. 6.1. The scheme is divided into three main modules: data preparation, address clustering, and cross-chain provenance tracking.

In the data preparation module, we use the dataset of Ethereum transactions. The module includes data acquisition, data cleaning, and data extraction. We clean and organize the data into a format that can be read efficiently. It is worth mentioning that we stored and indexed the transaction data in chunks, which will greatly improve the efficiency of our subsequent testing algorithms and is also meaningful for practical applications.

In the address clustering module, we cluster addresses that may belong to the same user based on the deposit address reuse heuristic. This heuristic can be described as follows: two or more addresses are considered to belong to the same entity if they send currency to the same deposit address controlled by the exchange. We creatively use chunk and timestamp indexing, which allows our algorithm to more intuitively and accurately match the time of transactions.

In the cross-chain provenance tracking module, we trace the transaction coins by means of the tracking coins algorithm. This heuristic can be described as: recovering the relationship between the two parties of a transaction by matching the transaction time and amount of the transaction input with the transaction output. Our algorithm uses the timestamp of the transaction and the amount of the transaction as the basis for judgment, so our algorithm can be applied to all digital currencies and is easily scalable to other historical transaction datasets with different data structures. This may provide support for subsequent research on traceability.

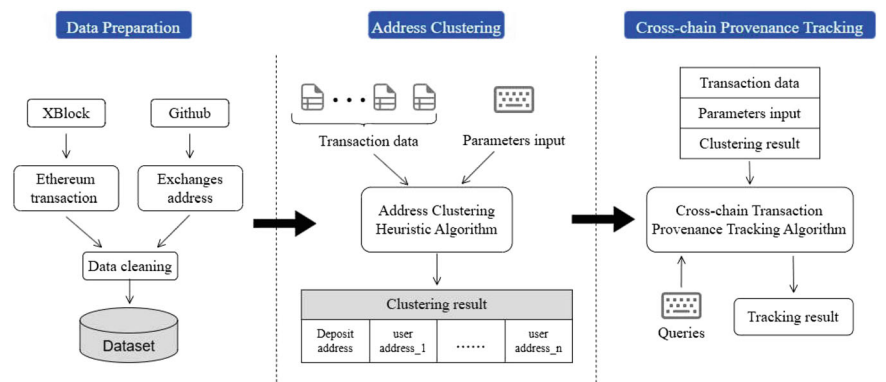


Fig. 6.1 Overview of Cross-chain Provenance Tracking Scheme

### 6.3 Detailed Design of Cross-Chain Provenance Tracking Scheme

#### 6.3.1 Data Preparation Module

This module prepares the necessary data before running the algorithm. The module can be divided into three parts: data acquisition, data cleaning, and data extraction. The detailed processing flowchart is shown in Fig. 6.2.

The data acquisition part is to obtain the data needed for our scheme from the network: Ethereum transaction data, Ethereum block data, and exchange address

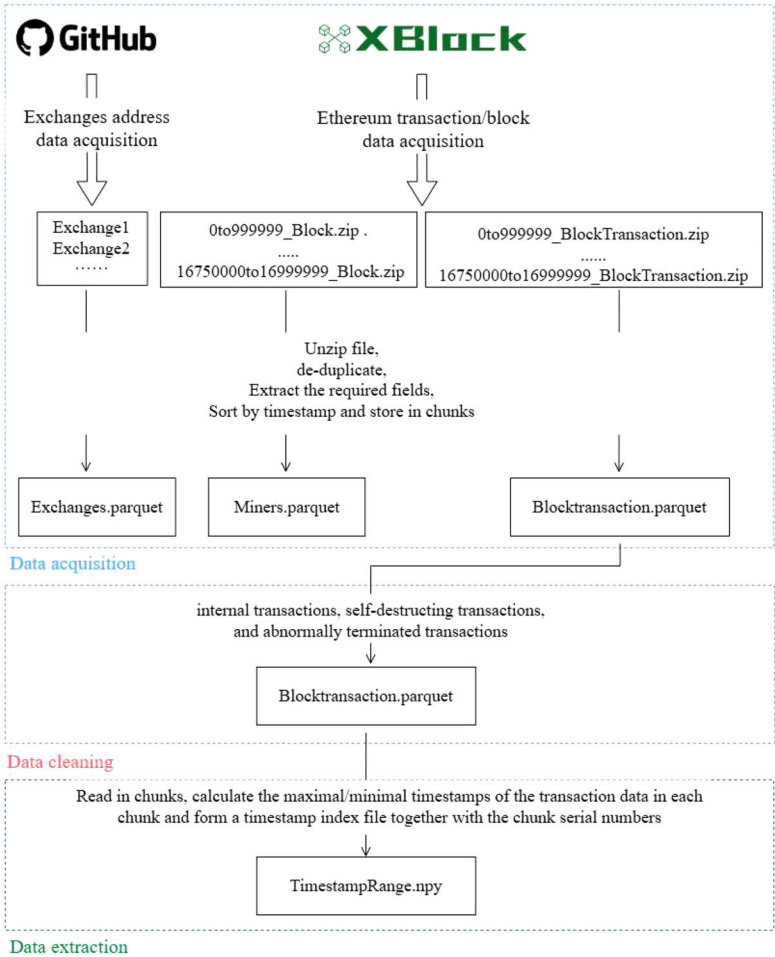


Fig. 6.2 Detailed process design drawing of data preparation module



data. The data cleaning part mainly includes the elimination of invalid data such as internal transactions, self-destructing transactions, and abnormally terminated transactions from the Ethereum transaction data. The data extraction part includes extracting the required fields as well as generating an index of the Ethereum transaction data. We sort and chunk the transaction data in advance based on timestamps and store it in chunks. The range of timestamps of transactions in each block does not intersect. So we can use block number and range of timestamps as an index of the transaction data chunks and store it separately as a file that can be searched quickly. With the data extraction part, we will improve the efficiency of our algorithm operation.

First, we explain the data sources for the data acquisition part:

- The initial Ethereum transaction data and block data are from Zheng et al. (2020). The authors processed a large volume of Ethereum data and produced a well-structured dataset, which is available for download from their website.<sup>1</sup> We download and utilize the transaction data and block data from number 0 to 16,999,999 in that website, which are organized into multiple files, including BlockTransaction.zip and Block.zip.
- The initial exchange address data are from Shamsi et al. (2022), Hall (2020), Victor (2020b), consisting of approximately 332 exchange addresses.

The initial data file format we acquired is CSV file. Since Python's read and write speeds for CSV files are very slow, we need to convert the data to a more lightweight storage format during the data extraction process. We choose a Parquet file as the storage format for the large files of the dataset. Parquet is a columnar storage format, which can be applied with a variety of encoding and compression algorithms, resulting in smaller memory and faster read and write speed of the files.

In the subsequent address clustering and cross-chain provenance tracking module, the algorithms need to perform traversal lookup operations on the Ethereum transaction data several times. However, since the number of Ethereum transaction data reaches 10 million, traversing the lookup will take much time and occupy a huge amount of running memory, which will affect the efficiency of algorithm testing. To this end, we propose a solution in the data processing module: store and read the Ethereum transaction data in chunks, with an estimated block size of 1,000,000 pieces of data; at the same time, just reading in chunks cannot effectively reduce the total time consumed by traversal lookups. Therefore, we generate a timestamp index file of the transaction data in advance. The index includes the serial number of each chunk and the timestamp range of the transaction data in the chunk. Using chunk reading combined with chunk indexing, we can ignore the data chunks whose timestamp ranges do not match according to the timestamp of the transaction to be queried and only need to perform a traversal in the data chunks whose timestamp ranges match. This approach greatly reduces the memory consumption and improves the speed of our algorithms.

---

<sup>1</sup> <https://xblock.pro/#/dataset/14>

**Table 6.1** Field description of Ethereum transaction data file

| Field name      | Data type | Description                         |
|-----------------|-----------|-------------------------------------|
| BlockNumber     | Int64     | Number of block                     |
| Timestamp       | Int64     | Timestamp of transactions           |
| TransactionHash | Object    | Hash of transactions                |
| From            | Object    | Source address of transactions      |
| To              | Object    | Destination address of transactions |
| Value           | Object    | Value of transactions               |

In summary, this module eventually generates four types of data files: Ethereum transaction data files, exchange address data file, miner address data file, and index file. The following describes how each file is generated:

1. Ethereum transaction data file. We unzip BlockTransaction.zip in turn and read the BlockTransaction.csv file in it. The fields of the raw data include: blockNumber, timestamp, transactionHash, from, to, toCreate, fromIsContract, toIsContract, value, gasLimit, gasPrice, gasUsed, callingFunction, isError, eip2718type, baseFeePerGas, maxFeePerGas, and maxPriorityFeePerGas. Based on the fromIsContract, toIsContract, to and isError fields, we can clean invalid data, including internal transactions, self-destructing transactions, and abnormally terminated transactions. We will also clean redundant fields, i.e., fields that need not be used by subsequent algorithms. We finally extract the fields shown in Table 6.1 and store them in the data types that are allowed by the system and have the smallest memory footprint, respectively. In addition, the transaction data file should be sorted by the value of the timestamp field in ascending order and stored in chunks to facilitate the generation of subsequent index files.
2. Exchange Address Data File: This file originates from the initial address data file of the exchange. This module will read the exchange addresses from these files according to their data structure and merge them into a single Exchange.csv file, which contains one field: exchangeAddress.
3. Miner Address Data File: This file comes from the initial Ethereum block data Block.zip, specifically from the Block\_MinerReward.csv file. Its fields include: blockNumber, timestamp, miner, and reward. This module will extract the miner address field and merge it into a single Miners.csv file, which contains one field: minerAddress.
4. Timestamp Index File: This file is derived from processed Ethereum transaction data files (BlockTransaction.parquet). Based on chunks (with a chunk size of 1,000,000), it calculates the minimum and maximum timestamps within each chunk and sequentially numbers each chunk (starting from 0). The chunk number, maximum timestamp, and minimum timestamp are stored in a Numpy array format in the TimestampRange.npy file.

6.3.2 Address Clustering Module

This module implements the Address Clustering Heuristic Algorithm: two or more addresses are considered to belong to the same entity if they send coins to the same deposit address controlled by the exchange. In addition to the data file, this module requires two input parameters: the maximum time difference between receiving and forwarding:  $t_{max}$  and the maximum amount difference between what was received and forwarded:  $a_{max}$ . This module will output the clustering results and generate a clustering results file that contains the fields shown in Table 6.2.

To trade Ether or other coins, users typically send them to an exchange. To credit assets to the correct account, exchanges usually create the so-called deposit addresses to receive coins first and then transfer the received coins to a main exchange address. These deposit addresses are often created separately for each user.

This module explains the working principle of deposit address reuse in more detail according to Fig. 6.3. In this figure, different colors represent different entities. Users attempting cross-chain transactions through an exchange firstly send coins to a deposit address, which is not an exchange address. The deposit address then sends the coins to an exchange address. It can be seen that user addresses sending coins to the same deposit address are likely to have some connection.

- The following issues need to be addressed in the algorithm:
- Selection of Objects for Algorithm Analysis: First, the objects analyzed by the algorithm should exclude invalid addresses or those that may interfere with

Table 6.2 Field description of address clustering result

| Field name | Data type | Description                                                                        |
|------------|-----------|------------------------------------------------------------------------------------|
| Deposit    | Object    | A storage address                                                                  |
| Addr       | Object    | Must be concatenated by at least two users. The concatenation character is a comma |

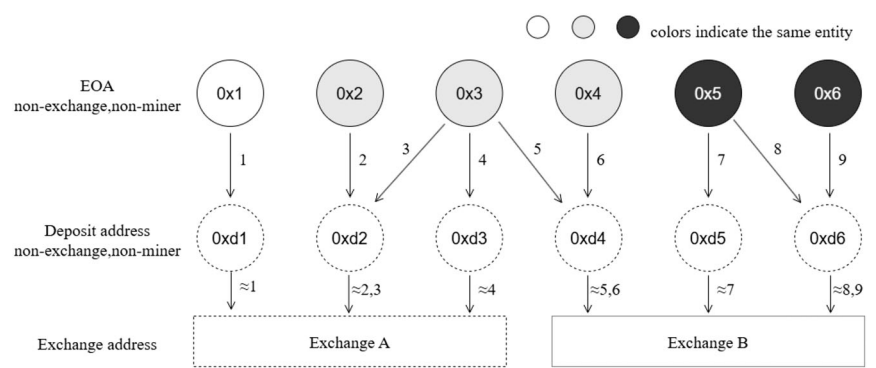


Fig. 6.3 Exchange deposit address reuse principle diagram

clustering results. We categorize Ethereum addresses into Externally Owned Account (EOA) addresses, smart contract addresses, miner addresses, and exchange addresses. The valid objects for our algorithm should be active EOA addresses.

This algorithm excludes miner addresses and exchange addresses because when users transact directly with exchanges without using deposit addresses, exchange or user addresses might be mistakenly identified as deposit addresses. Additionally, in mining pools, miners' rewards are often sent directly to a deposit address before being distributed, which could also affect our algorithm's analysis. Smart contract addresses are excluded because they differ significantly from EOA addresses in terms of generation method and functional characteristics. They contain code and data for handling functions and logic questions defined in contracts, which do not meet the requirements for the user addresses of entities. Inactive EOA addresses are excluded because it is pointless to analyze them with our algorithm. An address is considered inactive if it has not initiated a transaction, i.e., it is not the source address of any transaction, e.g., 0x0000000000000000000000000000000000000000000000000000000000000000.

- **Identifying Deposit Addresses:** The characteristic of deposit addresses is that they transfer received coins to an exchange address. The forwarded amount is usually slightly less than the received amount due to transaction fees required by the exchange, and the forwarding time is typically shortly after receiving the coins. The amount difference and time difference are defined as  $a_{max}$  and  $t_{max}$ , respectively. Additionally, identifying deposit addresses requires excluding known exchange addresses and miner addresses for reasons similar to those explained in filtering EOA addresses. It should be noted that while most deposit addresses are EOA addresses, they can also be smart contracts. For example, when depositing tokens on the Kraken cryptocurrency exchange, users are instructed to send them to a given smart contract address, often with zero transaction fees since fees cannot be paid with tokens.
- **Excluding Invalid Transactions:** Transactions involved in the algorithm must be valid. Transaction data that need to be excluded includes internal transactions, self-destruct transactions, and transactions terminated abnormally. For ease of implementation, this part has been realized in advance within the data preparation module.

After understanding how to handle the aforementioned three issues, the basic pseudocode for the Address Clustering Heuristic is shown in Algorithm 1.

### 6.3.3 Cross-Chain Provenance Tracking Module

This module implements the Cross-chain Transaction Provenance Tracking Algorithm: it recovers the correlation between the participating parties by matching the amount and time between the input transaction and the output transaction, and the

**Algorithm 1** Address Clustering Heuristic Algorithm

---

**Input:** exchange addresses list  $E$ ; miner addresses list  $M$ ; smart contract addresses list  $C$ ; transaction data  $Txs$ ; maximum amount difference  $a_{max}$ ; maximum time difference  $t_{max}$

**Output:** clustering result  $CR[deposit, [users]]$

//toEx: transactions where the destination address is an exchange address

- 1:  $toEx = Txs[Txs[to] \text{ is in } E]$
- 2:  $toEx = toEx[toEx[from] \text{ not in } E \text{ or } M]$
- //ptDepo: potential deposit addresses, which had initiated transactions with the exchange
- 3:  $ptDepo = toEx[from]$
- //toPtDepo: transactions where the destination address is a potential deposit addresses
- 4:  $toPtDepo = Txs[Txs[to] \text{ in } ptDepo]$
- 5:  $toPtDepo = toPtDepo[toPtDepo[from] \text{ not in } E \text{ or } M \text{ or } C]$
- 6: **for** each path  $toPtDepo[from] \rightarrow toPtDepo[to] \rightarrow toEx[to]$  **do**
- 7:    $tx_1 = toPtDepo[from] \rightarrow toPtDepo[to]$
- 8:    $tx_2 = toPtDepo[to] \rightarrow toEx[to]$
- //depositTx: transactions that from an EOA address to a deposit address
- 9:   **if**  $tx_1[value] - tx_2[value] \in [0, a_{max}]$  **and**  $tx_2[time] - tx_1[time] \in [0, t_{max}]$  **then**
- 10:      $depositTx.add(user = toPtDepo[from], deposit = toPtDepo[to])$
- 11:   **end if**
- 12: **end for**
- //Organize user addresses related to the same deposit address together
- 13: **for** each  $deposit^*$  **in**  $depositTx[deposit]$  **do**
- 14:    $users = depositTx[depositTx[deposit] == deposit^*][user]$
- 15:    $CR.add(deposit^*, users)$
- 16: **end for**
- 17: **return**  $CR$

---

algorithm is based on the clustering results. In addition to using the data from the data preparation module and the clustering result, this module also requires three input parameters: time difference between the deposit transaction and the withdraw transaction:  $t_{max}$ , amount difference between the deposit transaction and the withdraw transaction:  $a_{max}$  and query parameter:  $s$ .

We design a Cross-chain Transaction Provenance Tracking Algorithm based on the transaction time and amount. Based on the Address Clustering Heuristic Algorithm, for a transaction with timestamp  $t$  and amount  $a$ , in order to find its previous transaction on the tracking chain, we first find out whether the originating address of the transaction belongs to a certain cluster. If the originating address belongs to a certain cluster, we iterate through all the transactions whose destination address belongs to this cluster and look for the transactions whose timestamp  $t^* \in [t - t_{max}, t]$  and amount  $a^* \in [a, a + a_{max}]$  as candidate transactions. Finally, we select the most probable transaction among the candidate transactions as the previous transaction on the tracking chain.

The pseudocode for the Cross-chain Transaction Provenance Tracking Algorithm is shown in Algorithm 2.

**Algorithm 2** Cross-chain Transaction Provenance Tracking Algorithm

---

**Input:** a transaction hash  $txHash$  with timestamp  $t$ ; address clustering result  $CR[deposit, [users]]$ ; all transaction data  $Txs$ ; maximum amount difference  $a_{max}$ ; maximum timestamp difference  $t_{max}$

**Output:** tracking result  $TR[Tx_1, Tx_2, \dots Tx_n]$

```

//Find the first transaction data according to the hash
1:  $preTx = Txs[Txs[transactionHash] == txHash]$ 
2:  $preValue = preTx[value]$ 
3:  $preTime = preTx[time]$ 
4:  $TR.add(preTx)$ 
//Find the cluster related to the originating address of  $preTx$ 
5: while (1) do
6:    $deposit* = CR[preTx[from] \text{ is in } CR[users]][deposit]$ 
7:    $clustAddrs = clusterResult[clusterResult[deposit] = deposit*][users]$ 
8:   if  $clustAddrs == empty$  then
9:      $clustAddrs = [preTx]$ 
10:  end if
//Find all transactions whose destination address is in the cluster  $clustAddrs$ 
11:   $clustAddrsTx = Txs[Txs[to] \text{ is in } clustAddrs]$ 
//Find the optimal transaction according to maximum amount and time difference
12:   $clustAddrsTx[valueDiff] = clustAddrsTx[value] - preValue$ 
13:   $clustAddrsTx[timeDiff] = clustAddrsTx[time] - preTime$ 
14:   $qualified = clustAddrsTx[timeDiff \in [0, t_{max}] \text{ and } clustAddrsTx[valueDiff] \in a_{max}]$ 
//If no qualified transaction, the tracking ends
15:  if  $qualified == empty$  then
16:    return  $TR$ 
17:  end if
//Choose the transaction with the smallest  $valueDiff$  as the previous transaction
18:   $preTx = qualified[qualified[valueDiff] ismin]$ 
19:   $preValue = preTx[value]$ 
20:   $preTime = preTx[time]$ 
21:   $TR.add(preTx)$ 
22: end while

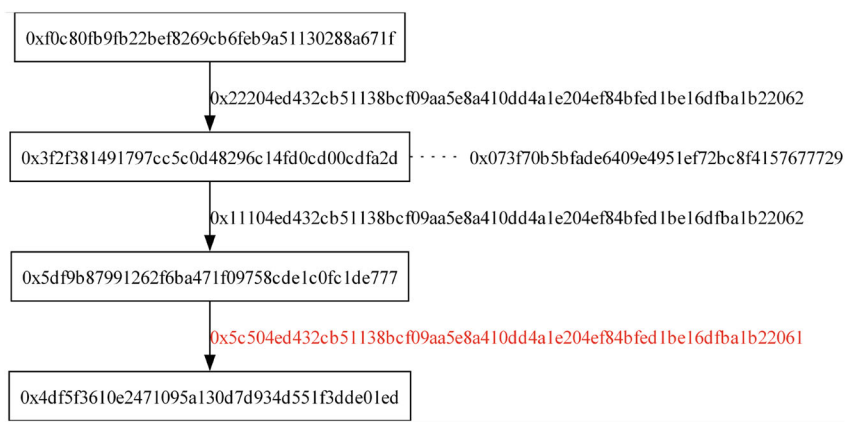
```

---

In order to facilitate the observation of the clustering and tracking results of the algorithms, this module provides two queries:

- **Provenance Tracking Query:** The query parameter  $s_1$  is a transaction hash, and  $s_2$  is its timestamp. It returns all addresses and transaction hashes on the provenance tracking chain.
- **Cluster Query:** The query parameter  $s_3$  is a user address. It returns all addresses within the same cluster.

For both query options, this module uses Graphviz for visual output. Graphviz is an open-source graph visualization toolkit that allows you to define graph structures using textual descriptions and then automatically generate corresponding graphical outputs. It supports various types of graphs, including directed graphs, undirected graphs, and tree diagrams.



**Fig. 6.4** Visualization result of provenance tracking query (the transaction hash for the query is 0x5c50...2061)



**Fig. 6.5** Visualization result of cluster query (the address for the query is 0x03c6...777e)

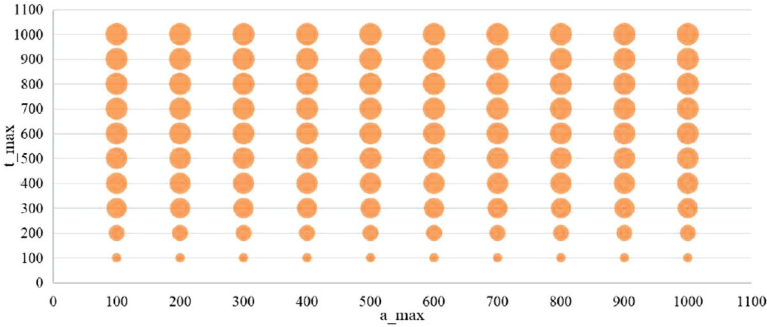
This module generates directed graphs and undirected graphs for the results of provenance tracking query and cluster query, respectively. The exported images are shown in Figs. 6.4 and 6.5.

### 6.4 Performance Evaluation and Analysis

In addition to the data, the performance of our algorithm is mainly related to the choice of the values of parameters  $t_{max}$  and  $a_{max}$ . In this section, we will observe and evaluate the performance of the Address Clustering Heuristic Algorithm by testing different combinations of the parameters  $t_{max}$  and  $a_{max}$ . To improve the efficiency of algorithm testing, we test on about 700,000,000 Ethereum transactions data.

**Table 6.3** Test result with  $a_{max} = 500$ ,  $t_{max} \in [100, 1000]$ 

| $a_{max}$ | $t_{max}$ | Number of clusters | [2,5) | [5,10) | [10, $\infty$ ) | Maximum cluster |
|-----------|-----------|--------------------|-------|--------|-----------------|-----------------|
| 500       | 100       | 84                 | 83    | 1      | 0               | 5               |
| 500       | 200       | 252                | 247   | 5      | 0               | 6               |
| 500       | 300       | 404                | 391   | 13     | 0               | 7               |
| 500       | 400       | 446                | 429   | 17     | 0               | 8               |
| 500       | 500       | 463                | 444   | 18     | 0               | 8               |
| 500       | 600       | 475                | 457   | 10     | 0               | 8               |
| 500       | 700       | 480                | 462   | 18     | 0               | 8               |
| 500       | 800       | 484                | 466   | 18     | 0               | 8               |
| 500       | 900       | 485                | 467   | 18     | 0               | 8               |
| 500       | 1000      | 487                | 469   | 18     | 0               | 8               |

**Fig. 6.6** The parameter test results for the range [100,1000]

We test the values of  $t_{max}$  and  $a_{max}$  in the range of [100, 1000] with a step size of 100, and we focus on these results: the number of clusters, the number of clusters containing addresses in the intervals [2, 5), [5, 10)) and [10,  $+\infty$ ), and the number of addresses contained in the largest cluster. For example, if  $a_{max} = 500$ , the detailed test results are shown in the Table 6.3:

We plot the full test results for  $a_{max}, t_{max} \in [100, 1000]$  as a bubble plot, as shown in Fig. 6.6.

It can be seen that when  $t_{max} \geq 300$ , the number of clusters exceeds 400, and the variation with changes in  $a_{max}$  is relatively small. Considering that  $t_{max} = 300$  already represents a significant time difference between transactions, these clusters are likely to be false positives.

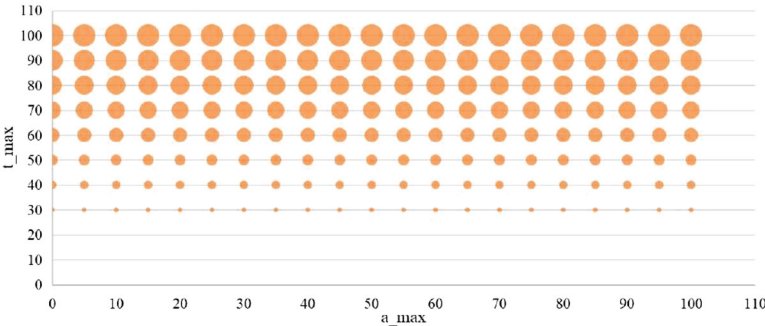
Next, we will reduce the values of  $a_{max}$  and  $t_{max}$  and test their algorithm results within the interval [0, 100] with a step size of 10. Taking the  $a_{max} = 50$  as an example, we provide detailed data in Table 6.4.

It can be observed that when  $a_{max}, t_{max} \in [0, 100]$ , the clustering results show a more noticeable trend of variation with changes in  $a_{max}$  while  $t_{max}$  is fixed. This further indicates that when  $a_{max}, t_{max} > 100$ , a significant portion of the resulting



**Table 6.4** Test result with  $a_{max} = 50, t_{max} \in [0, 100]$

| $a_{max}$ | $t_{max}$ | Number of clusters | [2,5) | [5,10) | [10, $\infty$ ) | Maximum cluster |
|-----------|-----------|--------------------|-------|--------|-----------------|-----------------|
| 50        | 0         | 0                  | 0     | 0      | 0               | 0               |
| 50        | 10        | 0                  | 0     | 0      | 0               | 0               |
| 50        | 20        | 0                  | 0     | 0      | 0               | 0               |
| 50        | 30        | 4                  | 4     | 0      | 0               | 2               |
| 50        | 40        | 12                 | 12    | 0      | 0               | 4               |
| 50        | 50        | 20                 | 19    | 1      | 0               | 5               |
| 50        | 60        | 34                 | 33    | 1      | 0               | 5               |
| 50        | 70        | 50                 | 49    | 1      | 0               | 5               |
| 50        | 80        | 60                 | 59    | 1      | 0               | 5               |
| 50        | 90        | 73                 | 72    | 1      | 0               | 5               |
| 50        | 100       | 84                 | 83    | 1      | 0               | 5               |



**Fig. 6.7** The parameter test results for the range [0,100]

clusters may be due to coincidence. We also plot the full test results for  $a_{max}, t_{max} \in [0, 100]$  as a bubble plot, as shown in Fig. 6.7.

From Figs. 6.6 and 6.7, it can be observed that the clustering results do not vary with changes in  $a_{max}$  within a certain range when the value of  $t_{max}$  is fixed. This may indicate that the value of  $t_{max}$  has a more sensitive impact on the clustering algorithm, making it particularly important to determine an appropriate value of  $t_{max}$ .

### 6.5 Summary

This chapter proposes a provenance tracking scheme for cross-chain systems including an account-based model via deposit address reuse clustering heuristic. Our scheme not only standardizes the dataset derived from the network but also proposes the Address Clustering Heuristic Algorithm and the Cross-chain Transaction Provenance Tracking Algorithm. We clustered user addresses that may

belong to the same entity based on the principle of deposit address reuse and applied the clustering results to the latter algorithm. We implemented our scheme in Python and visualized the output using Graphviz. Through parameter testing, we found that the time difference between the transactions has a more significant impact on the algorithm.

## References

- Altschuler S (2022) Should centralized exchange regulations apply to cryptocurrency protocols? *Stan J Blockchain Law Pol'y* 5:92
- Biryukov A, Feher D (2019) Privacy and linkability of mining in zcash. In: 2019 IEEE Conference on Communications and Network Security (CNS). IEEE, pp 118–123
- Bjelajac Z, Bajac MB (2022) Blockchain technology and money laundering. *Law Theory Prac* 39:21
- Chen L, Xu L, Shah N, Diallo N, Gao Z, Lu Y, Shi W (2017) Unraveling blockchain based cryptocurrency system supporting oblivious transactions: a formalized approach. In: Proceedings of the ACM Workshop on Blockchain, Cryptocurrencies and Contracts, pp 23–28
- de Boissieu E, Kondrateva G, Baudier P, Ammi C (2021) The use of blockchain in the luxury industry: supply chains and the traceability of goods. *J Enterprise Inf Manag* 34(5):1318–1338
- Dutta P, Choi TM, Somani S, Butala R (2020) Blockchain technology in supply chain operations: Applications, challenges and research opportunities. *Transp Res E Logist Transp Rev* 142:102067
- Hall H (2020) Labelled ethereum addresses. <https://www.kaggle.com/datasets/hamishhall/labelled-ethereum-addresses/data>
- Haslhofer B, Karl R, Filtz E (2016) O bitcoin where art thou? insight into large-scale transaction graphs. In: SEMANTiCS (Posters, Demos, SuCESS)
- Kappos G, Yousaf H, Maller M, Meiklejohn S (2018) An empirical analysis of anonymity in zcash. In: 27th USENIX Security Symposium (USENIX Security 18), pp 463–477
- Longfei C, Zhongyuan Y, Heng P, Gaoyuan Q, Xueming S (2023) Cross-chain review: mechanisms protocols applications and challenges. *J Comput Appl* 43(10):3017
- Meiklejohn S, Pomarole M, Jordan G, Levchenko K, McCoy D, Voelker GM, Savage S (2013) A fistful of bitcoins: characterizing payments among men with no names. In: Proceedings of the 2013 Conference on Internet Measurement Conference, pp 127–140
- Möser M, Böhme R, Breuker D (2013) An inquiry into money laundering tools in the bitcoin ecosystem. In: 2013 APWG eCrime Researchers Summit. IEEE, pp 1–14
- Möser M, Soska K, Heilman E, Lee K, Heffan H, Srivastava S, Hogan K, Hennessey J, Miller A, Narayanan A, et al. (2017) An empirical analysis of traceability in the monero blockchain. *arXiv preprint arXiv:170404299*
- Palamara P (2016) Tracing and tracking with the blockchain
- Poskriakov F, Chiriaeva M, Cavin C (2020) Cryptocurrency compliance and risks: A european kyc/aml perspective. *Blockchain & Cryptocurrency Regulation 2020*
- Shamsi K, Victor F, Kantarcioglu M, Gel Y, Akcora CG (2022) Chartist: Labeled graph datasets for utxo and account-based blockchains. *Adv Neural Inf Process Syst* 35:34926–34939
- Siddiqui MS, Syed TA, Nadeem A, Nawaz W, Albouq SS (2020) Blocktrack-l: A lightweight blockchain-based provenance message tracking in iot. *Int J Adv Comput Sci Appl* 11(4)
- Victor F (2020a) Address clustering heuristics for ethereum. In: Bonneau J, Heninger N (eds) *Financial Cryptography and Data Security - 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers*. Springer, Lecture Notes in Computer Science, vol 12059, pp 617–633
- Victor F (2020b) Github - etherclust/etherclust. <https://github.com/etherclust/etherclust>

- Wang X, Yu G, Liu RP, Zhang J, Wu Q, Su SW, He Y, Zhang Z, Yu L, Liu T, et al. (2021) Blockchain-enabled fish provenance and quality tracking system. *IEEE Internet Things J* 9(11):8130–8142
- Wu L, Hu Y, Zhou Y, Wang H, Luo X, Wang Z, Zhang F, Ren K (2021) Towards understanding and demystifying bitcoin mixing services. In: *Proceedings of the Web Conference 2021*, pp 33–44
- Yousaf H, Kappos G, Meiklejohn S (2019) Tracing transactions across cryptocurrency ledgers. In: Heninger N, Traynor P (eds) *28th USENIX Security Symposium, USENIX Security 2019*, Santa Clara, CA, USA, August 14–16, 2019, USENIX Association, pp 837–850
- Zhang Z, Li W, Liu H, Liu J (2020) A refined analysis of zcash anonymity. *IEEE Access* 8:31845–31853
- Zhang Z, Yin J, Hu B, Gao T, Li W, Wu Q, Liu J (2022) Cltracer: A cross-ledger tracing framework based on address relationships. *Comput Secur* 113:102558
- Zheng P, Zheng Z, Wu J, Dai H (2020) Xblock-eth: Extracting and exploring blockchain data from ethereum. *IEEE Open J Comput Soc* 1:95–106

# Chapter 7

## Generic Privacy-Aware Cross-Chain Framework



### 7.1 Problem Statement

In the cross-chain systems, the user identity and the asset are usually transferred between different blockchains. Due to no protection on the transaction, the transaction is fully exposed, incurring serious economic loss. For example, Wormhole, a cross-chain interoperability protocol, was hacked with attackers forging messages from “guardians” to mint large amounts of whETH, which resulted in a loss of approximately \$326 million (Wormhole security program 2023). Harmony’s official cross-chain bridge, Horizon, was also attacked, causing a total loss of about \$100 million due to a private key leakage (Cross-chain bridge attacks and payouts 2023). Therefore, privacy leakage resistance in cross-chain transactions is a critical and desirable issue.

Traditional single-chain privacy-preserving schemes are difficult to support cross-chain transactions (Miers et al. 2013). Two possible methods for cross-chain transaction privacy are token converting (Guo et al. 2024) and transaction mixing (Bonneau et al. 2014). The former is to convert transactional assets into a privacy token, which hides the specific amount of the transaction and the identity of the user, but by analyzing the timing, frequency, and related data of the transaction, it is still possible to infer the pattern of the transaction and the behavior of the participants. The latter is to mix transactions for less leakage, which has transaction delay and heavy complexity. Furthermore, it does not completely eliminate the problem of transactions tracing from unauthorized participants. There are also cross-chain solutions that use other ways to protect privacy, e.g., Zhu et al. (2024) uses commitment to hide cross-chain transaction information, and Chen et al. (2024) uses adaptor signature to achieve a secret transfer channel between several users. However, they either loss ability to audit historical transactions or have requirements for the underlying architecture of the chains involved in the transactions.

In the previous chapters, we present kinds of solutions to cross-chain regulation, i.e., detecting abnormal transactions in the cross-chain network. These solutions

can effectively identify abnormal transactions in aspects of the corresponding accounts, nodes, and network traffic. In fact, the regulation mechanisms and privacy preserving are opposite. The regulation means to open while privacy means to hide. A balance between transaction privacy and regulation is struck. In this chapter, we focus on how to protect the cross-chain privacy.

For cross-chain systems, the diversity of involved blockchains brings about a blizzard of underlying structures and running rules. Even, each cross-chain system has its unique features and functions. The difference of privacy goals demands distinct protection methods. For example, some cross-chain system needs to just protect the user privacy while another cross-chain system has to protect both transaction amount privacy and the user privacy. In this case, they adopt different privacy-preserving technologies. The used concrete mechanisms rest with the prospective privacy level and objectives. So far, it still lacks the generalization of the privacy technology in cross-chain transaction systems. How to design a generic privacy-preserving cross-chain framework is an important issue.

In this chapter, we consider two types of generic privacy-preserving frameworks for cross-chain transactions. From the underlying structure, we present a sharding-based cross-chain framework and an encrypted cross-chain bridge framework.

## 7.2 Sharding-Based Cross-Chain Framework

### 7.2.1 *Cross-Chain Sharding*

Sharding a blockchain into isolated shards can mitigate this problem (Wang et al. 2019; Luu et al. 2016). However, there are still two major technical challenges for the cross-chain systems. Firstly, the independence of the business chains causes storage conflicts and unbalanced sharding. Traditional sharding methods usually directly divide the ledger based on account addresses (Wang et al. 2019; Luu et al. 2016). However, in cross-chain supervision scenarios, data originating from different business chains are defined independently. Similar naming conventions not only substantially increase the conflict probability but also result in unbalanced data sharding in the supervision chain. Secondly, the sharding of data storage makes it difficult to validate inter-shard transactions (Kokoris-Kogias et al. 2018; Zamani et al. 2018) due to the isolation of the ledger and the network among shards. Some researchers (Chen and Wang 2019) have proposed deploying a rootchain to enable inter-shard transaction validation, but this approach suffers from significant performance degradation as the number of transactions increases. In addition, some researchers (Kokoris-Kogias et al. 2018; Huang et al. 2021) have proposed transaction splitting methods based on UTXO transactions, but they are all designed for crypto-currency transactions on public chains. They can identify the related shards from the input addresses, and then the nodes in related shards validate the intra-shard transactions. However, related shards can only be identified through

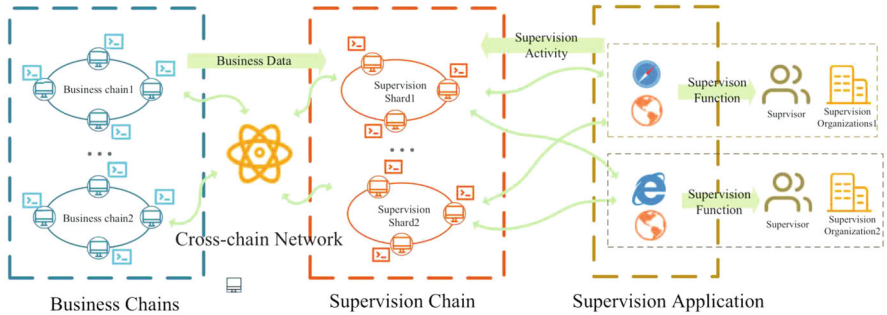


Fig. 7.1 Sharding-based cross-chain system

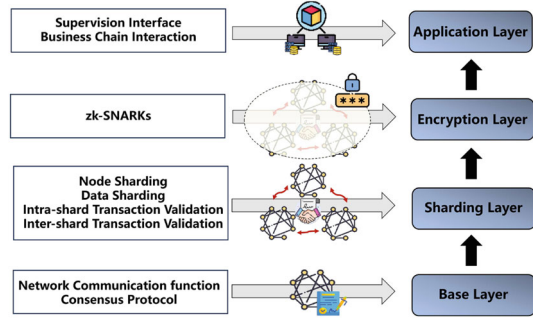
operations such as calculation and querying the ledger. Thus, the shards involved in a transaction can only be determined after validating the sub-transactions. Therefore, the transaction splitting method based on the UTXO format cannot be applied to the cross-chain supervision scenario described earlier. Besides, in most sharding-based cross-chain schemes, users can freely view the cross-chain transactions, which increases the risk of private information disclosure and hinders the applications of the cross-chain technology.

Here, we consider to apply the blockchain sharding technology into the underlying architecture of the cross-chain system. Figure 7.1 describes a sharding-based cross-chain system, including a supervision chain with multiple business chains. The supervision chain is built by multiple supervision organizations. Nodes in the supervision chain are crash-only (Androulaki et al. 2018). A cross-chain network is deployed for data transmission. Cross-chain networks can be implemented using various methods, including relay chains, oracle machines (Sober et al. 2021), and so on.

Any feasible cross-chain communication method can be used between the supervision chain and the business chains. In Fig. 7.1, the supervision chain uses the sharding technique to improve scalability. In the aspect of node sharding, nodes are spread among shards evenly via off-chain negotiation.

Organizations provide supervision services by deploying applications. A user accesses the nodes in the supervision chain via the application and initiates supervision activities. Supervision activities are divided into two categories: static and dynamic supervision. Static supervision means that the supervision chain requests data from the business chain and stores it on its ledger. These data include account snapshot information, block information, transaction information, etc. Dynamic supervision means that the supervision chain re-executes a transaction that has already occurred on the business chain to validate its correctness. As the majority of business chains are built with consortium blockchain technology that supports smart contracts, this validation process is also the process of re-running the smart contract by the nodes on the supervision chain.

**Fig. 7.2** Privacy-preserving sharding-based framework



### 7.2.2 Privacy-Preserving Sharding-Based Cross-Chain Framework Overview

We propose a privacy-preserving sharding-based cross-chain framework that involves zero-knowledge proof into inter-shard transaction. Figure 7.2 shows the basic components of the proposed framework, which is divided into four layers: the base layer, the sharding layer, the encryption layer, and the application layer.

The base layer provides three basic functions for the supervision chain. The first is the network communication function, which includes communication between nodes in the chain and communication with nodes in the business chain. Supervision chains can choose from a variety of communication protocols. Here we choose the Gossip protocol between nodes and use agent nodes as a cross-chain communication scheme. The second is the consensus protocol. Theoretically, it allows any kind of consensus protocol. It supports various operations on the ledger, including querying and updating the status of accounts and storing and accessing transaction data.

The sharding layer is the core and provides four functions: node sharding, data sharding, intra-shard transaction validation, and inter-shard transaction validation. In the aspect of node sharding, it requires that all nodes are randomly and evenly divided into different shards. This goal can be achieved by off-chain negotiation between supervisory organizations. More shards are necessary when the business chain increases. Data sharding means that the supervision chain divides the ledger into different shards. To avoid the problem of storage conflicts and unbalanced sharding caused by large-scale business chains, we propose a unified label-based data sharding method. We define the intra-shard transaction and the inter-shard transaction the same as Omniledger (Kokoris-Kogias et al. 2018).

If a transaction only involves data on the ledger within a shard, it's an intra-shard transaction. We can use the same validation method as other blockchains, and this is not our focus. When the data of a transaction are distributed among different shards, it's an inter-shard transaction. To deal with that, we propose a smart contract splitting algorithm and design an inter-shard transaction validation method based on it. As aforementioned, the unified label-based data sharding method, the smart

contract splitting algorithm, and the intershard transaction validation method are the core work of it.

The encryption layer runs cross-chain privacy protection protocols based on zero-knowledge proofs for cross-chain exchange, which preserve privacy without requiring any trusted third party. The private information of every transaction in every shard is protected in this layer by being encapsulated by zkSNARKs, thus no plaintext is shown in supervision chain.

The application layer is open to supervisors and business blockchains. It is wrapped as a distributed application established. Supervision interfaces, including static and dynamic supervision activities, are exposed to the supervisor.

Meanwhile, business blockchains can query its information and states via the application layer. More specific functions can be implemented according to the demand.

### **7.2.3 Framework Analysis**

Scalability is a central feature of the proposed framework, achieved primarily through the use of sharding. As the number of business chains grows, the framework can dynamically adjust the number of shards to balance the load and maintain performance. This flexibility ensures that the system can scale in response to increasing data and transaction volumes. By segregating transactions into intra-shard and inter-shard categories, the framework optimizes the validation process. Inter-shard transactions are handled quickly using standard methods, while intra-shard transactions are processed using specialized algorithms that ensure consistency without sacrificing performance.

The framework's design prioritizes data integrity and privacy through several key mechanisms. By dividing data and processing workloads across multiple shards, the framework can handle a higher volume of transactions efficiently. This division reduces the burden on individual nodes and prevents bottlenecks, ensuring smooth and scalable operations. The use of zk-SNARKs in the encryption layer is a significant privacy feature. It allows the validation of transactions without revealing any sensitive information. This is particularly important in a multi-chain environment where data privacy is a primary concern. The zk-SNARKs ensure that all transaction data remain confidential, protecting the privacy of users and organizations involved.

The privacy-preserving sharding-based framework for cross-chain supervision is a comprehensive solution designed to address the challenges of data privacy, scalability, and security in a multi-chain environment. Its layered architecture ensures a modular approach to system design, allowing for flexible and scalable deployment. The use of zk-SNARKs provides robust privacy protections, while the sharding and consensus mechanisms support high transaction throughput and data integrity. The application layer facilitates user and business chain interactions, providing a user-friendly interface for monitoring and managing cross-chain transactions.



### 7.2.4 Summary

In summary, the proposed framework integrates zk-SNARKs and sharding technologies to address the challenges of privacy, security, and scalability in cross-chain transactions. zk-SNARKs ensure that transaction details and user identities remain confidential throughout the cross-chain process. Sharding enables parallel transaction processing, enhancing the network's ability to handle high transaction volumes efficiently.

By leveraging these technologies, the framework provides a robust solution for privacy-preserving, secure, and scalable cross-chain transactions, facilitating broader adoption and more complex use cases in the blockchain ecosystem.

## 7.3 Encrypted Cross-Chain Bridge Framework

### 7.3.1 Methodology Overview

We present CPAP, a privacy-preserving cross-chain asset transfer protocol, which augments cross-chain bridge with smart contract and cryptographic mechanisms. The CPAP's methodology overview is illustrated in Fig. 7.3.

We deploy the Way Network architecture as the fundamental component of CPAP, ensuring the confidentiality of transaction data and preventing unauthorized access to sensitive information. The integration of the smart contract not only ensures the security and reliability of cross-chain asset transfers but also provides automatic verification, supporting the scalability. For stronger privacy, CPAP incorporates both encryption and zero-knowledge proof into the cross-chain bridge, ensuring that all of the assets transferred between both chains and stored in the bridge are in the encrypted format against transaction exposure. For auditability, CPAP records the ciphertext of every historical transaction along with its ZKP, and the information of the problematic transaction will be disclosed when necessary.

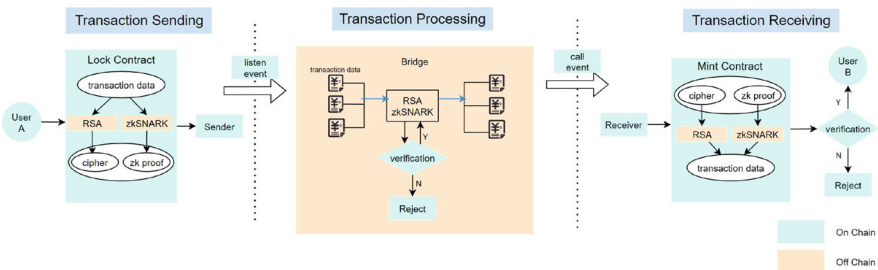


Fig. 7.3 Sharding-based cross-chain scenario

Also, CPAP removes the transaction mixing such as to accelerate the transaction and mitigate the traceability of participants.

CPAP can be summarized with the following main technical points. CPAP is a new cross-chain transaction architecture, which integrates encrypted data transmission in Way Network to prevent information leakage during communication between users and the relayer. Unlike traditional cross-chain architecture, the CPAP architecture delivers messages in ciphertext and uses zero-knowledge proof tools to prove the trustworthiness of messages. Under this architecture, we propose a new cross-chain transaction protocol based on the new architecture, named CPAP, which leverages RSA for secure message encryption to ensure confidentiality during data transmission and zkSNARKs to validate transaction integrity without disclosing sensitive information. We also conduct an experimental test of CPAP protocol based on ChainMaker. The experimental results show that CPAP is capable of throughput rates up to 4TPS, and uploading a piece of 200-byte data in size consumes only 0.4 seconds and less than  $3 \times 10^5$  gas. Besides, the upload time and gas consumption grow linear-size with message length.

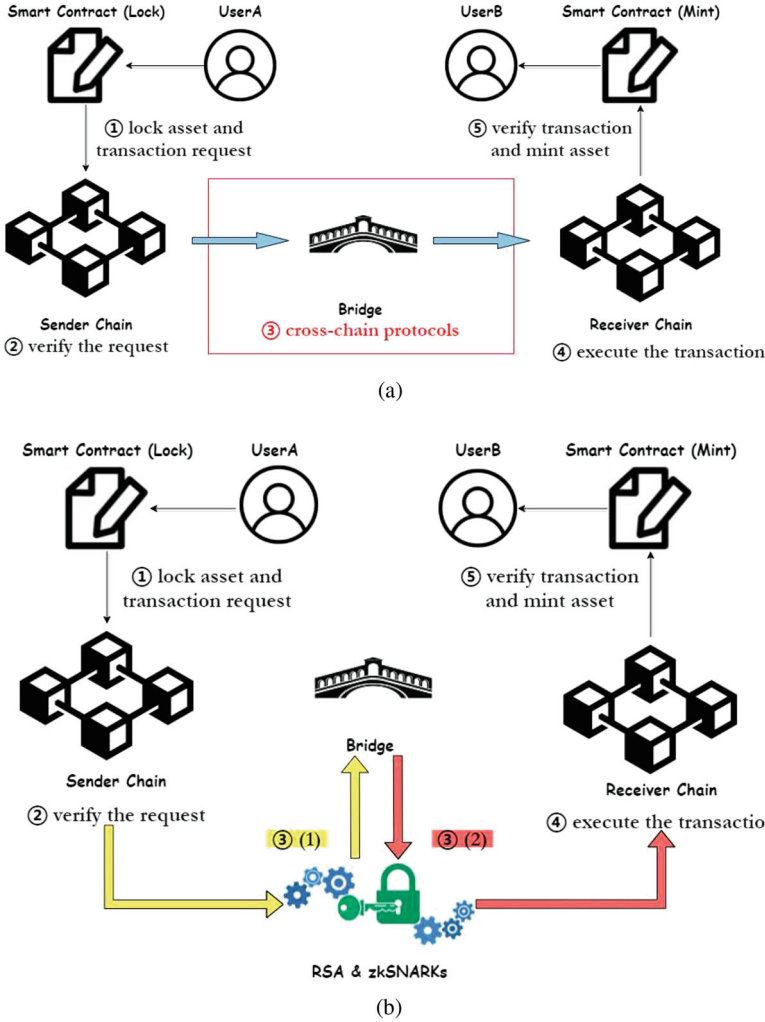
## 7.3.2 System Overview

### 7.3.2.1 CPAP Architecture

Figure 7.4 presents the typical cross-chain architecture, utilizing a bridge to facilitate the transfer and forward information between two distinct blockchains. It involves three main entities: User A on Sender Chain, User B on Receiver Chain, and Bridge. Components of CPAP include:

- **Smart Contract (Lock):** This smart contract is deployed by User A on Sender Chain. It is responsible for locking the specified assets and generating a zero-knowledge proof (ZKP) that proves the assets are locked without revealing details about the assets or the transaction.
- **Smart Contract (Mint):** This smart contract is deployed by User B on Receiver Chain. It verifies the zero-knowledge proof received via the Bridge and releases the corresponding assets to User B if the proof is valid.
- **Zero-Knowledge Proof System:** It contains a Proof Generation algorithm, which demonstrates that the assets are locked without revealing any details about the assets or the transaction and a Proof Verification algorithm, which verifies the ZKP.
- **Encryption System:** It plays a critical role in ensuring the security and integrity of the cross-chain transaction process, which encrypts privacy data to provide secure communication.

**Workflow** Figure 7.4a presents the traditional cross-chain structure, which typically relies on intermediary entities such as centralized relays or notaries to transfer assets and data between two blockchain networks. However, this approach often



**Fig. 7.4** Traditional cross-chain architecture v.s. CPAP. (a) Traditional structure. (b) CPAP structure

suffers from security vulnerabilities, including the leakage of privacy information, reliance on trust in third parties, and limited scalability. We propose the CPAP structure as shown in Fig. 7.4b, sending the encrypted transaction message along with a ZKP to prove the credibility of data.

In the CPAP structure, all the participants start with initializing public parameters, public keys and private keys, and then disclose the public keys. When two users (A and B) agree to complete an asset transfer, A needs to publish a smart contract to lock his asset on Sender Chain and send messages to the Bridge concerning the

transaction information, such as addresses of participants and amount of transaction. The Bridge checks the message and forwards it to Receiver Chain. The smart contract published by user B on Receiver Chain receives and verifies the message, then mints asset on Receiver Chain.

### 7.3.2.2 System Goals

CPAP aims to provide a privacy-preserving solution for cross-chain transactions with the following design goals:

- *Anonymity*. The identities of the transaction participants can remain anonymous, i.e., the transaction record does not disclose information about the identities of the participants involved.
- *Confidentiality*. The content of the transactions, including the amount and the assets being exchanged, is kept confidential from any unauthorized parties.
- *Authenticity*. The protocol ensures that all parties involved in the transaction are legitimate and authorized.
- *Auditability*. Transaction records on the blockchain are open and transparent, and anyone can view and verify the validity of a transaction.
- *Resistance to Double-spend Attacks*. Transaction records cannot be spent twice by exploiting a vulnerability in the blockchain consensus mechanism.

## 7.3.3 Detailed CPAP Design

### 7.3.3.1 Building Blocks

#### 7.3.3.1.1 Public Key Encryption

Public key encryption (PKE) (Cramer and Shoup 1998) is a fundamental cryptographic technique used to ensure secure communication over an insecure network. It involves two keys: a public key, which can be shared openly, and a private key, which remains confidential. Messages encrypted with the public key can only be decrypted by the corresponding private key, ensuring that only the intended recipient can access the message content. A public key encryption scheme is defined by three probabilistic polynomial-time algorithms: **KeyGen**, **Enc**, and **Dec**.

- **KeyGen**( $\lambda$ )  $\rightarrow$  ( $pk, sk$ ). It takes the security parameter  $\lambda$  as input, generates a public key  $pk$  and a private key  $sk$ .
- **Enc**( $pk, M$ )  $\rightarrow C$ . It takes public key  $pk$  and a plaintext message  $M$  as input, outputs a ciphertext  $C$  for the message.
- **Dec**( $sk, C$ )  $\rightarrow M$ . It takes private key  $sk$  and a ciphertext  $C$  as input, outputs the original message  $M$ .

**Correctness** For a PKE scheme to be correct, the decryption of a ciphertext using the corresponding private key should yield the original plaintext message. Formally, for every key pair  $(pk, sk)$  generated by  $\text{KeyGen}(1^\lambda)$ , and for every message  $M$ , the following condition must hold:

$$\text{Dec}(sk, \text{Enc}(pk, M)) = M.$$

**Security** A PKE scheme is confidential if it ensures that an adversary who intercepts the ciphertext cannot learn anything about the original plaintext message. A PKE scheme is complete if the decryption function always correctly recovers the original message from the ciphertext, given the correct private key. A PKE scheme is unforgeable if any adversary cannot generate a valid ciphertext that decrypts to a meaningful message without access to the corresponding private key.

#### 7.3.3.1.2 Zero-Knowledge Proof

Zero-knowledge proof (ZKP) (Bitansky et al. 2012) refers to cryptographic protocol that allows a prover to demonstrate knowledge of a secret without revealing any information about the secret itself. A ZKP scheme contains the following algorithms:

- **Setup** $(\lambda) \rightarrow (pk, sk)$ . It takes the security parameter  $\lambda$  as input, generates system parameters for the ZKP protocol, including proving key  $pk$  and verification key  $vk$ .
- **Prove** $(pk, x, w) \rightarrow \pi$ . It takes the proving key, statement  $x$ , and witness  $w$  as input, generates a proof  $\pi$  for the given statement and witness.
- **Verify** $(vk, x, \pi) \rightarrow \text{true/false}$ . It takes the verification key, statement  $x$ , and proof  $\pi$  as input, outputs *true* or *false* based on the verification result.

**Completeness** A valid proof convinces the verifier of the truthfulness of the statement being proved. For every statement  $x$  and witness  $w$  such that  $(x, w) \in \mathcal{R}$ , an honest prover  $\mathcal{P}$  can generate a proof  $\pi$  that convinces the verifier  $\mathcal{V}$ . Formally:

$$\Pr[\text{Verify}(vk, x, \pi) = 1 \mid \pi \leftarrow \text{Prove}(pk, x, w)] = 1.$$

**Soundness** An invalid statement cannot be proven as true with overwhelming probability. If  $\pi$  is a valid proof for statement  $x$  under verification key  $vk$ , then  $x$  satisfies:  $\Pr[\text{Verify}(vk, x, \pi) = 1 \mid x \notin \mathcal{L}] \leq \epsilon$ , where  $\mathcal{L}$  is the language of statements, and  $\epsilon$  refers to negligible probability.

The main properties of ZKPs are:

- *Completeness*: A valid proof convinces the verifier of the truthfulness of the statement being proved.
- *Soundness*: An invalid statement cannot be proven as true with overwhelming probability.

- *Zero-Knowledge*: The proof leaks no information about the secret being proved, except for its validity.

#### 7.3.3.1.3 Smart Contract

A smart contract is defined as a self-executing piece of code deployed on a blockchain network (Wang et al. 2018). The algorithm of a smart contract is an automated protocol that defines and enforces the terms of the contract through code without human intervention to ensure the execution of the contract. The core algorithmic structure usually consists of three parts: initialization (initializing contract variables, participant addresses, etc.), execution logic (defining the specific terms and conditions of the contract, such as transfer of funds, status update, etc.), and termination or status update (updating the status of the contract or transferring funds to the appropriate address). Smart contract algorithms typically use the consensus mechanism of the blockchain to validate and execute the code, ensuring that the terms of the contract are open, transparent, and tamper-proof.

An example of a smart contract is shown in Fig. 7.5, concerning creating address, assigning balance, and transferring balance (Ethereum 2024).

#### 7.3.3.2 CPAP Construction

Way Network is a zero-knowledge omnichain interoperability protocol, a universal solution to enable interchain transaction with trustless and valid delivery (Way network architecture 2023). CPAP is designed based on Way Network to facilitate secure and private cross-chain transactions, ensuring that assets are exchanged atomically between different blockchain networks while preserving user privacy through ZKPs. Unlike traditional cross-chain architectures, CPAP leverages ZKPs and cryptographic tools to provide enhanced privacy, authenticity, and auditability. Note that in CPAP, we assume that Bridge is fully honest, will follow the workflow, and keep the sensitive information undisclosed and untampered.

*Setup* All the participants run the algorithms **PKE.KeyGen** and **ZKP.Setup**, generating necessary PKE key pairs  $(pk_A, sk_A)$  for User A,  $(pk_B, sk_B)$  for User B,  $(pk_C, sk_C)$  for Bridge, and a zero-knowledge proof key pair  $(pk, vk)$ . User A and User B agree offline to conduct a cross-chain transaction. They negotiate the terms, including the amount of assets to be exchanged and the addresses on Sender Chain and Receiver Chain. They also exchange their public keys  $(pk_A, pk_B)$  and blockchain addresses. User A provides  $addr_A$  and  $pk_A$ , while User B provides  $addr_B$  and  $pk_B$ .

*Locking Assets on Sender Chain* As shown in Fig. 7.6a, user A deploys a smart contract  $SC_A$  on Sender Chain, locking the specified amount of assets, then generating a ZKP  $\pi_A$  through algorithm **ZKP.Prove** to prove the lock. The transaction details are encrypted using  $pk_C$  and sent to the Bridge together with  $\pi_A$ . If the ZKP

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.10;
3
4  contract Token {
5      address public owner;
6
7      mapping (address => uint) public balances;
8      // this `mapping` assigns an unsigned integer (the token
9      balance) to the address (the token holder).
10
11      event Transfer(address from, address to, uint amount);
12
13      // set `owner` as the address of the contract creator
14      constructor() public {
15          owner = msg.sender;
16          // all smart contracts rely on external transactions to
17          trigger their functions.
18      }
19
20      function mint(address receiver, uint amount) public {
21
22          require(msg.sender == owner, "You are not the owner.");
23          // only the contract creator can call this function
24
25          require(amount < 1e60, "Maximum issuance exceeded");
26
27          balances[receiver] += amount;
28          // increase the balance of `payee` by `amount`.
29      }
30
31      // sends a certain number of tokens to an address.
32      function transfer(address receiver, uint amount) public {
33          require(amount <= balances[msg.sender], "Insufficient
34          balance.");
35
36          balances[msg.sender] -= amount;
37          balances[receiver] += amount;
38
39          emit Transfer(msg.sender, receiver, amount);
40          // triggers a previously defined event.
41      }
42  }

```

**Fig. 7.5** Token contract

$\pi_A$  generation fails, the transaction is aborted, and any locked assets are returned to User A.

*Verification and Relay by Bridge* As shown in Fig. 7.6b, Bridge decrypts the message using  $sk_C$ , verifies the ZKP  $\pi_A$ , ensures that the assets are indeed locked on Sender Chain, and generates a ZKP  $\pi_B$  to prove the lock. If ZKP verification succeeds, the transaction details are re-encrypted using  $pk_B$  and sent to Receiver

**Fig. 7.6** CPAP details. (a) Lock contract  $SC_A$ . (b) Verify contract  $Verifier$ . (c) Mint contract  $SC_B$

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./verifier.sol";

contract SC_A {
    mapping(address => uint256) public balances;
    event Locked(address indexed user, uint256 amount, bytes zkProof);

    function lockAssets(uint256 amount, bytes calldata zkProof) external {
        require(amount > 0, "Amount must be greater than zero");
        balances[msg.sender] += amount;
        emit Locked(msg.sender, amount, zkProof);
    }
}
```

(a)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Verifier {
    function verifyProof(
        bytes calldata proof,
        uint256[1] calldata publicInputs
    ) public view returns (bool) {
        // Implement verification logic generated by snarkjs
        return true; // Placeholder
    }
}
```

(b)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./verifier.sol";

contract SC_B {
    mapping(address => uint256) public balances;
    event AssetsReleased(address indexed user, uint256 amount);

    function releaseAssets(address user, uint256 amount, bytes calldata proof) external {
        require(Verifier.verifyProof(proof, [amount]), "Invalid proof");
        balances[user] += amount;
        emit AssetsReleased(user, amount);
    }
}
```

(c)

Chain together with  $\pi_B$ . If ZKP verification fails, the Bridge sends a failure message to User A and aborts the transaction.

*Minting Assets on Receiver Chain* As shown in Fig. 7.6c, user B deploys a smart contract  $SC_B$  on Receiver Chain, decrypts the message using  $sk_B$ , verifies the ZKP  $\pi_B$ , and handles the received message. If ZKP verification succeeds,  $SC_B$  releases the assets to User B, completing the transaction. If ZKP verification fails, the transaction is aborted, and the Bridge initiates a rollback on Sender Chain. The rollback mechanism in  $SC_A$  unlocks the assets and returns them to User A.



### 7.3.4 Framework Analysis

The analysis shows that CPAP satisfies anonymity, confidentiality, authenticity, and auditability and can resist double-spend attacks. It should be noted that the security of the protocol mainly depends on the public key encryption and zero-knowledge proof algorithm we use, which aligns with our intended goal to provide privacy-preserving for cross-chain transactions by using cryptographic tools.

**Theorem 7.1** *CPAP satisfies anonymity.*

**Proof** In the context of the protocol, a degree of anonymity is maintained for users A and B. This is achieved through the utilization of cryptographic public keys instead of revealing their real identities. Furthermore, the incorporation of zero-knowledge proofs in CPAP offers an additional layer of privacy protection, particularly for user A, as these proofs do not disclose specific asset information. The process of encrypting and decrypting messages at various stages, from user A to the intermediary Bridge and then to user B, further obfuscates the identities involved.

**Theorem 7.2** *CPAP satisfies confidentiality.*

**Proof** Confidentiality is a crucial aspect of secure communication and transaction protocols, ensuring that sensitive information is not exposed to unauthorized parties. In the context of CPAP, confidentiality primarily pertains to ensuring that the details of a transaction, such as the identities of the participants and the specifics of the assets involved, remain private throughout the cross-chain transaction process. In CPAP, ZKPs are used to prove the validity of asset locking and other critical operations without revealing the underlying details, such as the amount of assets locked or the identities of the participants. This ensures that even though the transaction is validated, the specifics remain confidential. Besides, CPAP uses encryption to secure messages relayed between Sender Chain, Receiver Chain, and Bridge. By encrypting transaction details, CPAP ensures that intermediaries, including the Bridge, cannot access sensitive information. Only the intended recipient (e.g., the smart contract on the Bridge or Receiver Chain) can decrypt and process the transaction.

**Theorem 7.3** *CPAP satisfies authenticity.*

**Proof** Authenticity in the context of blockchain protocols refers to the assurance that the entities involved in a transaction are who they claim to be, and that the data or messages exchanged during the transaction have not been tampered with. In CPAP, ZKPs are used to prove that certain conditions (like asset locking) have been met without exposing the details. The proof itself is tied to the transaction's authenticity because it confirms that the transaction was performed correctly and by the legitimate party.

**Theorem 7.4** *CPAP satisfies auditability.*

**Proof** Auditability is a crucial aspect of CPAP, allowing for the verification of transactions without compromising privacy. In CPAP, Auditability in CPAP is maintained through encrypted records and verifiable proofs. Let  $E_k(m)$  represent the encryption of message  $m$  with key  $k$ . The Bridge stores  $E_{k_C}(T)$ , where  $k_C$  is the public key of the Bridge. This allows for auditability without revealing transaction details. Meanwhile, any auditor with the necessary cryptographic keys can verify the ZKP  $\pi$  without learning the underlying details of  $T$ .

**Theorem 7.5** *CPAP resists double-spend attacks if the ZKP protocol used in CPAP satisfies completeness, soundness, and zero-knowledge properties, and the cryptographic primitives are secure.*

**Proof** The completeness of the ZKP ensures that if the prover is honest and the assets are correctly locked on Sender Chain, the proof  $\pi$  will always verify correctly on Receiver Chain. The soundness of the ZKP ensures that if the prover attempts to generate fraudulent proof for a double-spend, the verification on Receiver Chain will fail with overwhelming probability. This implies that an adversary cannot generate two valid proofs  $\pi_1$  and  $\pi_2$  for two different transactions  $tx_1$  and  $tx_2$  using the same locked assets on Sender Chain. The encryption scheme used in CPAP ensures that the transaction information  $C$  relayed through the relay chain is confidential and cannot be tampered with. Any modification by the adversary will be detected, and the altered message will be rejected by Receiver Chain.

### 7.3.5 Performance Evaluation and Analysis

#### 7.3.5.1 Implementation Setting

The experiments are conducted on a Windows operating system with the following hardware specifications: Intel(R) Core(TM) i5-10210U CPU and 12GB of RAM. Python version 3.9 is used for executing off-chain components and cryptographic operations. For the on-chain component, Solidity code with version 0.8.2 is employed. The compilation environment is the Remix-Ethereum IDE, and the smart contract functions are tested using Remix test accounts. The code is compatible with Solidity compilers with versions no lower than 0.8.2 and no higher than 0.9.0 to ensure correct functionality. In the experiment, the RSA asymmetric encryption algorithm (Rivest et al. 1978) is utilized for encryption, with 1024-bit public and private keys. The ECDSA digital signature algorithm (Koblitz 1987) is adopted for signing operations. Additionally, Groth16 (Groth 2016) is used as the zero-knowledge proof (ZKP) protocol for secure verification, ensuring privacy-preserving and integrity in cross-chain transaction operations.

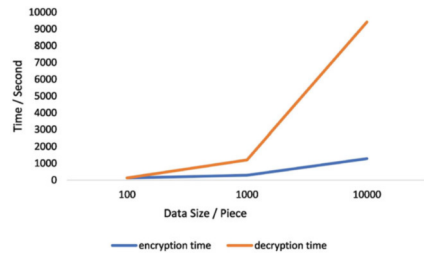
### 7.3.5.2 Generation Test

In the context of our system, we have implemented the RSA encryption algorithm, a fundamental component that underpins secure data transmission. The RSA algorithm involves two primary operations: powering and modular arithmetic. For the experimental setup, the RSA algorithm generates two large prime numbers of 512 bits each for a 1024-bit key, which are used to form the modulus. The space required for storing the keys includes the public and private key components, both totaling 1024 bits for each user. This results in a space consumption of 128 bytes per key (public/private). The plaintext is divided into pieces of 117 bytes each.

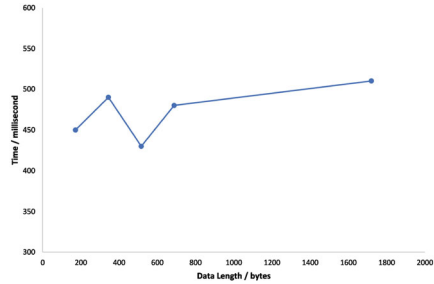
To assess the performance of our encryption process, we conducted runtime measurements, and the results are illustrated in Fig. 7.7. As observed, the encryption time increases linearly with the amount of data being encrypted. This is due to the nature of the RSA algorithm, where the encryption operation is directly dependent on the length of the plaintext. As we can see, when the data size comes to 10,000 pieces, CPAP consumes up to about 10 seconds on the encryption and decryption part of the process. This is the longest part of the CPAP transaction process. The decryption time grows significantly larger than the encryption time as the data size increases. This discrepancy arises because the time complexity of the encryption operation is logarithmically correlated with the public exponent, which is typically small (e.g.,  $e = 65537$ ) to optimize encryption speed. In contrast, the decryption operation is logarithmically correlated with the private exponent, which is substantially larger to ensure security.

The decryption process is computationally intensive because the private exponent, typically large (close to the modulus size), needs to be processed. This ensures robust security, making the algorithm resistant to attacks such as brute force or factorization. The trade-off here is between security and performance: while increasing the key size enhances security by making it computationally infeasible for an attacker to break the encryption, it also significantly increases the decryption time and space consumption due to larger key sizes.

**Fig. 7.7** Transaction encryption execution time



**Fig. 7.8** Transaction uploading time



**7.3.5.3 Uploading Test**

In the completion of a transaction within our system, the relevant transaction information is securely stored on the blockchain in an encrypted format. This process is integral to the immutability and transparency of our data records.

The efficiency of data upload to the blockchain is crucial for CPAP’s overall performance. As indicated in Fig. 7.8, as the data length grows from 172 bytes to 1720 bytes, data upload time grows slowly and linearly from about 4 seconds to 5 seconds, which exhibits some stability. In our implementation, the data are uploaded from the initial address (the sender’s account) to the destination address (the smart contract address on the blockchain). Regardless of the size of the data contained in the transaction, miners always need to perform certain work to package the transaction into a block and execute consensus verification. Therefore, the upload time depends more on factors such as the performance of the blockchain network, congestion, miners’ activity, and the priority of transactions, but less on data length.

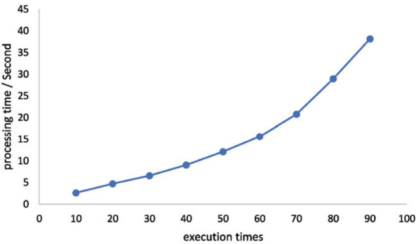
This integrated process of securely storing transaction information on the blockchain is a vital component of CPAP’s operational efficiency and reliability, ensuring the integrity and accessibility of transaction data while maintaining a smooth user experience throughout the transaction lifecycle.

**7.3.5.4 Throughput Test**

To assess the performance and efficiency of CPAP, throughput testing was conducted. Throughput, in the context of this chapter, refers to the system’s ability to handle a specific volume of transactions within a given time frame.

Consider a simple cross-chain scenario with only two chains, one of which sends a transaction request containing 6 bytes to the other. As shown in Fig. 7.9, as the number of transaction batch executions grows from 10 to 100, the total time taken to process a transaction increases approximately exponentially from about 2.6 seconds to 40 seconds. Nearly four transactions are executed every second, which demonstrates the system’s capability to handle a significant transaction volume while maintaining acceptable confirmation times.

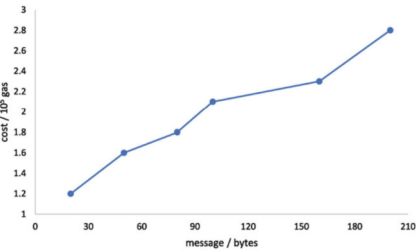
**Fig. 7.9** Transaction processing time



**Table 7.1** User parameters

|          | Address                                    |
|----------|--------------------------------------------|
| Sender   | 0xDA0bab807633f07f013f94DD0E6A4F96F8742B53 |
| Receiver | 0xd9145CCE52D386f254917e481eB44e9943F39138 |

**Fig. 7.10** Gas cost



7.3.5.5 Gas Discussion

Gas consumption is a crucial metric in blockchain systems, representing the computational effort and resource utilization required for executing transactions. Here we only consider the exchange of messages between participants, excluding cryptographic operations such as encryption, decryption, and zero-knowledge proofs. In our implementation, the sender address and receiver address are shown Table 7.1.

As shown in Fig. 7.10, the gas consumed per transaction increases linearly with the number of bytes carried by the transaction. As the message size grows from 20 bytes to 200 bytes, the gas consumed per transaction gradually increases from  $1.2 \times 10^5$  to  $2.8 \times 10^5$ .

- Several factors contribute to the observed gas consumption:
- **Message complexity:** Gas consumption is influenced by the complexity of messages being exchanged. Larger or more intricate messages result in higher gas costs.
  - **Blockchain network conditions:** The overall gas cost can be affected by the current state of the blockchain network, including congestion, miner activity, and gas prices.

- **Transaction priority:** Gas costs may vary based on the priority assigned to transactions. Higher-priority transactions might experience faster processing but potentially at an increased gas cost.

While the gas consumption for message exchange is within an acceptable range, ongoing optimization efforts can further enhance the protocol's efficiency. Exploring gas-efficient messaging patterns and utilizing advanced techniques to reduce gas costs will be considered in future protocol iterations.

### 7.3.6 Summary

We propose CPAP to ensure privacy-preserving and auditability in cross-chain transactions. Leveraging a relay-based cross-chain structure and integrating cryptographic techniques including RSA encryption and zkSNARKs, CPAP ensures that transaction data, such as participant identities and transaction amounts, remain private in cross-chain message transferring while enabling transparent audits and resistant to unauthorized access. This work has been submitted to ACM BSCI 2025 (Liu et al. 2025).

## References

- Androulaki E, Barger A, Bortnikov V, Cachin C, Christidis K, Caro AD, Enyeart D, Ferris C, Laventman G, Manevich Y, Muralidharan S, Murthy C, Nguyen B, Sethi M, Singh G, Smith K, Sorniotti A, Stathakopoulou C, Vukolic M, Cocco SW, Yellick J (2018) Hyperledger fabric: a distributed operating system for permissioned blockchains. In: Oliveira R, Felber P, Hu YC (eds) Proceedings of the thirteenth EuroSys conference, EuroSys 2018, Porto, April 23–26, 2018. ACM, New York, pp 30:1–30:15
- Bitansky N, Canetti R, Chiesa A, Tromer E (2012) From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In: Proceedings of the 3rd innovations in theoretical computer science conference. Association for Computing Machinery, New York, NY, ITCS '12, pp 326–349. <https://doi.org/10.1145/2090236.2090263>
- Bonneau J, Narayanan A, Miller A, Clark J, Kroll JA, Felten EW (2014) Mixcoin: anonymity for bitcoin with accountable Mixes. In: Christin N, Safavi-Naini R (eds) Financial cryptography and data security. Springer, Berlin, pp 486–504
- Chen C, Yang G, Li Z, Xiao F, Chen Q, Li J (2024) Privacy-preserving multi-party cross-chain transaction protocols. Cryptography 8(1). <https://doi.org/10.3390/cryptography8010006>, <https://www.mdpi.com/2410-387X/8/1/6>
- Chen H, Wang Y (2019) SSChain: a full sharding protocol for public blockchain without data migration overhead. Pervasive Mob Comput 59
- Cramer R, Shoup V (1998) A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In: Proceedings of the 18th annual international cryptology conference on advances in cryptology, CRYPTO '98. Springer, Berlin, pp 13–25
- Cross-chain bridge attacks and payouts (2023). <https://www.panewslab.com/zh/index.html>
- Ethereum (2024) Sample code for a token contract. <https://ethereum.org/zh/developers/docs/smart-contracts/anatomy/>

- Groth J (2016) On the size of pairing-based non-interactive arguments. In: Fischlin M, Coron JS (eds) *Advances in cryptology – EUROCRYPT 2016*. Springer, Berlin, pp 305–326
- Guo H, Liang H, Huang J, Ou W, Han W, Zhang Q, Zhang R (2024) A framework for efficient cross-chain token transfers in blockchain networks. *J King Saud Univ Comput Inform Sci* 36(2):101968
- Huang C, Wang Z, Chen H, Hu Q, Zhang Q, Wang W, Guan X (2021) Repchain: a reputation-based secure, fast, and high incentive blockchain system via sharding. *IEEE Internet Things J* 8(6):4291–4304
- Koblitz N (1987) Elliptic curve cryptosystems. *Math Comput* 48:203–209. <https://api.semanticscholar.org/CorpusID:14288427>
- Kokoris-Kogias E, Jovanovic P, Gasser L, Gailly N, Syta E, Ford B (2018) Omniledger: a secure, scale-out, decentralized ledger via sharding. In: 2018 IEEE symposium on security and privacy, SP 2018, Proceedings, 21–23 May 2018, San Francisco, CA. IEEE Computer Society, Washington, pp 583–598
- Liu J, Jiang P, Zhu L (2025) Cpap: Cross-chain privacy-preserving auditable protocol. In: BSCI '25: proceedings of the 7nd ACM international symposium on blockchain and secure critical infrastructure, co-located with AsiaCCS 2025 Ha Noi, Vietnam, August 25–29, 2025. Submitted
- Luu L, Narayanan V, Zheng C, Baweja K, Gilbert S, Saxena P (2016) A secure sharding protocol for open blockchains. In: Weippl ER, Katzenbeisser S, Kruegel C, Myers AC, Halevi S (eds) *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, Vienna, October 24–28, 2016. ACM, New York, pp 17–30
- Miers I, Garman C, Green M, Rubin AD (2013) Zerocoin: anonymous distributed e-cash from bitcoin. In: 2013 IEEE symposium on security and privacy, pp 397–411. <https://doi.org/10.1109/SP.2013.34>
- Rivest RL, Shamir A, Adleman L (1978) A method for obtaining digital signatures and public-key cryptosystems. *Commun ACM* 21(2):120–126. <https://doi.org/10.1145/359340.359342>
- Sober M, Scaffino G, Spanring C, Schulte S (2021) A voting-based blockchain interoperability oracle. In: Xiang Y, Wang Z, Wang H, Niemi V (eds) *2021 IEEE international conference on blockchain, blockchain 2021*, Melbourne, December 6–8, 2021. IEEE, Piscataway, pp 160–169
- Wang S, Yuan Y, Wang X, Li J, Qin R, Wang FY (2018) An overview of smart contract: architecture, applications, and future trends. In: 2018 IEEE intelligent vehicles symposium (IV), pp 108–113. <https://doi.org/10.1109/TVS.2018.8500488>
- Wang G, Shi ZJ, Nixon M, Han S (2019) Sok: sharding on blockchain. In: *Proceedings of the 1st ACM conference on advances in financial technologies*, AFT 2019, Zurich, October 21–23, 2019. ACM, New York, pp 41–61
- Way network architecture (2023). <https://way-networks-organization.gitbook.io/way-network/concepts/way-network-architecture>
- Wormhole security program — end-of-year update (2023). <https://wormholecrypto.medium.com/wormhole-security-program-end-of-year-update-212116ecfb91>
- Zamani M, Movahedi M, Raykova M (2018) RapidChain: scaling blockchain via full sharding. In: Lie D, Mannan M, Backes M, Wang X (eds) *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, CCS 2018, Toronto, ON, October 15–19, 2018. ACM, New York, pp 931–948
- Zhu J, Gai K, Jiang P, Zhu L (2024) Lightweight privacy-preserving mechanisms for cross-chain transactions. In: 2024 IEEE 11th international conference on cyber security and cloud computing (CSCloud), pp 59–64. <https://doi.org/10.1109/CSCloud62866.2024.00018>

# Chapter 8

## Lightweight Cross-Chain Privacy-Preserving Protocol



### 8.1 Background and Motivation

As aforementioned, the privacy protection in the cross-chain system has been a desirable issue. For some privacy-related scenarios, a non-interactive zero-knowledge (NIZK) technique can be used to protect the users' sensitive data (Lu et al. 2018, 2020), allowing one party to prove to another party that they know a certain piece of information without revealing the information itself. Zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) is one of the earliest and most popular zero-knowledge proof implementations. In order to launch a zkSNARK network, a trusted setup is required, in which multiple parties each generate a partial key to launch the network and then destroy the key afterward (Ben-Sasson et al. 2013; Groth 2016). zkSNARK can be further improved by replacing the costly setup with an updatable and universal setup, such as Sonic (Maller et al. 2019), PLONK (Gabizon et al. 2019), and Lunar (Campanelli et al. 2021). However, the universal setup is still not practical without a trusted third party, which breaks the decentralized property of the blockchain systems. The zero-knowledge scalable and transparent argument of knowledge (zkSTARK) (Ben-Sasson et al. 2018) uses public and random parameters to generate proofs, which completely removes the need for a trusted third party or a costly multi-party computation. Unfortunately, the proof size of zkSTARK is much larger than existing approaches. As the proofs need to be transmitted over the whole network and stored for a long time, zkSTARK incurs high communication and storage overhead. How to achieve lightweight privacy preserving for cross-chain transaction?

To address the transaction data privacy issues of single blockchain, employing zero-knowledge proof technology is a commonly effective way in recent years (Lu et al. 2018, 2020). Yang and Li (2020) designed a digital identity management system using smart contracts and zero-knowledge proof. It adopts the framework of off-chain computation and on-chain verification, which effectively prevents the exposure of ownership between user entities and attributes in distributed ledgers.



Gabay et al. (2020) applied zero-knowledge proof into blockchain to achieve anonymous identity verification of electric vehicle users. Wan et al. (2019) proposed a blockchain-based zero-knowledge proof scheme for identity verification by improving the classical zkSNARK scheme, supporting both authenticity and privacy of uploaded data. Ben-Sasson et al. (2014) issued a new cryptocurrency named Zerocash, aiming to achieve anonymity and transaction privacy simultaneously, which is at the expense of high computational costs in the transaction proof generation.

For cross-chain transactions, privacy becomes more serious since independent blockchains use different privacy-preserving policy, and it lacks interconnection. Moreover, the complexity of cross-chain structure makes the privacy protection more difficult. Malavolta et al. (2019) and Deshpande and Herlihy (2020) proposed privacy-preserving HTLC-based protocols, which set up to protect transaction privacy. Thyagarajan et al. (2022) proposed a universal atomic swap scheme considering protecting the private information of cross-chain exchanges via transferring the process of cross-chain exchange to offline. But it only guarantees order privacy when multiple cross-chain exchanges are initiated simultaneously. To address privacy concerns, Garoffolo et al. (2020) introduced Zendoo, a cross-chain protocol employing zkSNARK for safeguarding exchange-related private data. However, it meets greater computational overheads. Li et al. (2022) designed a sidechain privacy-preserving cross-chain solution for Monero and other cryptocurrencies named ZeroCross. Payers can properly conduct cross-chain exchanges, preventing others from conducting remote side-channel attacks. Jiang et al. (2024) proposed PCP protocol to balance both privacy and regulation for cross-chain transactions.

As smart contract has emerged where users can arbitrarily define programs running in the blockchain, it is important to consider the programmability aspect without exposing transactions and data in plaintext to the public (i.e., any party not involved in the contract). Hawk (Korpela et al. 2017), presented by Kosba et al., is the first work to simultaneously provide transactional privacy and programmability in the blockchain. This method is based on the idea of Zerocash and the smart contract system, users send encrypted and committed information to the smart contract and rely on the NIZK proofs to enforce the correctness of contract execution and funds' transfer.

The aforementioned techniques all use zero-knowledge proofs to achieve privacy and anonymity, while ignoring the high complexity of them. In this chapter, we focus on lightweight realization of cross-chain transaction privacy without heavy-cost zero-knowledge proof.

We consider a new and succinct mechanism named Bulletproof (Bünz et al. 2018) to address the heavy cost problem. Bulletproof is a primitive with a logarithmic-size range proof to show a secret  $v \in [0, 2^n - 1]$ , which is short like a bullet, with bulletproof security assumptions. Bulletproofs take some of the advantages of zkSNARK and zkSTARK. That is, Bulletproofs neither require a trusted setup to run and function nor rely on massive proof sizes. Besides, Bulletproofs bring range proofs to NIZK, where the prover first commits to a value using a commitment scheme and then proves that a committed value lies in a given interval. Using a

**Table 8.1** Comparison with other privacy techniques

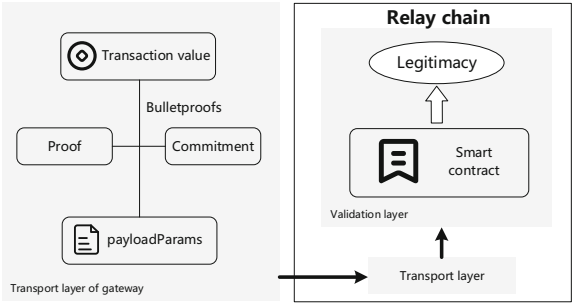
| NIZK         | Proof size   | Time complexity | Trusted setup | Transaction fee |
|--------------|--------------|-----------------|---------------|-----------------|
| zkSNARK      | 288 B        | $O(n \lg n)$    | Yes           | High            |
| zkSTARK      | 48 KB–200 KB | $O(n)$          | No            | High            |
| Bulletproofs | 800 B        | $O(4 \lg n)$    | No            | Low             |

range proof, each transaction can be confidentially transferred without disclosing the amount transferred by including only the zero-knowledge proof validity of the transaction. The concrete comparison with other proof-type privacy preserving techniques (Ben-Sasson et al. 2013, 2018) is shown in Table 8.1. From Table 8.1, we can see that bulletproofs have shorter proof length, lower time complexity, and lower transaction fee without relying on a trusted setup. It seems to provide an elegant way to protect the cross-chain transaction. We propose a lightweight cross-chain transaction privacy-preserving method named PTCross, which cryptographically embeds both Bulletproofs and Pedersen commitment to hide the cross-chain transaction value. The proposed PTCross design is deployed over the relaychain structure and consists of four phases, integrated with off-chain computation and on-chain contract verification.

8.2 PTCross Overview

PTCross borrows bulletproofs to achieve privacy of cross-chain transactions, as shown in Fig. 8.1. PTCross is built on top of PieBridge, where four main entities are involved, i.e., source chain, the gateway of source chain, the relay chain, and the smart contract for verification on the rely chain. The transaction willing to exchange assets with the other chain is initiated by the source chain. When the transaction is sent to the gateway, its transaction value is hidden in the transport layer using Pedersen commitment, denoted as *Com*. In the meantime, a succinct proof  $\pi$  is added to ensure the legitimacy of the transaction value. Then the relevant commitment *Com* and proof  $\pi$  are packed as the *payloadParams* in the cross-chain transaction,

**Fig. 8.1** The structure of PTCross



thereafter sent to the relay chain. In this way, no plaintext data will be exposed on the relay chain. In the verification layer of the relay chain, a smart contract will be deployed to verify the correctness of the transaction value. Finally, the successfully verified cross-chain transaction data are deposited into a new block for cross-chain process.

### 8.3 Detailed Design of PTCross

The proposed PTCross comprises four phases, they are **Initialize**, **Conceal**, **Pack**, and **Verify**, the flow is demonstrated in Fig. 8.2 and details are depicted as follows:

#### Phase 1. Initialize

Denote  $q$  as the field module. Choose  $g, p \in \mathbb{G}_q$  randomly then the public parameter is  $pp = (g, p)$ .

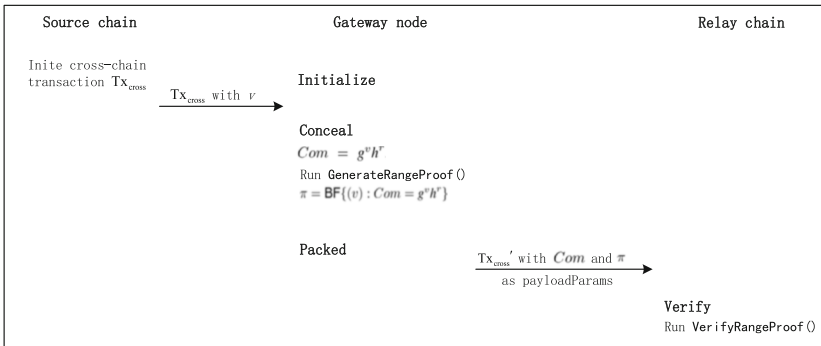
#### Phase 2. Conceal

In the transport layer of the gateway node between the source chain and the relay chain, the transaction value  $v$  is extracted from the raw cross-chain transaction. Multiply the transaction value  $v$  with a generator point  $g$  on the additive group of the elliptic curve as  $g^v$ . Then select a random number  $r$  as the blinding factor and multiply it with a random base point  $h$  on the additive group of elliptic curves, denoted as  $h^r$ . Next, generate a promise  $Com = g^v h^r$ , whereby the transaction value  $v$  is hidden in the commitment.

After hiding the transaction value in the Pedersen commitment, then it uses Bulletproofs to prove the arithmetic result is non-negative as follows:

$$\pi = \text{BF}\{(v) : Com = g^v h^r\}.$$

The concrete instantiation can be seen in Fig. 8.3.



**Fig. 8.2** The workflow of PTCross

---

**Initialization:** Elliptic curve group  $\mathbb{G}_q$ ; Generators of  $\mathbb{G}_q$   $g, h$ ; Value to be committed and proven with range  $v$ ; Random binding factor  $r, r_1, r_2$

**Input:** Range  $Range$

**Output:** Proof  $\pi$

```

 $(1, u) \leftarrow Range$  // Extract lower and upper bounds of the range
 $t_1, t_2 \leftarrow$  Generate polynomial commitments for range proof
 $T_1 \leftarrow g^{t_1} h^{r_1}$  //Compute commitment for  $t_1$ 
 $T_2 \leftarrow g^{t_2} h^{r_2}$  //Compute commitment for  $t_2$ 
 $x \leftarrow$  Challenge from the verifier
 $\tau \leftarrow r + x * r_1 + x^2 * r_2$  //Compute the proof scalar  $\tau$ 
 $\mu \leftarrow \text{SecretScalar}()$  //Additional secret for the proof
 $L, R \leftarrow$  Compute vectors  $L$  and  $R$  for the inner product argument
 $\pi \leftarrow (t_1, t_2, T_1, T_2, \tau, \mu, L, R)$ 
return  $\pi$ 

```

---

Fig. 8.3 Generate range proof

---

**Initialization:** Elliptic curve group  $\mathbb{G}_q$ ; Generators of  $\mathbb{G}_q$   $g, h$ ; Value to be committed and proven with range  $v$ ; Random binding factor  $r, r_1, r_2$

**Input:** Commitment  $Com$ ; Proof  $\pi$

**Output:**  $Valid$

```

 $(1, u) \leftarrow Range$  // Extract lower and upper bounds of the range
 $(t_1, t_2, T_1, T_2, \tau, \mu, L, R) \leftarrow \pi$ 
 $x \leftarrow$  Challenge from the prover
 $T_{1check} \leftarrow g^{t_1} h^{x r_1}$  //Recompute commitment for  $t_1$ 
 $T_{2check} \leftarrow g^{t_2} h^{x^2 r_1}$  //Recompute commitment for  $t_1$ 
if  $T_1 \neq T_{1check}$  or  $T_2 \neq T_{2check}$  then
    return false
end if
 $valid \leftarrow \text{InnerProductArgumentVerification}(Com, \tau,$ 
 $\mu, L, R, x)$  //Verify the inner product argument
return  $valid$ 

```

---

Fig. 8.4 Verify range proof

### Phase 3. Pack

When the gateway node builds the cross-chain transaction, the proof  $\pi$  and commitment  $Com$  are encapsulated in the payloadParams field within the payload of it. This payload, devoid of any plaintext data, as the secure transmission of transaction information across disparate blockchain networks. The gateway node then issues the cross-chain transaction on the relay chain, ensuring that sensitive data remain safeguarded throughout the cross-chain interaction process.

### Phase 4. Verify

When the miner receives the cross-transaction, he/she needs to verify the correctness of the transaction. He/she checks the legitimacy of transaction value by verifying whether the proof in the payloadParams is correct using  $\text{BF.Verify}(Com) \stackrel{?}{=} 1$ . (The concrete verification process can be seen in Fig. 8.4.) If the check pass, it means the original transaction value  $v$  is legitimacy on the condition that  $v$  is known by nobody.

8.4    Performance Evaluation and Analysis

In this section, we implement the proposed PTCross over a cross-chain environment where ChainMaker and Bitcoin are two exchanging chains and ChainMaker is the relay chain. We take some measurements about the proof size, the concealing time, and the verification time of it. All experiments are undertaken on an AMD Ryzen 5 5600U CPU @ 2.30 GHz and 16 GB of RAM, and the operating system is Ubuntu 20.04 installed on the VMware Workstation Pro with 4 GB RAM. ChainMaker is set as single-chain 4-node cluster, and the smart contract is written in Golang language. More details about the experiments can be seen in Table 8.2.

We first change the range sizes of Bulletproofs from 5 bits to 100 bits and test the proof size. As shown in Fig. 8.5a, the experiment shows that the proof size is about 0.65 KB constantly, which is lightweight enough for cross-chain scenarios with expensive storage space.

For the implementation, we utilized an optimized Bulletproofs library, ensuring that the setup and proof generation were consistent across varying bit ranges. The testing environment was standardized to ensure that the results were not affected by external factors such as hardware discrepancies or background processes. Each test was repeated 50 times to obtain average values and minimize the impact of outliers.

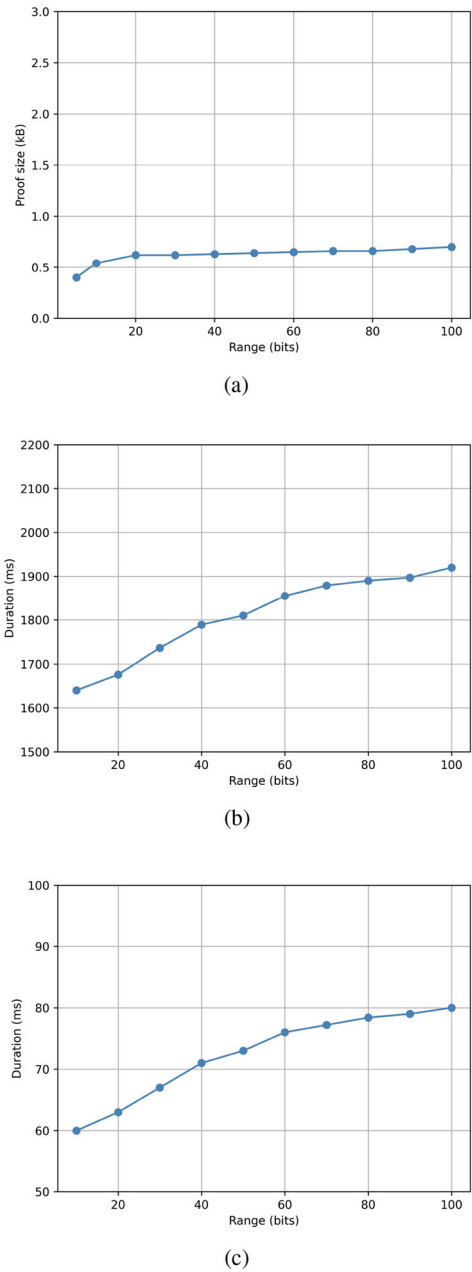
Figure 8.5b, c show the durations of the Conceal phase and the Verify phase, respectively. As we can see, the bigger the range of Bulletproofs, the longer the duration. But the longest concealing time is no longer than 1950 ms, and the longest verification time is no longer than 80 ms, considering the biggest range is no larger than 100 bits.

During the Conceal phase, the time complexity grows as the bit range increases due to the linear relationship between the range size and the number of operations required to generate the proof. The observed duration remains within acceptable limits for practical applications, even at the upper bound of 100 bits. This efficiency is crucial for real-time applications where rapid proof generation is necessary. In the Verify phase, although the duration also increases with the bit range, it remains significantly shorter than the Conceal phase. Verification is more computationally efficient, which is advantageous in scenarios where quick validation is necessary to maintain system performance and responsiveness.

**Table 8.2**    Details of cross-chain environments

|                             | Address                                  |
|-----------------------------|------------------------------------------|
| Bitcoin (parallel chain)    | 1GQHspZg5JwmRapqdsVFayutvvSWx9Euq        |
| ChainMaker (parallel chain) | 73c2b7e910784a4d8a5618d1ea8211fe6f9a91aa |
| Smart contract              | 23f6a637e29f63e5af3aa7faf4e15f8e1a930460 |

**Fig. 8.5** Performance of PTCross. **(a)** Proof sizes of different ranges. **(b)** Duration of Conceal phase. **(c)** Duration of Verify phase



## 8.5 Summary

Privacy asset and information transfer of cross-chain transactions are becoming vital concern with the development of blockchain interconnection. To address this challenge, we introduce PTCross, an innovative method that leverages Bulletproofs and Pedersen commitment to conceal transaction value while improving system efficiency by combining off-chain computation with on-chain verification. Practical implementation of PTCross between ChainMaker and Bitcoin demonstrates that the proof size of Bulletproofs is about 0.65 KB constantly, and the longest verification time is no longer than 80 ms, which achieves the lightweight property. This work has been published in CSCloud 2024 (Zhu et al. 2024).

## References

- Ben-Sasson E, Chiesa A, Genkin D, Tromer E, Virza M (2013) Snarks for C: verifying program executions succinctly and in zero knowledge. In: Canetti R, Garay JA (eds) CRYPTO 2013. Lecture Notes in Computer Science, vol 8043. Springer, Berlin, pp 90–108
- Ben-Sasson E, Chiesa A, Garman C, Green M, Miers I, Tromer E, Virza M (2014) Zerocash: decentralized anonymous payments from bitcoin. In: SP 2014. IEEE Computer Society, Washington, pp 459–474
- Ben-Sasson E, Bentov I, Horesh Y, Riabzev M (2018) Scalable, transparent, and post-quantum secure computational integrity. In: IACR 2018, p 46
- Bünz B, Bootle J, Boneh D, Poelstra A, Wuille P, Maxwell G (2018) Bulletproofs: short proofs for confidential transactions and more. In: SP 2018. IEEE Computer Society, Washington, pp 315–334
- Campanelli M, Faonio A, Fiore D, Querol A, Rodríguez H (2021) Lunar: a toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. In: Tibouchi M, Wang H (eds) ASIACRYPT 2021, LNCS, vol 13092. Springer, Berlin, pp 3–33
- Deshpande A, Herlihy M (2020) Privacy-preserving cross-chain atomic swaps. In: Bernhard M, Bracciali A, Camp LJ, Matsuo S, Maurushat A, Rønne PB, Sala M (eds) FC 2020. Lecture Notes in Computer Science, vol 12063. Springer, Berlin, pp 540–549
- Gabay D, Akkaya K, Cebeci M (2020) Privacy-preserving authentication scheme for connected electric vehicles using blockchain and zero knowledge proofs. *IEEE Trans Veh Technol* 69(6):5760–5772
- Gabizon A, Williamson ZJ, Ciobotaru O (2019) PLONK: permutations over lagrange-bases for ocumenical noninteractive arguments of knowledge. *IACR* 2019, p 953
- Garofalo A, Kaidalov D, Oliynykov R (2020) Zendo: a zk-snark verifiable cross-chain transfer protocol enabling decoupled and decentralized sidechains. In: ICDCS 2020. IEEE, Piscataway, pp 1257–1262
- Groth J (2016) On the size of pairing-based non-interactive arguments. In: Fischlin M, Coron JS (eds) Advances in cryptology – EUROCRYPT 2016. Springer, Berlin, pp 305–326
- Jiang P, Zhu J, Zhu L (2024) Balancing privacy and regulation of cross-chain transaction systems via sok-assisted policy enhancement. *IEEE Trans Inf Foren Secur* 19:5620–5629
- Korpela K, Hallikas J, Dahlberg T (2017) Digital supply chain transformation toward blockchain integration. In: Bui T (ed) HICSS 2017, ScholarSpace/AIS Electronic Library (AISEL), pp 1–10
- Li Y, Weng J, Li M, Wu W, Weng J, Liu J, Hu S (2022) Zerocross: a sidechain-based privacy-preserving cross-chain solution for monero. *J Parallel Distributed Comput* 169:301–316

- Lu Y, Tang Q, Wang G (2018) Zebralancer: private and anonymous crowdsourcing system atop open blockchain. In: ICDCS 2018. IEEE Computer Society, Washington, pp 853–865
- Lu Y, Tang Q, Wan G (2020) Dragoon: private decentralized hits made practical. In: ICDCS 2020. IEEE, Piscataway, pp 910–920
- Malavolta G, Moreno-Sanchez P, Schneidewind C, Kate A, Maffei M (2019) Anonymous multi-hop locks for blockchain scalability and interoperability. In: NDSS 2019. The Internet Society, Reston
- Maller M, Bowe S, Kohlweiss M, Meiklejohn S (2019) Sonic: zero-knowledge snarks from linear-size universal and updateable structured reference strings. IACR 2019, p 99
- Thyagarajan SAK, Malavolta G, Moreno-Sanchez P (2022) Universal atomic swaps: secure exchange of coins across all blockchains. In: SP 2022. IEEE, Piscataway, pp 1299–1316
- Wan Z, Guan Z, Zhou Y, Ren K (2019) zk-authfeed: how to feed authenticated data into smart contract with zero knowledge. In: ICB 2019. IEEE, Piscataway, pp 83–90
- Yang X, Li W (2020) A zero-knowledge-proof-based digital identity management scheme in blockchain. *Comput Secur* 99:102050
- Zhu J, Gai K, Jiang P, Zhu L (2024) Lightweight privacy-preserving mechanisms for cross-chain transactions. In: 2024 IEEE 11th international conference on cyber security and cloud computing (CSCloud), pp 59–64. <https://doi.org/10.1109/CSCloud62866.2024.00018>



# Chapter 9

## Balance of Privacy and Regulation for Cross-Chain Transactions



### 9.1 Background and Motivation

In the sidechain-based cross-chain system, the receiver in one underlying blockchain is identical to the sender in another side, and the time when the cross-chain exchange transactions are published in two interactive blockchains is close enough to link up. Such an inherent security disadvantage of linkability makes the violation of a chain's privacy to compromise the privacy of another interacting chain, such as obtaining the exchange value of the transaction in chain B based on that in chain A. Privacy leakage may cause malicious merchants obtaining customers' identity and account information for illegal trading, while the linkability of cross-chain exchanges between various blockchain may give rise to tremendous chain reactions of privacy leakage.

Many works (Li et al. 2022; Xie et al. 2022) have used cryptographic tools like commitment (Li et al. 2022) and signature of knowledge (Xie et al. 2022) to hide the amounts and enable the payers to prove that they indeed construct the exchanging transaction correctly as agreed without revealing the identity and transaction amounts. However, a direct application of commitment and signature of knowledge still has to suffer from the remote side-channel attack, which uses the lock mechanism and the delayed response time of Monero's wallet to identify the real receiver.

The previous privacy-preserving techniques take no regulatory requirements into consideration. Anonymity of cryptocurrencies affords a cover for various illegal activities, such as contraband trading, racketeering, and money laundering, as adversaries are indistinguishable and hard to be punished. The abuse of anonymous cryptocurrencies for illegal purposes is gaining momentum these years, while the full anonymity of cross-chain exchange provides a great space for the crime. In the WannaCry event (Mohurle and Patil 2017), the Bitcoins given by victims have been exchanged by the criminals for Monero, which employs the one-time address and is awfully hard to trace. Blockchain surveillance is important, while anonymity makes

inspecting illegal transactions difficult. Due to the requirements of supervision, Lei et al. (2021) designed a supervised chain under the sidechain for the storage of cross-chain data and the traceability, while on the supervised chain, the account information and the transaction data are still plaintext. The regulation requirement to some extent means the sacrifice of the privacy level, for it needs the incomplete anonymity. So, we are trying to design a new solution to balance privacy and traceability in cryptocurrency, especially for the cross-chain process.

In this chapter, we introduce a privacy-preserving policy-enforcement cross-chain transaction protocol (PCP) with the underlying sidechain framework. PCP can be applied to any cross-chain structure with a condition that one side is Monero. Due to that, traceability of PCP is achieved with the stealth address and the linkable ring signature, Monero is required to be one side. The other side can be any type of cryptocurrencies, e.g., any UTXO-based cryptocurrency or any account-based cryptocurrency. In this work, we just present a concrete construction by taking Bitcoin as an example.

In the proposed PCP protocol, selective users and coins can be traced in the Monero side. It enables payers to proactively prove to the sidechain that they have indeed constructed the cross-chain transaction correctly as agreed. We apply an accumulator to give a membership proof without disclosing the specific transaction. We construct another proof stating that the cross-chain exchange transaction is in a confirmation block without revealing which confirmed transaction is the actual transaction they used and the relationship of the transactions related to the same cross-chain exchange. This proof also guarantees that the miners cannot deduce the identification of payers in one underlying blockchain via the identification in another captured by launching the remote side-channel attack. At the same time, a tracing authority exists and can revoke the anonymity of payers if he/she misbehaves. It's worth noting that the tracing authority is optimistic and passive. It will not interfere with any normal transaction and only intervenes when an investigation is required.

## 9.2 PCP Overview

Figure 9.1 shows a PCP overview. PCP involves four entities: Payer A and Payer B from different blockchains in the cross-chain system, cross-chain contract in the sidechain, and the tracing authority.

Payer A creates an account storing certain exchanging Monero via **KeyGen** and **KeyExchange**. When Payer A generates one-time key pairs, also the partial key pairs of the joint account for Payer B, a tag is also produced for tracing. To swap Monero, Payer A hides the amount of cryptocurrency using homomorphic commitment in **SwapMonero.mint()**, employs the linkable ring signature to sign the transaction with hidden identity, and broadcasts the transaction into the Monero network via **SwapMonero.spend()**. When the transaction is confirmed in the Monero side, a proof for the confirmation of the transaction being in the block can be submitted to the cross-chain contract. Upon witnessing the proof, in **Swap Bitcoin**,



When tracing the long-term public key of a payer, the tracing authority can decrypt the ciphertext in the tag using its private key. To trace the one-time address of a payer, the tracing authority decrypts the ciphertext in the transaction, revealing the index  $s$  in the ring. Additionally, by exposing the real input account, the tracing authority can backtrack to accounts that transferred to the current account and move forward to locate subsequent accounts that received payments originating from the current account. This enables the tracing authority to trace the flow of money.

**Definition 9.1 (Privacy-Preserving Policy-Enforcement Cross-Chain Transaction Protocol (PCP))** The PCP is defined by the following algorithms/protocols:

- **Setup:** On input the security parameter  $\lambda$ , output public parameters  $pp$  (implicit input to all algorithms).
- **KeyGen:** For the tracing authority, generate a key pair  $(\omega, h_0^\omega)$ . Take the long-time public key  $(A_b, B_b)$  of Payer B as inputs, generate the public key of the joint account  $pk_{out}$ , with a corresponding tag for tracing.
- **KeyExchange:** Payer A encrypts and sends one partial private key  $sk_b$  with a correctness proof  $(R_{ae}, c_a, s_a)$  to Payer B. Cross-chain contract encrypts and sends the other partial private key  $sk_a$  with a correctness proof  $(R_{be}, c_b, s_b)$  to Payer A.
- **SwapMonero.mint:** Take as input the transaction amount  $a_{out}$  and output the Pedersen commitment of the value  $c_{out,l}$ .
- **SwapMonero.spend:** Construct transactions to joint accounts in  $A_{out}$  with corresponding encrypted value  $c_{out,l}$ . Output  $CT = (ct_1, ct_2)$  for tracing and proofs  $\pi_{tx}$  and  $\pi_{bl_m}$  to show the transaction is spent properly and is confirmed.
- **Swap Bitcoin:** In this protocol, transaction is constructed to transfer the corresponding amount of Bitcoin to  $addr_A$ , with the correctness proof  $\pi_{bl_b}$ .
- **Final Determination:** In normal situation, Payer B retrieves  $r_{be}$  from cross-chain contract and decrypts  $s_a$  for the partial key  $sk_b$ , taking control of the joint account. In unnormal situation, Payer A asks cross-chain contract for the partial key  $sk_b$  and takes the ownership of the joint account.
- **Trace:** To trace the one-time public key, on input  $CT = (ct_1, ct_2)$ , output the payer's index  $s$ . To trace the long-term public key, on input  $ct$ , output the payer's public key  $B$ .

### 9.3 Detailed Design of PCP

#### Phase 1. Initialization

Denote the description of the accumulator as  $\text{desc} = \text{ACC.GEN}(1^\lambda)$  and let  $\text{par} = \text{SoK.GEN}(1^\lambda)$ . Choose  $g, h_0, h_1, h_2, \tilde{h}, u \in \mathbb{G}_q$  randomly then the public parameter is  $pp = (1^\lambda, \text{desc}, \text{par}, g, h_0, h_1, h_2, \tilde{h}, u, H, H_{\text{prime}})$ , where  $H : \{0, 1\}^* \rightarrow \mathbb{G}_q$  denotes a collision resistant hash function,  $H_{\text{prime}}$  represents a hash function with division intractable outputs.

The miners in the relay blockchain run a verifiable secret sharing (VSS) scheme to create a specific key  $(sk_m, pk_m)$  using Algorithm **VSS.KeyGen** $(1^\lambda)$ .

*Key Generation* Tracing authority generates its key using Algorithm **KeyGen.ta**(pp).

---

**Algorithm 3** KeyGen.ta(pp)

---

- 1: Randomly Choose  $\omega \in \mathbb{Z}_q$ ;
  - 2: Compute  $z = h_0^\omega$ ;
  - 3: Output private/public key as  $(\omega, h_0^\omega)$
- 

When Payer A needs to pay to Payer B with long-term public key  $(A_b = h_0^{ab}, B_b = h_0^{bb})$ , Payer A generates a one-time public key with a corresponding tag for Payer B using Algorithm **KeyGen.A2B**(pp). When generating the tag,  $ac$  is the number of accounts to conceal the actual account in the “OR” proof.

---

**Algorithm 4** KeyGen.A2B(pp)

---

- 1: Randomly choose  $r_a \in \mathbb{Z}_q$ ;
  - 2: Compute  $R_a = h_0^{r_a}$ ;
  - 3: Let  $h = H(A_b^{r_a}, B_b)$ ,  $pk_b = B_b h_0^h$ ,  $sk_b = b_b + H(R_a^{a_b}, B_b)$ ;
  - 4:  $ct = z^h$ ; // Compute the tag
  - 5: tag =  $SoK\{((A_i, B_i), r_a): \bigvee_{i=1}^{ac} R_a = h_0^{r_a} \wedge pk_b = B_{b,i} h_0^{H(A_{b,i}^{r_a}, B_{b,i})} \wedge ct = z^{H(A_{b,i}^{r_a}, B_{b,i})}\}$ ;
  - 6: Send  $pk_b, ct, tag$  to Cross-chain Contract
- 

The Cross-chain Contract samples a licensing key  $r_b$  for Payer B, then the public key of the joint account is generated by licensing keys from both Payer A and Payer B.

---

**Algorithm 5** KeyGen.C2B(pp)

---

- 1: Randomly choose  $r_b = sk_a \in \mathbb{Z}_q$ ;
  - 2:  $pk_{out} = g^{sk_a} \cdot g^{sk_b}$ ; //For the joint account
  - 3: Send  $pk_{out}$  to Payer A
- 

*Key Exchange* The exchanging Monero is first locked in a joint account  $(g^a, g^{sk_a} \cdot g^{sk_b})$ , where  $a$  is the view key for just checking the balance of the account. The private key  $(sk_a, sk_b)$  of the account is shared by both payers at first, neither of the payers can spend cryptocurrencies in this account with only partial private key. Finally, Cross-chain Contract will assign the complete private key to one of the payers according to the exchange’s execution process. Before that, Payer A needs to send the partial private key in ciphertext to Payer B with a correctness proof using

Algorithm KeyExchange.A2B(pp,  $sk_b$ ) (Cross-chain to Payer A respectively, using Algorithm KeyExchange.C2A(pp,  $sk_a$ )).

---

**Algorithm 6** KeyExchange.A2B(pp,  $sk_b$ )
 

---

- 1: Randomly choose  $r_{ae} \in \mathbb{Z}_q$ ;
  - 2: Compute  $R_{ae} = g^{r_{ae}}$ ,  $c_a = H(R_{ae}, A_a)$ ,  $s_a = r_{ae} + sk_b c_a$ ;
  - 3: Send  $R_{ae}$ ,  $c_a$ ,  $s_a$ ,  $A_a$  to Payer B;
- 

---

**Algorithm 7** KeyExchange.C2A(pp,  $sk_a$ )
 

---

- 1: Randomly choose  $r_{be} \in \mathbb{Z}_q$ ;
  - 2: Compute  $R_{be} = g^{r_{be}}$ ,  $c_b = H(R_{be}, A_b)$ ,  $s_b = r_{be} + sk_a c_b$ ;
  - 3: Send  $R_{be}$ ,  $c_b$ ,  $s_b$  to Payer A
- 

They can verify the correctness of the ciphertext using  $g^s \stackrel{?}{=} A^c R_e$ .

If the verification succeeds, Payer A encrypts the encryption key of  $r_{ae}$  and submits it to the Cross-chain Contract in ciphertext  $e_a$  using Algorithm KeyExchange.A2C(pp,  $r_{ae}$ ), so that miners can decrypt it, verify its correctness using the proof  $R_{ae}$ , and send it to Payer B at the last phase.

---

**Algorithm 8** KeyExchange.A2C(pp,  $r_{ae}$ )
 

---

- 1:  $e_a \leftarrow \text{Enc}_{pk_m}(r_{ae})$ ;
  - 2: Send  $R_{ae}$ ,  $e_a$  to Cross-chain Contract
- 

## Phase 2. Construct Cross-Chain Transactions

In this phase, the cross-chain transactions are constructed in the Monero side and in the Bitcoin side successively.

*Swap Monero* Suppose there are  $n$  groups of input accounts.

$$A = \{(pk_{in,i}^{(k)}, cn_{in,i}^{(k)})\}_{1 \leq i \leq n, 1 \leq k \leq m}$$

In the system and the real account of Payer A is the  $s$  th group denoted by  $A_{in} = \{(pk_{in,s}^{(k)}, cn_{in,s}^{(k)})\}_{1 \leq k \leq m} = \{(pk_a^{(k)}, cn_a^{(k)})\}_{1 \leq k \leq m}$ . The public keys within the accounts can be seen as a matrix. Each group of public keys forms a column, and the individual public keys in each row  $k$  are combined into a single value  $v_k$  using an accumulator function  $f$ . Each group of public keys are set as  $\{pk_{in,i}^{(k)} \cdot u^i\}$ . In the final step of the matrix, an additional row is computed as  $\tilde{pk}_i \cdot u^i$  to guarantee the overall balance for every transaction.

Run KeyGen and KeyExchange for  $m$  times identical to the number of real input accounts of Payer A. The number of joint accounts is also  $m$ , and they are

denoted by  $A_{out} = \{(pk_{out}^{(l)}, cn_{out}^{(l)})\}_{1 \leq l \leq m}$ , where  $cn_{out}^{(l)}$  is the Pedersen commitment of the value of the transaction, generated by Algorithm SwapMonero.mint(pp,  $a_{out}$ ).

---

**Algorithm 9** SwapMonero.mint(pp,  $a_{out}$ )
 

---

- 1: Randomly choose  $r_{out,l} \in \mathbb{Z}_p$ ;
  - 2: Compute  $cn_{out}^{(l)} = c_{out,l} = \text{Com}(ctk, a_{out,l}; r_{out,l}) = h_0^{r_{out,l}} \cdot h_1^{a_{out,l}}$ , where  $\sum_{k=1}^m a_{in,s}^{(k)} = \sum_{l=1}^m a_{out,l} = x$ ;
  - 3: Send the coin key  $ck_{out,l} = (r_{out,l}, a_{out,l})$  corresponding address  $pk_{out}^{(l)}$  to Cross-chain Contract.
- 

Then Payer A constructs transactions from accounts in  $A_{in}$  to accounts in  $A_{out}$  with corresponding encrypted  $cn_{out}^{(l)}$  of value  $a_{out}$  and generates proofs to show that the transactions are spent properly.

---

**Algorithm 10** SwapMonero.spend(pp,  $m$ ,  $A$ ,  $A_{in}$ ,  $cn_{in}$ )
 

---

- 1: Sets  $\tilde{sk}_s = \sum_{k=1}^m sk_{in,s}^{(k)} + \sum_{k=1}^m r_{in,s}^{(k)} - \sum_{l=1}^m r_{out,l}$ ;
  - 2: Compute  $\tilde{pk}_i = \prod_{k=1}^m pk_{in,i}^{(k)} \cdot \prod_{k=1}^m cn_{in,i}^{(k)} / \prod_{l=1}^m cn_{out}^{(l)}$
  - 3: **for**  $k = 1$  to  $m$  **do**
  - 4:   Compute  $v_k = \text{ACC.Eval}(\text{desc}, \{pk_{in,i}^{(k)} \cdot u^i\})$ ;
  - 5:   Compute  $w_k = \text{ACC.Eval}(\text{desc}, \{pk_{in,i}^{(k)} \cdot u^i \mid i \neq s\})$ ;
  - 6:   Compute  $s_k = \tilde{h}^{sk_{in,s}^{(k)}}; // \text{serial number}$
  - 7: **end for**
  - 8: Randomly choose  $\tilde{a}$ ;
  - 9: Compute  $ct_1 = z^{\tilde{a}}, ct_2 = h_2^s \cdot h_0^{\tilde{a}}$ ;
  - 10: Compute  $\pi_{tx} = \text{SoK}(\{\{\omega_k, z_k, sk_s^{(k)}\}_{k=1}^m, s, \tilde{a}\} : ct_1 = z^{\tilde{a}} \wedge ct_2 = h_2^s h_0^{\tilde{a}} \wedge f(\omega_{m+1}, z_{m+1}) = v_{m+1} \wedge z_{m+1} = h_0^{sk_s^{(m+1)}} u^s \wedge f(\omega_1, z_1) = v_1 \wedge z_1 = h_0^{sk_s^{(1)}} u^s \wedge s_1 = \tilde{h}^{sk_s^{(1)}} \wedge \dots \wedge f(\omega_m, z_m) = v_m \wedge z_m = h_0^{sk_s^{(m)}} u^s \wedge s_m = \tilde{h}^{sk_s^{(m)}}\}(tx_m)$ ;
  - 11: Issue  $tx_m$  and send  $\pi_{tx}, S = \{s_1, s_2, \dots, s_m\}, \text{CT} = (ct_1, ct_2)$  to Cross-chain Contract;
- 

When  $tx_m$  is issued, miners in the Monero can verify it by computing  $\tilde{pk}_i = \prod_{k=1}^m pk_{in,i}^{(k)} \cdot \prod_{k=1}^m cn_{in,i}^{(k)} / \prod_{l=1}^m cn_{out}^{(l)}$ ,  $v_k = \text{ACC.Eval}(\text{desc}, \{pk_{in,i}^{(k)} \cdot u^i\})$  and  $v_{m+1} = \text{ACC.Eval}(\text{desc}, \{\tilde{pk}_i \cdot u^i\})$ . Then verify the SoK with  $(v_1, \dots, v_{m+1})$ , serial number  $S$ , transaction  $tx_m$ , and proof  $\pi_{tx}$  by checking

$$\text{SoK.Verf}(tx_m, ct_1, ct_2, (v_1, \dots, v_{m+1}, s_1, \dots, s_m), \pi_{tx}) \stackrel{?}{=} 1.$$

When  $tx_m$  is verified and included in the global ledger, Payer A then constructs a proof  $\pi_{bl_m}$  to prove that the exchanging transaction is packed in a confirmation block, where

$$\begin{aligned}\pi_{bl_m} &= SoK\{(tx_m, Blk_1) : ACC.Verify(wit, Acc, e) = 1 \\ &\wedge Com(ctk, tx_m; r_{tx_m}) = c_{tx_m} \wedge Com(ctk, Blk_1; r_{blk_1}) = c_{blk_1} \\ &\wedge H_{prime}(tx_m) = e\},\end{aligned}$$

and  $Acc$  is denoted as an accumulator of the set  $\{H_{prime}(tx_i)\}_{i \in Blk_1}$ .

*Swap Bitcoin* Payer B starts to issue  $tx_b$  transferring  $u$  BTC to the address  $addr_A$  owned by Payer A. First Payer B checks the total balance of the joint accounts with view key. If the balance equals  $x$  XMR, Payer B will create the transaction  $tx_b$  and compute the commitment  $c_{tx_b}$ , upload  $c_{tx_b}$  to the Cross-chain Contract and send  $tx_b$  and the decommitment message  $dc_{tx_b}$  to Payer A. Payer A verifies whether  $tx_b$  is sent to  $addr_A$  and the value of it is  $u$ . If the verification succeeds, Payer B sends  $tx_b$  to the global ledger. After  $tx_b$  is confirmed, Payer B generates a proof to state that  $tx_b$  is included in a confirmation block using

$$\begin{aligned}\pi_{bl_b} &= SoK\{(tx_b, Blk_2) : ACC.Verify(wit, Acc, e) = 1 \\ &\wedge Com(ctk, tx_b; r_{tx_b}) = c_{tx_b} \wedge Com(ctk, Blk_2; r_{blk_2}) = c_{blk_2} \\ &\wedge H_{prime}(tx_b) = e\},\end{aligned}$$

where  $Acc$  is denoted as an accumulator of the set  $\{H_{prime}(tx_i)\}_{i \in Blk_2}$ .

Payer B sends  $\pi_{bl_b}$  to the Cross-chain Contract.

*Final Determination* If both payers perform operations without interruption as agreed, Payer A has transferred  $x$  XMR in the joint accounts and each payer has owned one half private key and another half private key in the form of ciphertext. Meanwhile, Payer B has stored  $z$  BTC in Payer A's account. Thus, the possessor of the joint account should be delivered to Payer B enabling him to access  $x$  XMR. Miners will decrypt the encryption key  $r_{ae}$  with their  $sk_m$  and upload it to the Cross-chain Contract. Payer B retrieves  $r_{ae}$  from the contract and decrypts  $s_a$  for the partial key  $sk_b$ . Payer B can also get the coin key  $ck_{out,l} = (r_{out,l}, a_{out,l})$  corresponding to address  $pk_{out}^{(l)}$  from the contract. Now Payer B owns the whole private key of the joint accounts and can spend cryptocurrencies in it.

If Payer B is corrupted to make some misbehaviors, he/she could redeem his/her Monero by asking the Cross-chain Contract for the partial key  $sk_a$  and take the ownership of the joint accounts.

### Phase 3. Trace Monero

To trace the one-time public key using the ciphertext  $CT = (ct_1, ct_2)$  from the transaction  $tx_m$ , the tracing authority computes  $p = ct_2/ct_1^{\omega^{-1}}$  for decryption using his secret key  $\omega$ . He searches for  $s \in [1, n]$  by testing which  $s$  satisfies  $p = h_2^s$ ,



enabling the tracing of the corresponding public key. To prove that  $s$  is the payer's index within the group, the tracing authority can generate a proof  $\phi'$  using  $SoK\{(\omega) : z = h_0^\omega \wedge ct_1 = (ct_2/h_2^s)^\omega\}(s)$ .

To trace the long-term public key, tracing authority can decrypt the tag by computing  $B = pk/ct'^{\omega^{-1}}$  and then would get the corresponding long-term public key  $B$ . By producing the  $SoK\{(\omega) : z = h_0^\omega \wedge ct' = (pk/B)^\omega\}$  as a proof  $\phi$ , the tracing authority can prove himself that the tag is opened correctly and  $B$  indeed represents the payer.

The proof  $\phi$  is publicly verifiable. With the inputs including the payer's public key, the proof generated by the tracing authority, the tracing authority's public key  $z$ , and the ciphertext, anyone can validate the proof  $\phi$  and  $\phi'$  by checking  $\text{Verf}(B, (z, pk, CT), \phi) \stackrel{?}{=} 1$  and  $\text{Verf}(s, (z, ct_1, ct_2), \phi') \stackrel{?}{=} 1$ , respectively.

## 9.4 Security Analysis

**Theorem 9.1 (Anonymity)** *The proposed scheme achieves anonymity if SoK satisfies simulatability and extractability and commitment is perfect hiding.*

**Proof** We employ game-based framework to present our proof. We denote  $Pr[\text{Win}]$  as the winning probability of an adversary guessing correctly the actual sender. We denote the challenge transaction as  $(tx^*, \pi^*, S^*, C^*)$ , where  $tx^* = (A, A_{in}^{(b)})$ ,  $S^* = (s_1^{(b)}, s_2^{(b)}, \dots, s_m^{(b)})$  and  $C^* = (ct_1, ct_2^{(b)})$ . The adversary outputs a guess  $b'$ , then  $Pr[\text{Win}] = Pr[b' = b]$ .

The simulator runs the **Gen** algorithm to generate **par** and the **Sign** algorithm to generate the SoK signature, respectively. During the challenge phase, the *Spend* and challenge queries are computed without using the witness, while the simulator chooses the output address  $pk_{out,j} \in R$  randomly. Specifically, for a challenge query  $(m, A_{s0}, A_{s1}, A, R)$ , the simulator randomly chooses  $\alpha$  and sets  $h_1 = h_0^\alpha$ , then changes the output address  $pk_{out,j}$  by picking a random  $\hat{r}_j$  and setting  $cn_{out,j} = h_0^{\hat{r}_j}$  for  $1 \leq j \leq t$ , thus the output account  $A_{out} = (pk_{out,j}, cn_{out,j})_{j=1}^t$  is independent of  $b$ , while calculating the statement and the corresponding witness as normal, while computes the SoK:  $\pi^* = \text{Sim}(tx^*, ct_1, ct_2^{(b)}, (v_1, v_2, \dots, v_{m+1}^{(b)}), s_1^{(b)}, \dots, s_m^{(b)})$  without the knowledge of witness. In this case, we also have  $\tilde{pk}_i = \prod_{k=1}^m pk_{in,i}^{(k)} \cdot \prod_{k=1}^m cn_{in,i}^{(k)} / \prod_{l=1}^m cn_{out}^{(l)}$  independent of  $b$  and so as  $v_{m+1} = \text{ACC.Eval}(\text{desc}, \{pk_{in,i}^{(k)} \cdot u^i\}_{i=1}^n)$ . The adversary also can be used to construct algorithms to violate **Ext** of SoK and break the perfect hiding property of commitment.

Due to the extractability security of our employed SoK and the commitment is perfect hiding, the challenge transaction  $(tx^*, \pi^*, S^*, C^*)$  is independent of  $b$ , thus this setting leaks no information about  $b$  to the adversary. So, the probability of the adversary in winning the game  $Pr[\text{Win}]$  is  $1/2 + \epsilon$ , where  $\epsilon$  is negligible.  $\square$

**Theorem 9.2 (Traceability)** *The proposed scheme achieves traceability if the SoK is sound and the encryption and SoK are with perfect correctness.*

**Proof** Take one wallet from the group as an example to prove the traceability. If the SoK  $\pi_{tx}$  and  $\pi_{bl}$  is sound and valid, it implies that  $ct_1$  and  $ct_2$  are in the required form, thus the perfect correctness guarantees that  $h_2^s$  could be uniquely recovered. Moreover,  $s$  is the index of the one-time public key in the group, which is a small set and can be identified easily. Once the one-time public key  $pk_b$  is decided, by the soundness of SoK in *KeyGen*, we can extract the element  $r_a$  and the key pair  $(A_b, B_b)$  and  $R_a, pk_b, ct$  are in the required form. We now need to prove that given a one-time key  $pk_b = B_b h_0^{H(A_b^{r_a}, B_b)}$ , it is computational infeasible to find another key pair  $(A'_b, B'_b)$  as a collision that satisfies  $pk_b = B'_b h_0^{H(A_b^{r_a}, B'_b)}$ . We can see from the equation that, for a fixed  $pk_b$ , one can choose an arbitrary  $B'_b$ , and the possibility to find a  $A'_b$  s.t.  $\log_{h_0}(y/B'_b) = H(A_b^{r_a}, B'_b)$  is negligible, if we treat the hash function  $H$  as a random oracle. In this manner, we affirm that  $(A_b, B_b)$  constitutes the real payee, validated by this proof. Furthermore, due to the accuracy of variant ElGamal encryption, we can confirm the correctness of decryption, allowing us to obtain  $B = pk/ct^{\omega^{-1}}$ . The other aspect of the Trace Monero phase's output is a SoK proof  $\phi$ , generated by the tracing authority using their private key. This proof attests to the proper generation of the tracing output according to the protocol. The accuracy of  $\phi$  can be verified based on the correctness and soundness of the SoK scheme. Therefore, our proposal ensures traceability if SoK is sound as well as the encryption scheme and SoK are with perfect correctness.  $\square$

**Theorem 9.3 (Unlinkability)** *The proposed scheme achieves unlinkability if SoK satisfies extractability, homomorphic commitment is perfect hiding.*

**Proof** Based on the SoK and homomorphic commitment, the proposed scheme achieves anonymity, which means that users can state the transactions correlated with a cross-chain exchange are correct and included in confirmed blocks without revealing the actual transactions. Since the adversaries cannot disclose the specific identity of the sender in both Monero and Bitcoin side and then link the identical identity related to the same cross-chain exchange in another blockchain, they cannot utilize the identity information to destroy privacy. So, the probability of adversary in winning the unlinkability game is  $1/2 + \epsilon$ , where  $\epsilon$  is negligible.  $\square$

## 9.5 Performance Evaluation and Analysis

In this section, we assess and analyze the computation cost of our scheme in each phase. All experiments are undertaken on a laptop with 64-bit Win 10 operating system and 16.0 GB RAM. The processor is AMD Ryzen 5 5600U @ 2.30 GHz. The programs are written in JAVA language with IntelliJ IDEA. We use the Fiat–Shamir paradigm incorporated with zero-knowledge protocol to instantiate SoK and the

**Table 9.1** Computation cost (ms) in each phase

| Phase               | Payer A | Payer B | Miners  | Tracing authority |
|---------------------|---------|---------|---------|-------------------|
| Initialization      | 341.10  | 2.20    | 15.80   | 0.79              |
| Swap Monero         | 150.53  | -       | 3291.95 | –                 |
| Swap Bitcoin        | –       | 39.21   | 2249.49 | –                 |
| Final determination | –       | -       | 3.30    | –                 |
| Trace Monero        | –       | -       | -       | 6.33              |

Java Pairing-Based Cryptography Library (JPBC) to implement the basic operation of the signature and encryption scheme. It is worth stressing that all experiments are conducted 200 times in the testing environment.

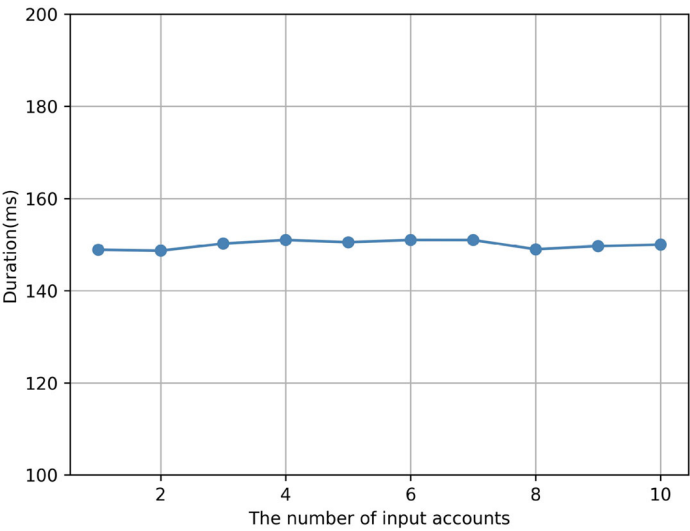
When swapping Monero, one can determine how many accounts would be accumulated in a group to hide the real public key of Payer A. And the system would compute a SoK for every cross-chain transaction (not for the ordinary transfer transaction) confirmed in the block, so the number of cross-chain transactions in the underlying blockchain would also influence the computation cost. After comprehensively considering the average throughput of cryptocurrency and the realizability of the experiment, we set the basic configuration as 5 for the number of input accounts in the side of Monero, 100 for the number of transactions in each block of each underlying blockchain, and 50% for the proportion of transactions related to the cross-chain exchange payment in each block.

The experimental results are shown in Table 9.1. The main cost for Payer A is in computing the accumulator and the SoK in the Initialization phase and the Swap Monero sub-phase, while for miners, the primary computation cost is caused by proof verification in the Swap Monero sub-phase and the Swap Bitcoin sub-phase. The whole process for one round of exchange between Monero and Bitcoin costs about 60.94 s. For the tracing authority, it just costs 7.12 ms to trace the public key of Payer A.

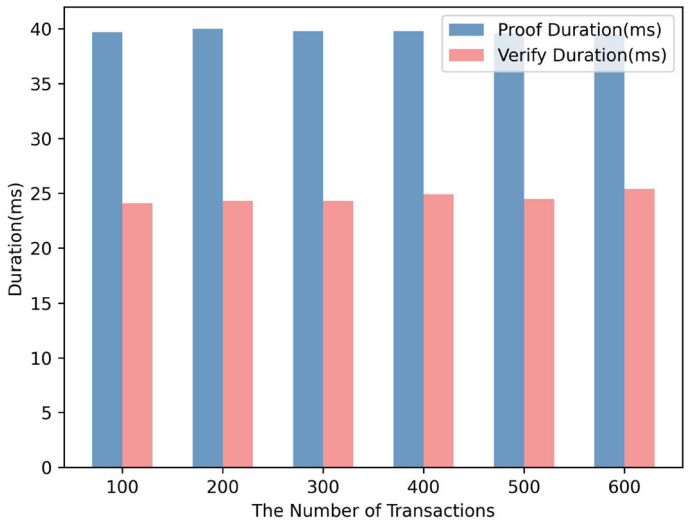
To explore the relationship between the number of input accounts in a group of Payer A in Monero and the computation cost of the Swap Monero phase, we change the number from 1 to 10. We can see from Fig. 9.2 that the overheads of the Swap Monero phase, also the time cost for generating the SoK in Swap Monero phase are almost constant. This aligns with our practical observations, as the overhead is generated on the column number  $s$ , and it’s not conditional on the number of input accounts.

To measure the overhead of increasing the number of transactions in the underlying blockchain, we increased the number of transactions from 100 to 600 per block and evaluated the change in proof generation time and verification time. As shown in Fig. 9.3, the verification time and proof time exhibit minimal changes as the number of transactions increases. So, the cost can be negligible.

Further, we increased the percentage of transactions related to cross-chain exchange payments in a block from 10% to 100%, as a measure of miners’ verification time, while the number of transactions range from 100 to 600. As



**Fig. 9.2** The time cost of Swap Monero phase with different number of input accounts



**Fig. 9.3** The impact of the number of transactions in each block for miners to verify the proof

depicted in Fig.9.4, the execution time rises proportionally with the increase in proportion, and it also increases with the number of transactions bit by bit. Specifically, even if all quantities reached their maximum, the verification time is acceptable.

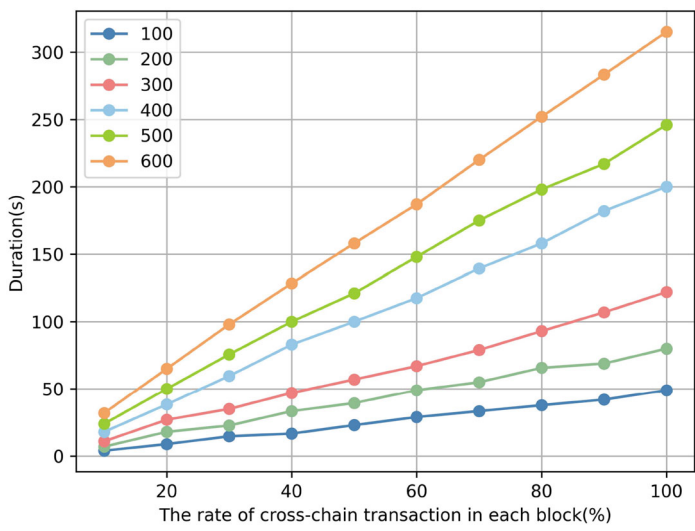


Fig. 9.4 The verification time of miners with the proportion of transactions

9.6 Summary

Motivated by the major challenge in decentralized cryptocurrencies of balancing users’ privacy and traceability, we propose PCP, a privacy-preserving policy-enforcement cross-chain cryptocurrency exchange scheme between Monero and Bitcoin. By leveraging the protocol, payers from different blockchain network can exchange cryptocurrencies without disclosing any details of the transaction, while the anonymity can be revoked if the payer misbehaves. Then we define the security of PCP and prove it achieves anonymity, unlinkability, and traceability. Finally, we evaluate the efficiency of PCP, the experiment results show that the protocol is cost-reasonable with constant 150 ms for Swap Monero phase, 40 ms for proof generation, and 24 ms for proof verification. This work has been published in IEEE TIFS 2024 (Jiang et al. 2024).

References

Jiang P, Zhu J, Zhu L (2024) Balancing privacy and regulation of cross-chain transaction systems via sok-assisted policy enhancement. *IEEE Trans Inf Forensics Secur* 19:5620–5629

Lei Z, Zhu Y, Zhang J, et al (2021) Design of a supervised blockchain cross chain platform. *Comput Digit Eng* 49(12):2544–2550

Li Y, Weng J, Li M, Wu W, Weng J, Liu J, Hu S (2022) Zerocross: a sidechain-based privacy-preserving cross-chain solution for monero. *J Parallel Distributed Comput* 169:301–316

- Mohurle S, Patil M (2017) A brief study of wannacry threat: ransomware attack 2017. *Int J Adv Res Comput Sci* 8(5):1938–1940
- Xie T, Zhang J, Cheng Z, Zhang F, Zhang Y, Jia Y, Boneh D, Song D (2022) zkbridge: trustless cross-chain bridges made practical. In: Yin H, Stavrou A, Cremers C, Shi E (eds) *CCS 2022*. ACM, New York, pp 3003–3017